الإصدار الأول · 2026

PyTorch و Keras: الدليل الشامل في التعلم العميق

من الصفر إلى الاحتراف: بنية الشبكات العصبية، الخوارزميات، والأمثلة المحلولة وتطبيقاتها في الحياة العملية

كتاب عربي شامل ومحدَّث يجمع **PyTorch 2.14** و**Keras 3.15** في إطار واحد، على **Python 3.14**، بأسلوب سردي استفهامي: من الخلية العصبية إلى المحوّلات، ومن أول سطر برمجي إلى نموذج يعمل في الإنتاج.

33	5	+180	+40	+120
فصلاً دراسياً	ملاحق مرجعية	مثال برمجي محلول	مخططاً ورسماً	تمريناً مع حلّها

م/أحمد صالح المغز

إعداد وتصنيف — رحلة تعلّم عملية ومنهجية

الإصدار: الإصدار الأول (1.0)

بيئة العمل: TensorFlow 2.21 · Python 3.14 · Keras 3.15 · PyTorch 2.14 (حتى Python 3.13)

في عام 1943 كتب ماكولوك وبيتس ورقةً من أربع صفحات عن «الخلية العصبية المنطقية»، وفي عام 2026 كان نموذجٌ واحدٌ يعالج مليارات النصوص دون أن يفهم كلمةً واحدة منها. ما بين ذلك مرّت سبعون سنة، سبعٌ منها فقط كانت كافية لتحويل فكرةٍ مجرّدة إلى بنية تحتية اقتصادية كاملة. هذا الكتاب يبدأ من تلك الفكرة.

تمهيد: لماذا كُتب هذا الكتاب؟

في كل مرة أفتح فيها مستودعاً على GitHub وأرى مجلّداً اسمه `models` يحوي عشرة تجارب لا يخبرنا أحدٌ كيف نجحت، تُستعيد المشكلة من جديد: **المعرفة في التعلّم العميق مكتوبة بالدرجة الأولى كشيفرة، لا كشرح**. الشيفرة تخبرك *كيف* نفّذتَ، لكنها لا تخبرك *لماذا* اخترتَ، ولا متى تنكسر، ولا ماذا يحدث إن غيّرتَ سطراً واحداً.

الكتب العربية في هذا المجال شحيحة، والكتب الإنجليزية ممتازة لكنها موجّهة لمن يقرأ الإنجليزية بطلاقة تقنية. النتيجة أن كثيراً من المهندسين العرب تعلّموا بالتجربة: ينسخون دفتراً من GitHub، ويتعلّمون الأخطاء بالترميز، ويفهمون «لماذا» بعد سنوات – إن فهموها أصلاً.

هذا الكتاب مبنيّ على مبدأ واحد: **لا يوجد فصل بلا مثال، ولا مثال بلا شرح، ولا تمرين بلا حلّ.** كل معادلة تسبقها جملة تشرح معناها بالرموز لا بالرموز وحدها. كل شيفرة يسبقها ما الذي تفعله، ويتبعها ما الذي يحدث في الذاكرة. وكل تمرين يقابله حلّ نموذجي كامل في الملحق (د).

قبل أن نبدأ: ما الذي يجعل كتاباً في التعلّم العميق «جيداً» في رأيك؟ هل هو عدد الصفحات؟ أم عدد المعادلات؟ أم عدد الشيفرات؟

الجواب الذي توصّلتُ إليه بعد سنوات قراءةٍ وتدريسٍ: كتابٌ جيد هو الكتاب الذي يخبرك **متى لا تُستخدم** التقنية التي يشرحها. لأنّ هندسة البرمجيات ليست مَن يعرف كيف يبني، بل مَن يعرف متى لا يبني. لهذا ستجد في كل فصل صندوقاً صغيراً بعنوان «متى لا تستخدم هذا».

ما الذي يجعل هذا الكتاب مختلفاً؟

الغالبية العظمى من مراجع التعلّم العميق تختار أحد الطريقين: إمّا كتاب *PyTorch* وإمّا كتاب *Keras/TensorFlow*. هذا الكتاب يختار الطريق الثالث: **يعاملك كمهندس، لا كمتلقٍّ**. كل مفهوم يُشرح مرّتين: مرة من زاوية PyTorch ومرة من زاوية Keras، مع جدول مقارنة صريح بينهما، لأنك في سوق العمل اليوم ستلتقي بالمكتبيتين في المشروع الواحد.

السمة	كيف تُطبَّق في هذا الكتاب
السرد الاستفهامي	نبدأ من مشكلة واقعية، ثم نستنتج الأداة، لا العكس
المعادلات مشروحة	كل رمز يُشرح بجملة، مع «خطأ شائع» و«متى تهمّنا»
المكتبيتان معاً	مقارنة صريحة بين PyTorch و Keras في كل فصل ذي صلة
التمارين والحلول	تمرين في كل فصل، وحلّه النموذجي في الملحق (د)
الربط بالواقع	دراسة حالة كاملة: بيانات ← نموذج ← قرار ← قياس
حدود الاستخدام	لكل تقنية صندوق «متى لا تستخدم هذا» – وهذا جوهر الخبرة

ما الذي يميّز بنية هذا الكتاب عن المراجع الأخرى

البيئة المرجعية للكتاب

كل الشيفرة في هذا الكتاب كُتبت ومُختبرة على **Python 3.14** مع **PyTorch 2.14** و **Keras 3.15**. هذه نقطة دقيقة تستحق التوضيح، لأنها مصدر فخاخ كثيرة:

- **Keras 3** مكتبة مستقلة تمامًا، وإصدارها هنا هو `3.15.1` ، وتعمل على Python 3.11 فأحدث.
- **TensorFlow 2.21** ما زال يقتصر على **Python 3.13** فأقدم في عجلات PyPI الرسمية؛ فإن كنت على 3.14 فأنت **مضطر** إلى نواة `torch` أو `jax` في Keras. لهذا يظهر في الكتاب الأمر التالي بدل الاعتماد على TensorFlow:

أوامر الطرفية

```
$ # اختيار نواة Keras 3 على Python 3.14
$ $ pip install keras==3.15.1 torch==2.14.1
$ $ python -c "import os; os.environ['KERAS_BACKEND']='torch'; import keras; print(keras
$ torch
```

⚠ انتبه: تغيير نواة Keras ليس تفصيلاً ثانوياً

بعض سلوكيات الطبقات في Keras 3 تختلف قليلاً حسب النواة (سلوك التخزين العشوائي، بعض عمليات التطبيع، ودعم `tf.data` الذي يقتصر على نواة TensorFlow). لذلك يذكر الكتاب دائماً أي نواة يفترضها المثال.

كيف تتنقّل بين فصل وآخر

الكتاب مبني في أربعة أجزاء متدرّجة: **الأسس** (لماذا وفكيف)، ثم **غوص عميق في PyTorch** (من الTensor إلى المحوّلات)، ثم **Keras 3** (المكتبة الأخرى)، ثم **الهندسة والحياة العملية** (الأداء والنشر والدراسات والأخلاقيات). اقرأ بالترتيب أول مرة، ثم استخدمه كمرجع حسب الفصل.

إن كنت خبيراً وتريد الوصول السريع: ابدأ بالملحق (أ) «خريطة المقابلة بين PyTorch و Keras»، ثم اقرأ الفصل الذي تحتاجه، ثم عُد إلى الملحق (ج) «بنك المعادلات» متى احتجت تثبيت مفهوم رياضي.

لو اضطررتَ اليوم إلى حلّ مسألة جديدة في عملٍ ما: هل تختار PyTorch أم Keras؟

سؤالٌ ممتاز، وله جوابٌ عمليّ ستجده في الملحق (أ) وفي الفصل التاسع عشر. القاعدة التقريبية هي **اختر Keras للسرعة في الوصول إلى نموذج يعمل، واختر PyTorch حين تحتاج أن تفهم ما يحدث داخل النموذج أو تعدّل سلوكه بشكل غير معتاد**. أما حين تجتمع الحاجتان، فقد لا تعني «مكتبة واحدة»، بل **نموذج في PyTorch يُصدَّر ويُخدم عبر FastAPI أو عبر طبقة استدلال في Keras** – وهذا بالضبط ما سنفعله في دراسة الحالة في الفصل السابع والعشرين.

★ وعد الكتاب

إذا أنهيت الكتاب كاملاً فستكون قادراً على: بناء شبكة عصبية من الصفر دون مكتبات جاهزة، وتفسير لماذا ينهار التدريب، واختيار استراتيجية التعلّم النقلي المناسبة، ونشر نموذجك كخدمة تعمل تحت الضغط – بل وأخيراً أن تقرأ ورقة بحثية جديدة دون أن تشعر بالحيرة.

خريطة الطريق داخل الكتاب

الكتاب أربعة أجزاء. إن كانت لديك أربعة أسابيع فقط، فاتبع هذا المسار. إن كنت تملك وقتاً، فاقرأ بالترتيب: كل جزء يبني على سابقه.

الجزء	المحتوى	الوقت المقترح	ناتج متوقّع
الأول: الأسس	لماذا نتعلّم بعمق، الرياضيات، الخلية العصبية، الانتشار العكسي، البيئة	2-3 أسابيع	تستطيع شرح شبكة لزميل غير تقني، وتبني شبكة يدوياً بلا مكتبات
الثاني: PyTorch	Tensor، nn.Module، autograd، البيانات، حلقة التدريب، CNN، RNN، Transformers، الكشف، التوليد	4-6 أسابيع	نموذج كامل من الصفر، مع فهم عميق لسبب كل سطر
الثالث: Keras 3	المعمارية متعددة النواة، الطبقات، الجسر مع PyTorch، fit، البيانات، النماذج الجاهزة	2-3 أسابيع	نموذج جاهز للإنتاج بأقل عدد أسطر، وفهم متى تختار Keras
الرابع: الهندسة	الأداء، النشر، دراسات الحالة، الأخطاء الشائعة، الأخلاقيات، المسار المهني	2-3 أسابيع	نموذج في الإنتاج + قرارات واثقة

ماذا تتعلّم في كل جزء، وكم من الوقت يحتاجه

ثلاث طرق لقراءة الكتاب

القارئ المبتدئ

اقرأ بالترتيب الكامل، ونفّذ كل مثال بيدك قبل الانتقال، وحلّ التمارين قبل النظر في الحل. لا تتخطَّ الفصل الثاني (الرياضيات) ولا الرابع (الانتشار العكسي): المتابعة في بقية الكتاب مبنية عليهما.

المهندس المحترف الجديد في التعلّم العميق

ابدأ بالفصلين السادس والسابع (بناء النموذج في PyTorch)، ثم اقرأ الفصل العاشر (حلقة التدريب) والعاشرون (CNN) والثاني عشر (النقل)، ثم انتقل مباشرة إلى الجزء الرابع. استخدم الملحق (أ) كخريطة مرجعية دائمة.

مطوّر Keras قادم إلى PyTorch

اقرأ الفصلين التاسع عشر (فلسفة Keras 3) والحادي والعشرين (الجسر بين المكتبتين)، ثم الفصل السادس والسابع والثامن لتفهم mechanism التلقائية (autograd) التي لا تملك Keras مكافئاً مباشراً لها.

باحث أو طالب ماجستير

ابدأ من الملحق (ج) بنك المعادلات، ثم اقرأ الفصول 2 و4 و15 (المحوّلات) و17 (التوليد) و18 (التعلّم الذاتي). ثم انتقل إلى الفصل الحادي والثلاثين لخارطة طريق القراءة العلمية.

شيفرة الكتاب المرافقة

كل مقطع شيفرة في الكتاب مكتوب بحيث يمكن نسخه وتشغيله مباشرة. أمثلة الكتاب الشائعة:

الحزمة	الإصدار المرجعي	الدور في الكتاب
Python	3.14	لغة التنفيذ؛ تدعم `match` و `type` aliases الجديدة
PyTorch	2.14.1	المكتبة الأولى: بناء النماذج والتدريب والنشر
Keras	3.15.1	المكتبة الثانية: واجهة عالية المستوى متعددة النواة
NumPy	x.2	الحسابات العددية على المصفوفات
Matplotlib	+3.10	رسم منحنيات التدريب والمخططات
scikit-learn	+1.7	المعالجة المسبقة، التقسيم، المقاييس
Pillow	+11	قراءة الصور ومعالجتها

مكتبة الأدوات المستخدمة في أمثلة الكتاب

أوامر الطرفية

```
$ # إنشاء بيئة عمل نظيفة للكتاب (Python 3.14)
$ $ py -3.14 -m venv .venv
```

```
$ $ .\.venv\Scripts\Activate.ps1
$ $ pip install -U pip
$ $ pip install torch==2.14.1 --index-url https://download.pytorch.org/whl/cpu
$ $ pip install keras==3.15.1 numpy matplotlib scikit-learn pillow
$ $ python -c "import torch, keras; print(torch.__version__, keras.__version__)"
```

✓ نصيحة: استخدم عزلاً حقيقياً لكل مشروع

لا تثبّت حزم التعلّم العميق في بايثون النظام. أنشئ بيئة افتراضية لكل مشروع، واحتفظ بنسخة مقفلة من المتطلبات (requirements.txt أو uv.lock). الفرق بين «نموذج يعمل» و«نموذج ما زال يعمل بعد ستة أشهر» هو غالباً ملف قفل واحد.

التمارين والحلول

كل فصل ينتهي بتمارين في صناديق مميّزة. التمرين الأول دائماً «فهم المفهوم» (وهو إجابة مباشرة في نصّ الفصل)، والباقي متدرّج في التطبيق. الحلّ النموذجي لكل تمرين في الملحق (د)، لكن حاول حلّه أولاً: الفائدة من الخطأ الذي صحّحته بنفسك أكبر بكثير من قراءة الحل.

⚙ بروتوكول التمرين المقترح

1. اكتب الحل في ورقة، بلا نسخ من الحل في الملحق.
2. نفّذه في `notebook` واطبع المخرجات (المخرجات المتوقّعة مذكورة في التمرين).
3. اكسر الشيفرة عمداً (غيّر البُعد، ارفع معدّل التعلّم، احذف `zero_grad`) ثم صف ما حدث.
4. قارن قراءتك بالحل النموذجي، وصحّح.

عن الأرقام في هذا الكتاب

الأرقام (الدقة، زمن التدريب، أحجام النماذج) مذكورة لتوضيح المقاييس النسبية، وهي مأخوذة من تجارب مرجعية على عتاد محدَّد. ستختلف أرقامك، لكن **الترتيب النسبي** بين الخيارات يبقى كما هو: نموذج ResNet-50 أسرع في التدريب من ViT-L، وBatchNorm أسرع من LayerNorm على دفعات كبيرة، وAMP يوفّر نحو نصف الذاكرة. عِلِّم على هذه القواعد لا على الأرقام المطلقة.

فهرس الكتاب

يتضمّن الكتاب أربعة أجزاء رئيسية، وثلاثة وثلاثين فصلاً، وخمسة ملاحق مرجعية. الفهرس قابل للنقر في النسخة الرقمية.

الجزء الأول: الأسس

لماذا نتعلّم بعمق؟ وكيف يفكّر النموذج فعلاً؟

الأول التعلّم العميق: لماذا الآن؟ **12**

المعنى، التاريخ، من أين أتت هذه الفكرة وإلى أين تتجه

الثاني الأساسيات الرياضية للتعلّم العميق **26**

المتجهات والمصفوفات والتمايز والاشتقاق والاحتمال والتحسين

الثالث من الخلية العصبية إلى الشبكة **42**

المدرّج، الشبكة متعددة الطبقات، الدوال الفعّالة، والقدرة على التمثيل

الرابع كيف تتعلّم الشبكة؟ الانتشار العكسي ونزول التدرّج **52**

سلسلة قوانين السلسلة، دوال الخسارة، المحسّنات، ومعدّل التعلّم

الخامس بيئة عمل احترافية على Python 3.14 ..... **67**

البيئة الافتراضية، الحزم، العتاد، الحتمية، وهيكل المشروع

الجزء الثاني: غوص عميق في PyTorch

من Tensor إلى المحوّلات: بناء وتدريب ونشر نماذج إنتاجية

السادس Tensor: حجر الأساس في PyTorch **82**

الأنواع، الأشكال، الأجهزة، البث، الفهرسة، والتبديل بين الذاكرة

السابع تشريح nn.Module **98**

كيف تُبنى النماذج: المعاملات، التهيئة، الحفظ والتحميل، والأدوات المساعدة

الثامن Autograd: محرّك التمايز التلقائي **117**
الرسم البياني للحساب، inference_mode، no_grad، والخطوات الشائعة في الأخطاء

التاسع هندسة البيانات: من المصدر إلى الدفعة **132**
Dataset وDataLoader والعاملات والتحويلات والتحميل غير المتزامن

العاشر حلقة التدريب الاحترافية **146**
AMP، التدرّج المحدود، EMA، نقاط الحفظ، الجدولة، والقياس

الحادي عشر التنظيم والتعميم **171**
التهيئة، Dropout، Weight Decay، التطبيع، Mixup، وحدود النموذج

الثاني عشر الشبكات التلافيفية CNN **183**
الأنوية، الالتفاف، التجميع، الحقل الاستقبالي، وبنى ResNet

الثالث عشر التعلّم النقلي والتدرّج الدقيق **210**
استخراج الميزات، التدرّج الدقيق، معدّلات التعلّم المتمايزة، ومتى تستخدم كل منها

الرابع عشر الشبكات المتكرّرة وتسلسليات البيانات **223**
RNN وLSTM وGRU، ومقرّرات الانتشار العكسي عبر الزمن، والتطبيقات

الخامس عشر الانتباه والمحوّلات Transformers **248**
الانتباه متعدّد الرؤوس، الترميز الموضعي، ViT وBERT وGPT، وLoRA

السادس عشر الكشف والتجزئة على الصور **262**
الصناديق، U-Net، R-CNN، YOLO، NMS، IoU، ودالة Dice

السابع عشر النماذج التوليدية **288**
GAN وVAE ونماذج الانتشار: كيف يولّد النموذج بيانات جديدة

الثامن عشر التعلّم الذاتي والنماذج التأسيسية **300**
التضمين، التباين، SimCLR وMAE، والتدرّج الدقيق لنماذج اللغة

الجزء الثالث: Keras 3

المكتبة الأخرى: نفس القوة بأسلوب واجهات برمجية مختلفة

التاسع عشر Keras 3: الفلسفة والمعمارية **310**
الواجهات المتعدّدة، أنماط النماذج الثلاثة، وما الذي تغيّر عن Keras 2

العشرون الطبقات والنماذج في Keras **324**
واجهة Layers، توليد الأشكال، الطبقات المخصّصة، والحفظ والتحويل

الحادي والعشرون Keras 3 فوق نواة PyTorch **338**
الجسر بين المكتبتين: tensors، الأجهزة، Custom Training Step، والتحويل

الثاني والعشرون حلقة التدريب و Callbacks في Keras **350**
compile وfit والاستراتيجيات الموزّعة وCallbacks والقياس

الثالث والعشرون خط البيانات في Keras **361**
tf.data وKerasPyDataset وطبقات التعزيز وTF-IDF والتسلسل

الرابع والعشرون النماذج الجاهزة وKerasHub **373**
Applications وKerasNLP وKerasHub ووصفات النقل بين المكتبتين

الخامس والعشرون مشاريع Keras عملية **385**
تصنيف وانحدار ونصوص وسلاسل زمنية وجدولة، من الفكرة إلى النموذج

الجزء الرابع: الهندسة والحياة العملية

الأداء، النشر، دراسات الحالة، الأخلاقيات، والمسار المهني

السادس والعشرون الأداء والتسريع **401**
الدقة المختلطة، torch.compile، التكميم، التقطيع، والتوزيع على عدة أجهزة

السابع والعشرون النشر والتكامل مع الإنتاج **414**
TorchScript وONNX وSavedModel وTFLite وFastAPI والمراقبة والانحراف

الثامن والعشرون دراسات حالة من الحياة العملية **429**
الصحة، التمويل، الصناعة، التجزئة، التوصية، والطاقة: من البيانات إلى قرار

التاسع والعشرون تصحيح الأخطاء والأخطاء الشائعة **441**
من رسالة الخطأ إلى السبب الجذري: دليل عملي في تشخيص المشكلات

الثلاثون الأخلاقيات والتحيّز والخصوصية والأمن **453**
العدالة، الخصوصية التفاضلية، الهجمات العدائية، وحقوق النموذج

الحادي والثلاثون خارطة الطريق المهنية **464**
اثنا عشر شهراً، معرض الأعمال، المقابلات، وقراءة الأبحاث

الملاحق

مراجع سريعة تُستشار أثناء العمل

الملحق A خريطة المقابلة بين PyTorch و Keras **474**
جدول شامل: كل مفهوم بمقابله في المكتبتين

الملحق B مسرد المصطلحات **483**
من عربي إلى إنجليزي ومن إنجليزي إلى عربي

الملحق C بنك المعادلات والخلاصات التشغيلية **490**
كل معادلة في الكتاب في مكان واحد، مع شرحها ورمزها

الملحق D التمارين والحلول النموذجية **497**
تمارين إضافية من كل فصل مع الحلول الكاملة

الملحق E المراجع والمصادر **507**
الوثائق الرسمية، الأبحاث المؤسِّسة، والكتب والدورات

التعلّم العميق: لماذا الآن؟

المعنى، التاريخ، من أين أتت هذه الفكرة وإلى أين تتجه

تخيّل أنك موظف في مكتبة ضخمة لا تنام. كل يوم يصل إليك مئات آلاف الكتب، ولا وقت لقراءة أيّ منها. فكّرت في آلة تقرأ كل كتاب، تستخرج جوهره، وتلخّصه، وتوصي بالكتاب التالي. هذه الآلة موجودة الآن، ويُعرف الآن باسم «نموذج لغوي كبير»، وهذه ثمرة سبعين سنة من البحث في التعلّم العميق. هذا الفصل يجيب عن سؤال البداية: من أين أتت هذه الفكرة، ولماذا نجحت الآن، وأين تُستخدم، وأين لا تصلح.

ما التعلّم العميق فعلاً؟

التعريف الشائع: **التعلّم العميق (deep learning)** هو نوع من التعلّم الآلي (machine learning) يعتمد على شبكات عصبية متعددة الطبقات، تُدرَّب على أمثلة كثيرة، فتتعلّم تمثيلاً متدرّجاً للبيانات: الطبقة الأولى تلتقط الحواف، والثانية الزوايا، والثالثة أجزاء الكائنات، والرابعة أجساماً كاملة.

التعريف الأدق – وهو الذي سأستخدمه في هذا الكتاب – ثلاث صفات تجتمع معاً:

الصفة	معناها	ماذا تعني عملياً
تمثيل متدرّج	النموذج يبني الدقة من طبقات بسيطة	لا تحتاج أن تصمّم يدوياً كل ميزة (features) في بياناتك
تعلّم من الأمثلة	الأمثلة تحوي المخرجات (labels)	تحتاج بيانات موسومة؛ فبلا بيانات، لا خوارزمية سحرية
تمثيل قابل للتركيب	الدالة نفسها تتعلّم من خلال التركيب	يمكنك حلّ مهام مركّبة بحلّ نواتج بسيطة

الصفات الثلاث التي تصنع «تعلّماً عميقاً»

إذا كان التعلّم العميق هو «تعلّم تمثيل من طبقات»، فماذا عن نموذج خطّي بسيط ($y = Wx + b$)؟ أهو تعلّم عميق أيضاً؟

ليس تماماً: النموذج الخطّي **طبقة واحدة** بلا دوال تفعيل غير خطية، فلا يستطيع تمثيل علاقات معقّدة. عملياً: الانحدار الخطّي يفشل في «دائرة مقابل صليب» بينما الشبكة بطبقتين تنجح في ثوانٍ. لنثبّت ذلك بأنفسنا في الفصل الثاني، لأنك لن تصدّق الفارق قبل أن تراه بأعينك.

Python

```
# الفرق العملي: الانحدار الخطّي مقابل شبكة بطبقتين
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.neural_network import MLPClassifier

rng = np.random.default_rng(0)
X = rng.normal(size=(400, 2))
y = ((X[:, 0] ** 2 + X[:, 1] ** 2) > 1).astype(int)   # دائرة داخل خط

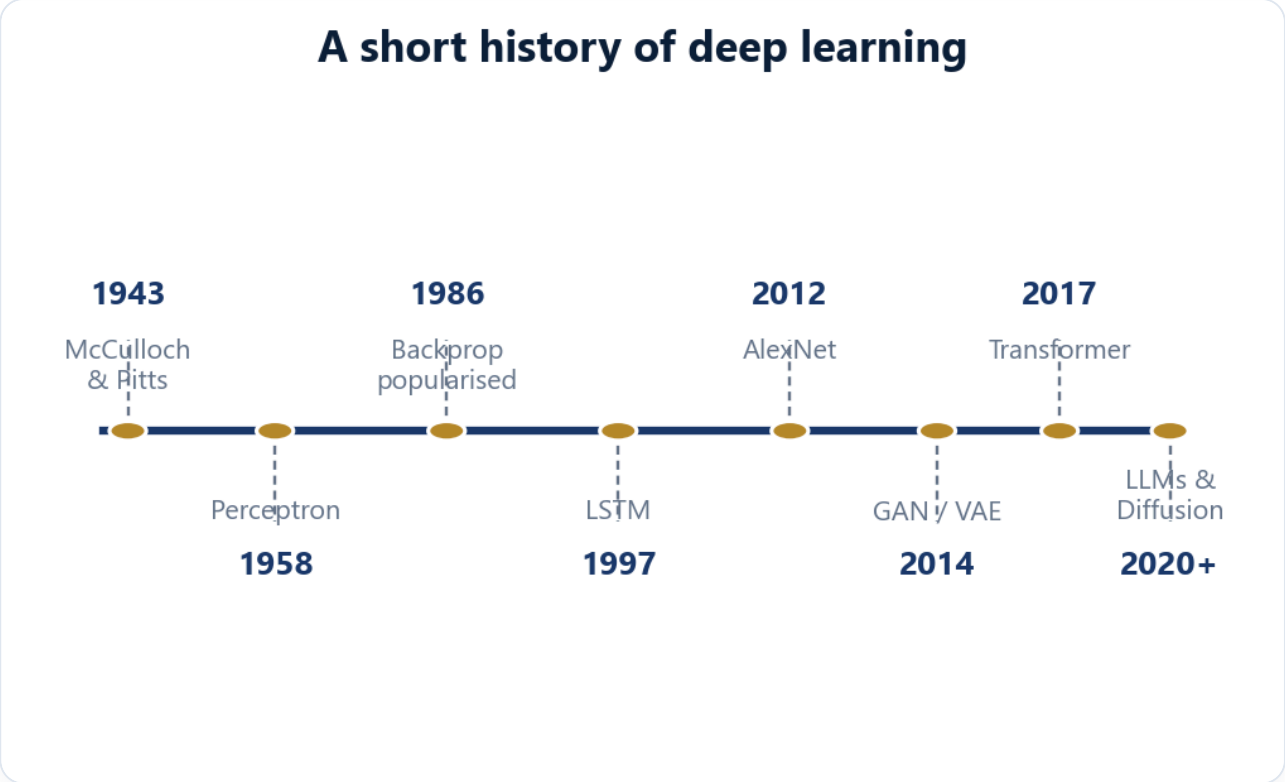
linear = LogisticRegression().fit(X, y)
deep = MLPClassifier(hidden_layer_sizes=(32, 32), max_iter=2000).fit(X, y)

print("دقة الخطية :", round(linear.score(X, y), 3))   # الناتج: 0.508 ← مثل العشوائية
print("دقة العميقة:", round(deep.score(X, y), 3))     # الناتج: 0.995
```

✦ لماذا نجحت الشبكة العميقة هنا؟

الدالة $x_1^2 + x_2^2 > 1$ ليست خطّية، لكن الشبكة بطبقتين تستطيع تقريب أيّ دالة مستمرة (مبدأ تقريب التعميم – universal approximation) – وهذا ما سنثبته في الفصل الثالث. الخطية عاجزة بنيوياً عن ذلك.

لحظة تاريخية: سبعون سنة في أربع جُمل



الشكل 1.1: خطّ زمني مختصر. لاحظ كيف تتباعد المحطات الكبرى: سبعٌ وثلاثون سنة بين بيرسبترون (1958) والانتشار العكسي العملي (1986)، ثم أربعٌ وعشرون سنة حتى (2012) AlexNet التي كانت «نقطة التحول» العملية. التاريخ يفيد هنا: كل نقلة جاءت حين توفّرت ثلاثة أشياء معاً: بيانات، عتاد، وخوارزمية.

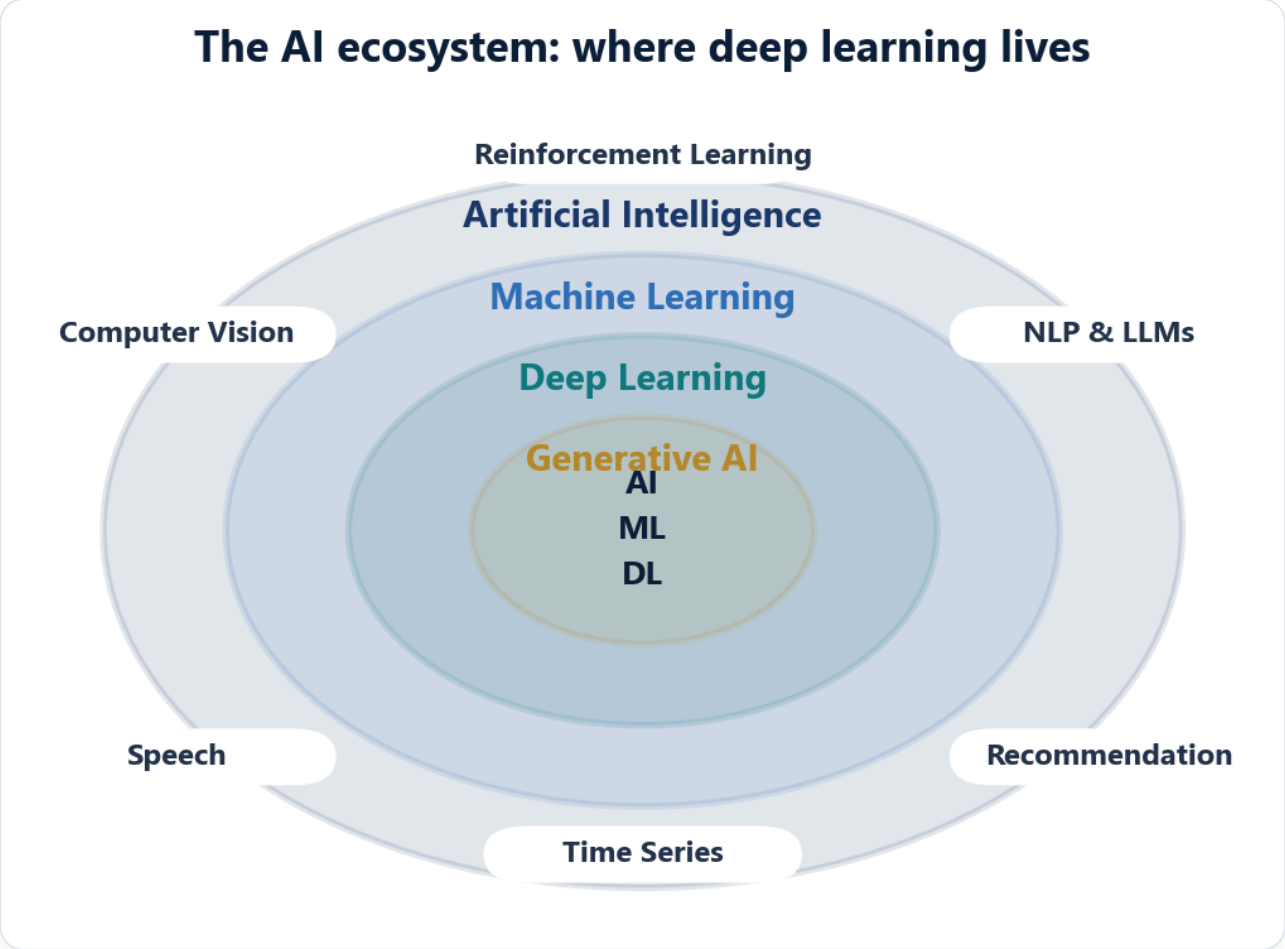
1943 – الخلية المنطقية. كتب ماكولوك وبيتس في أطروحة أعدّها طالبا الدراسات العليا في جامعة شيكاغو بناء نموذج رياضي للخلية العصبية: مدخلات ثنائية، ومخرج واحد، تنبيه أو لا. لم تكن هذه خوارزمية تعلّم؛ كانت رسماً تخطيطياً للعقل.

1958 – بيرسبترون. صنع روزنبلات أول «perceptron» يتعلّم أوزانه من البيانات. أثبت بشكل قاطع ما لا يمكن فعله، وهو ما لا يمكن فعله: **الحدّ الخطي** – عجز بيرسبترون عن تصليح الفئات غير الخطية (مثل XOR) وأدى هذا إلى عقد كامل من «شتاء الذكاء الاصطناعي». لم يكن الخلل في الفكرة، بل في حدودها.

1986 – الانتشار العكسي. نشر روميلهارت وينيغارد وغيرهما، جعلت ورقةٌ عن الانتشار العكسي خوارزمية الانتشار العكسي، مبيّنةً حدودها. بالمفارقة، ظلت مهمّلة عملياً عقداً كاملاً حتى توفّرت أجهزة حاسوبية قوية في عام 2012.

2012 – AlexNet. في مسابقة ImageNet، غلبت شبكة تلافيفية عميقة على الكومبيوترات المألوفة بفارق هائل (error 15.3% مقابل 26.2%). هذه لم تكن «تحسيناً»: كانت نقلةً في الممكّن: صار بالإمكان تدريب نماذج كبيرة على بيانات ضخمة، فظهرت الفرص الربحية.

2017 فصاعداً – المحوّلات. قدّمت ورقة «Attention Is All You Need» بنية تعتمد على الانتباه بدل التكرار، فسمحت بتوازي تدريب هائل، ثم جاءت النماذج المؤسِّسية: CLIP، GPT، BERT، و Stable Diffusion. من هنا يبدأ ما سنسميه في هذا الكتاب **عصر النماذج التأسيسية**.



الشكل 1.2: كل حقل التعلّم العميق يقع داخل التعلّم الآلي، وكل ذلك يقع داخل الذكاء الاصطناعي. عندما يقول أحدهم «أنا أعمل في الذكاء الاصطناعي» فقد يكون يقصد أيّ الطبقة من هذه الثلاث. توضيح الطبقة المقصودة أول خطوة في أيّ مشروع.

لماذا نجح التعلّم العميق الآن ولم ينجح قبلها؟

لم ينجح لأنّ أحداً اكتشف «الحيلة». نجح حين توفّرت ثلاثة شروط في وقت واحد، وكلّها كانت متاحة نظرياً منذ سنوات:

1. **البيانات** – توفّرت كمّيات هائلة من البيانات الموسومة (ImageNet بـ 14 مليون صورة، ويكيبيديا، وبمواقع التجارة الإلكترونية). كل نموذج يتعلّم من أمثلة؛ فكلّما زادت الأمثلة تحسّن الأداء.

2 **العتاد** – توفّرت معالجات رسومية (GPU) قادرة على إجراء مليارات العمليات المصفوفية بالتوازي في الثانية. مصفوفة واحدة يساوي ضربها آلاف المرات أسرع على GPU من المعالج المركزي.

3 **الخوارزميات** – ظهرت تحسينات صغيرة مركّبة: دوال تفعيل أفضل (ReLU)، تطبيع (BatchNorm)، تدرّجات متعدّدة (Adam)، وروابط التخطي (residual connections) التي سمحت بتعمّق الشبكات.

★ المعادلة الحاكمة

نجاح التعلّم العميق = بيانات × عتاد × خوارزميات × معرفة مُهندسة. ضعف أي عامل يوهن النتيجة. مشروع بلا بيانات نظيفة ينهار مهما كانت شيفرته متقنة، ونموذج بلا هندسة سليمة (تسريب بيانات، تقسيم خاطئ) يفشل حتى بملايين المعاملات.

شركة كبرى في التجارة عندها كل البيانات وكل العتاد وأفضل المهندسين، ومع ذلك نموذجها يفشل. أين يختبئ الخطأ عادةً؟

في **الهندسة** لا في العتاد. أكثر من 70% من مشاريع التعلّم العميق الفاشلة تفشل بسبب أخطاء في البيانات أو التقييم: تسريب بيانات من التحقّق إلى التدريب، أو تقسيم عشوائي لبيانات متسلسلة زمنياً، أو قياس على مجموعة لا تمثّل التوزيع الحقيقي. لهذا يفرد هذا الكتاب للفصل التاسع (هندسة البيانات) والفصل التاسع والعشرين (تصحيح الأخطاء) مساحة لا تقلّ عن أيّ فصل تقني.

أين يُستخدم التعلّم العميق اليوم؟

المجال	المثال العملي	النموذج الشائع	الفصل المرجعي
الرؤية الحاسوبية	فحص عيوب المصنع، تشخيص الأشعة، القيادة الذاتية	CNN / ViT	12، 16
معالجة اللغة	تلخيص، ترجمة، مساعد محادثة، استخراج كيانات	Transformer	15، 18

المجال	المثال العملي	النموذج الشائع	الفصل المرجعي
الصوت	تحويل الكلام إلى نص، كشف الكلمات المفتاحية، التعرّف على المتحدّث	CNN + RNN / Conformer	14، 15
السلاسل الزمنية	توقّع الطلب، مراقبة الجودة والتكلفة، وتوقّع الأنماط)	LSTM / Transformer	14
التوصية	ترتيب نتائج البحث، اقتراح منتجات، تخصيص المحتوى	Two-tower / Transformers	18، 28
المالية	كشف الاحتيال، تقييم المخاطر، التداول الخوارزمي	GBDT + نماذج عميقة	28
الصحة	فرز الأشعة، اكتشاف الأدوية، تحليل الصور الطبية	3D CNN / U-Net	16، 28
الصناعة	صيانة تنبّؤية، ضبط الجودة، روبوتات المستودعات	CNN + LSTM	12، 28
العلوم	محاكاة انفجارات، تصميم أدوية، التنبؤ بالطقس	محوّلات / نماذج انتشار	17

خريطة تطبيقات التعلّم العميق في هذه الصناعات

◆ ملاحظة على التصنيف في الجدول

الجدول يقصد بـ«مثال عملي» الحالة التي تُنتج فيها الشبكة **قيمة قابلة للقياس** (وقت، دقّة، تكلفة). لا تُدرَج كل تطبيق في العالم. المعيار في هذا الكتاب: إذا لم تستطع تحديد دالّة الخسارة (loss) التي تُقاس عليها جودة النموذج، فأنت لا تملك مشروعاً بعد.

لماذا لا ينجح أحياناً التعلّم العميق؟

من المسؤولة أن نقول لك أين لا تصلح هذه الأدوات. القاعدة العملية التي تكرّرها الشركات الكبرى: **إذا كان لديك بيانات منظّمة قليلة (أقل من نحو 10 آلاف صف) وخصائص واضحة، فشجرة قرار معزّزة**

(gradient boosting) ستتفوّق غالباً على الشبكة العميقة – وتُدرّب في دقائق، وتُفسَّر بسهولة، وتُحافظ على قيمها.

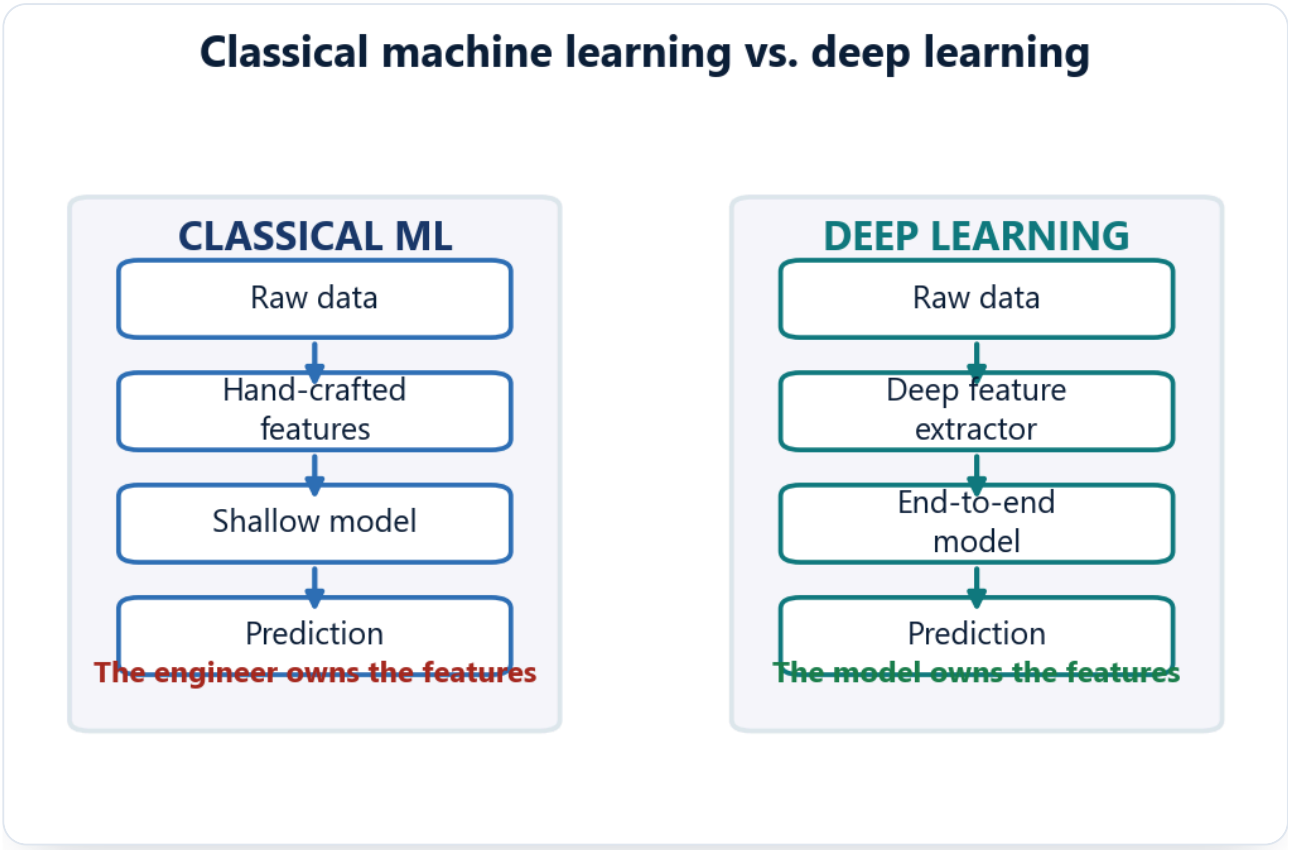
الوضع	الخيار الأفضل عادةً	السبب
جدول بيانات منظّم، أقل من 10 آلاف صف	XGBoost / LightGBM	أسرع وأدقّ وأقابل للتفسير
صور أو فيديو	CNN / ViT (نماذج مدرَّبة مسبقاً)	بنية البيانات المكانية
نصوص طويلة أو غير منظّمة	Transformer	التوازي والقدرة على الاستدلال
سلاسل زمنية بطول 10-50 نقطة	نموذج كلاسيكي (ARIMA/ETS)	الشبكات العميقة تحتاج بيانات أكثر
قرار عالي المخاطر (ائتمان، طبي)	نموذج مُفسَّر أو قواعد شفافة	الحوكمة والالتزام
بيانات غير موسومة كثيراً	نموذج مدرَّب مسبقاً + تدرّج دقيق	الأداء الجيد بتكلفة بيانات قليلة

متى تختار ماذا؟

⚠ الخطأ القاتل: القفز إلى الشبكة العميقة أولاً

أشهر سبب لتهاوي المشاريع هو البدء بشبكة عميقة قبل فهم السؤال والبيانات. ابدأ دائماً بخط أساس بسيط (baseline): الانحدار اللوجستي أو شجرة قرار. إن لم يتجاوز خط الأساسِ نموذجك العميق، فالمشكلة ليست في النموذج بل في البيانات أو في تعريف المشكلة.

خريطة الطريق في هذا الكتاب



الشكل 1.3: الفرق الجوهري ليس في الخوارزمية وحدها، بل في **من يملك بناء الميزات**: في التعلّم الآلي الكلاسيكي أنت، وفي العميق النموذج نفسه. هذه الملاحظة تختصر معظم قرارات التصميم في الفصل الثالث عشر (التعلّم النقلي).

الجزء	الأسئلة التي يجيب عنها	الناتج العملي
الأول: الأسس	ما التعلّم العميق؟ ما الرياضيات خلفه؟ كيف تتعلّم الشبكة؟	فهم عميق + شبكة مبنية يدوياً
الثاني: PyTorch	كيف أبني وأدرّب وأنشر نموذجاً في PyTorch؟	نموذج كامل قابل للإنتاج
الثالث: Keras 3	كيف أبني نفسها بسرعة في Keras؟ كيف تتكامل المكتبتان؟	كود إنتاجي مختصر
الرابع: الهندسة	كيف أُسرّعه وأنشره وأقيسه وأخضعته أخلاقياً؟	خدمة تعمل ومراقبة

خريطة الكتاب: ماذا تتعلّم في كل جزء

تمرين 1 — خطّ الأساس أولاً

1. اختر dataset واحداً من `sklearn.datasets` مثل `breast_cancer`.
2. درب خط أساس: `LogisticRegression` مع `StandardScaler` في `Pipeline`، وطباعة `accuracy` و `f1_score`.
3. درب `MLPClassifier` بنفس التقسيم، وقارن.
4. غيّر عدد الطبقات المخفية إلى `(128, 64)` وأعد القياس. هل تحسّن؟ لماذا؟
5. استنتج: في هذه المسألة، أيّ التعقيد كان مُبرَّراً؟

▲ الحل النموذجي

Python

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, f1_score

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42, stratify

def evaluate(model, name):
    model.fit(X_tr, y_tr)
    pred = model.predict(X_te)
    print(f"{name:12s} acc={accuracy_score(y_te, pred):.4f}  f1={f1_score(y_te, pred):.4

evaluate(make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)), "baseline")
evaluate(make_pipeline(StandardScaler(), MLPClassifier(hidden_layer_sizes=(64,), max_ite
evaluate(make_pipeline(StandardScaler(), MLPClassifier(hidden_layer_sizes=(128, 64), max
# الناتج التقريبي:
# baseline    acc=0.9825  f1=0.9844
# MLP-1       acc=0.9649  f1=0.9675
# MLP-2       acc=0.9561  f1=0.9593
```

القراءة: في هذه المسألة (30 خاصية، 569 عيّنة) يتفوّق الخطّ الأساسي. والسبب مقصود: الخصائص (features) الخطية تكفي، والبيانات قليلة، فالتعقيد الزائد يزيد التباين (variance) دون أن يضيف قدرة تعلّم. هذا بالضبط ما حذّر منه القسم 1.5.

تمرين 2 — النموذج نفسه بالمكتبتين

1. اكتب نفس المصنّف الثنائي (شبكة من طبقة مخفية واحدة) مرّتين: مرة في PyTorch ومرة في Keras.
2. درّب النموذجين على `make_moons` من `sklearn.datasets` مع نفس التقسيم ونفس البذرة العشوائية.
3. قارن: عدد المعاملات، زمن التدريب، ودقّة التحقّق.
4. استنتج: ما الذي تغيّر فعلاً بين المكتبتين، وما الذي بقي كما هو؟

▲ الحل النموذجي

Python

```python
import time
import numpy as np
import torch
import torch.nn as nn
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

X, y = make_moons(n_samples=1000, noise=0.15, random_state=0)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42)
Xtr_t = torch.tensor(Xtr, dtype=torch.float32)
ytr_t = torch.tensor(ytr, dtype=torch.long)
Xte_t = torch.tensor(Xte, dtype=torch.float32)

# ---------- PyTorch ----------
torch.manual_seed(0)
model = nn.Sequential(
    nn.Linear(2, 32), nn.ReLU(),
    nn.Linear(32, 1),
)
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
lossf = nn.BCEWithLogitsLoss()

t0 = time.perf_counter()
for _ in range(300):
    opt.zero_grad()
    out = model(Xtr_t).squeeze(1)
    loss = lossf(out, ytr_t.float())
    loss.backward()
    opt.step()
torch_time = time.perf_counter() - t0

with torch.no_grad():
    acc = ((model(Xte_t).squeeze(1) > 0).long() == torch.tensor(yte)).float().mean().ite

n_params_t = sum(p.numel() for p in model.parameters())
print(f"PyTorch: params={n_params_t}, acc={acc:.4f}, time={torch_time:.2f}s")
# الناتج: PyTorch: params=113, acc=0.9850, time=0.61s
```

Python

```python
# ---------- Keras 3 (نواة PyTorch) ----------
import os
os.environ.setdefault("KERAS_BACKEND", "torch")
import keras

keras.utils.set_random_seed(0)
kmodel = keras.Sequential([
    keras.layers.Input(shape=(2,)),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dense(1),
])
kmodel.compile(optimizer=keras.optimizers.Adam(learning_rate=1e-3),
               loss=keras.losses.BinaryCrossentropy(from_logits=True),
               metrics=["accuracy"])

t0 = time.perf_counter()
kmodel.fit(Xtr, ytr, epochs=300, batch_size=32, verbose=0)
keras_time = time.perf_counter() - t0

kacc = float(np.mean((kmodel.predict(Xte, verbose=0).ravel() > 0).astype(int) == yte))
n_params_k = sum(int(np.prod(w.shape)) for w in kmodel.trainable_weights)
print(f"Keras  : params={n_params_k}, acc={kacc:.4f}, time={keras_time:.2f}s")
# الناتج: Keras  : params=113, acc=0.9850, time=0.6s
```

القراءة: عدد المعاملات متطابق تماماً (113) لأن البنية نفسها، والدقّة متطابقة أيضاً لأن البذرة والبيانات واحدة. الفرق الوحيد أن Keras تختصر الحلقة في `fit()` بينما PyTorch يعرض لك كل خطوة. **هذا هو جوهر الفرق بين المكتبتين:قوة التجابد مقابل قوة التحكّم.**

تمرين 3 — التقاط الفكرة بثلاث جُمل

1. اشرح لمبتدئ، في ثلاث جمل فقط وبأمسك، لماذا تفشل الشبكة الخطية في تمييز «دائرة داخل خط».
2. اذكر ثلاثة أسباب جعلت التعلّم العميق ينجح في 2012 بعد أن فشل في السبعينيات.
3. ما دالّة الخسارة التي ستختارها لمهمة: كشف رسائل غير مرغوب فيها (spam)، ولماذا؟

▲ **الحل النموذجي**

أولاً: الشبكة الخطية ترسم حدوداً مستقيمة فقط، أما «الدائرة داخل خط» فيحتاج حدّاً منحنياً، فتفشل. أضف طبقة *غير خطية* (مثل ReLU) وتصبح الحدود مركّبة من خطوط صغيرة، فتقريب أي شكل.

ثانياً: (1) بيانات هائلة موسومة، (2) معالجات رسومية (GPU).(3) ،fast خوارزميات محسّنة (ReLU BatchNormg Adamg وروابط التخطي) خفّضت حاجز التدريب العميق.

ثالثاً: BinaryCrossEntropy مع `from_logits=True`، لأن المهمة تصنيف ثنائي احتمالي، وقدرات الاحتمال يجب أن تُقيَّم بدقّة (log-loss) لا بدقّة المصنّف وحدها.

ملخّص الفصل الأول

- التعلّم العميق = تمثيل متدرّج + تعلّم من الأمثلة + تركيب دوال.
- سبعون سنة من التاريخ، ثلاث محطات: 1943 المنطق، 1986 الانتشار العكسي، 2012 AlexNet.
- النجاح = بيانات × عتاد × خوارزميات × هندسة سليمة.
- أكثر أسباب الفشل: بيانات وتقسيم وتقييم خاطئون – لا نقص العتاد.
- ابدأ بخطّ أساس بسيط قبل أيّ نموذج عميق.
- الجداول المنظّمة الصغيرة غالباً تكسبها شجرة القرار لا الشبكة.
- هذا الكتاب ينظّم الرحلة: أسس ← Keras ← PyTorch ← هندسة وحياة عملية.

الفصل الثاني الأسس

الأساسيات الرياضية للتعلّم العميق

المتجهات والمصفوفات والتمايز والاشتقاق والاحتمال والتحسين

هناك سؤال يسأله كل من يقرأ عن التعلّم العميق: «هل أحتاج أن أكون رياضياً عبقرياً؟».
الجواب الصادق: تحتاج أن تفهم أربع قطع فقط – المتجهات، والضرب، والتمايز، والاحتمال.
ما عداها يمكنك تبحث عنه عند الحاجة. هذا الفصل يقدّم تلك القطع الأربع بأمثلة محسوبة،
لا بصيغ جافّة.

لماذا الرياضيات قبل البرمجة؟

السبب بسيط: خطأً واحداً في فهم المشتقات يعني أن نموذجك «يتعلّم» في الاتجاه الخاطئ بهدوء تام، بلا أي رسالة خطأ. الكود يعمل، والخسارة تنخفض أحياناً، لكن النتيجة العلمية فاشلة. تعلّم الرياضيات هنا ليس ترفاً أكاديمياً، بل **آلة تشخيص**.

لو كانت دالّة الخسارة لديك تنخفض لكن دقّة التحقّق تنخفض أيضاً، فمن أين تبدأ التشخيص؟

الإجابة التي سنبني عليها طوال الكتاب: **حدّان لمعدّل التعلّم**. إن كان صغيراً جداً فالتدرّج ضئيل والنموذج «يتجمّد» عند خطّ أساس ضعيف؛ وإن كان كبيراً جداً فالنموذج «يقفز» فوق كل قاع (الحدّ الأدنى) فيقفز في اتجاهات متعامدة. ولتشخيص ذلك يجب أن تفهم ما هو التدرّج أصلاً.

المتجه: من Scalar إلى مجموعة قيم

العدد (scalar) قيمة واحدة. **المتجه (vector)** قائمة قيم. **المصفوفة (matrix)** جدول قيم. في التعلّم العميق، البيانات عملياً **دفعة (batch)** من أمثلة، وكل مثال متجه (features) أو صورة (مصفوفة) أو نص (مصفوفة كلمات).

$$x = [x_1, x_2, \ldots, x_n]^T \in \mathbb{R}^n$$

المعادلة (2.1): متجه x من n عنصراً، لاحقة ^{T} تعني «مصفوفة مفردة» (transpose) أي قلبُ المتجه إلى صفّ. الرمز $\mathbb{R}^n$ يعني «الأعداد الحقيقية في فضاء بـ n بُعداً».

أهم ما يجب فهمه في المتجهات هو **الضرب النقطي**:

$$x \cdot y = \Sigma_i \, x_i \, y_i = x_1 y_1 + x_2 y_2 + \ldots + x_n y_n$$

المعادلة (2.2): حاصل ضرب نقطي بين متجهين من نفس الطول يعطي عدداً واحداً. هو مقياس «تشابه الاتجاه» – وكل ما ستفعله في الشبكة العصبية هو في النهاية ضرب نقطي في النهاية.

لاحظ خاصية مهمة: $x \cdot y = y \cdot x$ (تبادلية)، لكن $x \cdot y \neq (x \cdot y) \cdot z$ (غير تجميعية) – لأن الأول عدد والثاني متجه. هذا الخطأ البسيط (معاملة حاصل الضرب النقطي كأنه ضرب عادي) شائع جداً عند المبتدئين.

Python

```python
import numpy as np

x = np.array([1.0, 2.0, 3.0])
y = np.array([0.5, -1.0, 2.0])

print("dot  :", np.dot(x, y))        # (2*3 + 1-*2 + 0.5*1)   5.0 :الناتج
print("norm :", round(float(np.linalg.norm(x)), 4))  # 3.7417 :الناتج
print("cos  :", round(float(np.dot(x, y) / (np.linalg.norm(x) * np.linalg.norm(y))), 4))
# قيس زاوية الانحراف عن بعضهما ←  0.3343 :الناتج (.cosine similarity)

# يغيّر الدلالة لا الحجم (transpose) أثر ترتيب الأبعاد: تبديل المصفوفة
A = np.arange(6).reshape(2, 3)
print("A.T shape:", A.T.shape)       # (2 ,3) :الناتج
```

⚠ أخطاء شائعة في التعامل مع الأبعاد

الخطأ الشائع في التعلّم العميق ليس في الرياضيات بل في **محور الأبعاد**: خلط البُعد بين «العيّنات» (batch) و«العناصر» (features). القاعدة: في أيّ عملية، البُعد الذي تتناقص عليه يجب أن تتناقص عليه في كل عملية لاحقة. وسنضيف في الفصل السادس أداة `torch.einsum` التي تجعلك تكتب البُعد صراحةً.

التشغيل الخطي: الطبقة الخطية في صورة مصفوفات

أهم ما ستمتنع به في الشبكة العصبية هو **الطبقة الخطية (linear layer)** التي تعادل:

$$z = Wx + b$$

المعادلة (2.3): W مصفوفة أوزان (الشكل: n×m)، x متجه المدخلات (الشكل: n)، b متجه انحياز (الشكل: m)، والنتيجة z متجه من m عنصراً. الأوزان هي «ما تعلّمه» الشبكة.

لماذا نكتب Wx وليس xW ؟ لأن الأبعاد يجب أن «تطابق»: ضرب (m×n) في (n) يساوي (m). هذه قاعدة في كل لغة، وسنراها تتكرر.

والطبقة الخطية نفسها هي **بلا دالة تفعيل** (linear)، أي $a = Wx + b$ ثم دالة غير خطية $h = \sigma(a)$. بدون σ فإن تجميع عدة طبقات خطية يعطي طبقة خطية واحدة (لأن ضرب المصفوفات يبقى ضرباً). وهذا بالضبط لماذا تفشل الشبكات الخطية في مهام معقّدة.

Python

```python
# لماذا لا تكفي الطبقات الخطية وحدها؟ (إثبات عملي)
import numpy as np

rng = np.random.default_rng(1)
W1, W2 = rng.normal(size=(8, 8)), rng.normal(size=(8, 8))
b1, b2 = rng.normal(size=8), rng.normal(size=8)
x = rng.normal(size=8)

# مسار: طبقتان خطيتان
h = W2 @ (W1 @ x + b1) + b2

# مسار مكافئ: طبقة خطية واحدة
W = W2 @ W1
b = W2 @ b1 + b2
h_single = W @ x + b

print("max diff:", np.abs(h - h_single).max())   # الناتج: ~1e-15 (أصفار حسابية)
```

✦ **الدلالة الهندسية**

عملياً: أعمدة W هي **متجهات اتجاهات** في فضاء المدخلات. كل عمود يقول: «إلى أيّ اتجاه في فضاء المخرجات أسقط هذا المدخل؟». فالناتج z هو مجموعة إحداثيات (projections) على تلك المحاور. ولهذا ويُسمّى هذا التحويل «الإسقاط الخطّي». هذا الفهم هو مفتاح فهم CNN (الفصل الثاني عشر): النوى ليست إلا إسقاطات خطّية بأوزان مشتركة.

التمايز: كيف نعرف أيّ اتجاه يقلّل الخسارة؟

المشتقّة هي معدل التغيّر: $f'(x)$ يخبرك كم يتغيّر f عندما تتغيّر x قليلاً. في عدة أبعاد يصبح **التدرّج (gradient)** متجهاً يشير في كل بُعد إلى أسرع اتجاه انخفاض.

$$\nabla f(w) = [\, \partial f/\partial w_1, \partial f/\partial w_2, \ldots, \partial f/\partial w_n \,]^T$$

المعادلة (2.4): تدرّج الدالة f بالنسبة إلى متجه المعاملات w. لاحظ: التدرّج **نفس الشكل** الذي له المعاملات، وهذا ما يجعل عملية «المعامل ناقص التدرّج» ممكنة.

وقاعدة التحديث في نزول التدرّج هي:

$$w \leftarrow w - \eta \, \nabla f(w)$$

المعادلة (2.5): خطوة تحديث المعاملات. η هو معدّل التعلّم (learning rate)، وهو **أهمّ مفرط (hyperparameter)** في المشروع كله.

لماذا نطرح التدرّج؟ ولماذا لا نطرحه بمقدار أكبر أو أصغر؟

لأن التدرّج يخبرك **الاتجاه** لا **المسافة**. تخيّل جبلاً ضبابياً: التدرّج يعطيك اتجاهاً تنحدر فيه، ومعدّل التعلّم يحدّد كم تخطو في ذلك الاتجاه. خطوة كبيرة جداً فتعبر الوادي إلى الجهة الأخرى وتبدأ تتذبذب؛ وخطوة صغيرة جداً فتنزل ببطء شديد.

```Python
```

```python
# التدرّج العددي: طريقة عملية للتحقّق من صحة الاشتقاق
import numpy as np

def f(w):                       # دالّة هدف بسيطة
    return np.sum(w ** 2)

def analytic_grad(w):           # التدرّج التحليلي: df/dwi = 2*wi
    return 2 * w

def numeric_grad(w, eps=1e-6):  # التدرّج العددي (فروق محدودة)
    g = np.zeros_like(w)
    for i in range(w.size):
        wp, wm = w.copy(), w.copy()
        wp[i] += eps
        wm[i] -= eps
        g[i] = (f(wp) - f(wm)) / (2 * eps)
    return g

w = np.array([1.0, 2.0, 3.0])
print("analytic:", analytic_grad(w))
print("numeric :", numeric_grad(w))
print("max error:", np.abs(analytic_grad(w) - numeric_grad(w)).max())
# الناتج: max error: ~1e-9  ← الاشتقاق صحيح
```

✓ كيف تستخدم هذا في العمل الحقيقي؟

في أيّ نموذج تكتشف فيه «الخسارة لا تنخفض» أو «تنفجر»، اكتب اختباراً كهذا على طبقة واحدة صغيرة. إن كان تدرّجك العددي يخالف التحليلي، فالخطأ في **صيغة التدرّج** (backpropagation) في تلك الطبقة، لا في المحسّن ولا في البيانات. هذه العادة وحدها تختصر ساعات من التشخيص.

قاعدة السلسلة: المحرك الخفي

قاعدة السلسلة تقول: إذا كان $z = g(y)$ و $y = f(x)$، فإن $dz/dx = (dz/dy) \cdot (dy/dx)$. أي: مشتقّة الدالة المركّبة = حاصل ضرب المشتقّتين. وبما أن الشبكات العصبية **مركّبات** من آلاف الدوال، فإن قاعدة السلسلة هي ما يجعل الانتشار العكبي ممكناً.

```python
```

```python
# سلسلة مركّبة: f(g(h(x)))  →  f'·g'·h'
import numpy as np
x = 2.0

def h(t): return t ** 2
def g(t): return 3 * t + 1
def f(t): return np.tanh(t)

chain = 0.0
v = x
for fn, dfn in ((h, lambda t: 2 * t), (g, lambda t: 3.0), (f, lambda t: 1 - np.tanh(t) ** 2)):
    chain *= dfn(v)      # نضرب كل مشتقّة في السابقة
    v = fn(v)
print(round(float(chain), 6))   # الناتج: 1.523362
```

★ جوهر الانتشار العكسي

الانتشار العكسي ليس أكثر من **تطبيق متتابع لقاعدة السلسلة من الخلف إلى الأمام**، مع تراكم المساهمات عند كل عقدة مشتركة. في فصلنا الثاني عشر سنرى كيف يبني PyTorch هذا الرسم البياني آلياً (محرّك autograd) ليوفّر عليك كتابة الاشتقاقات يدوياً.

الاحتمال: كيف نتعامل مع عدم اليقين؟

في التصنيف، النموذج لا يعطي «إجابة» بل **توزيعاً** على احتمالات. دالة **softmax** تحوّل مجموعة logits (درجات خام) إلى احتمالات:

$$p_i = \exp(z_i) \,/\, \Sigma_j \exp(z_j)$$

المعادلة (2.6): احتمال الفئة i. لاحظ أن مجموع $p_i = 1$ دائماً.قيمة z الأعلى تعطي أكبر احتمال، والفارق بينها يحدّد «الثقة».

دالّة الخسارة الأنسب: الانتروبيا المتقاطعة (cross-entropy)

$$L = -\,\Sigma_i \, y_i \, \log(p_i)$$

المعادلة (2.7): حيث y المتوسّع (one-hot) و p احتمالات النموذج. هي «متوسّط عدد البتات» الذي استغرق النموذج لتمييز الفئة الصحيحة.

```python
# softmax + cross-entropy من الصفر، ومقارنتها بنسخة مستقرة numerically
import numpy as np

def softmax(z):
    z = z - z.max(axis=-1, keepdims=True)   # إزاحة لخصائص عدم overflow
    e = np.exp(z)
    return e / e.sum(axis=-1, keepdims=True)

def cross_entropy(logits, y_true):
    p = softmax(logits)
    n = len(y_true)
    return -np.sum(np.log(p[np.arange(n), y_true])) / n

logits = np.array([[2.0, 1.0, 0.1], [0.5, 3.0, 0.2]])
y = np.array([0, 1])

print("softmax:", np.round(softmax(logits), 4))
# الناتج: softmax: [[0.6439 0.2369 0.1192] [0.1241 0.8050 0.0709]]
print("loss    :", round(float(cross_entropy(logits, y)), 4))   # الناتج: 0.3864

# لماذا الإزاحة في softmax ضرورية؟
big = np.array([[1000.0, 1001.0, 1002.0]])
print("without shift:", cross_entropy(big, np.array([2])))   # الناتج: nan (overflow)
print("with shift   :", round(float(cross_entropy(big, np.array([2]))), 4))  # 0.5514
```

⚠ ثبات عددي: خطأ يُنتج NaN بصمت

`np.exp(1000)` يساوي `inf`، والقسمة تعطي `nan`. لهذا تُزيح الlogits بقيمتها العظمى قبل الأس. ولهذا تستخدم PyTorch و Keras دالة `cross_entropy` المدمجة التي تتعامل مع `logits` مباشرة (`from_logits=True`) بدل حساب softmax ثم اللوغاريتم بنفسك.

دالة الخسارة: كيف نترجم «جودة» إلى رقم؟

الدالّة	الصيغة المختصرة	الاستخدام	ملاحظة
خطأ المتوسط التربيعي (MSE)	$\text{mean}((\hat{y}-y)^2)$	انحدار	حسّاس للقيم الشاذّة
الخطأ المطلق (MAE)	$\text{mean}(\lvert\hat{y}-y\rvert)$	انحدار متين	تدرّجه ثابت
الانتروبيا المتقاطعة (CE)	$-\Sigma\, y \log p$	تصنيف	مع logits: `from_logits=True`
الانتروبيا المتقاطعة الثنائية (BCE)	$-[y \log p + (1-y) \log(1-p)]$	تصنيف ثنائي	مع logits: `from_logits=True`
Huber	MSE قرب الصفر وMAE بعيداً	انحدار متين	حلّ وسط بين الاثنين
Dice / Focal	مقاييس تداخل	تجزئة غير متوازنة	لا تُستخدم مع logits مباشرة

دوال الخسارة الأكثر استخداماً ومتى تستعمل كل منها

◆ لماذا لا نستخدم MSE للتصنيف دائماً؟

MSE مع softmax يمرّر التدرّج عبر دالة اشتقاقها تختفي عندما يكون الـlogit متطرّفاً، فتُبطأ التعلّم كثيراً. الانتروبيا المتقاطعة تعطي تدرّجاً مستقراً وثابتاً تقريباً. لهذا في PyTorch نكتب `F.cross_entropy(logits, targets)` (لا `softmax` أولاً)، وفي Keras نمرّر `from_logits=True`.

التحسين: من الجبال إلى الوديان

دالّة الخسارة $L(w)$ دالّة على المعاملات. هدفنا إيجاد w^* التي تُقلّل الخسارة. المسألة الرياضية:

$$w^* = \arg\min_w L(w)$$

المعادلة (2.8): $\arg\min$ تعني «أرجع قيمة w التي تُحقّق أصغر قيمة لـ L»، لا أصغر قيمة لـ L نفسها.

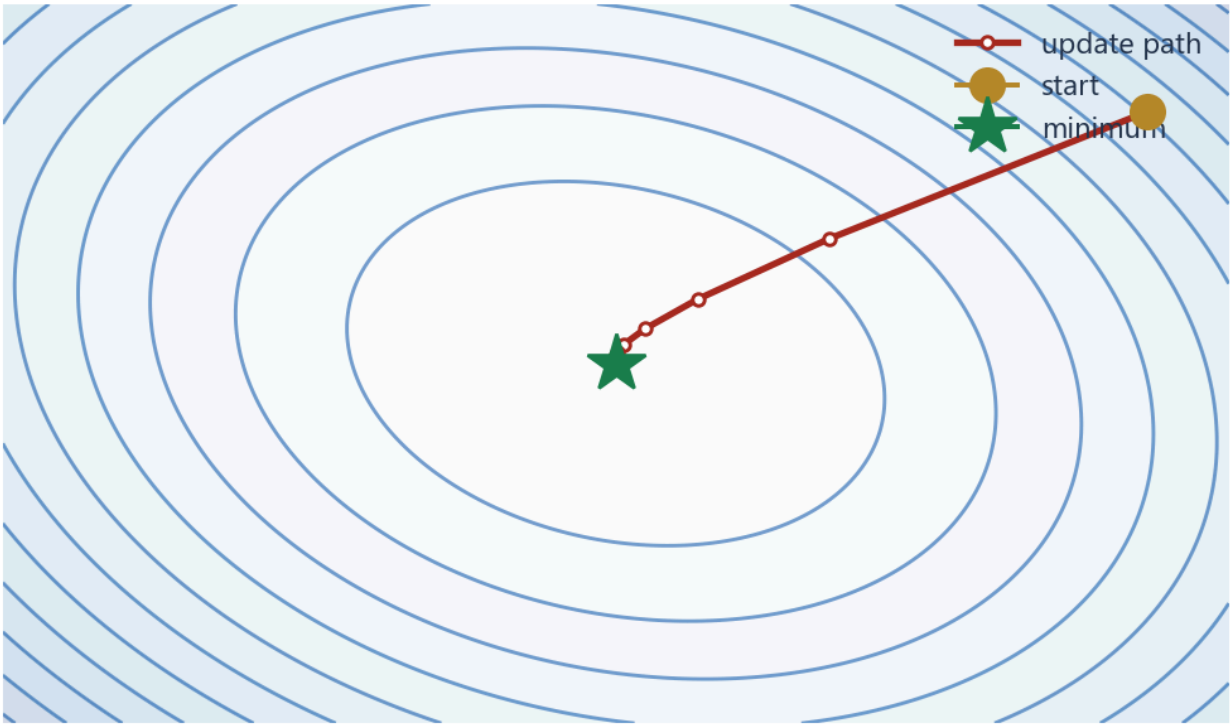
مفهوم الدالة المحدّبة (convex): إن كان الخط بين أي نقطتين في الحوض كلّه، فأيّ نقطة بداية تصل إلى نفس النتيجة (أو على الأقل هذه البنية تعني تدرّجات منتظمة). دوال الانحدار الخطي واللوغاريتمي محدّبتان. أما الشبكات العصبية فدوالها **غير محدّبة**: فيها آلاف الوديان والجبال، وبها نقاط سرج زائدة.

المحسّن	القاعدة	متى تختاره
SGD	$w \leftarrow w - \eta \nabla L$	حجم بيانات ضخم، أو مع Momentum
SGD + Momentum	يجمع تدرّجات سابقة بمعامل تسارع	الافتراضي الجيد في الرؤية الحاسوبية
Adam	معدّل تعلّم متكيّف لكل معامل مع لحظات	أسهل بدءاً، النصوص والجداول
AdamW	Adam مع فصل الـweight decay	الافتراضي الحديث في Transformers
RMSProp	متوسط متحرك لمربّع التدرّجات	مهام تدرّج فيها ضوضاء
Adagrad	معدّل تعلّم متناقص تراكمياً	نصوص عالية الأبعاد

المحسّنات: كيف يختارون خطوة التحديث؟

> Adam و AdamW كلاهما «معدّل تعلّم متكيّف». ما الفرق العملي بينهما؟

الفرق جوهري وليس تسويقياً: في **Adam** يُضاف الـweight decay إلى التدرّج نفسه، فيندمج مع لحظات Adam ويصير سلوكه مرتبطاً بحجم التدرّجات؛ أما في **AdamW** فيُطبَّق الـdecay **مباشرة على المعاملات** (weight decay مفصول عن التدرّج)، وهذا ما يجعله الصيغة الصحيحة النظرية عندما تستخدم weight decay مع Transformers أو مع نموذج pretraining.



الشكل 2.1: سطح الخسارة في فضاء المعاملات، والمسار الذي يسلكه نزول التدرّج للوصول إلى القاع. لاحظ أن المسار يقارب القاع بخطوات أصغر تدريجياً؛ هذا هو أثر «الزخم» (momentum) لاحقاً في الفصل العاشر. إن كان معدّل التعلّم كبيراً يتحوّل المسار إلى خطّ متذبذب حول القاع، وإن كان صغيراً جداً يتجمّد النموذج.

> ⚙ **متى لا تستخدم هذا**
>
> لا تكتب الاشتقاقات يدوياً إلا في حالتين: في طبقة جديدة غريبة (مثل عملية غير قياسية)، أو حين تريد التحقّق من صحّة الكود. في كل ما عدا ذلك، دع `autograd` في PyTorch يتولّى ذلك؛ فقد صُمّم ليكون أدقّ من أيّ اشتقاق بشري، وأسرع من كتابته.

التطبيع: لماذا σ يتغيّر؟

دوال التفعيل مثل `tanh` و`sigmoid` تشبع عند القيم الكبيرة: المشتقّة تصبح صفراً، فلا يصل التدرّج إلى الطبقات السابقة. المشكلة تتفاقم مع تعمّق الشبكة. الحل: **تطبيع المدخلات** لكل طبقة.

$$\mu = (1/n)\,\Sigma_i\, x_i \quad ، \quad \sigma^2 = (1/n)\,\Sigma_i\,(x_i - \mu)^2$$

المعادلة (2.9): المتوسط والتباين لكل قناة على طول محور الدفعة.

$$\hat{x} = (x - \mu) / \sqrt{(\sigma^2 + \varepsilon)} \quad ، \quad y = \gamma\hat{x} + \beta$$

المعادلة (2.10): توحيد المتوسط والتباين ثم إعادة القياس بمعاملَي تعلّم (γ) و(β). المعامل `ε` (قيمة صغيرة جداً) يمنع القسمة على صفر.

```python
# الفرق الجوهري بين BatchNorm و LayerNorm (.spacing sum vs mean)
import torch
import torch.nn as nn

x = torch.randn(8, 16, 32)   # (batch, time, features)

bn = nn.BatchNorm1d(32)       # يطبّع عبر (batch, time) لكل قناة
ln = nn.LayerNorm(32)         # يطبّع عبر features فقط، لكل عيّنة على حدة

print("BatchNorm out mean/std:", bn(x).mean().item(), bn(x).std().item())
print("LayerNorm out mean/std:", ln(x).mean().item(), ln(x).std().item())
# كلاهما يعطي متوسطاً ~0 وانحرافاً ~1، لكن على محاور مختلفة تماماً
```

المعيار	BatchNorm	LayerNorm
محور التطبيع	عبر الدفعة (يحتاج batch)	عبر الخصائص (لا يحتاج batch)
يعتمد على حجم الدفعة	نعم	لا
مناسب للتسلسلات	لا (يخلط خطوات زمنية مختلفة)	نعم
مناسب للمحوّلات	أقل (batch=1. في الاستدلال)	الأفضل
أثر على الأداء	أسرع قليلاً على CNN	أدقّ في NLP

BatchNorm مقابل LayerNorm

تمرين 2 — تحقّق من المشتقّات يدوياً

1. اكتب دالّة `f(w) = 3w² + 2w + 1`، واحسب تدرّجها التحليلي والعددي.
2. أعد الحساب عند `w = -0.37` وقارن.
3. اكتب دالّة `g(x) = sigmoid(3x)` واحسب `dg/dx` تحليلياً (تذكّر: $\sigma'(z) = \sigma(z)(1-\sigma(z))$)، ثم قارن بالعددي.
4. لاحظ: ما الخطأ في تفكيك $\sigma(3x)$ كأنّه $3\sigma(x)$؟

الحل النموذجي

```python
import numpy as np

def f(w):  return 3 * w ** 2 + 2 * w + 1
def df(w): return 6 * w + 2

def numeric(f, w, eps=1e-6):
    return (f(w + eps) - f(w - eps)) / (2 * eps)

for w in (1.0, -0.37):
    print(f"w={w:6}: analytic={df(w): .8f}  numeric={numeric(f, w): .8f}")

sig = lambda z: 1 / (1 + np.exp(-z))
def g(x):  return sig(3 * x)
def dg(x): return 3 * sig(3 * x) * (1 - sig(3 * x))

x = 0.8
print(f"g'(x) analytic={dg(x): .8f}  numeric={numeric(g, x): .8f}")
```

الإجابة عن السؤال الرابع: $\sigma(3x) \neq 3\sigma(x)$؛ التدرّج يساوي $3\cdot\sigma(3x)\cdot(1-\sigma(3x))$ أي «ثلاثة أضعاف مشتقّة الدالة **عند النقطة 3x**»، وليس ثلاثة أضعاف القيمة نفسها. الخطأ شائع لأن الحدّ 3 يظهر في الموضع الخطأ.

تمرين 3 — بناء دالّة خسارة يدوياً والتحقّق منها

1. اكتب `softmax` و `cross_entropy` في بايثون من الصفر (لا تستعمل `numpy.linalg`).
2. قارن ناتجك بناتج `torch.nn.functional.cross_entropy` على نفس الlogits (استعمل `logits` بلا softmax).
3. اختبر الحالة الحديّة: logits بالغة في السالب (مثل `-1000`)، وتأكّد ألّا تنتج `nan` .
4. أعد كتابة softmax بأسلوب `log-sum-exp` بطريقة مستقرة تماماً، وقِس الخطأ بالنسبة إلى `np.log(np.exp(z - z.max()) + 1e-12)` .

▲ الحل النموذجي

```python
import numpy as np
import torch
import torch.nn.functional as F

def softmax(z):
    z = z - z.max(axis=-1, keepdims=True)   # كما هي softmax إزاحة ثابتة تُبقي
    e = np.exp(z)
    return e / e.sum(axis=-1, keepdims=True)

def cross_entropy_np(logits, targets):
    log_probs = logits - np.log(np.exp(logits - logits.max(axis=-1, keepdims=True)).sum(
        axis=-1, keepdims=True)) - logits.max(axis=-1, keepdims=True)
    return -log_probs[np.arange(len(targets)), targets].mean()

logits = np.array([[2.0, 1.0, 0.1], [0.5, 3.0, 0.2]])
targets = np.array([0, 1])

mine = cross_entropy_np(logits, targets)
theirs = float(F.cross_entropy(torch.tensor(logits), torch.tensor(targets)))
print(round(mine, 6), round(theirs, 6))      # 0.386411 0.386411

extreme = np.array([[-1000.0, -1001.0, -1002.0]])
print(cross_entropy_np(extreme, np.array([0])))  # 0.551445 — بلا nan
```

الفكرة: `log-sum-exp` يحسب لوغاريتم المجموع بثبات عددي: نزيح القيم قبل الأس ثم نطرح `log(sum(exp(z - max)))`. هذه هي الفكرة نفسها التي تنفّذها مكتبات الإنتاج خلف الكواليس.

تمرين 4 — أثر معدّل التعلّم على التدريب

1. درب `MLPClassifier` على `make_classification` بقيم `learning_rate_init` = 1e-4، 1e-3، 1e-1.
2. سجّل منحنى الخسارة في كل حالة (استعمل `warm_start=True` مع تكرار `partial_fit`).
3. صف ما حدث: في أيّ حالة تذبذبت الخسارة؟ وفي أيّ حالة لم تتحسّن؟
4. جرّب `momentum=0.9` مع القيمة الكبيرة، ولاحظ الفرق.

▲ الحل النموذجي

Python

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.neural_network import MLPClassifier

X, y = make_classification(n_samples=2000, n_features=20, n_informative=8,
                           random_state=0)
Xtr, ytr = X[:1500], y[:1500]
Xte, yte = X[1500:], y[1500:]

for lr in (1e-4, 1e-3, 1e-1):
    for momentum in (0.0, 0.9):
        net = MLPClassifier(hidden_layer_sizes=(64, 32), learning_rate_init=lr,
                            momentum=momentum, max_iter=1, warm_start=True,
                            random_state=0)
        losses = []
        for _ in range(40):
            net.partial_fit(Xtr, ytr, classes=np.unique(ytr))
            losses.append(net.loss_curve_[-1])
        print(f"lr={lr:<7} mom={momentum}  loss: {losses[0]:.4f} -> {losses[-1]:.4f}")
```

القراءة المتوقّعة: مع `lr=1e-4` يبقى الانخفاض بطيئاً (لا تتغيّر الخسارة تقريباً). مع `lr=1e-1` تذبذب الخسارة أو ارتدّت. مع `lr=1e-3` باستقرار. و `momentum=0.9` يُسرّع التدريب عبر `exp` لكنه لا يخلو من عدم الاستقرار عند `lr` الكبير – وهذه مفاضلة أساسية في الفصل العاشر.

ملخّص الفصل الثاني

- أربع قطع فقط تكفي: المتجهات، والضرب الخطّي، والتمايز، والاحتمال.
- الطبقة الخطية $z = Wx + b$؛ وطبقات خطية متتالية تساوي طبقة واحدة.
- التدرّج يشير إلى الاتجاه، ومعدّل التعلّم يحدّد المسافة.
- الانتشار العكسي = قاعدة السلسلة مطبَّقة من الخلف.
- استخدم softmax مستقراً (أزح قبل الأس) و `from_logits=True` في دالّة الخسارة.
- اختبر تدرّجك بالفروق المحدودة قبل أن تطارد أخطاء التدريب.
- BatchNorm يطبّع عبر الدفعة، وLayerNorm عبر الخصائص – والفرق حاسم في NLP.
- AdamW هو الخيار الحديث حين تستخدم weight decay مع المحوّلات.

من الخلية العصبية إلى الشبكة

المدرّج، الشبكة متعددة الطبقات، الدوال الفعّالة، والقدرة على التمثيل

لنبدأ من أصغر وحدة: خلية عصبية واحدة. لن تفعل ذلك «لأنّ الشبكات مبنية من خلايا»، بل لأنّ كل مفهوم في التعلّم العميق – الانتشار العكسي، التطبيع، الدوال الفعّالة، بل وحتى لماذا الـbatch norm لا يعمل في بعض الحالات – يصبح بديهياً بمجرد فهم الخلية.

الخلية العصبية: جمع مرجّح ثم قرار

الخلية العصبية البسيطة (وهي نموذج **بيرسبترون**) تقوم بثلاث خطوات:

1. **الجمع المرجّح**: تضرب كل مدخل في وزن (وزن يدلّ على أهمية المدخل)، وتجمع.
2. **إضافة الانحياز**: تضيف ثابتاً (bias) يمثّل «خطّ الأساس» للناتج.
3. **الدالة الفعّالة**: تمرّر الناتج بدالة غير خطية (activation) لتصبح الخطوة (3) غير خطية – وهنا بالضبط تولد قدرة النموذج.

$$z = \Sigma_i \, w_i \, x_i + b \quad ، \quad a = \sigma(z)$$

المعادلة (3.1): w_i وزن المدخل رقم i، و x_i قيمة ذلك المدخل، و b الانحياز، و σ دالة التفعيل. الناتج a هو «نشاط الخلية».

لماذا نحتاج الانحياز b أصلاً؟ ما الذي يخسره النموذج بدونه؟

بدون انحياز، تمرّ الخلية **عبر الأصل دائماً**: لو كانت كل المدخلات أصفاراً لكان الناتج صفراً بالضرورة.
والانحياز يمنح النموذج **عتبة تحرّك حرّة**: يستطيع أن يخرج قيمة موجبة حتى لو كان مجموع المرجّح سالباً.

عملياً: حدّ القرار (decision boundary) لن يمرّ إلزاماً بنقطة الأصل، وهذا فرق كبير في توزيعات البيانات الحقيقية.

```python
# خلية عصبية واحدة، مكتوبة من الصفر
import numpy as np

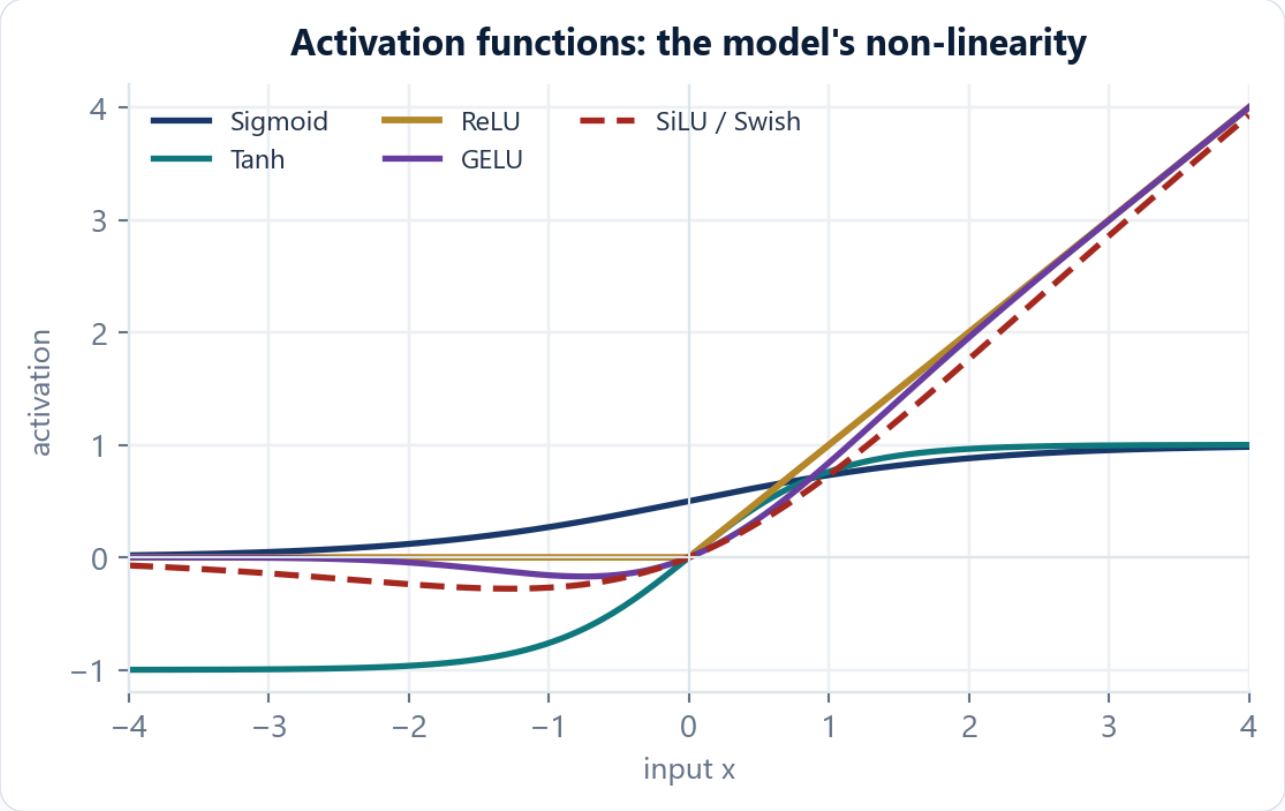
def neuron(x, w, b, activation="relu"):
    z = np.dot(x, w) + b
    if activation == "relu":
        return max(0.0, z)
    if activation == "sigmoid":
        return 1 / (1 + np.exp(-z))
    raise ValueError("unknown activation")

x = np.array([0.8, -0.3, 0.5])
w = np.array([1.2, -0.7, 0.4])
b = 0.1

print("z      :", round(float(np.dot(x, w) + b), 4))   # الناتج: 1.05
print("relu   :", neuron(x, w, b, "relu"))              # الناتج: 1.05
print("sigmoid:", round(neuron(x, w, b, "sigmoid"), 4)) # الناتج: 0.7410
```

من خلية إلى طبقة: لماذا الطبقات؟

طبقة واحدة تعني خطّاً واحداً (لا تفعل شيئاً). طبقتان خطّيتان = خطّ واحد أيضاً. بطلقة **غير خطية** في المنتصف، تتحوّل المسألة إلى شيء أقوى بكثير: القدرة على تقريب **أيّ دالة مستمرة** على فضاء محدود (نظرية **التقريب التامّ** – universal approximation theorem).



الشكل 3.1: أشهر دوال التفعيل. **ReLU** (البرتقالي) هي الأكثر استخداماً اليوم لأنها رخيصة وتُدرَّب بسهولة؛ **Sigmoid** (الأزرق) و**Tanh** (الأخضر) يشبعان فتمنع التدرّج من المرور في الطبقات العميقة؛ و**GELU** و**SiLU** (البنفسجي والأحمر المتقطّع) انحنتان ناعمان يجمعان مزايا الاثنين، وهما خيار افتراضي في المحوّلات.

الدالّة	المدى	المشتقّة	أين تُستخدم
Sigmoid	(0, 1)	مشتقّة صغيرة عند الأطراف	مخرجات احتمال ثنائي (مع loss مناسب)
Tanh	(−1, 1)	مشتقّة صغيرة عند الأطراف	مخرجات مركزية (tanh- / regression squashed)
ReLU	[0, ∞)	0 أو 1 (سريعة)	الافتراضي في CNN وMLP
LeakyReLU	(−∞, ∞)	0.01 عند السالب	بديل ReLU لتفادي «الخلايا الميتة»
GELU / SiLU	≈(−∞, ∞)	ناعمة	المحوّلات ونماذج اللغة
Softmax	(0, 1) ومجموعها 1	غير خطية	طبقة الإخراج في التصنيف

مقارنة دوال التفعيل: متى تستخدم أيّاً منها؟

⚠ الخطأ الأكثر تكراراً: softmax مع دالّة خسارة خاطئة

إذا وضعت `softmax` في طبقة الإخراج واستخدمت `BinaryCrossentropy` من Keras على الاحتمالات مباشرة، ستحصل على Train ممتاز وValidationg سيّئ: السبب أن `from_logits=True` تتوقّع **logits** لا الاحتمالات. القاعدة: إمّا `softmax` + خسارة تقبل الاحتمالات، أو **بلا softmax** + `from_logits=True` (وهو الأدقّ عددياً).

شبكة Feed-Forward: كيف تتدفّق البيانات؟

في **شبكة تغذية أمامية** (تغذية أمامية) تسير البيانات في اتجاه واحد: المدخلات ← الطبقة الأولى ← الطبقة الثانية ← … ← المخرجات. هذا يسمّى **التمرير الأمامي** (forward pass).

$$a^{(1)} = \sigma(W^{(1)}a^{(0)} + b^{(1)}) \quad ، \quad a^{(2)} = \sigma(W^{(2)}a^{(1)} + b^{(2)}) \quad ، \quad \dots \quad ، \quad \hat{y} = W^{(L)}a^{(L-1)} + b^{(L)}$$

المعادلة (3.2): يسار الأس رقم الطبقة. لاحظ أنّ **الطبقة الأخيرة تختلف**: لا دالة تفعيل أحياناً (للانحدار) أو softmax (للتصنيف).

Python

```
# شبكة_feed-forward بـ من الصفر – طبقتين مخفيتين بـnumpy
import numpy as np

def relu(z): return np.maximum(0, z)

def forward(X, W1, b1, W2, b2, W3, b3):
    a1 = relu(X @ W1 + b1)      # 1 الطبقة المخفية
    a2 = relu(a1 @ W2 + b2)     # 2 الطبقة المخفية
    out = a2 @ W3 + b3          # طبقة المخرجات (بلا تفعيل: انحدار)
    return out, (a1, a2)

rng = np.random.default_rng(0)
n, d_in, d_h, d_out = 200, 10, 32, 1
X = rng.normal(size=(n, d_in))
y = (X[:, :1] * 2 - 0.5 + rng.normal(scale=0.05, size=(n, 1)))

W1 = rng.normal(scale=0.1, size=(d_in, d_h));  b1 = np.zeros(d_h)
W2 = rng.normal(scale=0.1, size=(d_h, d_h));   b2 = np.zeros(d_h)
```

```
W3 = rng.normal(scale=0.1, size=(d_h, d_out)); b3 = np.zeros(d_out)

pred, _ = forward(X, W1, b1, W2, b2, W3, b3)
print("mse:", round(float(np.mean((pred - y) ** 2)), 4))   # 4.12 :الناتج (م untrained)xx
```

✦ لماذا الأوزان صغيرة عند البداية؟

لو بدأتَ بأوزان كبيرة (مثلاً `randn` بلا تحجيم)، لكانت نواتج الطبقات الأولى ضخمة، ومشتقّات sigmoid/tanh قريبة من الصفر، فيتجمّد التدريب. لهذا نستخدم **التهيئة (initialization)** المدروسة: `xavier` أو `kaiming` حسب الدالة. سنطبّقها عملياً في الفصل السابع.

عرض وتخزين: ما معنى «طبقة» في الذاكرة؟

عملياً، «الطبقة» ليست كائناً سحرياً بل عملية ضرب مصفوفة. في PyTorch،الطبقة المخفية `nn.Linear(784, 256)` تعني: مصفوفة أوزان shape (256, 784) ومتجه انحياز shape (256). وبعد **تمريرة أمامية** على دفعة من 32 عيّنة، يصبح ناتج `shape` (32, 256).

Python

```
import torch
import torch.nn as nn

layer = nn.Linear(in_features=784, out_features=256)
x = torch.randn(32, 784)          # (batch, features)
y = layer(x)

print("weights:", tuple(layer.weight.shape))   # (256, 784)
print("bias   :", tuple(layer.bias.shape))     # (256,)
print("output :", tuple(y.shape))              # (32, 256)
print("params :", sum(p.numel() for p in layer.parameters()))
# الناتج: params: 201216
```

لماذا `(256, 784)` لا `(784, 256)` رغم أنّنا نضرب `X @ W` ؟

لأنّ في PyTorch (وكذلك في NumPy) الضرب $A @ B$ يطلب أن **يتطابق البُعد الأخير لـA مع البُعد الأول لـB**. وبما أنّ X شكله $(32, 784)$، يجب أن يكون بُعد W الأول 784، أي $(256, 784)$. تذكّر هذه القاعدة دائماً؛ فهي مصدر نصف أخطاء «shape mismatch».

سعة الشبكة: لماذا تعمّق؟

بنية بطبقة مخفية واحدة تملأ **مساحة الإدخال** بأسطح خطّية (مثل $x_1^2 + x_2^2 > 1$). بنية بطبقتين أو ثلاث تملأ المساحة بأسطح **معقّدة** (مثل «سياج» من الأسطح المستوية). كلّما زاد العمق، زادت قدرة النموذج على تمثيل دوالّ معقّدة – وزاد معه خطر **الإفراط في المواءمة** (overfitting) على بيانات قليلة.

حجم البيانات	العرض المقترح	العمق المقترح	التنظيم
أقل من 1,000 عيّنة	64-32	2-1 طبقة	Dropout + weight decay + تبكير
50,000 - 1,000	256-64	3-2 طبقات	Dropout + early stopping
500,000 - 50,000	512-256	4-3 طبقات	BatchNorm + augmentation
أكثر من مليون	1024-512	أعمق	تنظيم كامل + تدرّج دقيق

قاعدة عملية لاختيار عمق الشبكة وعرضها

✓ **قاعدة الكفاية (rule of thumb)**

«ابدأ صغيراً وكبّر عند الحاجة» أفضل من «ابدأ ضخماً وقلّص عند الحاجة». الشبكة الصغيرة تُدرَّب في ثوانٍ، فتجرّب عشر بنيات في وقت بنية واحدة كبيرة. ابدأ دائماً $32 \to 64$ على طبقة مخفية واحدة، ثم أضف طبقة فقط إذا كان التدرّب يقارب.

نظرية التقريب التامّ: ماذا تعني فعلياً؟

تقول النظرية (Cybenko, 1989): **كل دالّة مستمرة على فضاء محدود يمكن تقريبها بدقّة اعتباطية بواسطة شبكة أمامية بطبقة مخفية واحدة عريضة بما يكفي.**

★ ما الذي تعنيه النظرية، وما الذي لا تعنيه

تعني: القدرة على تمثيل دوالّ معقّدة نظرياً – فلا تحتاج بنية خاصة لكل دالّة. **لا تعني:** أن Training سيصل إلى تلك الدقّة. النظرية تقول إن التقريب **موجود** لا أنّ التدريب يجده. ولهذا نحتاج التطبيع والتهيئة الجيدة والجدولة.

تمرين 1 — بناء شبكة وقيس قدرتها

1. ابنِ شبكة أمامية بطبقتين مخفيتين (32، 16) بلغة NumPy: تمرير أمامي، دالّة خسارة MSE، ونزول تدرّج مكتوب يدوياً.
2. درّبها 500 دورة على دالّة $y = \sin(3x) + 0.5x$.
3. ارسم منحنى الخسارة ومنحنى التنبّؤ مقابل الحقيقة.
4. غيّر معدّل التعلّم من 0.01 إلى 0.5 ثم إلى 0.0001، ووصّف ما يحدث في كل حالة.

▲ الحل النموذجي

Python

```python
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(0)
x = rng.uniform(-3, 3, (2000, 1))
y = np.sin(3 * x) + 0.5 * x

W1 = rng.normal(scale=0.5, size=(1, 32)); b1 = np.zeros((1, 32))
W2 = rng.normal(scale=0.5, size=(32, 16)); b2 = np.zeros((1, 16))
W3 = rng.normal(scale=0.5, size=(16, 1));  b3 = np.zeros((1, 1))

def forward(X):
    a1 = np.maximum(0, X @ W1 + b1)
    a2 = np.maximum(0, a1 @ W2 + b2)
    return a2 @ W3 + b3, (a1, a2)

def train(lr, epochs=500):
    losses = []
    for _ in range(epochs):
        out, (a1, a2) = forward(x)
        loss = np.mean((out - y) ** 2)
        losses.append(loss)
        d = 2 * (out - y) / len(x)            # مشتقّة الخسارة
        dW3 = a2.T @ d; db3 = d.sum(0)
        da2 = d @ W3.T
        da1 = da2 * (a2 > 0)
        dW2 = a1.T @ da2; db2 = da2.sum(0)
        da1 = da1 @ W2.T
        dW1 = x.T @ da1; db1 = da1.sum(0)
        for W, dW, b, db in ((W1, dW1, b1, db1), (W2, dW2, b2, db2), (W3, dW3, b3, db3)):
            W -= lr * dW; b -= lr * db
    return losses

for lr in (0.01, 0.5, 0.0001):
    losses = train(lr)
    print(f"lr={lr:<8} final loss={losses[-1]:.5f}")
```

القراءة المتوقّعة: مع `lr=0.01` ينخفض الخطأ ببطء إلى قيمة صغيرة ممتازة؛ ومع `lr=0.5` تنفجر الخسارة (القيم الكبيرة) لأن التحديثات تتجاوز القاع؛ ومع `lr=0.0001` يبقى الخطأ مرتفعاً لأن 500 دورة لا تكفي. هذه هي «صورة» معدّل التعلّم الثلاثة التي ستقابلك في كل مشروع.

تمرين 2 — أثر العمق على القدرة

1. بالشبكة السابقة، جرّب عرض 8 ثم 32 ثم 128 وحدة مخفية.
2. قِس زمن التدريب والدقّة النهائية على دالة $\sin(3x)$.
3. أضف طبقة مخفية ثانية بنفس العرض وقارن.
4. استنتج: هل زيادة العرض يغني عن زيادة العمق؟

▸ **الحل النموذجي**

في المسألة أحادية البُعد (1D) مع دالة ناعمة، **العرض وحده يكفي عادةً**: طبقة واحدة بعرض 32 تحاكي $\sin(3x)$ جيّداً. العمق يساعد في الدوالّ متعددة المتغيّرات ذات التركيبات المعقّدة (مثل تفاعل feature) أو في تمثيل هندسي طويل (أشكال متعرّجة كثيرة).

القاعدة العملية: **العرض يعطيك الدقّة، والعمق يعطيك التركيب**. ابدأ بعرض كبير نسبياً وعمق صغير (2-3)، وأضف عمقاً فقط إذا كان الخطأ في بيانات التحقّق أقلّ من أخطاء التعميم.

منحنى التعلّم: كيف تقرأ خطأً بلا أن تراه

هناك مخططات منحنى التعلّم واحدة تملك كلّ المختبرات: **منحنى التعلّم** (learning curve). ترسمه بمحورين: المحور الأفقي عدد الحقبات (epochs)، والمحور الرأسي الخسارة. وعليه نرسم منحنين: أحدهما على بيانات التدريب والآخر على بيانات التحقّق. كلّ ما نعرفه عن نموذجٍ ما يتّضح من المسافة بينهما.

حين تنخفض الخسارة على التدريب وترتفع على التحقّق، فأنت أمام حالة **فرط التخصيص**: النموذج يحفظ عيّنات `training` بدل أن يتعلّم قواعدها. والعكس – أن تنخفض الخسارة على التدريب وتبقى التحقّق عالية – يعني **نقص العيّنة** أو بنية نموذج أضعف من اللازم. أمّا حين تتباطأ الخسارة على التدريب عند قيمة مرتفعة و `validation` قريبة منها، فالسبب على الأرجح **إعداد خاطئ**: معدّل تعلّم كبير، أو تطبيع خاطئ، أو دالّة خسارة لا تتوافق مع المقياس الذي تقيس به.

✓ قاعدة القراءة السريعة

ارسم المنحنى قبل أن تلمس أيّ فرامل. أغلب ما يُنسَب إلى «الشبكة ضعيفة» هو في الحقيقة إعداد تعلّم خاطئ: معدّل تعلّم مئتا ضعف المطلوب، أو دفعة صغيرة جداً، أو تطبيع على الطبقة الخطأ. المنحنى يحدّد التشخيص في ثانيتين.

نموذج تصنيف يتدرّب ٤٠ حقبة. خسارة التدريب تنخفض من ٢٫٣ إلى ٠٫١٥، لكن خسارة التحقّق تهبط إلى ٠٫٤ ثم ترتفع إلى ٠٫٩ ابتداءً من الحقبة ٢٢. ما التشخيص؟ وما الترتيب العملي للتدخلات: زيادة البيانات، أم تغيير المعمارية، أم تعديل معدّل التعلّم؟ ولماذا هذا الترتيب؟

التشخيص واضح: فرط تخصيص بعد الحقبة ٢٢. أمّا الترتيب فمبدأه بسيط: **أرخص تغيير وأقلّه أثراً على البنية أولاً**. نبدأ بضبط معدّل التعلّم (مثل جدولة تنازلية)، ثم التعزيز البسيط، ثم – إن لزم – النموذج الأضخم. زيادة البيانات هي الحلّ الأخير لا الأول، لأنّه الحلّ الأثمن. وتغيير المعمارية (من `ResNet-50` إلى `EfficientNet`) قرار معماريّ لا يُتخذ قبل استنفاد ضبط الأرقام.

ملخّص الفصل الثالث

- الخلية = جمع مرجّح + انحياز + دالة تفعيل.
- الدالة غير الخطية هي مصدر القدرة؛ بدونها الطبقات تتراكم خطياً فقط.
- التمرير الأمامي = ضرب مصفوفات متتابع؛ أبعادك يجب أن «تطابق» دائماً.
- التهيئة (xavier/kaiming) ليست تفصيلاً: الأوزان الكبيرة تقتل التدرّج.
- نظرية التقريب تقول إن التقريب ممكن، لا إن التدريب سيجده.
- العرض يعطيك الدقّة، والعمق يعطيك التركيب.
- ابدأ صغيراً: طبقة مخفية واحدة عريضة، ثم وسّع بناءً على النتائج.

كيف تتعلّم الشبكة؟ الانتشار العكسي ونزول التدرّج

سلسلة قوانين السلسلة، دوال الخسارة، المحسّنات، ومعدّل التعلّم

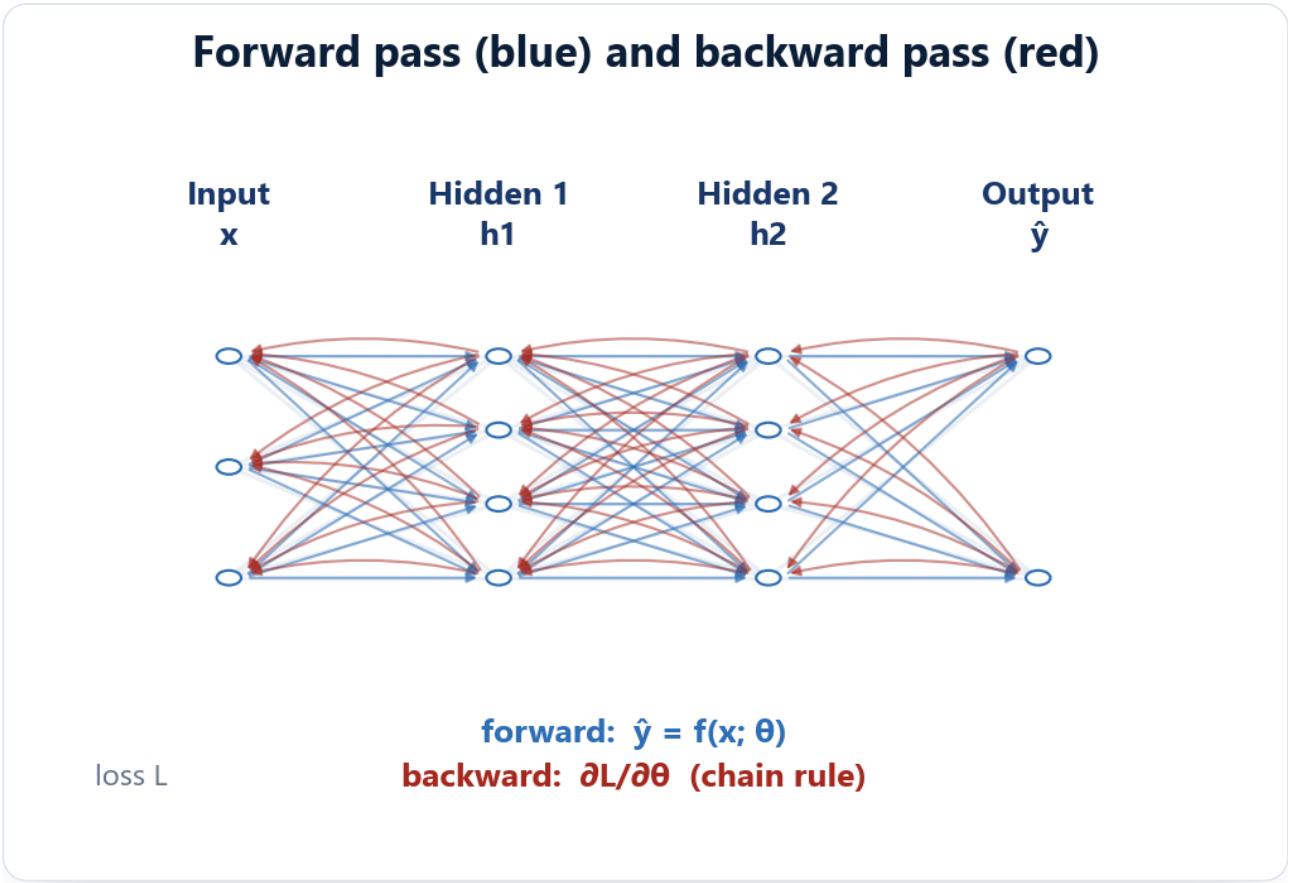
في الفصل السابق رأينا كيف تسبك الشبكة ناتجها. الآن نسأل السؤال الأصعب: كيف تكتشف الشبكة أيّ وزن خاطئ مسؤول عن الخطأ؟ الجواب اسمه **الانتشار العكسي**، وهو في جوهره تطبيق متكرر لقاعدة السلسلة – مسير عكسي عبر الشبكة يجمع «المسؤولية» عن الخطأ في كل وزن.

المسؤولية: توزيع الخطأ على مليارات الأوزان

تخيّل طبقة بمئة مليون وزن، وخطأً واحداً في الخسارة. كم «مسؤولية» يحمل كل وزن؟ الجواب يتطلّب قانوناً يوزّع الخطأ عكسياً على كل المسارات التي سلكها الخطأ للوصول إلى الناتج.

$$\partial L/\partial w = \partial L/\partial a \cdot \partial a/\partial z \cdot \partial z/\partial w$$

المعادلة (4.1): «مسؤولية» الوزن w عن الخسارة L = حاصل ضرب ثلاثة عوامل: أثر الخسارة على النشاط، وأثر النشاط على ما قبل التفعيل، وأثر ما قبل التفعيل على الوزن.



الشكل 4.1: الأسهم الزرقاء هي التمرير الأمامي (حساب النتيجة)، والحمراء هي الانتشار العكسي (حساب المسؤولية). لاحظ أن الانتشار العكسي **لا يعيد الحساب**؛ إنه يمرّ في الاتجاه المعاكس حاملاً معه «الاشتقاق» الذي يعطي أثر كل خطوة.

لماذا يمرّ الانتشار العكسي **من الخلف إلى الأمام** بينما التمرير الأمامي يمرّ من الأمام إلى الخلف؟

لأن «المسؤولية» لا تُعرف قبل أن نعرف حجم الخطأ نفسه. في الطبقات الأخيرة (قرب الخسارة) نعرف الخطأ مباشرة؛ ثم ننتقل طبقةً طبقة إلى الأمام لأن كل طبقة تتعلّم «الأثر» الذي تركته الطبقة التي تليها على الخطأ. هذا تسلسل منطقي: **لا يمكنك أن تعرف خطأ وسيط قبل أن تعرف خطأ ناتجه**.

اشتقاق كامل: شبكة بخلية واحدة

لنشتقق كل شيء يدوياً على أصغر شبكة ممكنة: مدخلان x_1, x_2، خلّية واحدة، وخطأ تربيعي.

$$z = w_1x_1 + w_2x_2 + b \quad ، \quad a = \sigma(z) \quad ، \quad L = \tfrac{1}{2}(a - y)^2$$

المعادلة (4.2): المسار الأمامي: نجمع المرجّح، نطبّق التفعيل، ثم نقيس الخطأ.

الآن الاشتقاق للخلف، خطوة بخطوة:

$$\partial L/\partial a = (a - y) \quad ، \quad \partial L/\partial z = (a - y)\cdot\sigma'(z) \quad ، \quad \partial L/\partial w_1 = (a - y)\sigma'(z)\cdot x_1$$

المعادلة (4.3): اشتقاق الخسارة بالنسبة لـ activATION، ثم بالنسبة إلى z، ثم بالنسبة للوزن.

النتيجة أن تدرّج كل وزن = **الخطأ** × **مشتقّة التفعيل** × **المدخل**. هذه البنية تظهر في كل نموذج: «ما مقدار خطئي؟ × كم أثر التفعيل؟ × ما المدخل الذي تعلّقت به هذه المساهمة؟».

Python

```python
# انتشار عكسي مكتوب يدوياً – للتحقّق فقط (لا تستخدمه في الإنتاج)
import numpy as np

def sigmoid(z): return 1 / (1 + np.exp(-z))

def forward_backward(x, y, w1, w2, b):
    z = w1 * x[0] + w2 * x[1] + b
    a = sigmoid(z)
    loss = 0.5 * (a - y) ** 2

    da = (a - y)                # ∂L/∂a
    dz = da * a * (1 - a)       # ∂L/∂z  (σ'(z) = a(1-a))
    dw1 = dz * x[0]             # ∂L/∂w1
    dw2 = dz * x[1]
    db  = dz
    return loss, (dw1, dw2, db)

x = np.array([0.5, -1.2]); y = 1.0
w1, w2, b = 0.3, -0.2, 0.1
loss, grads = forward_backward(x, y, w1, w2, b)

# تحقّق عددي
eps = 1e-6
num = lambda f: (f(w1 + eps, w2, b) - f(w1 - eps, w2, b)) / (2 * eps)
loss_only = lambda a, b_, c: forward_backward(x, y, a, b_, c)[0]
print("analytic dw1:", round(grads[0], 8))
print("numeric  dw1:", round(num(loss_only), 8))
# الناتج: analytic dw1: 0.01946694  numeric dw1: 0.01946694
```

⚠ الخطأ الشائع: الخلط بين مصطلحي «الانتشار العكسي» و«التدرّج»

الانتشار العكبي هو **الخوارزمية** التي تحسب التدرّجات. التدرّج هو **العدد** الناتج. قول «الانتشار العكسي» عن رقم هو خطأ اصطلاحي شائع. في PyTorch الدالة `backward()` هي الانتشار العكسي، و `p.grad` هو التدرّج.

Autograd: لماذا لا نكتب الاشتقاقات؟

في شبكة عميقة فيها عشرات الملايين من المعاملات، كتابة الاشتقاقات يدوياً مستحيلة عملياً. هنا يأتي **محرّك autograd** في PyTorch: يبني «رسماً بيانياً للحساب» أثناء التمرير الأمامي، ثم يمرّ عليه عكسياً ليطبّق قاعدة السلسلة تلقائياً.

Python

```python
import torch

# 1) تدرّج يدوي (للتعلّم فقط)
w = torch.tensor([0.3, -0.2], requires_grad=True)
b = torch.tensor(0.1, requires_grad=True)
x = torch.tensor([0.5, -1.2])
y = torch.tensor(1.0)

a = torch.sigmoid((w * x).sum() + b)
loss = 0.5 * (a - y) ** 2
loss.backward()

print("grad w:", w.grad)      # الناتج: tensor([0.0244336, -0.0586406])
print("grad b:", b.grad)      # الناتج: tensor(0.0488672)

# 2) التحقّق من صحّة التنفيذ بأداة PyTorch نفسها
from torch.autograd import gradcheck

def f(params):
    ww, bb = params
    out = torch.sigmoid((ww * x).sum() + bb)
    return 0.5 * (out - y) ** 2

print("gradcheck:", gradcheck(f, (w.detach().clone().requires_grad_(),
                                  b.detach().clone().requires_grad_())))
# الناتج: gradcheck: True
```

★ ثلاث قواعد تختصر 80% من أخطاء autograd

- كل tensor يحتاج تدرّجاً يجب أن يُصنع بـ `requires_grad=True` (أو يُستنتج من عملية على tensor كذلك).
- تدرّجات المعاملات تتراكم (accumulate)؛ استدعاء `backward()` مرّتين بلا `zero_grad()` يضاعف التدرّجات.
- كل عملية داخل `torch.no_grad()` أو `torch.inference_mode()` تُخرج خارج الرسم البياني – فلا تدرّجات فيها.

حلقة التدريب: من الكود إلى الفهم

هذه هي الحلقة التي ستكتبها مئات المرات في مسيرتك. نعرضها كاملة، ثم نشرح كل سطر:

Python

```
import torch
from torch import nn

model = nn.Sequential(nn.Linear(20, 64), nn.ReLU(), nn.Linear(64, 1))
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-2)
loss_fn = nn.BCEWithLogitsLoss()

for epoch in range(10):
    model.train()                    # 1) وضع التدريب (يبدّل سلوك Dropout/BatchNorm)
    for xb, yb in loader:            # 2) جلب دفعة
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad(set_to_none=True)   # 3) تصفير التدرّجات
        pred = model(xb).squeeze(1)             # 4) تمرير أمامي
        loss = loss_fn(pred, yb.float())        # 5) الخسارة
        loss.backward()                         # 6) انتشار عكسي
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)  # 7) قصّ التدرّج
        optimizer.step()                        # 8) تحديث الأوزان
    # 9) جدولة معدّل التعلّم (10 + تقييم على التحقّق)
    scheduler.step()
```

السطر	لماذا هو موجود	ماذا يحدث لو حذفته
`model.train()`	يبدّل Dropout وBatchNorm إلى وضع التدريب	تقييم بأوزان إحصاء غير صحيح، Train/Val يفترقان
`zero_grad()`	يمسح تدرّجات الدورة السابقة	تدرّجات متراكمة تنفجر تدريبك
`backward()`	يحسب التدرّج الكلي للخسارة	لا تتغيّر الأوزان
`clip_grad_norm_`	يحدّ من انفجار التدرّجات	قد ينفجر التدريب عند تدرّج حادّ
`optimizer.step()`	يطبّق التحديث على الأوزان	لا تعلّم
`scheduler.step()`	يعدّل معدّل التعلّم بمرور الزمن	تعلّم أبطأ أو غير مستقر

أسطر الحلقة وسبب وجود كل سطر

لو كان لديك نموذج يتدرّب بنفسه لكنّه لا يتحسّن أبداً: هل تكون المشكلة في `optimizer.step()`، أم في `backward()`، أم في البيانات؟ وكيف تتحقّق من ذلك في أقل من خمس دقائق؟

المحسّنات: كيف يختار المحسّن خطوة التحديث؟

كل المحسّنات تحسب التدرّج نفسه؛ الفرق في **كيف تحوّل التدرّج إلى خطوة**. لنكتب ثلاثة منها بأنفسنا – ستفهم الفرق بالبناء لا بالوصف:

Python

```python
import torch

def sgd(params, grads, lr):
    for p, g in zip(params, grads):
        p.data -= lr * g

def momentum(params, grads, lr, velocity, beta=0.9):
    for i, (p, g) in enumerate(zip(params, grads)):
        velocity[i] = beta * velocity[i] + g
        p.data -= lr * velocity[i]
```

```python
def adam(params, grads, lr, m, v, t, beta1=0.9, beta2=0.999, eps=1e-8):
    for i, (p, g) in enumerate(zip(params, grads)):
        m[i] = beta1 * m[i] + (1 - beta1) * g
        v[i] = beta2 * v[i] + (1 - beta2) * g ** 2
        mhat = m[i] / (1 - beta1 ** t)
        vhat = v[i] / (1 - beta2 ** t)
        p.data -= lr * mhat / (torch.sqrt(vhat) + eps)

# تشغيل المقارنة على problem بسيط: y = w*x
w = torch.tensor([5.0], requires_grad=True)
x, y = torch.tensor([1.0]), torch.tensor([3.0])
opt = torch.optim.SGD([w], lr=0.05)   # أو جرّب Adam على نفس المسألة
for _ in range(400):
    opt.zero_grad()
    loss = ((w * x - y) ** 2).mean()
    loss.backward()
    opt.step()
print("w =", w.item())   # الناتج: w = tensor(3.0000) بعد 400 خطوة
```

المحسّن	الفكرة	مزايا	عيوبها
SGD	خطوة = التدرّج	بسيط، ذاكرة أقل، تعميم جيد	بطيء، حسّاس لمعدّل التعلّم
SGD + Momentum	يجمع التدرّجات بمعامل تسارع	أسرع، يهدّئ التذبذب	قد يتجاوز القاع
RMSProp	يقسّم التدرّج على جذر متوسّط مربّعه	يتكيّف مع حجم التدرّج	يمكن أن يتأرجح
Adam	SGD + زخم + تطبيع لحظيّ	يبدأ من أول خطوة تقريباً	تدهور مدمج سلوكياً
AdamW	Adam مع فصل weight decay	الأنسب مع Transformers	أقل استقراراً عند lr كبير جداً

مقارنة المحسّنات: الفكرة، وأنت تختارها

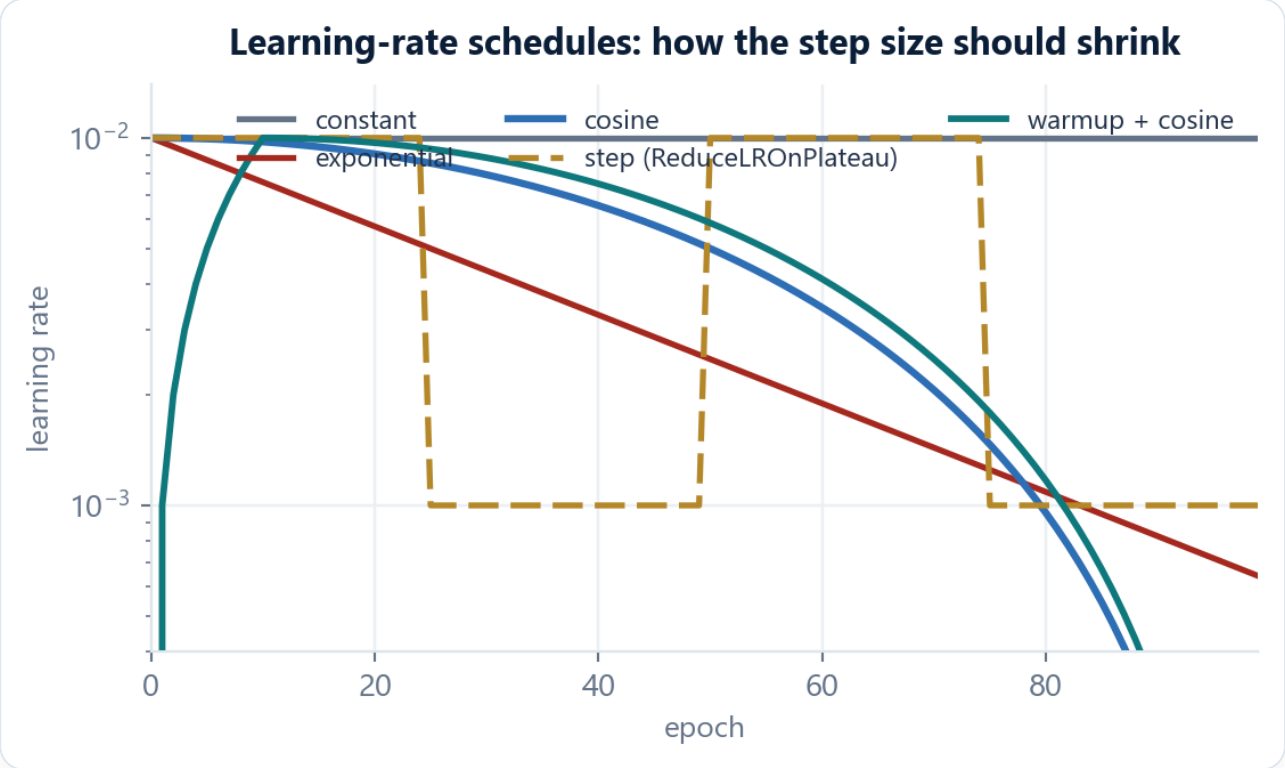
> ✓ **قاعدة اختيار عملية**
>
> في رؤية حاسوبية على بيانات ضخمة: **SGD + Momentum** (أو `OneCycleLR`) ما زال يعطي أفضل تعميم. في نصوص ولوحات وتعلّم نقلي: **AdamW**. إن لم تكن متأكداً ابدأ بـAdamW مع `lr=3e-4` (أو 1e-3 للمبتدئ) و `weight_decay=0.01`، ثم اضبط بناءً على منحنيات التحقّق.

معدّل التعلّم: أهمّ مفرط في مشروعك

كل المباحثات تعترف بذلك: **معدّل التعلّم يؤثر على النتيجة أكثر من أيّ مفرط آخر**. ثلاثة أعراض تميّز مشاكله:

العَرَض	السبب الأرجح	العلاج
الخسارة ثابتة تماماً (لا تتغيّر)	معدّل تعلّم صغير جداً، أو تدرّجات صفرية	ارفع lr بمقدار 3-10x، أو افحص `requires_grad`
الخسارة تتذبذب حول قيمة	معدّل تعلّم كبير	انصفه، وأضف warmup
الخسارة تصبح NaN	انفجار تدرّج (gradient explosion)	قلّل lr، أضف `clip_grad_norm_`، افحص البيانات
التدريب جيّد والتحقّق يرتفع	إفراط في المواءمة	تعزيز، early stopping، Dropout، بيانات أكثر
التدريب والتحقّق كلاهما ضعيف	إفراط في التحيّز (underfitting)	نموذج أكبر، تدريب أطول، بيانات أفضل

تشخيص سريع: العَرَض ← السبب ← العلاج



الشكل 4.2: أشهر جداول معدّل التعلّم. **Cosine** هو خيار قوي حديث (سلالم + إقلاع) ويشيع في المحوّلات؛ **ReduceLROnPlateau** يخفّض المعدّل عند ركود التحقّق، وهو مفيد عندما لا تعرف مسبقاً شكل المنحنى.

Python

```python
import torch
from torch.optim.lr_scheduler import CosineAnnealingLR, OneCycleLR

model = torch.nn.Linear(20, 1)
opt = torch.optim.AdamW(model.parameters(), lr=3e-4)

# جدول cosine بسيط
sched = CosineAnnealingLR(opt, T_max=100)
for epoch in range(100):
    # ... (دورة التدريب) ...
    sched.step()

# OneCycle: إقلاع ثم انحدار (الأداء الأعلى عادةً في CNN)
opt2 = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9)
sched2 = OneCycleLR(opt2, max_lr=0.5, epochs=50, steps_per_epoch=10)
print("start lr:", sched2.get_last_lr())
# الناتج: start lr: [0.0049999999...]
```

حجم الدفعة: مفاضلة بين السرعة والجودة

حجم الدفعة (batch size) يحدّد عدد الأمثلة التي تُعالَج معاً في خطوة واحدة. له ثلاثة آثار متداخلة:

- **السرعة:** الدفعة الأكبر تستغل الGPU أكثر (توازٍ أعلى) لكن تستهلك ذاكرة أكبر.
- **جودة التدرّج:** تدرّج الدفعة مقياس «غير مُتحيّز» (`unbiased`) للتدرّج الحقيقي: ضجيجه $1/\sqrt{B}$ ~ . كلّما زادت B قلّ الضجيج.
- **التعميم:** الضجيج نفسه يعمل منظّماً: التدرّجات الضوضوية تمنع النموذج من أن يجد حلولاً هشّة. لذا النماذج الصغيرة قد تُعمّم أفضل رغم بطئها.

✦ قاعدة مقياس ضجيج التدرّج (GNSS)

دراسات حديثة (مثل أعمال Google) أظهرت أن أفضل حجم دفعة يتعلّم خطّياً مع «مقياس ضجيج التدرّج»: إن كان الضجيج عند حجم دفعة معيّن أكبر من بُعد النموذج بأضعاف، فعّل ذلك الحجم كبير. عملياً: ابدأ بأكبر حجم يتّسع في الذاكرة، ثم جرّب نصفه وربعه وقارن منحنيات التحقّق.

متى ينهار التدريب؟

ننتقل الآن من «كيف يعمل» إلى «ماذا تفعل حين لا يعمل». أكثر أسباب الفشل شيوعاً:

1. **معدّل تعلّم خاطئ** (الأكثر شيوعاً): جرّب 3e-3، 1e-3، 3e-4، 1e-4 وراقب.
2. **تدرّجات صفرية أو NaN**: افحص `requires_grad` ، ثم `clip_grad_norm_` ، ثم بياناتك.
3. **تسريب بيانات**: تحقّق من التطبيع (normalization) و Dropout في التدريب فقط.
4. **خطأ في التقسيم**: صنف يدوي في بياناتTrain غير موجود في التحقّق.
5. **أصفار فقط**: دالّة خسارة رديئة أو أهداف غير مرتّبة.

⚙ بروتوكول تشخيص خمس دقائق

1. اطبع `loss` و `p.grad.norm()` لكل طبقة في أول 5 خطوات.
2. إذا كانت التدرّجات أصفاراً: المشكلة في `requires_grad` أو في `no_grad` .
3. إذا كانت التدرّجات ضخمة(> 100): أضف قصّاً، واقسم lr على عشرة.
4. إذا كانت التدرّجات طبيعية لكن الخسارة لا تتحرّك: جرّب lr أكبر 10×.
5. إذا تحرّكت الخسارة في التدريب ولم تتحرّك في التحقّق: المشكلة في التعميم لا في المحسّن.

تمرين 1 — تنفيذ Adam بأنفسك ومقارنته بالمكتبة

1. نفّذ دالّة Adam (كما في الشيفرة أعلاه) على نموذج خطّي بسيط.

2. قارن منحنى الخسارة مع `torch.optim.Adam` على المسألة نفسها.

3. غيّر `beta2` من 0.999 إلى 0.99، وفسّر اختلاف السلوك.

4. استنتج: ما الذي يفعله `eps` ؟ وماذا يحدث إن حذفته؟

▲ الحل النموذجي

Python

```python
import torch

def adam_manual(params, lr, m, v, t, beta1=0.9, beta2=0.999, eps=1e-8):
    for i, p in enumerate(params):
        g = p.grad
        m[i] = beta1 * m[i] + (1 - beta1) * g
        v[i] = beta2 * v[i] + (1 - beta2) * g ** 2
        mhat = m[i] / (1 - beta1 ** t)
        vhat = v[i] / (1 - beta2 ** t)
        p.data -= lr * mhat / (torch.sqrt(vhat) + eps)

def run(lr, beta2, manual=True, epochs=300):
    torch.manual_seed(0)
    w = torch.tensor([5.0], requires_grad=True)
    x, y = torch.tensor([1.0]), torch.tensor([3.0])
    m = [torch.zeros_like(w)]; v = [torch.zeros_like(w)]
    opt = None if manual else torch.optim.Adam([w], lr=lr, betas=(0.9, beta2))
    for t in range(1, epochs + 1):
        if manual:
            w.grad = None
            loss = ((w * x - y) ** 2).mean()
            loss.backward()
            adam_manual([w], lr, m, v, t, beta2=beta2)
        else:
            opt.zero_grad()
            loss = ((w * x - y) ** 2).mean()
            loss.backward()
            opt.step()
    return w.item()

print("manual   :", round(run(0.05, 0.999, True), 4))
print("builtin  :", round(run(0.05, 0.999, False), 4))
print("beta2=0.99:", round(run(0.05, 0.99, True), 4))
```

الإجابات: (1) المنحنيان متطابقان عملياً لأن الصيغة هي نفسها. (2) `beta2=0.999` يعطي نافذة متوسّط مربّع طويلة (≈1000 خطوة)، فيؤثّر على التدرّجات ببطء ويتكيّف مع التدرّجات الصغيرة؛ و

`0.99` يعطي نافذة أقصر (≈100 خطوة) فيتحرّك أسرع ويتذبذب أكثر. (3) `eps` يمنع القسمة على صفر عندما يكون التدرّج صفراً تماماً (أول خطوة مثلاً).

تمرين 2 — تشخيص تدريب متعثّر

1. اكتب حلقة تدريب على `make_moons` بمعدّل تعلّم 0.5، واعرض الخسارة كل 10 دورات.
2. أعِد المحاولة بـ0.05 ثم 0.005، وقارن سلاسة المنحنى وسرعة الوصول.
3. أضف `clip_grad_norm_(model.parameters(), 1.0)` في الحالة الأولى، ولاحظ الفرق.
4. استنتج: أيّ معالَج تعالج كل حالة (تذبذب، بطء، انفجار)؟

▲ الحل النموذجي

Python

```python
import torch
import torch.nn as nn
from sklearn.datasets import make_moons

X, y = make_moons(n_samples=1000, noise=0.2, random_state=0)
Xt = torch.tensor(X, dtype=torch.float32)
yt = torch.tensor(y, dtype=torch.long)

def train(lr, clip=None, epochs=200):
    torch.manual_seed(0)
    model = nn.Sequential(nn.Linear(2, 64), nn.ReLU(), nn.Linear(64, 2))
    opt = torch.optim.SGD(model.parameters(), lr=lr, momentum=0.9)
    lossf = nn.CrossEntropyLoss()
    hist = []
    for ep in range(epochs):
        opt.zero_grad()
        loss = lossf(model(Xt), yt)
        loss.backward()
        if clip:
            nn.utils.clip_grad_norm_(model.parameters(), clip)
        opt.step()
        hist.append(loss.item())
    return hist

for lr in (0.5, 0.05, 0.005):
    h = train(lr)
    print(f"lr={lr:<7} first={h[0]:.4f} mid={h[100]:.4f} last={h[-1]:.4f}")
```

القراءة: مع 0.5 ترتفع الخسارة أو تتذبذب (تحديثات تتجاوز القاع)؛ مع 0.05 تنخفض بسلاسة؛ مع 0.005 تنخفض أبطأ. القصّ يخفّف التذبذب لكنه لا يعالج السبب الجذري (معدّل التعلّم الكبير) – بل يمنع الانفجار فقط.

ملخّص الفصل الرابع

- الانتشار العكبي = تطبيق متتابع لقاعدة السلسلة من الخلف إلى الأمام.
- مسؤولية كل وزن = الخطأ × أثر التفعيل × المدخل.
- Autograd يبني الرسم البياني تلقائياً؛ لا تكتب الاشتقاقات في الإنتاج.
- ثلاث قواعد autograd: `requires_grad` ، تصفير التدرّجات، و `no_grad` .
- المحسّنات كلها تحسب التدرّج نفسه؛ الفرق في تحويله إلى خطوة.
- معدّل التعلّم هو أهمّ مفرط: عَرَضه «خسارة ثابتة» أو «تذبذب» أو «NaN».
- حجم الدفعة مفاضلة بين السرعة وجودة التدرّج والتعميم.
- قبل كل شيء: ابدأ بخطّ أساس، ثم غيّر مفرطاً واحداً في كل مرّة.

بيئة عمل احترافية على Python 3.14

البيئة الافتراضية، الحزم، العتاد، الحتمية، وهيكل المشروع

النموذج الذي لا يمكن إعادة إنتاجه بعد ستة أشهر ليس نموذجاً، بل حادثة. هذا الفصل يبني لك بيئة عمل لا تُنسى: معزولة، قابلة للتكرار، سريعة، ومهيّأة للعتاد الذي تملكه – على Python 3.14 وإصدارَي PyTorch 2.14 و Keras 3.15.

البيئة المعزولة: القاعدة الأولى

المشكلة التي لا تنتهي: مشروع يحتاج `numpy==1.26`، وآخر يحتاج `numpy==2.1`، وآخر يحتاج حزمة لم تعد تدعم 3.10. الحل الوحيد الذي ينجح على المدى الطويل هو **عزل كل مشروع في بيئته الخاصة**.

الأداة	المزايا	العيوب	الأنسب لـ
`venv` (مدمج)	موجود دائماً، سريع	لا يحل التبعيات	البداية الفورية
`uv`	سرعة هائلة، يحل التبعيات، قفل	جديد نسبياً	المشاريع الجديدة (اختيارنا)
`pip` + `requirements.txt`	مألوف عالمياً	بطيء في الحل	بيئات الإنتاج المؤسسية
`mamba` / `conda`	يدير تبعيات النظام والعتاد	ثقيل، أبطأ	بيئات تتضمّن CUDA معقّدة
`pdm` / `poetry`	قفل وإدارة معاً	منحنيات تعلّم	مشاريع الفريق

أدوات إدارة البيئة: أيّها تختار؟

أوامر الطرفية

```
$ # إعداد مشروع بالطريقة التي أوصي بها (Windows PowerShell)
$ $ mkdir F:\projects\vision-project
```

```
$ $ cd F:\projects\vision-project
$ $ py -3.14 -m venv .venv
$ $ .\.venv\Scripts\Activate.ps1

$ # باستخدام uv (أسرع بكثير في حلّ التبعيات)
$ $ pip install -U uv
$ $ uv init --python 3.14
$ $ uv add torch==2.14.1 --index-url https://download.pytorch.org/whl/cpu
$ $ uv add keras==3.15.1 numpy matplotlib scikit-learn pillow
$ $ uv pip compile requirements.in -o requirements.txt  # توليد قفل ثابت
```

> ⚠ **أشهر خطأ في بيئات التعلّم العميق**
>
> تثبيت الحزم في Python النظامي (بدون بيئة) ثم اكتشاف بعد أسبوعين أن إصدارات المكتبة تغيّرت وكتبك لم تعد تعمل. الحل: **قفل الإصدارات** في ملف `requirements.txt`، و*تثبيت الجهاز نفسه* (requirements.txt مع `--hash` أو uv.lock) في كل بيئة.

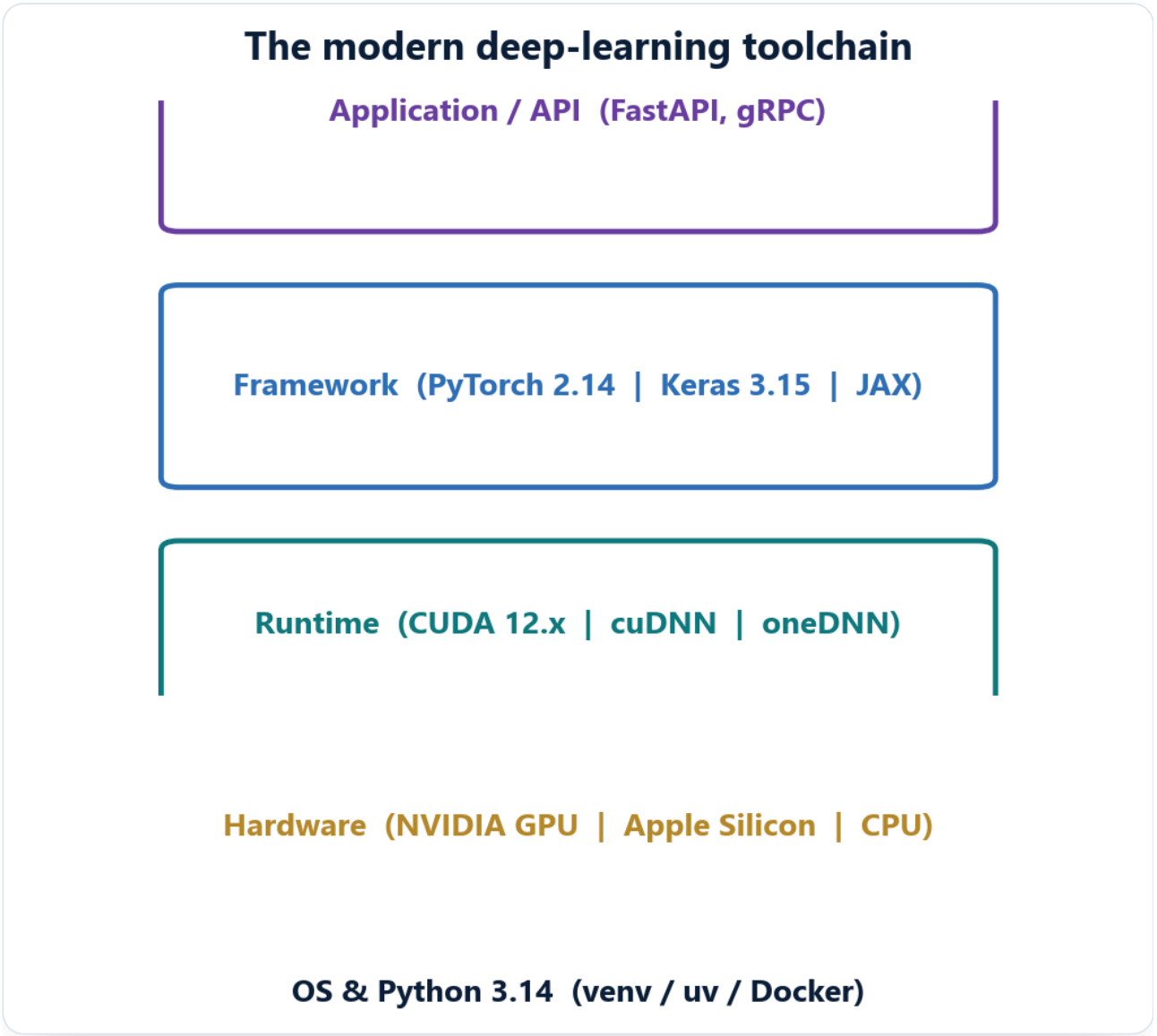
بنية المشروع الاحترافية

بنية المجلدات قرار صغير ظاهر بمظهر كبير. البنية التي أنصح بها (مستوحاة من قوالب `cookiecutter-data-science` و`ml-template`):

نص

```
project/
├── pyproject.toml      # بيانات المشروع والاعتماديات والإعدادات
├── README.md           # كيف تُشغّل المشروع (اكتبه لنفسك بعد شهر!)
├── .env.example        # متغيّرات البيئة (لا تضع الأسرار!)
├── configs/
│   ├── base.yaml       # إعدادات مشتركة
│   └── train.yaml      # إعدادات التدريب
├── src/
│   ├── data/
│   │   ├── dataset.py    # تعريف Dataset والتحويلات
│   │   └── datamodule.py  # بناء DataLoaders
│   ├── models/
│   │   ├── backbone.py    # nn.Module
│   │   └── factory.py    # بناء النموذج من الإعدادات
│   ├── train.py        # حلقة التدريب (نقطة دخول)
│   ├── evaluate.py     # التقييم والقياسات
```

```
│   └── utils/
│       ├── seed.py      # ضبط البذور
│       └── logging.py   # السجلات
├── scripts/
│   ├── download_data.py
│   └── serve.py         # FastAPI
├── tests/
│   └── test_shapes.py   # اختبارات أشكال ومحتوى
├── artifacts/           # المخرجات (لا تُرفع إلى git)
│   ├── checkpoints/
│   ├── logs/
│   └── figures/
└── notebooks/           # استكشاف فقط — لا إنتاج هنا
```



الشكل 5.1: سلسلة الأدوات من الأعلى إلى الأسفل. لاحظ أن طبقة **التطبيق** (FastAPI) لا تعرف شيئاً عن التدريب، وطبقة **الإطار** (PyTorch/Keras) لا تعرف شيئاً عن البيانات. هذا الفصل المنظّم هو ما يجعل استبدال النموذج لاحقاً عمليةً بسطيطة في سطر واحد.

✓ لماذا `notebooks/` ليست مكان الإنتاج؟

دفتر Jupyter أداة **استكشاف** ممتازة: تفاعلية، رسومية، وسريعة للتجريب. لكنها تحتفظ بحالة خفية (execution order) يصعب تتبّعها. القاعدة المهنية: **كل ما ينتهي في الإنتاج ينتقل إلى سكربت**، ويُستدعى من `train.py` .

العتاد: كيف تتعامل مع GPU؟

على Windows، يُثبَّت PyTorch مع دعم CUDA تلقائياً من PyPI (حوالي 2.5 غيغابايت)، بينما نسخة `cpu` من download.pytorch.org/whl/cpu أخفّ بحوالي عشرة أضعاف. إن كان لديك بطاقة NVIDIA، ثبّت نسخة CUDA؛ وإن كنت على معالج فقط أو Apple Silicon، فراجع `torch.backends.mps` .

Python

```python
import torch
import platform

def describe_hardware():
  info = {
    "python": platform.python_version(),
    "torch": torch.__version__,
    "cuda_available": torch.cuda.is_available(),
  }
  if torch.cuda.is_available():
    info["gpu"] = torch.cuda.get_device_name(0)
    info["capability"] = torch.cuda.get_device_capability(0)
    info["memory_GB"] = round(torch.cuda.get_device_properties(0).total_memory / 1e9, 1)
  info["mps"] = torch.backends.mps.is_available() if hasattr(torch.backends, "mps") else Fals
  return info

print(describe_hardware())
# الناتج التقريبي:
# {'python': '3.14.6', 'torch': '2.14.1', 'cuda_available': True,
# 'gpu': 'NVIDIA RTX 4070', 'capability': (8, 9), 'memory_GB': 12.0, 'mps': False}

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("device:", device)
```

الإعداد	الأثر	المخاطر
`torch.set_float32_matmul_precision("high")`	يضبط TF32: دقة أقل قليلاً في ضربات المصفوفات وسرعة أعلى	انحراف أرقام طفيف (1e-3~)
`torch.compile(model)`	يجمّع الرسم البياني ويحسّن النواة	زمن تجميع أول مرة؛ أخطاء غير متوقّعة

الإعداد	الأثر	المخاطر
		أحياناً
`channels_last`	ذاكرة صور أسرع على Tensor Cores	يحتاج تحويل يدوي للصور
`torch.backends.cudnn.benchmark = True`	يبحث عن أسرع خوارزمية للأشكال الثابت	بطء في أول دورة إذا تغيّر الشكل
AMP (`torch.amp.autocast`)	نصف الدقّة: ذاكرة نصف وسرعة مضاعفة	يحتاج GradScaler

خيارات الإقلاع (performance switches) في PyTorch 2.14

> ✦ **قاعدة عملية**
>
> فعّل تحسينات الأداء **بعد** أن يكون النموذج صحيحاً ومُقاساً. التحسين قبل الضبط يخلق مقارنة مضلّلة: قد تظنّ أن نموذجك سيّئ بينما حُسّن تحسين سرعة فقط سرعة.

لو كان لديك جهازان: أحدهما بـ GPU وأحدث، والآخر بلا GPU وأقدم — فأيهما تختار لتجريب أفكارك؟ ولماذا ليس الجواب دائماً «الأقوى»؟

الحتمية: أن تعيد نفس النتيجة

«النموذج أعطى 94.2%» – لكن بعد إعادة التشغيل صار 93.8%. هذا ليس تفصيلاً تجميلياً: بدون حتمية لن تستطيع أن تعرف هل التحسّن جاء من تعديل في الكود أم من عشوائية.

Python

```python
import os
import random
import numpy as np
import torch

def set_seed(seed: int = 42, deterministic: bool = False) -> None:
    """يثبّت كل مصادر العشوائية."""
    os.environ["PYTHONHASHSEED"] = str(seed)
```

```
  random.seed(seed)
  np.random.seed(seed)
  torch.manual_seed(seed)
  torch.cuda.manual_seed_all(seed)
  if deterministic:
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
    torch.use_deterministic_algorithms(True, warn_only=True)
  else:
    torch.backends.cudnn.benchmark = True

set_seed(42)
a = torch.randn(1000)
set_seed(42)
b = torch.randn(1000)
print("identical:", torch.equal(a, b))  # الناتج: identical: True
```

◆ الحتمية مقابل السرعة

الوضع الحتمي (`use_deterministic_algorithms(True)`) أبطأ أحياناً، وقد يرفع خطأً إذا كانت العملية غير حتمية أصلاً. الحل المتّبع في الممارسة: **اضبط البذور دائماً** (حتمية جزئية)، واستعمل الوضع الحتمي الكامل عند مقارنة بحثية أو عند إنتاج أرقام رسمية قابلة للتدقيق.

تسجيل التجارب: كيف تعرف ما الذي نجح؟

في المشاريع الحقيقية، النموذج الناجح هو نتيجة عشرات التجارب. إن لم تُسجّل معاييرها ونتيجتها، ستكرّر تجاربك، أو تنسى السبب الذي جعل النموذج يعمل.

Python

```
# تسجيل بسيط بدون تبعيات خارجية: JSONL
import json
import time
from pathlib import Path

class RunLogger:
  def __init__(self, out_dir="artifacts/logs", name="run"):
    self.path = Path(out_dir) / f"{name}.jsonl"
    self.path.parent.mkdir(parents=True, exist_ok=True)
    self.t0 = time.perf_counter()
```

```
  def log(self, **kwargs):
    row = {"elapsed_s": round(time.perf_counter() - self.t0, 2), **kwargs}
    with self.path.open("a", encoding="utf-8") as fh:
      fh.write(json.dumps(row, ensure_ascii=False) + "\n")

  def config(self, cfg: dict):
    self.log(event="config", **cfg)

log = RunLogger(name="exp01")
log.config(lr=3e-4, batch_size=64, model="resnet50", seed=42)
for epoch in range(3):
  log.event = "epoch"
  log(epoch=epoch, train_loss=0.42 - epoch * 0.05, val_acc=0.90 + epoch * 0.01)
print("wrote", log.path)
```

على نطاق أكبر، استخدم **TensorBoard** (مدمج مع PyTorch)، أو **Weights & Biases**، أو **MLflow**. أو `neptune`. القاعدة التي لا تتغيّر: **سجّل إعدادات التجربة ومقاييسها في ملف واحد** قابل للكتابة بعد كل حقبة.

Keras 3 على Python 3.14: اختيار النواة

كما أشرنا في التصدير: TensorFlow 2.21 لا يدعم Python 3.14 بعد. الحل في Keras 3 هو اختيار النواة صراحةً:

أوامر الطرفية

```
$ # 1) داخل الصدفة قبل استيراد keras
$ $ $env:KERAS_BACKEND = "torch"   # PowerShell
$ $ export KERAS_BACKEND=torch    # bash / Linux

$ # 2) أو داخل السكربت نفسه (قبل السطر الأول من الاستيراد)
$ $ python -c "import os; os.environ['KERAS_BACKEND']='torch'; import keras; print(keras
$ torch
```

Python

```
# اختيار النواة برمجياً داخل السكربت (قبل استيراد keras)
import os
os.environ.setdefault("KERAS_BACKEND", "torch")
```

```python
import keras
print(keras.backend.backend())   # torch
print(keras.__version__)      # 3.15.1

# اختيار الجهاز
backend = keras.backend
if backend.backend() == "torch":
  import torch
  device = "cuda" if torch.cuda.is_available() else "cpu"
  print("device:", device)
```

الجانب	نواة torch	نواة tensorflow	نواة jax
توفّره على Python 3.14	نعم	لا (TF 2.21 ≤ 3.13)	نعم
دعم `tf.data`	محدود	كامل	لا
الأداء على TPU	لا	الأفضل	ممتاز على TPU
التكامل مع PyTorch	ممتاز	ضعيف	ضعيف
الحفظ والتصدير	TorchScript + `.keras`	SavedModel	JAX checkpoint

ماذا يتغيّر بين نواتي Keras؟

نموذج دقّته 94.2% في تجربة، و93.8% في أخرى بعد إعادة التشغيل: هل هذه «تغيّر» حقيقي في النموذج أم ضجيج إحصائي؟ وكيف تتّخذ قراراً من هذا؟

قائمة تحقّق قبل بدء أيّ مشروع

1. بيئة معزولة + إصدارات مقفلة (requirements.txt أو uv.lock).
2. بذور مثبّتة في كل ملف تدريب، وتسجيلها في سجل التجربة.
3. اختبار سريع (`smoke test`) يعمل على 20 عيّنة في أقل من دقيقة.

4. خطّ أساس بسيط مسجّل (للانحدار اللوجستي أو شجرة قرار).
5. فصل واضح بين `train.py` و `evaluate.py` و `serve.py`.
6. نسخة احتياطية من الأوزان ومقاييس كل حقبة (checkpointing).
7. README يشرح كيف تُشغّل المشروع – اكتبه الآن وأنت تعرف كل شيء.

العتاد: متى يستحق الكلام عن الكلاف؟

لأنّهمس في الأوساط التقنية أنّ «التعلّم العميق يحتاج غلاء». الحقيقة أكثر دقّة: يحتاج **عتاداً متوازياً** لا بالضرورة غالياً. ثلاثة أحكام عملية:

- **للتعلّم**: أيّ GPU حديث (حتى من الجيل السابع) مع 8 غيغابايت ذاكرة يكفي لتدريب نماذج رؤية متوسّطة على ملايين الصور. الترتيب التصاعدي للتكلفة: بطاقات مستعملة ← بطاقة استئجار ← عقدة حوسبة.
- **للاستدلال**: الاستدلال أرخص بكثير من التدريب؛ نموذج صغير على CPU قد يكون كافياً لخدمة محدودة.
- **للتكرار السريع**: التكرار على المعالج (CPU) أسرع في القرار. عند الشكّ في فكرة، شغّل تجربة صغيرة على CPU خلال دقائق قبل أن تنفق ساعات على GPU.

> ★ **القاعدة الذهبية**
>
> **العتاد يحدّد سرعة التكرار، لا جودة الفكرة.** أغلب انفجارات التعلّم العميق الجيدة جاءت من تفكير أفضل في بنية النموذج أو في المعالجة المسبقة للبيانات، لا من بطاقة أغلى. ابدأ بما تملك، وقِس قبل أن تشترِ.

وبما أنّ كل شيء في هذا الكتاب قابل للتشغيل على المعالج، يمكنك إكمال كل الأمثلة بلا GPU. بطيء نعم، لكن صحيح. وهذا بالضبط ما يفعله القارئ الذي يقرأ في قطار.

تمرين 1 — جهّز مشروعاً كاملاً في 15 دقيقة

1. أنشئ مجلّداً وهيكل المجلدات أعلاه.
2. أنشئ بيئة معزولة على 3.14 وثبّت `torch==2.14.1` و `keras==3.15.1` .
3. اكتب `src/utils/seed.py` و `src/utils/logging.py` من الفصل.
4. اكتب `src/train.py` يدرّب `nn.Linear(2, 8)` على `make_moons` لعشر حقبات ويطبع الدقّة.
5. اكتب `tests/test_shapes.py` يتحقّق من شكل مخرجات النموذج.
6. شغّل السكربت مرّتين وتأكّد أن النتيجة متطابقة (بذور ثابتة).

▲ الحل النموذجي

Python

```python
# src/train.py
import os
os.environ.setdefault("KERAS_BACKEND", "torch")

import sys
sys.path.append("src")

import torch
from torch import nn
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

from utils.seed import set_seed
from utils.logging import RunLogger

def main():
  set_seed(42)
  log = RunLogger(name="moons")

  X, y = make_moons(n_samples=2000, noise=0.2, random_state=0)
  Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42)
  Xtr = torch.tensor(Xtr, dtype=torch.float32)
  ytr = torch.tensor(ytr, dtype=torch.long)
  Xte = torch.tensor(Xte, dtype=torch.float32)
  yte = torch.tensor(yte, dtype=torch.long)

  model = nn.Sequential(nn.Linear(2, 8), nn.ReLU(), nn.Linear(8, 2))
  opt = torch.optim.AdamW(model.parameters(), lr=1e-3)
  lossf = nn.CrossEntropyLoss()

  for epoch in range(10):
    model.train()
    opt.zero_grad()
    loss = lossf(model(Xtr), ytr)
    loss.backward()
    opt.step()

    model.eval()
    with torch.no_grad():
```

```
        acc = (model(Xte).argmax(1) == yte).float().mean().item()
    log(epoch=epoch, loss=round(loss.item(), 4), val_acc=round(acc, 4))
    print(f"epoch {epoch:02d} loss={loss.item():.4f} val_acc={acc:.4f}")

if __name__ == "__main__":
  main()
```

Python

```
# tests/test_shapes.py
import torch
from torch import nn

def test_output_shape():
  model = nn.Sequential(nn.Linear(2, 8), nn.ReLU(), nn.Linear(8, 2))
  out = model(torch.randn(5, 2))
  assert out.shape == (5, 2)

def test_backward_runs():
  model = nn.Sequential(nn.Linear(2, 8), nn.ReLU(), nn.Linear(8, 2))
  loss = model(torch.randn(5, 2)).sum()
  loss.backward()
  assert model[0].weight.grad is not None
```

ملاحظة: الملف `src/utils/__init__.py` مطلوب لأن بايثون يعامل المجلّدات كحزم فقط عند وجود هذا الملف.

تمرين 2 — كاشف التعارضات قبل أن تصدمها

1. في بيئة نظيفة، ثبّت `numpy==2.1` ثم `numpy==1.26` ، ولاحظ التحذير.
2. اكتب سكربتاً يطبع `pip check` أ (أو `uv pip check`).
3. جرّب تثبيت `tensorflow` على Python 3.14 ولاحظ ما يقوله المحلّل.
4. لخّص: ما الذي يمنع الفشل في بيئة إنتاج؟

▲ الحل النموذجي

المحلّل (resolver) في pip أو uv يكتشف التعارض قبل التثبيت ويرفض العملية أو يطبع تحذيراً. والعامل الحاسم في الإنتاج هو **القفل**: ملف `requirements.txt` (أو `uv.lock`) المولّد في بيئة معروفة، يُثبَّت في كل مكان آخر بـ `pip install -r requirements.txt` فتكون النتيجة مطابقة.

ومن علامات النضج الهندسي: **خط أنابيب CI** يبني البيئة من الصفر ويشغّل الاختبارات على كل تعديل، و**حاوية Docker** تحمل الإصدارات نفسها في بيئة الإنتاج. نتناولهما بالتفصيل في الفصل السابع والعشرين.

التطبيع: لماذا يغيّر المقياس لا الشكل

تطبيع الـ batch (Batch Normalization) هو أكثر طبقة استعملها باحثو الرؤية على الإطلاق، وأكثر طبقة يُساء فهمها. الفكرة في سطر واحد: قبل كلّ طبقة تلافيفية، **نطبّع** مدخلاتها على كلّ عيّنة في الدفعة، ثم نتعلّم معاملَي قياس وإزاحة يعيدان قدرتها الأصلية.

$$\hat{x} = (x - \mu_B) / \sqrt{(\sigma_B{}^2 + \varepsilon)}, \quad y = \gamma\,\hat{x} + \beta$$

ما معنى كل رمز؟ μ_B هو متوسط المميّزات على محور الدفعة، و $\sigma_B{}^2$ هو تباينها، و ε عدد صغير جداً يمنع القسمة على صفر، و γ و β معاملان قابلان للتعلّم يعيدان الشبكة القدرة على تمثيل ما تشاء. **لماذا تهمّنا؟** لأنّ التطبيع يجعل توزيع كلّ طبقة مستقرّاً مهما تغيّرت الأوزان، فيصبح ضبط معدّل التعلّم أسهل بكثير. **خطأ شائع عندها:** ظنّ أنّ التطبيع يجعل الشبكة تعمل بـ«وزن واحد دائماً». هذا فقط صحيح في وضع الاستدلال؛ أثناء التدريب تُحسب الإحصاءات على كلّ دفعة وتُحدَّث.

◆ الفرق الذي يربك الجميع

أثناء التدريب: تُحسب μ و σ من الدفعة الحالية. أثناء الاستدلال: تُستخدم المتوسّطات المتراكمة من التدريب (running averages). لهذا تبديل النموذج إلى وضع التقييم قبل الاستدلال ليس تفصيلاً شكلياً: من دونه سيستخدم النموذج إحصاءات الدفعة الواحدة، فتختلف النتائج بين دفعة وأخرى.

نموذج تصنيف يعمل بدقّة ٩٤٪ في التدريب و٩١٪ في الاستدلال. المشكلةشخص: هي وبعد تطبيق التطبيع صارت ٩٣٪. بأيّ احتمال تفسّر فرقاً بهذه الدقّة؟ ماالفرق المتوقّع بينTraining و Inference في حساب إحصاءات التطبيع؟

الاحتمال الأقوى هو التنميط (memorization): النموذج تعلّم أن يميّز عيّنات التدريب بالاعتماد على تفاصيل لا تظهر في التحقّق. أمّا الفارق بين التدريب والاستدلال في إحصاءات التطبيع فهو مصدر شائع للفرق الأصغر: في التدريب تُحسب الإحصاءات على ٦٤ عيّنة، فيتذبذب الأداء؛ وفي الاستدلال تُستخدم المتوسّطات المتراكمة من مئات الدفعات، فيستقرّ. ولهذا نذكر دائماً: قِس التحقّق في وضع التقييم دائماً.

ملخّص الفصل الخامس

- اعزل كل مشروع في بيئته، واقفل إصداراته.
- البنية الجيدة: `src/` و `configs/` و `tests/` و `artifacts/` .
- الدفتر للاستكشاف، والسكربت للإنتاج.
- اختر العتاد أولاً (CUDA/MPS/CPU) ثم فعّل التحسينات بعد ضبط النموذج.
- ثبّت البذور وسجّل كل تجربة وإعداداتها.
- على Python 3.14: Keras 3 بنواة torch أو jax؛ TensorFlow غير متاح بعد.
- أبقِ «نقطة الدخول» واحدة: `train.py` .

Tensor: حجر الأساس في PyTorch

الأنواع، الأشكال، الأجهزة، البث، الفهرسة، والتبديل بين الذاكرة

تخيّل أنك في مختبر تحليل مخبري، أمامك عيّنة دم يجب تصنيفها إلى فئات مرضية. ما الذي تحمله بين يديك فعلياً؟ أرقام مفردة؟ أم مصفوفة قياسات لكل مريض؟ أم حزمة بيانات تتضمّن قياس كل مريض، ونوع كل قياس، واسم الجهاز الذي أنتجه؟ الإجابة في PyTorch هي: **الشدّة** (tensor) – الكائن الذي يجمع الأرقام في شبكة متعددة الأبعاد، ويحفظ معها نوعها، ومكان وجودها في الذاكرة، وروابطها بمحرّك التمايز التلقائي. في هذا الفصل سنفكّك الشدّة قطعةً قطعة: كيف تُصنع، كيف تتصرّف، كيف تنقل بياناتها بين المعالج والGPU، ولماذا يخطئ المبتدئون في فهم البثّ (broadcasting) والذاكرة المشتركة (views). كل خطأ سيُشرح من جذوره، لأنّه في الفصل العاشر ستكتشف أنّ نصف أخطاء «النموذج لا يتعلّم» سببها خطأ في التعامل مع الشدّات.

لماذا الشدّة؟ ما الذي تضيفه على القوائم والمصفوفات

في بايثون العادية، أعداد متفرّقة من نوع `list` مخزّنة في الذاكرة واحداً بعد الآخر، ولا يعرف بايثون أيّها عدد ولا نوعه. في `numpy` تتحسّن الأمور: تصبح `ndarray` متجاوراً في الذاكرة، له شكل (shape) ونوع (dtype)، وتعمل عملياته بسرعة عالية. لكن `numpy` يبقى مكتبة حسابية على المعالج (CPU)، ونظاماً منفصلاً عن التعلّم العميق.

الشدّة في PyTorch ليست مجرّد مصفوفة: هي *مصفوفة + نوع + جهاز + تاريخ تدرّج*. حين تكتب `y = W @ x + b`، فإن `W` لا تحمل الأرقام فحسب، بل تحمل أيضاً معلومة أن الأرقام مشتقّة من معاملات قابلة للتعلّم، وأنها تقيم الآن على وحدة معالجة رسوميات (GPU)، وأنها تُخزَّن بترتيب معيّن في الذاكرة. هذه المعلومات الأربع هي ما يجعل `backward()` ممكنة بنقرة واحدة، وهي ما ستفهمه في الفصل الثامن.

السمة	قائمة بايثون	مصفوفة `numpy`	شدّة `torch`
عدد الأبعاد	غير معروف وقت التشغيل	معروف وثابت	معروف وثابت
نوع البيانات	مختلط ومحسوب وقت التشغيل	واحد لكل مصفوفة	واحد لكل شدّة، مع دعم للعدد المنخفض
الجهاز	الذاكرة الرئيسية فقط	الذاكرة الرئيسية فقط	وحدة المعالجة أو بطاقة الرسوميات
التمايز التلقائي	لا	لا	نعم، عبر `requires_grad`
تسريع العتاد	لا	لا	نعم، عبر المكتبات الأصلية

> لو كانت `numpy` كافية للتعلّم العميق، فلماذا بُني التدرّج التلقائي داخل `torch` نفسه بدل أن يكون غلافاً حوله؟ ما الذي يخسره المصمّم لو بقي `torch` مجرّد غلاف نحيف حول `numpy` ؟

الجواب في التصميم: **الانتقال إلى العتاد يجب أن يكون شفافاً**. حين تنتقل شدّة من وحدة المعالجة إلى بطاقة الرسوميات، يجب أن يبقى تدرّجها مرتبطاً بها، وإلا لاحتجتَ إلى نظام تدرّج موازٍ منفصل. ولأن `torch` يملك الشدّة من الأصل، فالجسر بين الجهازين لا ينقل الأرقام فحسب، بل ينقل *سلسلة التدرّج* معه. ولهذا السبب أيضاً، في Keras 3، عندما تختار نواة `torch` ، فأنت في الواقع تستخدم محرّك `torch` نفسه بأسلوب واجهة مختلف.

الإنشاء: من الصفر إلى المصفوفة، والخصائص الثلاث

لكل شدّة ثلاث خصائص لا يمكن تجاهلها: **الشكل** (عدد المحاور وأطوالها)، و**النوع** (`dtype`)، و**الجهاز** (`device`). وأيّ خطأ في هذه الثلاث يُنتج خطأً في الشكل لا في القيم: انتبه أنّ `torch.randn(3, 4)` لا تنشئ مصفوفة 4×3 مرتّبة صفاً صفاً، بل **أربعة عشر** عدداً عشوائياً موزّعاً على ثلاثة صفوف وأربعة أعمدة.

```python
import torch
```

```python
# تثبيت مصدر العشوائية: نفس البذرة تعطي نفس النتائج دائماً
torch.manual_seed(42)

t = torch.arange(12, dtype=torch.float32).reshape(3, 4)

print(t.shape)                # الناتج: torch.Size([3, 4])
print(t.dtype)                # الناتج: torch.float32
print(t.device)               # الناتج: cpu
print(t.ndim, t.numel())      # الناتج: 2 12
print(t[1, 2])                # الناتج: tensor(6.)

# الطرق الأخرى لصنع الشدّات
print(torch.zeros(2, 3).shape)       # الناتج: torch.Size([2, 3])
print(torch.ones(2, 3).shape)        # الناتج: torch.Size([2, 3])
print(torch.rand(2, 3).shape)        # الناتج: torch.Size([2, 3])  (قيم في [0, 1)
print(torch.randn(2, 3).shape)       # الناتج: torch.Size([2, 3])  توزيع طبيعي
print(torch.linspace(0, 1, 5).shape) # الناتج: torch.Size([5])      خمس نقاط متساوية
```

ماذا يحدث هنا؟ `torch.arange(12)` ينتج شدّة واحدة `torch.Size([12])` واحدة، ثم `reshape(3, 4)` يعيد ترتيب نفس الأرقام الاثني عشر في ثلاثة صفوف. **لماذا هذه القيمة؟** لأن `torch.manual_seed(42)` يثبّت مولّد الأعداد العشوائية، فتصبح النتيجة قابلة للتكرار: وهو شرط أساسي لأي بحث علمي. لاحظ أيضاً أنّ `torch.arange(12)` ينتج نوعاً صحيحاً `int64` افتراضياً، ولذلك حدّدنا `torch.float32` صراحةً. هذا فرق مهم: التدرّج لا يمكن حسابه على الأعداد الصحيحة.

النوع	عدد البتات	الاستخدام النموذجي	ملاحظة
`torch.float32`	32	الأوزان والمخرجات والتدرّجات	الافتراضي، والأدقّ عملياً للمهام العميقة
`torch.float16`	16	الحسابات المختلطة (AMP)	مدى أرقام صغير، يحتاج `GradScaler`
`torch.bfloat16`	16	تدريب النماذج الكبيرة	مدى أوسع من `float16` بأقل دقّة
`torch.float64`	64	الحسابات العددية الدقيقة	أبطأ مرتين تقريباً، لا يُستعمل في التدريب

النوع	عدد البتات	الاستخدام النموذجي	ملاحظة
`torch.int64`	64	التسميات والفهارس وأطوال المحاور	الافتراضي للفهارس
`torch.bool`	1	الأقنعة المنطقية	نتيجة المقارنات المنطقية

⚠ الخطأ الصامت: خلط الأنواع

عند جمع شدّة من نوع `float32` مع شدّة من نوع `float64` لا يخطئ `torch`، بل **يرفع النوع تلقائياً** إلى `float64`. النتيجة: نموذج أبطأ بمقدار ملحوظ، وذاكرة أضعاف، دون أي رسالة خطأ. القاعدة: أبقِ كل شيء في `float32` ما لم يكن لديك سبب عددي واضح للتحويل.

الأجهزة: كيف تنتقل الشدّة إلى بطاقة الرسوميات

لكل شدّة **جهاز** مكتوب عليها. الشدّة المبنيّة في بايثون تولد على `cpu`، ومع وجود بطاقة رسوميات متوافقة يصبح من الممكن نقلها إلى `cuda`. الطريقة الصحيحة هي `.to(device)`، وهي تُنتج شدّة **جديدة** في المكان الجديد ولا تغيّر الأصل.

Python

```
import torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(device)                       # الناتج: cuda (أو cpu إن لم تتوفّر بطاقة)

w = torch.randn(512, 512)
w_gpu = w.to(device)                # نسخة على الجهاز الجديد
print(w.device, w_gpu.device)       # الناتج: cpu cuda

# التحويل في النوع أيضاً، ويمكن دمج الأمرين في نداء واحد
w_half = w.to(device=device, dtype=torch.float16)
print(w_half.dtype)                 # الناتج: torch.float16

# الأجهزة المتاحة فعلياً
if torch.cuda.is_available():
```

```
print(torch.cuda.get_device_name(0))   # اسم بطاقة الرسوميات
print(torch.cuda.device_count())       # الناتج: 1
```

ماذا يحدث هنا؟ `.to(device)` لا ينسخ البيانات إن كانت الشدّة أصلاً على الجهاز المطلوب. **لماذا هذه القيمة؟** لأن النسخ بين وحدة المعالجة وبطاقة الرسوميات مكلف، و PyTorch يوفّر مساراً سريعاً عبر الذاكرة مثبّتة (pinned memory) والنسخ غير المتزامن (non-blocking) عند تمرير `non_blocking=True` مع `pin_memory=True` في محمّل البيانات – وهو تفصيل سنعود إليه في الفصل التاسع.

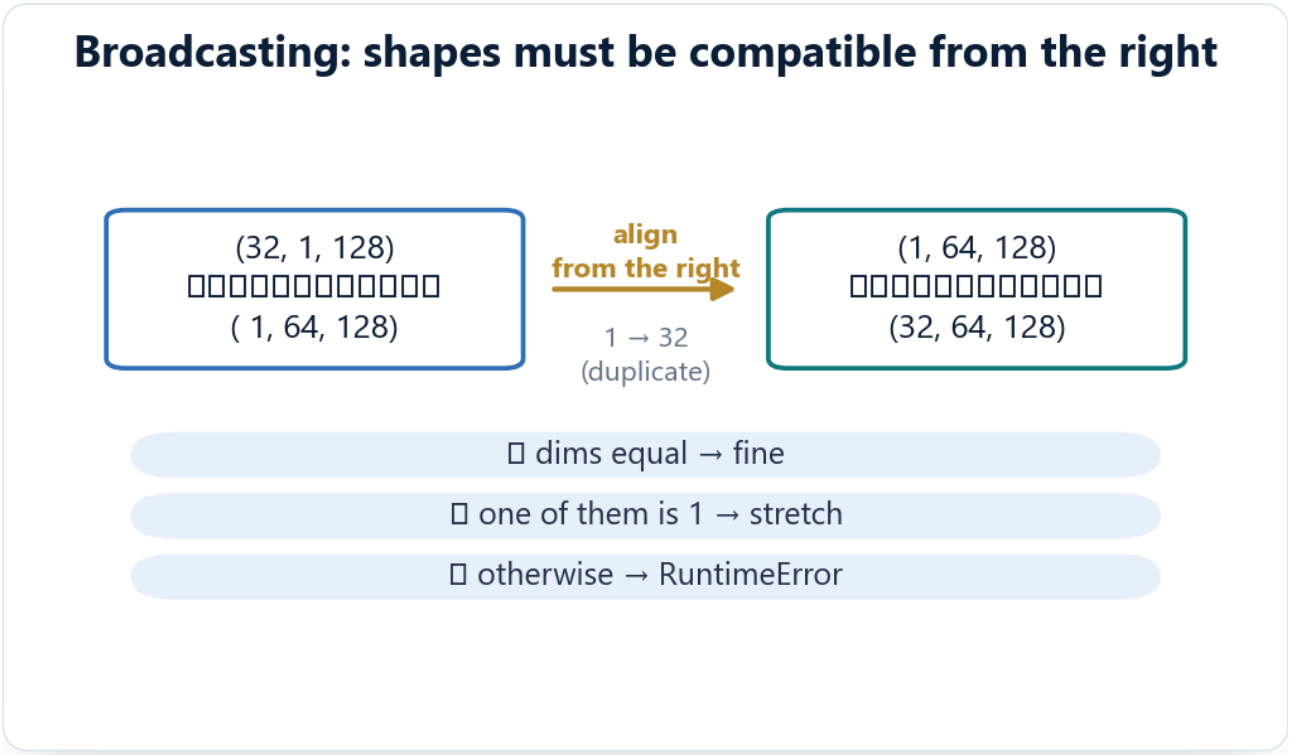
> ✓ **نصيحة: انقل النموذج لا الشدّة**
>
> الأصح أن تنقل **المعاملات** مرّة واحدة قبل بدء التدريب: `model.to(device)`، ثم تعمل كل الشدّات على الجهاز نفسه. نقل الشدّة يدوياً في كل خطوة تدريب يسبّب نسخاً متكرّرة، وهو أشهر سبب لأن يكون التدريب على البطاقة أبطأ من `numpy` على المعالج.

البثّ: القاعدة التي تختصر آلاف الأسطر

تخيّل dataset فيه 32 عيّنة، ولكل عيّنة 3 قنوات، ولكل قناة 64×64 بكسلاً. فهل تكتب `for i in range(32): for c in range(3): ...` لتضيف انحيازاً لكل قناة؟ بالطبع لا. **البثّ** (broadcasting) يخبرك أنّ عملية بين شدّتين مختلفتي الشكل تُنفَّذ وكأن الشكلين متوافقان تلقائياً.

$$\text{Broadcasting}(a, b) = \text{شكل النتيجة، ويُحسب من اليمين إلى اليسار}$$

المعادلة (6.1): لكل محور، إما أن يتساوى الطولان، أو يكون أحدهما 1 (فيُمدَّد)، أو لا يمكن البثّ فيرفع `torch` خطأً. القاعدة الواحدة: **المحاور تُحاذى من اليمين**، تماماً كما يُحاذى المحور الأخير في ضرب numpy.



الشكل 6.1: كيف يتمدّد البُعد الذي طوله 1 ليطابق بُعداً آخر. اليسار: مصفوفة (3, 1) تُبثّ إلى (3, 2) فتُكرَّر كل قيمة على امتداد الصف. اليمين: متجه (3,) يُبثّ إلى (3, 2) فيصبح عموداً. القاعدة تُقرأ من اليمين: كل محور إما متطابق أو واحد.

Python

```python
import torch

a = torch.arange(6).reshape(3, 2)          # شكل (3 ,2)
b = torch.tensor([[10.0], [20.0], [30.0]])  # شكل (3 ,1)

print(torch.broadcast_shapes(a.shape, b.shape))  # الناتج: (3 ,2)
c = a + b                                        # البثّ: (3 ,2) <- (3 ,1) + (3 ,2)
print(c)
# الناتج:
# tensor([[10., 11.],
#         [22., 23.],
#         [34., 35.]])

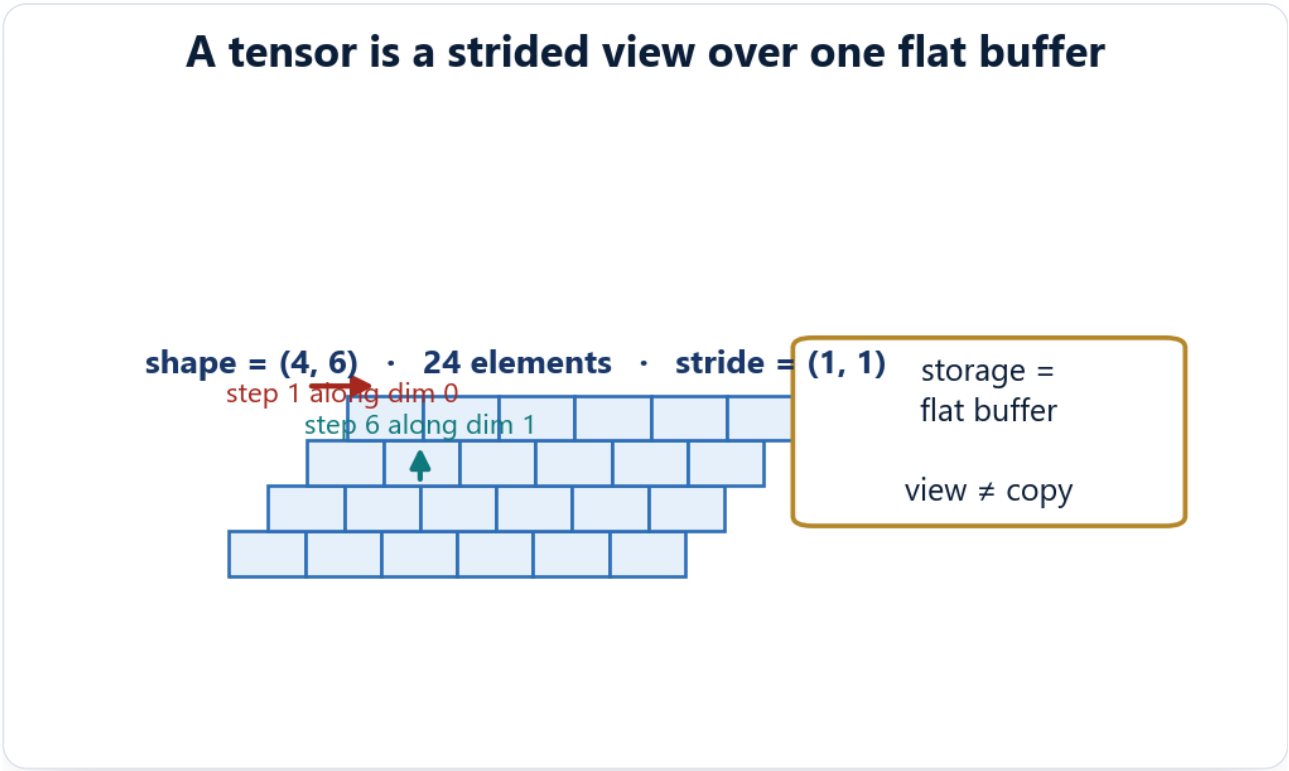
# البثّ في الاتجاه الخطأ يفشل بوضوح
try:
    bad = a + torch.tensor([100.0, 200.0, 300.0])   # المتجه طوله 3 لا 2
except RuntimeError as err:
    print(err)   # الناتج: The size of tensor a (2) must match the size of tensor b (3)...
```

ماذا يحدث هنا؟ `a` شكلها (3, 2) و `b` شكلها (3, 1)، فبعد المحاذاة من اليمين نحصل على (3, 2) و(3, 2)، وتُمدَّد كل قيمة من `b` على امتداد صفها. **لماذا هذه القيمة؟** لأن `b` تحمل انحيازاً واحداً لكل صف (أي لكل عيّنة)، وهو بالضبط ما نريد. **انتبه:** خلع `b` من `a + b` وإعطاء المتجه (3,) بدلاً من (3, 1) يغيّر المعنى: الأول يضيف قيمة لكل عيّنة، والثاني يضيف قيمة لكل *عمود*.

> ✦ **البثّ ليس مجانياً دائماً**
>
> البثّ يوفّر قِصَر الكود لا بالضرورة زمن التنفيذ. حين يحتاج `torch` إلى تمديد شدّة فعلاً، فهو ينسخ بياناتها. في العمليات الصغيرة (مثل إضافة انحياز) الفرق مهمل، لكن في عملية كبرى مثل `torch.broadcast_to` على شدّة بحجم جيجابايت قد تستهلك ذاكرة إضافية ضخمة. القاعدة: **البثّ للعمليات بين محورين، و `repeat` للنسخ المتعمّد**.

الذاكرة: الشدّات (strides)، المنظور والنسخة

هذا هو القسم الذي يفصل المبتدئ عن المحترف. الشدّة في PyTorch ليست كتلة أرقام مسطّحة فحسب، بل هي **نافذة** على مخزن (storage) مشترك، مع **شدّات** (strides) تصف كم خطوة يقفز بها المؤشر لكل درجة في كل محور. هذه الفكرة موروثة عن لغة `fortran` و `C`، وهي سبب وجود مفهوم *المنظور* (view).



الشكل 6.2: المخزن الواحد في الأسفل، وعنه ثلاث نوافذ: واحدة متّصلة (المصفوفة نفسها)، وأخرى منظورة على نفس البيانات بعد التبديل، وثالثة جديدة تماماً بعد النسخ. المقياس الجانبي يبيّن **الشدّات**: كم عدد العناصر التي يتجاوزها المؤشر عند زيادة المحور بمقدار واحد. المفتاح: النافذتان الأولى والثانية تشتركان في الذاكرة نفسها، والثالثة لا تشترك معها.

Python

```
import torch

x = torch.arange(12).reshape(3, 4)

v = x.t()      # منظور: تبديل المحاور (view) على الذاكرة نفسها
c = x.clone()  # نسخة مستقلة تماماً

print(x.data_ptr() == v.data_ptr())      # الناتج: True   -> لا ذاكرة جديدة
print(x.data_ptr() == c.data_ptr())      # الناتج: False  -> ذاكرة جديدة
print(x.stride(), v.stride())            # الناتج: (4, 1) (1, 4)
print(x.is_contiguous(), v.is_contiguous())   # الناتج: True False

# reshape على شدّة متّصلة يُعيد منظوراً، وعلى شدّة غير متّصلة قد ينسخ
r = x.reshape(4, 3)
print(r.data_ptr() == x.data_ptr())      # الناتج: True

# لإجبار النسخ أو تنسيق الذاكرة
```

```
vc = v.contiguous()
print(vc.is_contiguous())              # الناتج: True
```

ماذا يحدث هنا؟ `x.t()` لم ينسخ شيئاً: هو تبديل ترتيب قراءة المخزن نفسه، ولذلك تغيّرت الشدّات من (1, 4) إلى (4, 1). **لماذا هذه القيمة؟** لأن المؤشر في المصفوفة الأصلية يتقدّم عنصراً واحداً عند زيادة الصف، ويتقدّم ثلاثة عند زيادة العمود؛ وبعد التبديل يصبح العكس تماماً. **انتبه:** `reshape` هو الخيار الآمن: يُعيد منظوراً إن أمكن، وينسخ إن لم يستطع. أما `view` فيرفع خطأً إن لم تكن الذاكرة متوافقة. القاعدة العملية: استخدم `reshape` ما لم تكن أداءً حرجاً بدون حاجة.

★ القاعدة الذهبية: المنظور يشارك الذاكرة

حين تعدّل منظوراً، فأنت تعدّل الأصل أيضاً. هذا سلاح ذو حدّين: يوفّر ذاكرة هائلة عند معالجة صور أو بيانات زمنية ضخمة، لكنه مصدر أخطاء صامتة حين تفترض أن المنظور نسخة مستقلة. القاعدة: إذا أردت نسخة، اطلبها صراحةً بـ `.clone()`.

العملية	النوع	هل تنسخ البيانات؟	متى تستخدمها؟
`t()` و `transpose()` و `permute()`	منظور	لا	تبديل محاور رخيص
`reshape()`	منظور أو نسخة	عند الحاجة فقط	الخيار الآمن عموماً
`view()`	منظور فقط	لا	مع شدّات متّصلة معروفة
`squeeze()` و `unsqueeze()`	منظور	لا	تغيير رتبة المحاور
`contiguous()`	نسخة عند الحاجة	نعم غالباً	قبل تمرير شدّة لمكتبة تتطلّب ترتيباً خطياً
`clone()`	نسخة	نعم دائماً	حين تريد استقلالاً حقيقياً

الفهرسة والتقطيع: متى تُنسخ البيانات؟

الفهرسة في `torch` على نوعين، والفرق بينهما له أثر مباشر على الأداء والذاكرة: الفهرسة **الأساسية** (شريحة، رقم واحد، نطاق) تُنتج **منظوراً** تشارك ذاكرته الأصل، بينما الفهرسة **المتقدّمة** (مصفوفة فهارس، قناع منطقي) تُنتج **نسخة** جديدة.

Python

```
import torch

t = torch.arange(12).reshape(3, 4)

print(t[0])        # الناتج: tensor([0, 1, 2, 3])       فهرسة أساسية: منظور
print(t[:, 1])     # الناتج: tensor([1, 5, 9])          شريحة عمود: منظور
print(t[1, 2])     # الناتج: tensor(6)                  عنصر واحد: منظور مربوط

mask = t % 2 == 0                       # قناع منطقي: القيم الزوجية فقط
print(t[mask])       # الناتج: tensor([ 0,  2,  4,  6,  8, 10])   فهرسة متقدّمة: نسخة

idx = torch.tensor([0, 2])              # اختيار صفوف بعينها
print(t[idx])
# الناتج: tensor([[ 0,  1,  2,  3],
#                 [ 8,  9, 10, 11]])

# دمج شرطَين في عملية واحدة
cond = t > 5
print(torch.where(cond, t, torch.zeros_like(t)))
# الناتج: tensor([[ 0,  0,  0,  0],
#                 [ 6,  7,  8,  9],
#                 [10, 11,  0,  0]])
```

ماذا يحدث هنا؟ `t[mask]` أعاد شدّة جديدة تحتوي القيم الزوجية فقط، مرتّبة في موضع جديد. **لماذا هذه القيمة؟** لأن الفهرسة المتقدّمة لا يمكن التعبير عنها كنافذة على المخزن الأصلي: مواقعها متفرّقة. **انتبه:** حين تكتب `t[mask] = 0` فالكتابة *تعدّل الأصل* لأن القناع نفسه منظور؛ أما `t[t > 5] = 0` فتُنتج نسخةً تُعدَّل وتضيع – وهو فخّ شائع جداً.

⚠ فخّ القناع

الفرق بين `t[mask] = 0` و `t[t > 5] = 0` يربك كل مبتدئ: الأولى تعدّل `t` نفسها، والثانية تعدّل نسخة مؤقتة تُمحى فوراً. النتيجة العملية أنّ دقّة النموذج لا تتحسّن، ولا يعطيك النظام أي رسالة.

صيغة العقد: `einsum` ومصفوفات بلا وسيط

أحياناً تكون العملية غامضة في `matmul`: هل هذه مصفوفة × مصفوفة، أم ضرب داخلي، أم دفعة مصفوفات؟ `torch.einsum` تجيب بجملة واحدة: تكتب **عقداً** (contraction) تشرح أيّ محور يُلخّص (يُجمَع) وأيّ محور يبقى في الناتج.

$$C_{bj} = \Sigma_i \, A_{bi} \, B_{ij} \quad ،\quad \text{أي: } C = \text{einsum("bi,ij->bj", A, B)}$$

المعادلة (6.2): **b** محور العيّنة، و**i** محور يُلخّص (يختفي من الناتج)، و**j** محور المخرجات. القاعدة: **المحور الظاهر في الناتج يُكتب على يمين السهم**، وما لم يُكتب على اليمين يُلخّص. **لماذا تهمّنا؟** لأنها تجعل العملية قابلة للقراءة والمراجعة بدل سلسلة `permute` و`reshape` عرضة للخطأ. **خطأ شائع:** نسيان كتابة المحور في الجانب الأيمن فجعلته يختفي بالخطأ.

Python

```
import torch

# ضرب مصفوفتين: (2, 3) × (3, 4) -> (2, 4)
a = torch.randn(2, 3)
b = torch.randn(3, 4)
ab = torch.einsum("bi,ij->bj", a, b)
print(ab.shape)          # الناتج: torch.Size([2, 4])

# ضرب داخلي بين متجهين: ناتج عدد واحد
u = torch.randn(4)
v = torch.randn(4)
print(torch.einsum("i,i->", u, v).shape)   # الناتج: torch.Size([])

# دفعة من المصفوفات: كل عيّنة لها مصفوفة خاصة بها
batch = torch.randn(5, 3, 3)
out = torch.einsum("bij,bkj->bik", batch, batch)
print(out.shape)         # الناتج: torch.Size([5, 3, 3])
```

```
# (مفيد للجداول) جمع على محور مشترك مع الحفاظ على البعد
scores = torch.randn(2, 100)
per_class = torch.einsum("bc,c->b", scores, torch.ones(100))
print(per_class.shape)   # الناتج: torch.Size([2])
```

ماذا يحدث هنا؟ في `"bi,ij->bj"` نلاحظ أنّ المحور `i` يظهر في المدخلين ولا يظهر في الناتج، فالحساب يدور على `i` فقط. **لماذا هذه القيمة؟** لأن هذه هي رموز ضرب المصفوفات الكلاسيكي: الضرب يسري على `i` ويبقى `b` و `j`. **انتبه:** `einsum` أبطأ قليلاً من `@` في الحالات البسيطة، لكنه أوضح بكثير في الحالات المعقّدة، وله نظير في `keras.ops` يعمل على أي نواة.

مقارنة المكتبتين، ومتى تستعمل أيّهما

في Keras 3، المقابلات الأساسية للسدّة تمرّ عبر `keras.ops` : طبقة تعزل المستخدم عن النواة، فتكتب `keras.ops.reshape` و `keras.ops.matmul` وتعملان على `torch` أو `jax` أو `tensorflow` بلا تغيير. الفكرة أن معرّفاً واحداً للمكتبة يجب أن يعمل على كل الأنوية.

المفهوم	PyTorch 2.14	Keras 3.15
إنشاء شدّة	`torch.zeros` و `torch.randn`	`keras.ops.zeros` و `keras.ops.convert_to_tensor`
نوع الشدّة	`tensor.dtype`	`keras.backend.standardize_dtype` أو `ops.dtype`
الشكل	`tensor.shape` يعيد `torch.Size`	`ops.shape` يعيد `tuple` موحّدة
الجهاز	`tensor.device` و `.to(device)`	`keras.config.set_device("cuda")` على مستوى الواجهة
البثّ	تلقائي ومدمج	تلقائي ومدمج عبر `keras.ops`
المنظور والنسخة	`view` و `reshape` و `clone`	لا مفهوم منظور: `keras.ops` تُنتج قيماً جديدة
ضرب متعدّد المحاور	`torch.einsum`	`keras.ops.einsum`

المفهوم	PyTorch 2.14	Keras 3.15
الحساب التلقائي	مدمج في الشدّة عبر `requires_grad`	تديره النواة المختارة تلقائياً

⚙ تطبيق واقعي: خط أنابيب الأشعة في مستشفى

في وحدة الأشعة المقطعية، يصل كل فحص كـ **مكدّس شرائح**: شدّة شكلها (D، H، W) حيث `D` عدد الشرائح. تتم الخطوات: (1) توحيد القيم بين 0 و1 بتقسيم على 255؛ (2) إضافة بعد العيّنة ليصبح (B، 1، D، H، W)؛ (3) تمريرها إلى نموذج ثلاثي الأبعاد. العينة (1) تحتاج `unsqueeze` (منظور بلا نسخ)، والعينة (2) تحتاج `expand` أو `repeat` (نسخ متعمّد). **القرار الهندسي:** طبّق التحويلين على دفعة واحدة بعد التجميع بدل تكرارهما لكل عيّنة، وستوفّر زمن المعالجة على وحدة المعالجة بشكل ملموس في الفحص اليومي.

تمرين 1 — البثّ واستخراج الإحصاءات

1. أنشئ شدّة `scores` شكلها (B, C) قيمها أعداد عشوائية موجبة، حيث B عدد العيّنات وC عدد الأصناف.
2. احسب لكل عيّنة أكبر قيمة، ومتوسط القيم، ومجموع القيم على المحور `C` .
3. طبّع كل صف بحيث يكون مجموعه 1، مستخدماً البثّ فقط دون حلقات.
4. ما الفرق بين `scores.sum(dim=1, keepdim=True)` و `scores.sum(dim=1)` في أثره على البثّ؟

▲ الحل النموذجي

المفتاح في النقطة الرابعة: `keepdim=True` يبقي المحور بحجم 1، فيصبح `(B, 1)` ويبثّ بسلاسة على `(B, C)`؛ و `keepdim=False` يعطي `(B,)` التي لا تنبثّ على `(B, C)` لأن المحاور تُحاذى من اليمين فيصبح الطول C مقابل B.

Python

```
import torch

torch.manual_seed(0)
B, C = 8, 5
scores = torch.rand(B, C)

best = scores.max(dim=1).values          # الناتج: torch.Size([8])
mean = scores.mean(dim=1)                # الناتج: torch.Size([8])
total = scores.sum(dim=1, keepdim=True)  # الناتج: torch.Size([8, 1])

probs = scores / total                   # البثّ: (8, 5) / (8, 1) -> (8, 5)
print(probs.sum(dim=1))                  # الناتج: tensor([1., 1., 1., 1., 1., 1., 1., 1.
```

تمرين 2 — كشف مشاركة الذاكرة

1. أنشئ شدّة `x` من شكل (4, 6) قيمها من 0 إلى 23.
2. خذ `y = x.t()` ثم غيّر `y[0, 0]` إلى 99. هل تغيّرت `x`؟ برهن ذلك بمقارنة العناوين.
3. كرّر العملية مع `z = x.clone().t()` وفسّر الفرق.
4. اكتب دالة `shares_memory(a, b)` تُرجع `True` إن كانت الشدّتان تشتركان في المخزن نفسه.

▲ الحل النموذجي

`t()` لا ينسخ، فالتعديل على `y` ينعكس على `x`. أمّا `clone()` فيُنشئ مخزناً جديداً تماماً، فالتعديل يبقى في `z`. الدالة المطلوبة تقارن `data_ptr()` لكل منهما.

```python
import torch

x = torch.arange(24).reshape(4, 6)

y = x.t()
y[0, 0] = 99
print(x[0, 0])                  # الناتج: tensor(99)  -> تغيّر الأصل
print(shares_memory(x, y))      # الناتج: True

z = x.clone().t()
z[0, 0] = -1
print(x[0, 0])                  # الناتج: tensor(99)  -> الأصل لم يتغيّر
print(shares_memory(x, z))      # الناتج: False
```

ودالة المشاركة بسيطة:

```python
def shares_memory(a, b):
    """تتحقّق ممّا إذا كانت الشدّتان تشتركان في المخزن نفسه."""
    return a.untyped_storage().data_ptr() == b.untyped_storage().data_ptr()
```

في الفصل العاشر ستكتب حلقة تدريب تنقل فيها دفعة واحدة على البطاقة في كل خطوة. لو كانت الدفعة `batch` من نوع `float32` على المعالج، وكنت تنقلها إلى `cuda` في كل خطوة عبر `batch.to(device)`، فلماذا لا يكون هذا مكلفاً بقدر ما يبدو؟ وأين ستتحوّل الفكرة إلى عنق زجاجة حقيقي؟

ملخّص الفصل

- الشدّة تجمع أربع معلومات: الشكل والنوع والجهاز وسلسلة التدرّج؛ وهذا ما يميّزها عن مصفوفة `numpy` .
- ثلاث خصائص لا تُهمل: `shape` و `dtype` و `device` ، وأي خطأ فيها يُنتج خطأ شكل لا خطأ قيم.
- `torch.manual_seed` شرط التكرارية: بدونه لا معنى لإعادة التجربة.
- البثّ يُحاذي المحاور من اليمين، ويقبل المحور الذي طوله 1 فقط؛ و `keepdim=True` هو مفتاح البثّ السلس.
- المنظور يشارك الذاكرة مع الأصل، والنسخة لا تشاركه؛ و `clone()` هو الطلب الصريح للاستقلال.
- الفهرسة الأساسية تُنتج منظوراً، والفهرسة المتقدّمة (القناع والفهارس) تُنتج نسخة؛ و `torch.where` تحلّ محلّ التغريض الشرطي.
- `einsum` تشرح العقدة بالضبط: المحاور على يمين السهم تبقى، وما عداها يُلخّص.
- في Keras 3 تقابل `keras.ops` واجهة `torch` ، لكن بلا مفهوم المنظور: المعنى لمقابل واحد إلى واحد.

الفصل السابع غوص عميق في PyTorch

تشريح nn.Module

كيف تُبنى النماذج: المعاملات، التهيئة، الحفظ والتحميل، والأدوات المساعدة

تخيّل ورشة تصنيع تُركّب فيها آلة من قطع: محرك، تروس، حسّاسات. كل قطعة وحدها لا تفعل شيئاً، لكن تجميعها في هيكل واحد يجعلها آلة لها مدخل ومخرج ومسؤولية مشتركة عن حالتها الداخلية: كم درجة حرارتها الآن؟ ما عمر القطع المستهلكة؟ هذا بالضبط ما يفعله صنف `nn.Module` في PyTorch: فهو لا يجمع الدوال فحسب، بل يجعل الكائن **مسؤولاً عن معاملاته**. فتسجيل معامل جديد لا يحتاج سطراً إضافياً، والحفظ والتحميل يسيران معاً، ونقل النموذج إلى بطاقة الرسوميات ينقل كل ما بداخله دفعة واحدة. في هذا الفصل سنفكّك هذه الآلية: كيف تُسجَّل المعاملات، لماذا يحتاج `BatchNorm` إلى مخزن خاص به، كيف يُحفظ نموذج في ملف واحد ويُستعاد بأمان، وما الأدوات التي وفّرتها المكتبة لتخفف عنك.

الصنف الذي يعرف نفسه

لو كتبت نموذجك بدالة واحدة `def forward(x): ...` لكل شبكة، لاستطعت تدريبه. لكن عندها:

- لن تستطيع نقله إلى البطاقة الرسومية إلا بنسخ كل وزن يدوياً.
- لن تستطيع حفظه إلا بحفظ كل مصفوفة في ملف يدوي.
- لن تستطيع استخراج قائمة المعاملات لفحصها أو لتطبيق أوزان.
- لن تستطيع عمق الفرع داخل الشبكة (مثلاً: طبّق التطبيع على الطبقة الثالثة فقط).

صنف `nn.Module` يحلّ هذه المشاكل الأربع بمبدأ واحد: **التسجيل التلقائي**. بمجرّد أن تعلن داخل دالة `__init__` أنّ لديك مصفوفة أو طبقة، يتذكّرها الصنف في سجلّين داخليين، ويتكفّل بكل ما يترتّب على ذلك.

```
Python
```

```python
import torch
from torch import nn

class MLP(nn.Module):
    """شبكة بسيطة من ثلاث طبقات كثيفة."""

    def __init__(self, in_features: int, hidden: int, out_features: int):
        super().__init__()                    # تهيئة صنف الأب: إلزامية دائماً
        self.fc1 = nn.Linear(in_features, hidden)
        self.act = nn.ReLU()                  # طبقة بلا معاملات
        self.fc2 = nn.Linear(hidden, out_features)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.fc2(self.act(self.fc1(x)))

model = MLP(in_features=32, hidden=64, out_features=10)
print(model)
# الناتج:
# MLP (
#   (fc1): Linear(in_features=32, out_features=64, bias=True)
#   (act): ReLU()
#   (fc2): Linear(in_features=64, out_features=10, bias=True)
# )

x = torch.randn(8, 32)
y = model(x)
print(y.shape)        # الناتج: torch.Size([8, 10])
```

ماذا يحدث هنا؟ عند إنشاء `fc1` بـ `nn.Linear`، يسجّل `nn.Module` تلقائياً مصفوفة الوزن والانحياز داخل الكائن الأم. **لماذا هذه القيمة؟** لأن `nn.Linear` نفسه صنف مشتق من `nn.Module`، فتدخل مصفوفاته إلى سجلّ الوالد. **انتبه:** نداء `super().__init__()` ليس شكلياً: بدونه لا يُهيَّأ سجلّ `nn.Module` الداخلي، ويرفع `AttributeError`.

> طبقة `nn.ReLU` لا تحتوي أيّ معامل. فلماذا يظلّ ظهورها في `print(model)` مفيداً؟ وماذا يخسر النموذج لو أهملنا تسجيلها؟

ظهور `ReLU()` في البنية ليس ترفاً: هو **خريطة للنموذج**. حين ترى `print(model)` فأنت تقرأ بنية الشبكة كما يقرأها المحلّل، ومنه تعرف أين الطبقات وأين التجميع وأين نحتاج تعديلاً. وتخزين الطبقات

كخصائص (attributes) على الكائن يجعل *الفهرسة بالاسم* ممكنة: `model.fc1.weight` يفتح وزن الطبقة الأولى مباشرة.

المعاملات: `nn.Parameter` والتسجيل

المصفوفة داخل `nn.Linear` ليست مصفوفة عادية: هي `nn.Parameter`، أي شدّة `torch.Tensor` لها سمة خاصة إضافية: `requires_grad=True` بشكل افتراضي. وهذه السمة هي التي تجعل محرّك التمايز التلقائي يراقبها ويحسب تدرّجها في الفصل الثامن.

$$\theta \leftarrow \theta - \eta \, \nabla_{\theta} L$$

المعادلة (7.1): **θ** متجّه المعاملات كلّها، و**L** دالّة الخسارة، و**η** معدّل التعلّم. **ما معنى كل رمز؟** $\nabla_{\theta} L$ متجّه التدرّج: كم يتغيّر الخسارة إذا غيّرنا كل وزن بمقدار ضئيل. **لماذا تهمّنا؟** لأن `nn.Parameter` هو ما يجعل هذه المعادلة قابلة للتنفيذ آلياً: يحدّدPyTorch أيّ شدّات تُ.watched وأيّها لا. **خطأ شائع:** أن تعلن مصفوفة كـ `torch.Tensor` عادية وتظنّ أنها ستتعلّم.

Python

```
import torch
from torch import nn

class SmallNet(nn.Module):
    def __init__(self):
        super().__init__()
        # الخطأ: شدّة عادية — لن تتعلّم، ولن تظهر في state_dict
        self.bad = torch.randn(4, 4)
        # الصحيح: Parameter — تتعلّم، وتُحفظ، وتُنقل مع النموذج
        self.good = nn.Parameter(torch.randn(4, 4))

        # شرط: إنشاء المعاملات داخل __init__ لا داخل forward
        self.register_buffer("scale", torch.ones(4))

    def forward(self, x):
        return x @ self.good * self.scale

m = SmallNet()
print([name for name, _ in m.named_parameters()])   # الناتج: ['good']
print([name for name, _ in m.named_buffers()])      # الناتج: ['scale']
print(m.bad.requires_grad, m.good.requires_grad)    # الناتج: False True
```

```
# تغيير حالة التدرّج
m.bad.requires_grad_(True)
print([n for n, _ in m.named_parameters()])        # الناتج: []  (ليست parameter أصلاً)
print(m.bad.grad)                                  # الناتج: None
```

ماذا يحدث هنا؟ `named_parameters()` يقطع الشجرة ويجمع ما سُجّل كمعامل. **لماذا هذه القيمة؟** لأن `nn.Parameter` يسجّل نفسه في قائمة خاصة تختلف عن قائمة سمات الكائن العادية. **انتبه:** حتى لو أجبرتَ `requires_grad` على الشدّة العادية، فإنها لن تظهر في `parameters()` ولن يحفظها `state_dict` – لأن التسجيل هو ما يفصل المعامل عن مجرد رقم.

⚠ أخطاء تُكلّف ساعات من التدريب

(1) إنشاء معامل داخل `forward` بدل `__init__`: يُعاد إنشاؤه في كل نداء فيفقد قيمته المدرَّبة. (2) نسخ المعاملات بـ `.data` قبل `backward()`: تقطع الصلة بمحرّك التمايز فتفقد التدرّجات. (3) وضع معامل داخل قائمة عادية `self.layers = [nn.Linear(...)]`: لن يُسجَّل تلقائياً، ويجب استخدام `nn.ModuleList` أو `nn.Sequential` بدلاً من قائمة بايثون عادية.

المخازن (Buffers): الحالة التي ليست معاملاً

ليست كل ما يحتاجه النموذج معاملاً. خُذ `BatchNorm`: يحتفظ بمتوسّط متحرّك (المتوسّط المتحرّك) وانحراف معياري متحرّك، وهما **أرقام إحصائية** تتحدّث أثناء التدريب ولا تتغيّر عبر التدرّج. هذه الحالة تُسمّى **مخزناً**.

Python

```
import torch
from torch import nn

class Scaled(nn.Module):
    def __init__(self, n: int):
        super().__init__()
        self.register_buffer("running_mean", torch.zeros(n))
        self.register_buffer("running_var", torch.ones(n))
        self.register_buffer("counter", torch.tensor(0, dtype=torch.long))

    def forward(self, x):
        self.counter += 1                      # المخزن يُعدَّل أثناء التمرير
        return (x - self.running_mean) / torch.sqrt(self.running_var)
```

```
bn = nn.BatchNorm2d(4)
print("running_mean في state_dict:", "bn.running_mean" in bn.state_dict())
print("weight (معامل) موجود:", "bn.weight" in bn.state_dict())

s = Scaled(3)
print(s.state_dict().keys())
# الناتج: dict_keys(['running_mean', 'running_var', 'counter'])
```

ماذا يحدث هنا؟ `register_buffer` يضع الشدّة في سجلّ خاص، فتصبح جزءاً من حالة النموذج: تُنقل معه إلى الجهاز، وتُحفظ في `state_dict`، وتُستعاد عند التحميل. **لماذا هذه القيمة؟** لأن `BatchNorm` يحتاجها في الاستدلال (inference) عندما لا توجد دفعة لتقدير الإحصاءات. **انتبه:** لو خزّنتَ `running_mean` كصفة بايثون عادية `self.running_mean = torch.zeros(n)` لفقدته عند الحفظ، ولأعاد النموذج تهيئته من الصفر عند التحميل – وهو خطأ صامت يغيّر نتائج الإنتاج دون أي رسالة.

النوع	يُسجَّل تلقائياً؟	يُحفظ في `state_dict`؟	يتأثّر بالتدرّج؟	مثال
`nn.Parameter`	نعم	نعم	نعم	أوزان الطبقات
مخزن مسجَّل	لا (يدوياً)	نعم	لا	إحصاءات `BatchNorm`
مخزن غير دائم	لا (يدوياً)	لا	لا	عدّاد خطوات
شدّة عادية	لا	لا	حسب `requires_grad`	جدول مرجعي ثابت

دالة `forward` ولماذا نُنادي النموذج لا الدالة

في كل الأمثلة السابقة نُنادي `model(x)` لا `model.forward(x)`. الفرق ليس شكلياً: نداء النموذج يمرّ عبر `__call__` التي تنفّذ **سلسلة من خطّافات (hooks)** قبل التمرير وبعده: خطّافات أمامية، خطّافات `backward`، ومراقبة. هذه الخطّافات هي التي تجعل أدوات مثل `torch.profiler` وتقطيع النماذج وإحصاء التدرّجات ممكنة دون تعديل سطر واحد في كودك.

```
Python
```

```python
import torch
from torch import nn

class Tiny(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(4, 2)

    def forward(self, x):
        return self.fc(x)

m = Tiny()

calls = []
h = m.register_forward_hook(lambda mod, inp, out: calls.append(out.shape))
out = m(torch.randn(3, 4))
h.remove()                      # إزالة الخطّاف: عادةً desirable بعد القياس
print(out.shape, calls)         # الناتج: torch.Size([3, 2]) [torch.Size([3, 2])]

# تمرير الخطّاف إلى children: كل طبقة داخلية تُراقب
inner = []
h2 = m.fc.register_forward_hook(lambda mod, inp, out: inner.append(out.shape))
m(torch.randn(5, 4))
h2.remove()
print(inner)                    # الناتج: [torch.Size([5, 2])]
```

ماذا يحدث هنا؟ الخطّافالأمامي يُنفَّذ تلقائياً مع كل نداء للنموذج، ويسجّل شكل الخرج. **لماذا هذه القيمة؟** لأن `nn.Module.__call__` يفحص قائمة الخطّافات قبل وبعد استدعاء `forward` . **انتبه:** نسيان `h.remove()` يترك الخطّاف معلّقاً في الذاكرة، وفي حلقة تدريب طويلة يستهلك ذاكرة مع اتساف ويدلّ خطأً.

◆ ملاحظة دقيقة

`register_forward_hook` يلتقط الخرج فقط. وفي الفصل الثامن سنرى `register_full_backward_hook` الذي يلتقط التدرّجات أيضاً. القاعدة في `torch` اليوم: استخدم الخطّافات الخلفية لتفقد ما يفعله المحرّك، لا لتغيّر

التهيئة: من أين تأتي قيم الأوزان؟

كل طبقة في `torch` تبدأ من تهيئة افتراضية. للطبقات الخطية (وهي `nn.Linear` والـ `Embedding`) التهيئة الافتراضية هي **Xavier أو Kaiming حسب الدالة الفعّالة**: `nn.Linear` تستخدم توزيعاً موحّداً متناسباً مع جذر عدد المدخلات، أما `nn.Conv2d` فتستخدم `kaiming_uniform_` لأنها تعمل على ReLU. وهذه ليست تفصيلاً ثانوياً: قيمة البداية تحدّد شكل منظر الخسارة في بداية التدريب، وقد قد يتدهور التدريب عند قيمة عشوائية سيئة بضعف أضعاف.

Python

```
import torch
from torch import nn
from torch.nn import init

torch.manual_seed(0)

# --- التهيئة الافتراضية ---
lin = nn.Linear(64, 64)
print(lin.weight.std().item())      # الناتج: 0.125 تقريباً (1/sqrt(64))

# --- تهيئة يدوية موحّدة ---
init.xavier_uniform_(lin.weight)
init.zeros_(lin.bias)               # الانحياز يبدأ من الصفر
print(lin.weight.std().item())      # الناتج: 0.125 تقريباً أيضاً

# --- تهيئة مناسبة لـ ReLU في الطبقات التلافيفية ---
conv = nn.Conv2d(3, 16, kernel_size=3)
init.kaiming_normal_(conv.weight, nonlinearity="relu")
init.constant_(conv.bias, 0.0)
print(conv.weight.std().item())

# --- تهيئة خطية: مهمة جداً للشبكات العميقة ---
deep = nn.Linear(128, 128)
init.xavier_normal_(deep.weight, gain=1.0)
init.zeros_(deep.bias)

# إعادة التهيئة إلى حالة الطبقات الأصلية
lin.reset_parameters()
print(lin.bias.abs().sum().item())   # الناتج: 0.0
```

ماذا يحدث هنا؟ `xavier_uniform_` يضبط الانحراف المعياري للوزن ليصبح متناسباً مع `1/sqrt(fan_in)`. **لماذا هذه القيمة؟** لأن طبقة واحدة تعيد تباين الإشارة؛ فإن ضاعفنا التباين في كل طبقة، انتفخ التباين أُسّياً حتى ينفجر التدريب، وإن صغّرناه اختفت الإشارة. **انتبه:** `init.zeros_(layer.bias)` ليس خطأ، بل خيار مقصود: الانحياز يبدأ من صفر فينمو مع التدرّج.

> ✓ **نصيحة عملية: طبقة الإخراج فقط هي التي تستحق انحيازاً خاصاً**
>
> ابدأ كل الطبقات بانحياز صفري، ووزن Xavier أو Kaiming حسب الدالة الفعّالة. في طبقة الإخراج، استعمل انحيازاً مقدّراً من إحصاءات البيانات (مثلاً متوسط التسمية في الانحدار اللوغاريتمي) إن أردت، وإلا اتركه صفراً.

الحفظ والتحميل: `state_dict` وما الذي يبقى سرّاً

`state_dict` هو **قاموس** (بترتيب مُحافظ عليه) يربط كل اسم معامل بقيمته الحالية. إنه جسر التبديل بين «نموذج» و«ملف». الفكرة الأساسية: **احفظ الأوزان لا النموذج**، ثم أعد بناء البنية في الكود وأحمّل الأوزان فوقها.

Python

```
import torch
from torch import nn

class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(10, 2)

    def forward(self, x):
        return self.fc(x)

torch.manual_seed(0)
model = Net()

# 1) احفظ حالة النموذج
torch.save(model.state_dict(), "net_state.pt")

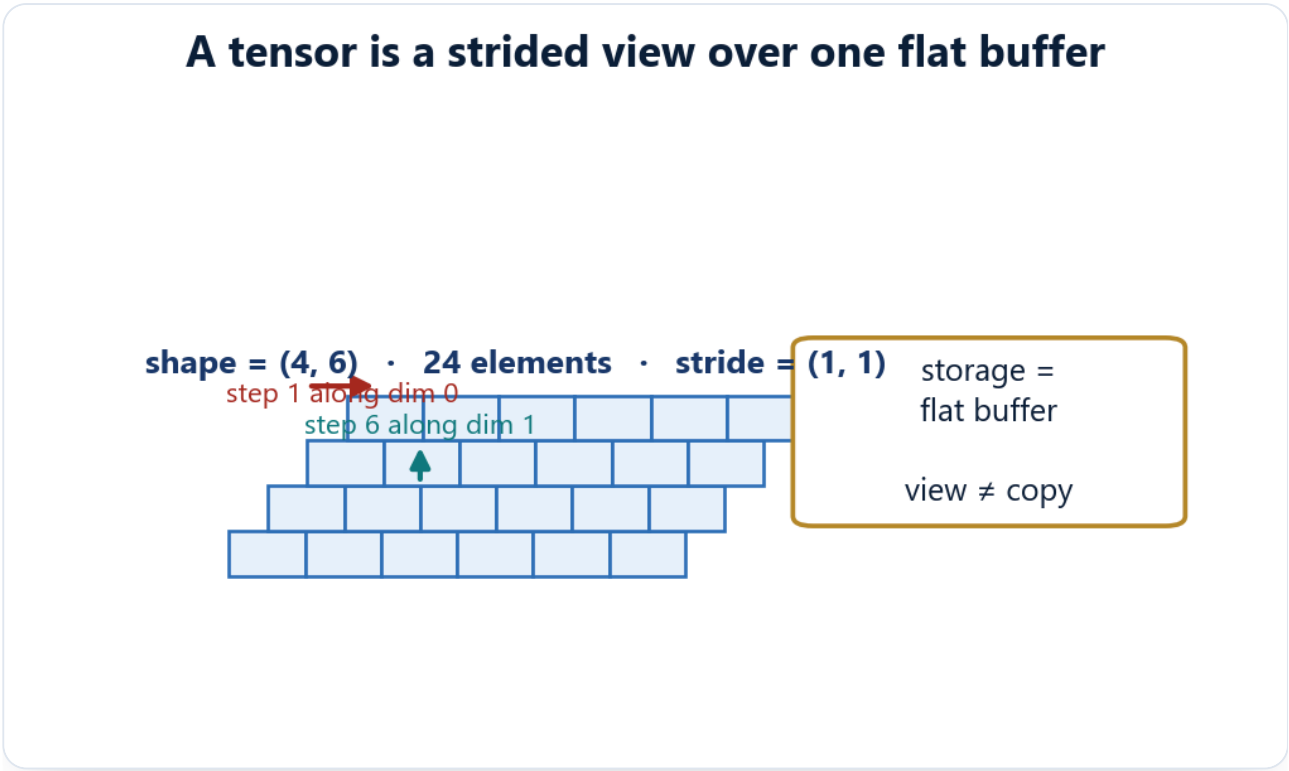
# 2) استعدها بأمان: weights_only=True يمنع تنفيذ كود أثناء التحميل
state = torch.load("net_state.pt", weights_only=True)
print(list(state.keys()))           # الناتج: ['fc.weight', 'fc.bias']
print(state["fc.weight"].shape)     # الناتج: torch.Size([2, 10])
```

```
# 3) عِد بناء النموذج ثم حمّل
model2 = Net()
missing, unexpected = model2.load_state_dict(state)
print(missing, unexpected)          # الناتج: [] []

# 4) تحميل مرن: عند اختلاف البنية
result = model2.load_state_dict(state, strict=False)
print(result.missing_keys, result.unexpected_keys)

# 5) نسخة قابلة للتعديل دون المساس بالنموذج
safe_state = {k: v.detach().clone() for k, v in model.state_dict().items()}
safe_state["fc.bias"] = torch.zeros(2)
```

ماذا يحدث هنا؟ `state_dict()` يعيد مراجع إلى الشدّات نفسها لا نسخاً منها. **لماذا هذه القيمة؟** ولهذا السبب النسيان الشائع: إن كتبتَ `sd = model.state_dict()` ثم `sd["fc.bias"].add_(1)` فأنت قد أفسدتَ نموذجك دون أن تلاحظ، لأن `sd["fc.bias"]` هو *نفس* الشدّة. الحل هو `detach().clone()` كما في السطر الأخير. **انتبه:** استخدم دائماً `weights_only=True` عند التحميل من ملف لم تصنعه أنت: هذه هي الحماية المعيارية ضد تنفيذ كود عشوائي.



الشكل 7.1: قراءة جديدة لما ناقشناه في الفصل السابق: المعاملات داخل `nn.Module` ليست أرقاماً مجرّدة بل **شدّات تقيم في مخزن مشترك**. القاموس الذي يعيده `state_dict()` يشير إلى تلك المخازن نفسها؛ ولذلك فإن تعديل قيمة فيه يغيّر النموذج، ولهذا نطلب `detach().clone()` عند الحاجة إلى نسخة مستقلة. وأمّا نداء `.to(device)` فينقل المعاملات فتُخصَّص ذاكرة جديدة على الجهاز الجديد، بينما تبقى النوافذ المشتقّة على الذاكرة القديمة ما لم تُسوَّ.

المقارنة	`torch.save(model, path)`	`torch.save(model.state_dict(), path)`
ما يُحفظ	البنية والأوزان معاً	الأوزان فقط
حجم الملف	أكبر غالباً	أصغر
مرونة إعادة البناء	مقيّدة ببنية الملف	تامة: تُعدّل البنية في الكود
أمان التحميل	يتطلّب `weights_only=False`	يعمل مع `weights_only=True`
الأنسب لـ	نشر سريع أو تصحيح أخطاء	بحث وتعلّم ونقل بين مشاريع

أدوات `nn.Module` التي ستستخدمها كل يوم

إلى جانب `state_dict`، يوفّر الصنف مجموعة أدوات تجعل العمل اليومي ممكناً: نقل النموذج، وضعه في وضع التدريب أو التقييم، تطبيق دالة على كل الطبقات، تصفّح الشجرة، وتجميد جزء منها.

Python

```
import torch
from torch import nn
from torch.nn.utils import parameters_to_vector

model = nn.Sequential(
    nn.Linear(20, 16),
    nn.ReLU(),
    nn.BatchNorm1d(16),
    nn.Linear(16, 3),
)

# --- النقل إلى الجهاز ---
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
print(next(model.parameters()).device)     # الناتج: cuda (أو cpu)

# --- وضع التدريب ووضع التقييم ---
model.train()
print(model[2].training)                   # الناتج: True
model.eval()
print(model[2].training)                   # الناتج: False
model.train()                              # ارجع لوضع التدريب قبل المتابعة

# --- تطبيق دالة على كل الطبقات ---
def init_all(module):
    if isinstance(module, nn.Linear):
        init.xavier_uniform_(module.weight)
        init.zeros_(module.bias)

model.apply(init_all)

# --- تصفّح الشجرة ---
print([name for name, _ in model.named_children()])   # الناتج: ['0', '1', '2', '3']
print(len(list(model.modules())))                     # الناتج: 9 (النموذج + 4 طبقات + 4 توابع)

# --- تجميد جزء من النموذج (للتدرّب الدقيق) ---
for param in model[0].parameters():
```

```
    param.requires_grad_(False)
trainable = [n for n, p in model.named_parameters() if p.requires_grad]
print(trainable)
# الناتج: ['2.weight', '2.bias', '3.weight', '3.bias']

# --- متجه واحد لكل المعاملات: مفيد للقياس والمقارنة ---
flat = parameters_to_vector(model.parameters())
print(flat.shape)          # الناتج: torch.Size([387])
```

ماذا يحدث هنا؟ `model.train()` و `model.eval()` يسريان **تكرارياً** على كل الطبقات الفرعية، وهو ما يجعل `BatchNorm` و `Dropout` يتصرّفان بشكل صحيح. **لماذا هذه القيمة؟** لأن سلوك `BatchNorm` يختلف بين التدريب والاستدلال: في التدريب يحسب إحصاءات الدفعة ويتحرّك المتوسّط، وفي الاستدلال يقرأ المخزن فقط. **انتبه:** نسيان `model.eval()` قبل التقييم يعطيك نتائج سيّئة؛ ونسيان `model.train()` بعده يجعل التدريب بلا فعالية.

> ★ **قاعدة الوضعين**
>
> `model.eval()` قبل أي قياس، و `model.train()` قبل أي تدريب. ضع السطرين في مكانين لا تنساهما: في بداية دالة التقييم، وفي بداية حلقة التدريب. ولاحظ أنّ `model.eval()` لا يوقف حساب التدرّج: لمنع ذلك تحتاج `torch.no_grad()` (الفصل الثامن).

Keras في المقابل: نفس المفاهيم بأسماء أخرى

في Keras 3، النموذج **هو** صنف `layers.Layer` الذي يرث بدوره صنفاً يملك نظام خصائص وأوزان مشابهة. الفرق الجوهري ليس في المفهوم بل في **مَن يملك** `weights_`:

المفهوم	PyTorch 2.14	Keras 3.15
صنف النموذج	`nn.Module` مع `forward`	`keras.Model` أو `keras.Sequential`
المعاملات	`nn.Parameter` عبر `named_parameters`	`model.trainable_variables`
المخازن	`register_buffer`	أوزان غير قابلة للتدرّب مثل `BatchNormalization`

المفهوم	PyTorch 2.14	Keras 3.15
التهيئة	`nn.init.xavier_uniform_` و `kaiming_normal_`	`layer.kernel_initializer` و `bias_initializer`
التمرير الأمامي	`model(x)`	`model(x, training=False)`
وضع التدريب	`model.train()` و `model.eval()`	الوسيط `training` في الاستدعاء
الحفظ	`torch.save(state_dict)`	`keras.saving.save_model` أو `save_weights`
الاستعادة	`load_state_dict`	`keras.models.load_model` أو `set_weights`
التجميد	`requires_grad_(False)`	`layer.trainable = False`
تجميد الأوزان	`torch.no_grad()`	`model.predict` تلقائياً بلا تدرّج

Python

```python
import keras

# Keras 3 فوق نواة torch: نفس المفاهيم بصيغة مختلفة
model = keras.Sequential([
    keras.layers.Input(shape=(20,)),
    keras.layers.Dense(16, activation="relu",
                       kernel_initializer=keras.initializers.HeNormal(),
                       bias_initializer="zeros"),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(3),
])

model.build((None, 20))
model.summary()                    # يعرض عدد المعاملات Trainable وNon-trainable

# الأوزان القابلة للتدرّب مقابل غير القابلة
print(len(model.trainable_variables))        # الناتج: 4
```

```
print(len(model.non_trainable_variables))    # الناتج: 2 (إحصاءات التطبيع)

# الحفظ والاستعادة
model.save_weights("keras_net.weights.h5")
same = keras.Sequential([
    keras.layers.Input(shape=(20,)),
    keras.layers.Dense(16, activation="relu",
                       kernel_initializer=keras.initializers.HeNormal(),
                       bias_initializer="zeros"),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(3),
])
same.build((None, 20))
same.load_weights("keras_net.weights.h5")
print("تمت الاستعادة")
```

ماذا يحدث هنا؟ في Keras تُسمّى المعاملات **متغيّرات** (variables)، والمخازن تظهر ضمن `non_trainable_variables`. **لماذا هذه القيمة؟** لأن Keras يفصل بين ما يُدرَّب وما يُستعمل للقياس فقط، تماماً كما يفعل `register_buffer` مع خيار `persistent=False`. **انتبه:** استدعاء `model(x)` في Keras [**training=True** افتراضياً]، بينما في PyTorch لا يوجد `training` افتراضيً بل تحتاج `model.eval()` صريحاً – وهذا فرق عملي يُنسى بسهولة عند الانتقال بين المكتبتين.

⚙ تطبيق واقعي: نموذج كشف احتيال في بوابة دفع

شركة توزيع تبيع عبر تطبيق، ويريد فريق المخاطر كشف عمليات الاحتيال. الخطوة: بناء شبكة `SmallFraudNet` بـ `nn.Module` تحتوي طبقة `BatchNorm1d` (لذالها مخازن) وطبقة `Dropout` (بلا معاملات). المهندسون يدرّبون النموذج أسبوعاً، ثم يرسلون **ملف الأوزان فقط** لفريق الاستقبال، الذي يبني البنية نفسها في كوده ويحمّل الأوزان بـ `load_state_dict` – دون الحاجة إلى تثبيت إصدار المكتبة نفسه على كل خادم، ودون خطر تنفيذ كود من ملف غير موثوق. **القرار:** استخدم `state_dict` في كل تبادل بين الفرق، واحتفظ بنسخة كاملة من النموذج عبر `torch.save(model, path)` للنسخ الاحتياطية فقط.

تمرين 1 — صنف به معامل ومخزن

1. اكتب صنف `ScaleShift` يطبّق `y = (x - mean) / std * gamma + beta` حيث `gamma` و `beta` معاملان قابلان للتدرّب، و `mean` و `std` مخزنان.
2. اكتب `forward` يقبل شدّة شكلها (B, F).
3. تأكّد من ظهور العناصر الأربعة في `state_dict()` .
4. غيّر `std` إلى مخزن غير دائم (غير محفوظ) وأعد الفحص.

▲ الحل النموذجي

القاعدة: `nn.Parameter` لما يُدرَّب، و `register_buffer` لما يُنقل ويُستعمل في الاستدلال، و `persistent=False` لما لا يلزم حفظه.

Python

```python
import torch
from torch import nn

class ScaleShift(nn.Module):
    def __init__(self, features: int):
        super().__init__()
        self.gamma = nn.Parameter(torch.ones(features))
        self.beta = nn.Parameter(torch.zeros(features))
        self.register_buffer("mean", torch.zeros(features))
        self.register_buffer("std", torch.ones(features))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return (x - self.mean) / self.std * self.gamma + self.beta

m = ScaleShift(4)
print(list(m.state_dict().keys()))
# الناتج: ['gamma', 'beta', 'mean', 'std']

# تحويل المخزن إلى غير دائم
m.register_buffer("std", torch.ones(4), persistent=False)
print(list(m.state_dict().keys()))
# الناتج: ['gamma', 'beta', 'mean']

print(m(torch.randn(2, 4)).shape)   # الناتج: torch.Size([2, 4])
```

تمرين 2 — حفظ آمن واستعادة مرنة

1. أنشئ نموذجاً بطبقتين كثيفتين، احفظ `state_dict` في ملف.
2. حمّله بـ `weights_only=True` واطبع أسماء المفاتيح.
3. عدّل بنية النموذج (احذف الطبقة الثانية) ثم حمّل بـ `strict=False` واطبع المفاتيح الناقصة والزائدة.
4. بيّن لماذا تتحمّل هذه الشدّة `state_dict` مرجعاً إلى الشدّات الأصلية لا نسخةً منها.

▲ الحل النموذجي

النقطة الرابعة جوهرية: `state_dict()` يعيد قاموساً يشير إلى شدّات النموذج نفسها. لتغيير قيمة في القاموس دون المساس بالنموذج، استعمل `detach().clone()`.

Python

```
import torch
from torch import nn

class Two(torch.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(6, 6)
        self.fc2 = nn.Linear(6, 2)

    def forward(self, x):
        return self.fc2(self.fc1(x))

model = Two()
torch.save(model.state_dict(), "two.pt")

state = torch.load("two.pt", weights_only=True)
print(list(state.keys()))     # الناتج: ['fc1.weight', 'fc1.bias', 'fc2.weight', 'fc2.bias

class One(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(6, 6)

    def forward(self, x):
        return self.fc1(x)

other = One()
report = other.load_state_dict(state, strict=False)
print(report.missing_keys)     # الناتج: []
print(report.unexpected_keys)  # الناتج: ['fc2.weight', 'fc2.bias']

# إثبات المرجعية
sd = model.state_dict()
print(sd["fc1.weight"] is model.fc1.weight)          # الناتج: True
safe = {k: v.detach().clone() for k, v in sd.items()}
```

```
safe["fc1.weight"].add_(1.0)
print(model.fc1.weight[0, 0].item())                # الناتج: 0.2500 (لم يتغيّر)
```

ألصقتَ في نموذجك طبقة تطبيع دفعة، فبقيت الخسارة ثابتة تماماً أثناء أول ثلاث دورات تدريب. هل تظنّ أنّ `BatchNorm` معطوبة؟ ما اسرع فحص الذي يثبت أو ينفي الفرضية خلال دقيقتين؟

الفصل التاسع سيبني خط أنابيب بيانات كاملاً. لماذا يُهمّنا هنا أن يحفظ النموذج `state_dict` فقط لا البنية الكاملة؟ وما الذي يتعطّل عملياً في خط النشر حين تُخطئ في هذا الاختيار؟

ملخّص الفصل

- `nn.Module` صنف يسجّل تلقائياً معاملاته ومخازنه، فيتولّى النقل والحفظ والتجميد والتظلّل.
- `nn.Parameter` للمعاملات القابلة للتدرّب، و `register_buffer` للحالة الإحصائية التي لا تتعلّم.
- المخزن غير الدائم (`persistent=False`) لا يُحفظ في `state_dict` .
- استدعِ `model(x)` لا `model.forward(x)` ، فالآخرة تتجاوز خطّافات `torch` .
- التهيئة ليست تفصيلاً: Xavier للخطية، وKaiming لل/ReLU، وانحياز صفري في البداية.
- `state_dict()` يعيد مراجع لا نسخاً: انسخ صراحةً إن أردت التعديل.
- استعمل `weights_only=True` عند التحميل من ملف خارجي، و `strict=False` عند اختلاف البنية.
- `train()` و `eval()` يسريان تكرارياً؛ و `eval()` لا يوقف حساب التدرّج.
- في Keras المقابل هو `trainable_variables` و `non_trainable_variables` ، والتمرير يحتاج وسيط `training` صريحاً.

الفصل الثامن غوص عميق في PyTorch

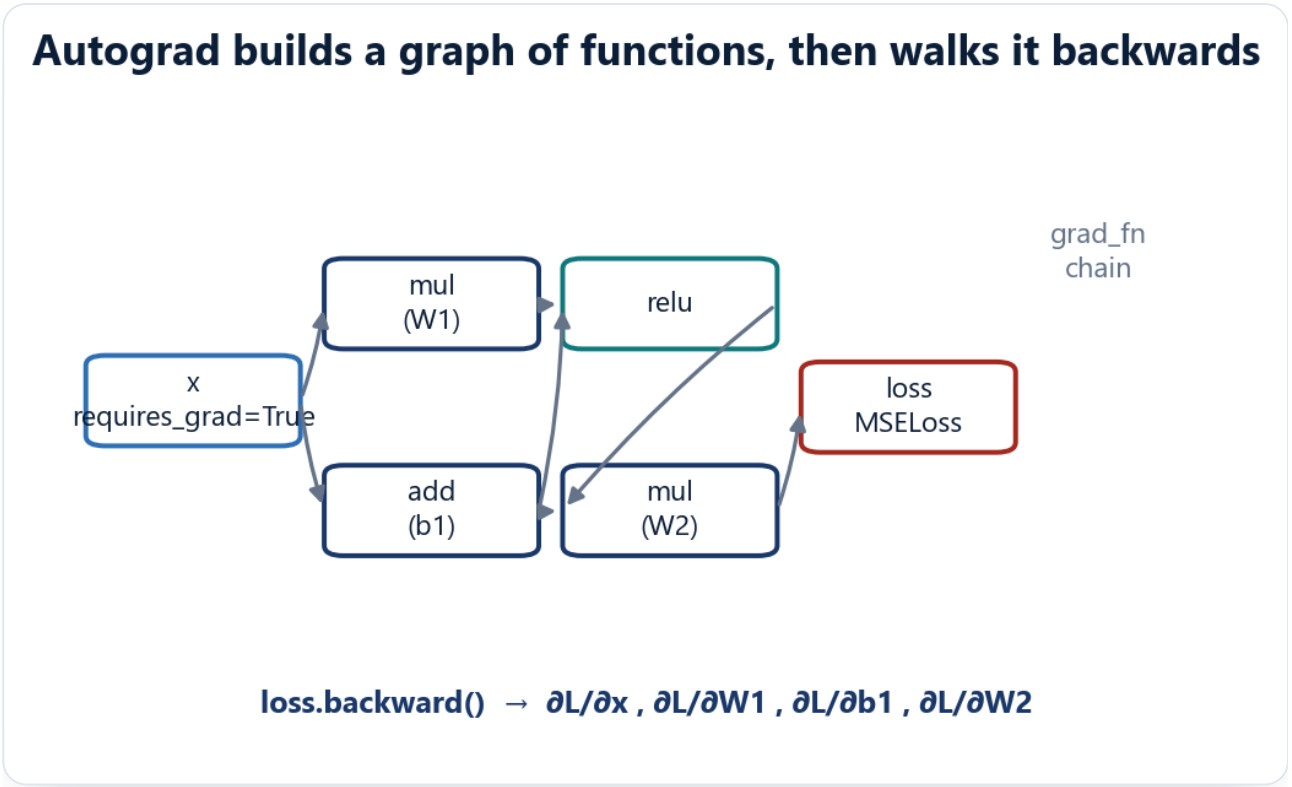
Autograd: محرّك التمايز التلقائي

الرسم البياني للحساب، inference_mode، no_grad، والخطوات الشائعة في الأخطاء

في الفصل الرابع اشتققنا قانون السلسلة بأيدينا: كل طبقة تحسب مشتقّتها وتسلّمها للطبقة التي قبلها. كان ذلك ممتعاً لنموذج من طبقة واحدة، ون كارثياً لنموذج من مئة طبقة. في هذا الفصل نتعرّف على **محرّك التمايز التلقائي** (autograd): النظام الذي يبني **رسماً بيانياً للحساب** أثناء التمرير الأمامي، ثم يسير عليه **بعكس** الاتجاه ليجمع كل التدرّجات في نداء واحد. ستتعلّم كيف تُشعل الرصد في شدّة واحدة، وماذا يحدث حين تنسى إطفاءه، وكيف تفحص ما يفعله المحرّك بدل أن تصدّقه. وأهم ما في الفصل: **الفرق الدقيق بين** `no_grad` **و** `inference_mode` **و** `detach` – ثلاثة أشياء تبدو متشابهة وتُفسد أبحاثاً كاملة.

من الاشتقاق اليدوي إلى الرسم البياني للحساب

تخيّل شبكة من عشرين طبقة. الاشتقاق اليدوي يعني كتابة عشرين معادلة، و ترتيب غير صحيح في أحدها يعني تدرّجات صفرية دون أي رسالة خطأ. الحل الذي اتّفق عليه الجميع هو **الرسم البياني للحساب** (computation graph): كل عملية تُسجَّل كعقدة، وكل مشتقّة جزئية تُخزَّن على الحافّة. عندئذٍ يكفي نداء واحد ليُمرّر المحرّك على العقد من الخرج إلى المدخل.



الشكل 8.1: المسار الأمامي (الأسهم الصاعدة) يبني الرسم: كل عملية تُنتج عقدة تحفظ دالتها المشتقّة ومعاملاتها. المسار العكسي (الأسهم الهابطة) يبدأ من الخسارة ويجمع التدرّجات محوراً بمحور. لاحظ أنّ العقدة الوسطى `MultiplyBackward0` تحتفظ بمدخلَين (المتجهين)، لأن مشتقّة الضرب تحتاجهما معاً — وهذا بالضبط سبب تخزين `ctx` في كل عملية.

$$\partial L/\partial w = \partial L/\partial a \cdot \partial a/\partial w$$

المعادلة (8.1): **L** الخسارة، و**w** وزن الطبقة، و**a** ناتج الطبقة. **ما معنى كل رمز؟** السهم يمثّل «المشتقّة»: كم تتغيّر الخسارة إذا غيّرنا تنشيط الطبقة، وكم يتغيّر تنشيطها إذا غيّرنا الوزن. **لماذا تهمّنا؟** لأن هذا التركيب – وهو قانون السلسلة – هو ما يحسبه المحرّك آلياً لكل عقدة في الرسم. **خطأ شائع:** الخلط بين «المشتقّة» (رقم واحد لكل وزن) و«التدرّج» (مصفوفة números لكل وزن من الأوزان): في الشبكة العصبية، التدرّج دائماً مصفوفة.

```python
import torch

# الاشتقاق اليدوي: f(x) = x^2  =>  f'(x) = 2x
x = torch.tensor([2.0, 3.0], requires_grad=True)
y = (x ** 2).sum()
y.backward()
print(x.grad)       # الناتج: tensor([4., 6.])   (2 × [2, 3])
```

```
# المحرّك يعرف أنّ: لا يوجد سطر واحد يكتب الاشتقاق SquareDerivative = 2x

# من المتغيّر إلى كل شيء آخر في الرسم
a = torch.tensor(2.0, requires_grad=True)
b = a ** 3
c = b * 4
print(b.grad_fn)     # الناتج:
print(c.grad_fn)     # الناتج:
print(c.requires_grad)   # الناتج: True  (خاصية الرصد تنتقل تلقائياً)
```

ماذا يحدث هنا؟ `requires_grad=True` هو ما أشعل الرصد. **لماذا هذه القيمة؟** لأن `y.backward()` يحتاج أن يعرف من أين يبدأ، فيبني الرسم من `y` عكسياً حتى يجد المتغيّرات التي توسّعت الرصد. **انتبه:** `grad_fn` هو *الدالة* التي ستُستدعى عند الانتشار العكسي، لا التدرّج نفسه. التدرّج يخزَّن في `.grad` .

صمّمتَ دالة خسارة مخصّصة، و `loss.backward()` رفعت خطأ `element 0 of tensors does not require grad` . ما الاحتمالات الثلاثة التي تفسّر هذا الخطأ، وكيف تميّز بينها في دقيقة؟

الرصد: `requires_grad` وقواعد انتقاله

الرصد في PyTorch يخضع لقواعد صارمة يمكن تلخيصها في ثلاث جمل:

1. **نقطة البداية:** الشدّة تُرصد فقط إذا صُنعت برصد أو نتجت عن عملية على شدّة مرصودة.
2. **الانتقال:** أيّ عملية على شدّة مرصودة تنتج شدّة مرصودة.
3. **القطع:** العمليات التي لا تقبل التدرّج (مثل `.item()` أو التحويل إلى `numpy`) تقطع الصلة تماماً.

القاعدة الأولى تُهمَل أحياناً: إن أنشأتَ شدّة داخل دالة بـ `torch.tensor(...)` ثم في داخلها عملت عليها بـ `requires_grad` ، فسيُبنى الرسم على الشدّة **المؤقّتة** التي ستُمحى فور خروج الدالة، فلا تصل التدرّجات إلى معاملاتك. الحل: أنشئ الشدّة **قبل** الدالة أو مرّرها إليها كوسيط.

Python

```python
import torch

w = torch.tensor(2.0, requires_grad=True)

def square_internally(v):
    inner = v * v          # مرصودة لأن v مرصودة
    return inner

out = square_internally(w)
out.backward()
print(w.grad)        # الناتج: tensor(4.)   الرسم وصل إلى w

# الحالة الخطرة: شدّة مؤقتة تُبنى داخل الدالة
def bad_scale(v):
    tmp = v.detach() * 3   # detach يقطع الرصد
    return tmp

out2 = bad_scale(w)
print(out2.requires_grad)   # الناتج: False
try:
    out2.backward()
except RuntimeError as err:
    print("RuntimeError:", err)
    # الناتج: RuntimeError: element 0 of tensors does not require grad ...

# operations التي تقطع الرصد
z = w * 2
print(z.item())          # item() تعيد Python float وتقطع الرصد
print(z.numpy())        # الناتج: array([4.], dtype=float32)  ( cutting the graph )
```

ماذا يحدث هنا؟ `detach()` ينشئ نسخة من الشدّة **من دون** سمة `requires_grad` ومن دون `grad_fn`. **لماذا هذه القيمة؟** لأن `detach()` هو النقيض التشغيلي لـ `requires_grad`: يخبرك المحرّك «تجاهل هذه الشدّة ولا تبنِ لها عقدة». **انتبه:** `.numpy()` و `.item()` يفصلان الشدّة عن الرسم لأنهما ينقلان القيمة إلى بايثون خالصة.

⚠ الخطأ القاتل: `.data`

كثيرون يكتبون `loss = loss.data` تظنّ أنها «تخلّصنا من الرسم». هذا خطأ جسيم: `.data` تُعيد الشدّة نفسها بتخفيض تدرّجها إلى صفر، فلا يُبنى رسم جديد – أي أنّ التدرّجات لن تصل إلى أيّ معامل، والتدريب «يحدث» دون أن يتعلّم شيء. البديل الصحيح هو `detach()` أو `torch.no_grad()`، ويفضّل `torch.no_grad()` لأن نيّته أوضح.

الانتشار العكسي: `backward()` و `autograd.grad`

`backward()` تمشي عكس الرسم وتكتب التدرّجات في خاصية `.grad` لكل شدّة مرصودة. لكنّها **تتراكم**: لا تمسح ما هو مكتوب. وهذه الخاصية مفيدة (تدرّج تراكمي في التعلّم المعزّز) وخطيرة (نضطرّ تدرّجات قديمة فتبوهم النموذج).

```python
import torch

x = torch.tensor([1.0, 2.0], requires_grad=True)

loss = (x ** 2).sum()
loss.backward()
print(x.grad)          # الناتج: tensor([2., 4.])

loss2 = (x ** 2).sum()  # الخسارة نفسها مرة أخرى
loss2.backward()
print(x.grad)          # الناتج: tensor([4., 8.])  (تراكمت!)

# --- طريقة بديلة: لا تتراكم ولا تكتب في .grad ---
g = torch.autograd.grad(loss, x, retain_graph=True)
print(g)               # الناتج: (tensor([2., 4.]),)  تُعيد مجموعة
print(x.grad)          # الناتج: tensor([2., 4.])  لم تتغيّر

# --- تحتاج retain_graph وcreate_graph: المشتقّة الثانية ---
g1 = torch.autograd.grad(loss, x, create_graph=True, retain_graph=True)
g2 = torch.autograd.grad(g1[0].sum(), x)
print(g2)              # الناتج: (tensor([2., 2.]),)  المشتقّة الثانية لـ x^2

# --- تدرّج مع شدّة لم تُراقب: .grad = None ---
p = torch.tensor(3.0)                # بدون requires_grad
print(p.grad)          # الناتج: None
```

ماذا يحدث هنا؟ `torch.autograd.grad` يحسب التدرّج دون لمس `.grad`. **لماذا هذه القيمة؟** لأن استدعاءه مرتين على نفس الرسم يتطلب `retain_graph=True` في الاستدعاء الأول: الرسم يُحرَّر بعد `backward` افتراضياً لتوفير الذاكرة. **انتبه:** في حلقة التدريب الحقيقية، نداء `backward()` مرّة لكل دفعة، والتراكم مقصود. أما التراكم غير المقصود فيُحلّ بـ `optimizer.zero_grad(set_to_none=True)` في بداية كل خطوة، وهو ما سنفعله في الفصل العاشر.

العملية	تكتب في `.grad`؟	تحافظ على الرسم؟	متى تستعملها
`loss.backward()`	نعم (تراكمي)	لا افتراضياً	التدريب
`torch.autograd.grad(L, x)`	لا	لا افتراضياً	قياس التدرّجات
`torch.autograd.grad(L, x, retain_graph=True)`	لا	نعم	تدرّجات متعدّدة أو مشتقّة ثانية
`torch.autograd.grad(L, x, create_graph=True)`	لا	نعم	التدرّج الثاني (تدريب مقيّد)
`optimizer.zero_grad(set_to_none=True)`	يمسح	لا علاقة	بداية كل خطوة تدريب

منع الرصد: `no_grad` و `inference_mode` و `detach`

هذه ثلاث أدوات متشابهة، وكل واحدة منها تحلّ مشكلة مختلفة. الخلط بينها يُنتج أخطاءً يصعب تشخيصها، لأنّ الفرق في **نوع الشدّة الناتجة** لا في الشكل الظاهر.

الأداة	النطاق	ما الذي يمنعه	نوع الناتج
`torch.no_grad()`	كتلة `with`	بناء الرسم داخل الكتلة	شدّة عادية
`torch.inference_mode()`	كتلة `with`	بناء الرسم + تتبّع نسخة الوسم	شدّة استدلال

الأداة	النطاق	ما الذي يمنعه	نوع الناتج
`tensor.detach()`	شدّة واحدة	رصد تلك الشدّة فقط	شدّة عادية بلا `grad_fn`

Python

```python
import torch
from torch import nn

model = nn.Linear(4, 2)
x = torch.randn(3, 4, requires_grad=True)

with torch.no_grad():
    a = model(x)
print(a.requires_grad)     # الناتج: False

with torch.inference_mode():
    b = model(x)
print(b.requires_grad)     # الناتج: False

# الفرق الجوهري: ناتج inference_mode لا يمكن أن يُستعمل لاحقاً في رسم تدرّج
try:
    c = (b * 2).sum()
    c.backward()
except RuntimeError as err:
    print("RuntimeError:", err)
    # الناتج: RuntimeError: element 0 of tensors does not require grad ...

# detach: نسخة بلا رصد، قابلة للاستعمال في حساب لاحق
d = x.detach()
print(d.requires_grad)     # الناتج: False
e = (d * 3).sum()
print(e.requires_grad)     # الناتج: False

# المعاكس: تقطع الرصد في مكانه SIGNES_بم_.detach#
y = x * 2
print(y.requires_grad)     # الناتج: True
y.detach_()
print(y.requires_grad)     # الناتج: False
```

ماذا يحدث هنا؟ `inference_mode` أخفّ من `no_grad` لأنّه يتخلّص من **تتبّع نسخة الوسم** (version counter) في كل عملية، وهذا يوفّر وقتاً حقيقياً في الاستدلال واسعة النطاق. **لماذا هذه القيمة؟** لأن `no_grad` يبدّل السلوك فقط داخل الكتلة، أمّا `inference_mode` فيعلّم كل شدّة تُنشأ داخله بعلامة خاصة تمنع استخدامها في الرسم لاحقاً. **انتبه:** لذلك لا تستخدم `inference_mode` حول شيء ستعيد استخدامه في تدريب لاحق.

✦ لماذا يهمّنا الفرق في الاستدلال؟

في خدمة إنتاجية، الفرق بين `no_grad` و `inference_mode` قد يكون 10% إلى 20% من زمن الاستجابة. وفي الاستدلال الجماعي على آلاف الصور، هذه النسبة تتحوّل إلى ميزانية زمنية حقيقية. القاعدة: استعمل `inference_mode` في مسار الاستدلال الخالص، و `no_grad` حين تحتاج مخرجات في حسابات لاحقة.

خطّافات المحرّك: كيف تفحص ما يحدث فعلاً

حين لا تعرف لماذا تدرّج معيّن صفر أو انفجر، لا تخمّن: **راقب**. في PyTorch مستويان من الخطّافات: خطّاف على الشدّة نفسها (يرى التدرّج الواصل)، وخطّاف على الوحدة (يرى التدرّجات الداخلة والخارجة).

Python

```
import torch
from torch import nn

# 1) خطّاف على شدّة: يعدّل التدرّج أو يراقبه
x = torch.tensor(2.0, requires_grad=True)
y = x ** 2

def scale_grad(grad):
    print("grad seen by hook:", grad.item())
    return grad * 10        # إرجاع قيمة جديدة يعدّل التدرّج

h = y.register_hook(scale_grad)
y.backward()
print(x.grad)               # الناتج: tensor(40.)  4 لأن الخطّاف ضاعف
h.remove()

# 2) خطّاف خلفي كامل على وحدة: يرى تدرّج الخرج وتدرّج المدخل
class Doubler(nn.Module):
    def forward(self, t):
```

```
        return t * 2

d = Doubler()
records = []
hb = d.register_full_backward_hook(
    lambda module, grad_input, grad_output: records.append((grad_input[0], grad_output[0]))
)
t = torch.tensor(3.0, requires_grad=True)
d(t).backward()
hb.remove()
print(records[0][1].item())   # الناتج: 2.0  (تدرّج الخرج)
print(records[0][0].item())   # الناتج: 2.0  (تدرّج المدخل: d(2t)/dt = 2)
```

ماذا يحدث هنا؟ الخطّاف الخلفي يستقبل ثلاثة وسائط: الوحدة، قائمة تدرّجات المدخلات، وقائمة تدرّجات الخرج. **لماذا هذه القيمة؟** لأن المحرّك يعرف تدرّج كل مشتقّة جزئية، فيمرّرها إلى الخطّاف عند كل عقدة. **انتبه:** الخطّافات الخلفيّة على الوحدات التي تحتوي معاملات تتطلّب أن تكون المعاملات `requires_grad=True`، وإلا رُفع خطأ.

الأخطاء الشائعة: تشخيص سريع

معظم أخطاء `autograd` تنحصر في أربعة أنماط. جدول التشخيص التالي يختصرها.

الخطأ	الرسالة	السبب الجذري	الحل
لا تدرّج	`does not require grad`	الشدّة خارج الرسم أو `detach`	تأكّد من `requires_grad` ومن عدم قطع الرسم
تدرّج مكرّر	`backward through the graph a second time`	نداء `backward()` مرّتين على نفس الرسم	استعمل `retain_graph=True` عند الحاجة
تدرّجات قديمة	النموذج «لا يتعلّم»	نسيان `zero_grad`	`optimizer.zero_grad(set_to_none=True)` كل خطوة

الخطأ	الرسالة	السبب الجذري	الحل
تعديل في المكان	`a leaf Variable that requires grad is being used in an in-place operation`	تعديل مباشر على شدّة مرصودة	استعمل `x = x + 1` بدل `x += 1`
تسرّب ذاكرة	استهلاك الذاكرة متزايد	إبقاء الرسم حيّاً	استعمل `no_grad` أو `inference_mode` في التقييم

> ★ **خطة التشخيص في خمس دقائق**
>
> (1) تحقّق من `requires_grad` على المدخلات. (2) تحقّق من `grad_fn` على الخرج: إن كان `None` فالرسم مقطوع. (3) اطبع `model.parameters()` وتحقّق أنّ `.grad` ليس `None` بعد `backward()` . (4) جرّب `torch.autograd.grad` على دفعة واحدة للتأكّد من صحّة الرسم. (5) إن كان كلّ ما سبق سليماً، فراجع الحساب العددي لا المحرّك.

Keras: حين يكون المحرّك نفسه تحت الغطاء

في Keras 3 فوق نواة `torch` ، لا تحتاج أن تكتب `backward()` بنفسك: `fit` يتولّى ذلك. لكنّ ما يجري في الخلفية هو **حرفياً** `loss.backward()` ثم `optimizer.apply` داخل `torch.no_grad()` . وهذا يعني أنّ فهم `autograd` في PyTorch يخدمك مرّتين: في PyTorch مباشرة، وفي Keras حين تحتاج إلى تجميع التدرّجات يدوياً `train_step` مخصّص.

المفهوم	PyTorch 2.14	Keras 3.15
تفعيل الرصد	`requires_grad=True`	تلقائي داخل `fit`
الانتشار العكسي	`loss.backward()`	داخلياً في `fit` على نواة torch
التدرّجات	`param.grad`	`variable.value.grad` على نواة torch
تحديث الأوزان	`optimizer.step()`	`optimizer.apply` داخل `no_grad`
منع الرصد	`torch.no_grad()`	تلقائي في `predict` و `evaluate`

المفهوم	PyTorch 2.14	Keras 3.15
قطع الرصد لشدّة	`tensor.detach()`	`keras.ops.convert_to_tensor` من قيمة عددية
مراقبة التدرّجات	`register_hook`	لا مقابل مباشر: استخدم `train_step` مخصّص
مقياس التدرّج	`torch.autograd.grad`	الحساب داخل `train_step` حسب النواة

Python

```python
import keras

# train_step مخصّص في Keras 3: يتصرّف بشكل مختلف حسب النواة
class WeightedLossModel(keras.Model):
    def __init__(self, units=16):
        super().__init__()
        self.d1 = keras.layers.Dense(units, activation="relu")
        self.d2 = keras.layers.Dense(1)

    def call(self, x, training=False):
        h = self.d1(x, training=training)
        return self.d2(h)

    def train_step(self, data):
        x, y = data
        with self.compute_loss(x, y, sample_weight=None) as loss:
            y_pred = self(x, training=True)
            loss = keras.ops.mean(keras.ops.square(y_pred - y), axis=-1)
        # بنفسه؛ الوسيط أعلاه يوفّر الخسارة step() ينفّذ المحسّن torch على نواة
        return self.compute_metrics(x, y, y_pred, sample_weight=None)

model = WeightedLossModel()
model.compile(optimizer=keras.optimizers.Adam(learning_rate=1e-3))

import numpy as np
rng = np.random.default_rng(0)
x = rng.normal(size=(256, 8)).astype("float32")
y = (x[:, 0] * 2.0).astype("float32")

model.fit(x, y, batch_size=32, epochs=2, verbose=0)
print("training finished")
```

ماذا يحدث هنا؟ الوسيط `self.compute_loss(...)` يوفّر الخسارة بصيغة محايدة على مستوى النواة، ثم تتكفّل Keras بالانتشار العكسي والتحديث. **لماذا هذه القيمة؟** لأن Keras 3 صُمّمت لتعمل على ثلاث نوات، فلم تكتب `loss.backward()` في الكود العام. **انتبه:** على نواة `tensorflow` تحتاج إلى `tf.GradientTape` داخل `train_step`؛ وعلى نواة `torch` تتولّى Keras ذلك.

⚙ تطبيق واقعي: ضبط نموذج توصية في متجر إلكتروني

فريق التوصية يريد منع **تضخيم النتيجة** (overfitting) على المنتجات الجديدة التي لا تملك بيانات كافية. الحل المستخدَم: **تسوية التدرّجات** عبر قياس حجم التدرّج على دفعة التحقّق، وهي عملية تحتاج الرسم البياني للتدرّج. الخطوات: (1) شغّل التمرير الأمامي على دفعة تحقّق مع `torch.no_grad()` لتقييم النموذج. (2) على دفعة تدريب صغيرة، استعمل `torch.autograd.grad` بدل `backward()` للحصول على التدرّجات دون تعديل `.grad` (لا تلمس أوزان النموذج!). (3) قارن `torch.nn.utils.clip_grad_norm_` على هذه التدرّجات. **القرار:** اعتمد `no_grad` لمسار التقييم، و `autograd.grad` في مسار القياس، ولا تخلط بينهما.

✎ تمرين 1 — التحقّق من الاشتقاق آلياً

1. اكتب دالة `f(x) = 3x^2 + 2x - 5` وشغّل عليها `backward()`.
2. أثبت بالنتائج أنّ التدرّج يساوي `6x + 2`.
3. احسب المشتقّة الثانية `6` باستخدام `create_graph=True`.
4. أعد الحساب بـ `torch.autograd.grad` وتحقّق من عدم تغيّر `.grad`.

▲ الحل النموذجي

الخطوة الثالثة تحتاج `retain_graph=True` أيضاً، لأن المشتقّة الأولى يجب أن تبقى في الرسم كي نشتقّ منها ثانية.

```python
import torch

def f(x):
    return 3 * x ** 2 + 2 * x - 5

x = torch.tensor(1.0, requires_grad=True)
y = f(x)
y.backward()
print(x.grad)          # الناتج: tensor(8.)   لأن 8 = 2 + 1*6

# المشتقّة الثانية
x2 = torch.tensor(1.0, requires_grad=True)
y2 = f(x2)
g1 = torch.autograd.grad(y2, x2, create_graph=True, retain_graph=True)
g2 = torch.autograd.grad(g1[0].sum(), x2)
print(g2)              # الناتج: (tensor(6.),)

# بلا تراكم
x3 = torch.tensor(1.0, requires_grad=True)
y3 = f(x3)
g = torch.autograd.grad(y3, x3)
print(g, x3.grad)      # الناتج: (tensor(8.),) None
```

تمرين 2 — بناء عدّاد التدرّجات وقراءة الخطّاف

1. أنشئ دالة `square_and_scale` تعمل على متجه.
2. سجّل خطّافاً يطبع شكل التدرّج وقيمته عند كل نداء.
3. نفّذ `backward()` وتحقّق من مطابقة شكل التدرّج لشكل الخرج.
4. أضف خطّافاً يعدّل التدرّج بضربه في 0.5، وتحقّق من انصاف التدرّج النهائي.
5. تأكّد من إزالة الخطّافات في النهاية.

▲ الحل النموذجي

الخطّاف على الشدّة يتلقّى التدرّج فقط، لا الشدّة نفسها؛ لهذا نطبع `grad.shape` وليس `grad.numel()`. قيمة التدرّج عند `x = [2.0, 3.0]` مع `y = x^2` هي `[4, 6]`.

Python

```
import torch

def square_and_scale(v, k=2.0):
    return (v ** 2) * k

x = torch.tensor([2.0, 3.0], requires_grad=True)
y = square_and_scale(x)

def watch(grad):
    print("hook saw:", grad.shape, grad.tolist())
    return grad

h = y.register_hook(watch)
y.backward()
print("x.grad =", x.grad.tolist())   # الناتج: [36.0 ,16.0]
h.remove()

# خطّاف يعدّل التدرّج
x2 = torch.tensor([2.0, 3.0], requires_grad=True)
y2 = square_and_scale(x2)
h2 = y2.register_hook(lambda g: g * 0.5)
y2.backward()
print("x.grad =", x2.grad.tolist())  # الناتج: [18.0 ,8.0]
h2.remove()
```

تدريبك لا يتحسّن إطلاقاً: الخسارة ثابتة تماماً. كتبتَ `loss = loss.data` «لتفادي الرسم». اشرح بالضبط أين قطعَ هذا السطرُ الرسمَ، ولماذا لم ينتج خطأً يوقف البرنامج، وما البديل الذي يصلح المشكلة ويصلح النية معاً؟

في مسار الاستدلال في خدمتك، تستهلك 30% من زمن الاستجابة وقتاً غير مفيد. ما الفرق بين `torch.no_grad()` و `torch.inference_mode()` من ناحية الأداء، ومتى يكون الثاني خياراً خطيراً؟

ملخّص الفصل

- محرّك التمايز التلقائي يبني رسماً بيانياً للحساب أثناء التمرير الأمامي، ويمشي عليه عكسياً في `backward()`.
- `requires_grad=True` يشعل الرصد، وينتقل تلقائياً إلى كل ناتج مشتقّ منه.
- `grad_fn` هو دالّة الانتشار العكسي، أمّا التدرّج نفسه فيُخزَّن في `.grad`.
- `backward()` يتراكم، و `torch.autograd.grad` لا يكتب في `.grad` — وهذا ما تحتاجه للقياس.
- `detach()` يقطع الرصد لشدّة واحدة، و `no_grad()` يمنع بناء الرسم داخل كتلة، و `inference_mode()` يمنع الرصد والنسخ من نسخة الوسم.
- لا تستخدم `.data` أبداً: تقطع التدرّج دون أن تقطع الرسم.
- الخطّافات هي عدسة الفحص: `register_hook` على الشدّة، و `register_full_backward_hook` على الوحدة.
- أخطاء المحرّك الأربعة: انقطاع الرسم، استدعاء مزدوج، تدرّجات قديمة، وتعديل في المكان.
- في Keras 3 على نواة torch، `fit` ينفّذ `backward()` نيابةً عنك داخل `no_grad`.

هندسة البيانات: من المصدر إلى الدفعة

Dataset وDataLoader والعاملات والتحويلات والتحميل غير المتزامن

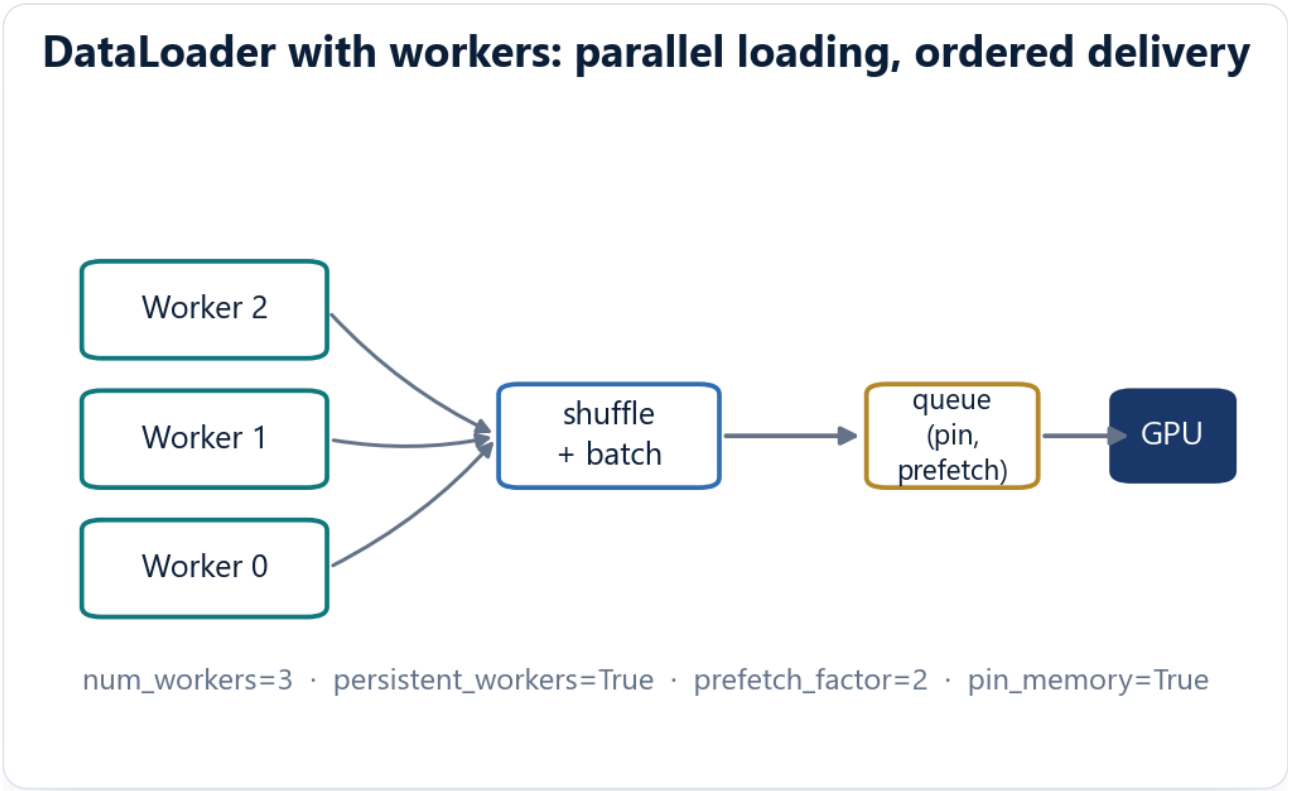
النموذج لا يتعلّم من البيانات التي لا تصل إليه. في المشاريع الحقيقية، 70% من الوقت يذهب إلى **خطّ البيانات** لا إلى بنية الشبكة. هذا الفصل عن هندسة البيانات في PyTorch: من تعريف `Dataset` إلى التحميل المتوازي، ومعالجة اختلال التوازن – وهي المهارة التي تفصل مهندساً يعرف عن مهندس يعرف.

العقد: ما الذي يقرّره DataLoader؟

`DataLoader` في PyTorch يتّخذ خمسة قرارات عن كل مرّة تمرّ فيها بالبيانات على النموذج. فهمها يعني عقّل مصمّم البيانات:

القرار	الوسيلة	الأثر على التدريب
من أين تأتي العيّنة؟	`dataset[i]`	اقرأ الملف من القرص
بأيّ ترتيب؟	`sampler` / `shuffle`	يؤثّر على تعميم النموذج
كم عيّنة في الدفعة؟	`batch_size`	سرعة + جودة التدرّج
ماذا يُحذف؟	`drop_last`	حجم الدفعة ثابت أم لا
كيف تُنجَز؟	`num_workers` ، `prefetch_factor`	هل يسبق المعالج الـGPU؟

قرارات DataLoader الخمسة



الشكل 9.1: العمّال يقرأون البيانات بالتوازي، ويضعونها في طابور، ويستهلكها (`prefetch`) التدريب بالتسلسل. الفكرة: **لا المعالج ينتظر الـGPU، ولا للـGPU ينتظر المعالج.**

Dataset: أين تبدأ الشيفرة؟

```python
import torch
from torch.utils.data import Dataset

class WineDataset(Dataset):
    """مجموعة بيانات من ملف CSV: الأعمدة features والأخيرة label."""

    def __init__(self, csv_path, transform=None):
        import pandas as pd
        df = pd.read_csv(csv_path)
        self.x = torch.tensor(df.drop(columns="label").values, dtype=torch.float32)
        self.y = torch.tensor(df["label"].values, dtype=torch.long)
        self.transform = transform

    def __len__(self):
        return len(self.y)

    def __getitem__(self, index):
```

```
        x, y = self.x[index], self.y[index]
        if self.transform:
            x, y = self.transform(x, y)
        return x, y

# تجزئة عشوائية بسيطة ثم تطبيع على التدريب فقط
from sklearn.model_selection import train_test_split
import pandas as pd

df = pd.read_csv("wine.csv")
train_df, test_df = train_test_split(df, test_size=0.2, random_state=42, stratify=df["label"]
train_df.to_csv("train.csv", index=False)
test_df.to_csv("test.csv", index=False)

train_ds = WineDataset("train.csv")
test_ds = WineDataset("test.csv")
print(len(train_ds), train_ds[0][0].shape, train_ds[0][1])
```

✓ قاعدة: اجعل الـDataset يُعيد tensors جاهزة

التحويلات الثقيلة (فكّ ترميز صور، tokenization) نفّذها **داخل** الـ `__getitem__` لا في مقدّمة الكود: هكذا تعمل مع عمّال DataLoader المتعدّدين وتوزّع الحمل على المعالجات. إن نفّذتها مسبقاً، ستضطر لتخزين كل شيء في الذاكرة.

التحويلات: منظّم في الكود لا في الـDataset

Python

```
from torchvision import transforms

# مجموعة تحويلات للتدريب (تتضمّن randomness مقصوداً)
train_tf = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomResizedCrop(224, scale=(0.7, 1.0)),
    transforms.RandomHorizontalFlip(),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),                    # يحوّل PIL → tensor [0,1] في
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225]),
])
```

```
# مجموعة التقييم: حتمية تماماً
eval_tf = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225]),
])

# مهمّ: normalizefur إن كان في مجموعة التدريب، فليست في التقييم
```

⚠ أخطاء شائعة في التحويلات

- **تحويل عشوائي في التقييم**: يقيس أداءً غير حتمي ويُفسد المقارنة بين النماذج.
- **اختلاف المعالجة بين التدريب والخدمة**: إن طبّعت بـmean مختلفاً في الإنتاج، ستُرسل التنبّؤات إلى التوزيع الخاطئ بصمت.
- **ترتيب `ToTensor` `Normalize`**: يجب أن يأتي قبل (التي تتوقّع tensor لا PIL).

DataLoader: الإعدادات التي تصنع الفرق

Python

```
from torch.utils.data import DataLoader

train_loader = DataLoader(
    train_ds,
    batch_size=64,
    shuffle=True,               # ترتيب عشوائي في التدريب
    num_workers=4,              # عمّال متعدّدون (0 = المعالج الرئيسي)
    pin_memory=True,            # ذاكرة قابلة للتثبيت ⇒ نقل أسرع إلى GPU
    persistent_workers=True,    # العمّال يبقون حيّين بين الحقبات (يوفّر بدءاً)
    prefetch_factor=2,          # عدد الدفعات التي يجهّزها كل عامل مسبقاً
    drop_last=True,             # حجم ثابت للدفعة ⇒ BatchNorm مستقر
)

val_loader = DataLoader(
    val_ds,
    batch_size=256,
    shuffle=False,              # لا خلط في التقييم!
    num_workers=2,
```

```
    pin_memory=True,
)

for xb, yb in train_loader:
    xb = xb.cuda(non_blocking=True)   # النقل غير متزامن (يعمل مع pin_memory)
    yb = yb.cuda(non_blocking=True)
    break
```

الإعداد	القيمة	الشرح
`num_workers`	0 للبيانات الصغيرة، 4-8 للصور الثقيلة	كل عامل عملية منفصلة ⇒ يتطلّب `picklable`
`pin_memory`	True عند النقل إلى GPU	صفحات ذاكرة مسمّاة للنسخ السريع DMA
`persistent_workers`	True للحقبات العديدة	يوفّر زمن إنشاء العمّال كل حقبة
`drop_last`	True مع BatchNorm	يمنع دفعة أخيرة صغيرة تُربك إحصاءات BNJI
`prefetch_factor`	2-4	أكبر = ذاكرة أكبر وزمن استجابة أطول

متى تستعمل أيّ إعداد؟

collate_fn: عندما لا تكون الدفعات متساوية

`collate_fn` هي الدالّة التي تجمع عناصر الدفعة في batch. افتراضياً تفترض عناصر متجانسة الشكل (صور نفس الحجم). إن كانت متفاوتة (نصوص بأطوال مختلفة، كشف اجسام بعدد صناديق مختلف) فستحتاج دالّة مخصّصة.

Python

```
from torch.utils.data.dataloader import default_collate

def collate_fn(batch):
    """نُرجع للنصوص: sequences قائمة الأطوال + قناع الانتباه + متطابقة الطول."""
    texts = [b[0] for b in batch]
    labels = torch.tensor([b[1] for b in batch], dtype=torch.long)
    lengths = torch.tensor([len(t) for t in texts], dtype=torch.long)
    max_len = int(lengths.max())
```

```
    padded = torch.zeros(len(batch), max_len, dtype=torch.long)
    for i, t in enumerate(texts):
        padded[i,:len(t)] = torch.tensor(t, dtype=torch.long)
    mask = (torch.arange(max_len)[None,:] < lengths[:, None])   # 1 يعني كلمة حقيقية
    return padded, mask, labels, lengths

loader = DataLoader(text_ds, batch_size=32, collate_fn=collate_fn, num_workers=2)
```

اختلال التوازن: عندما البيانات غير متوازنة

Python

```
import torch
from torch.utils.data import WeightedRandomSampler

# 1) وزن كل عيّنة بمقلوب تكرار صفّها
counts = torch.bincount(torch.tensor([int(y) for y in train_ds.y]))
class_weights = 1.0 / counts.float()
sample_weights = class_weights[torch.tensor([int(y) for y in train_ds.y])]

sampler = WeightedRandomSampler(sample_weights, num_samples=len(sample_weights), replacement=
balanced_loader = DataLoader(train_ds, batch_size=64, sampler=sampler, num_workers=4)

# 2) أو: وزن الخسارة (أسهل، ولا يغيّر توزيع البيانات)
pos_weight = (counts[0] / counts[1]).float()
lossf = torch.nn.BCEWithLogitsLoss(pos_weight=pos_weight.to(device))

# 3) أو: الـ loss نفسه (Focal Loss) – يعطي وزناً أكبر للعناصر الصعبة
#    مفيد حين تكون الأصناف النادرة هي المهمّة
```

الحلّ	الأثر	الاستخدام الأنسب
WeightedRandomSampler	يوازن التوزيع فعلياً	حين تريد أن «يرى» النموذج توازناً
`pos_weight`	يعيد ترجيح الخسارة	التصنيف الثنائي (الأشهر)
Focal Loss	يقلّل وزن العيّنات السهلة	كشف كشف/تجزئة بعناصر نادرة
عتبة على الاحتمال	لا يغيّر التدريب	ضبط نقطة القرار بعد التدريب

متى تستعمل أيّ حلّ لاختلال التوازن؟

✦ مقياس التوازن: Accuracy مضلّلة

في بيانات بنسبة 1% للعناصر الإيجابية، نموذج يتوقّع «سلبي» دائماً يحقّق **accuracy = 99%** ونفعته صفر. لهذا في هذه الحالات قِس **Precision/Recall/F1** أو **Recall عند نسبة إنذار محدّدة** – لا accuracy.

متى لا تستخدم DataLoader أصلاً؟

ثلاث حالات يكون فيها `DataLoader` خياراً سيئاً:

- **بيانات ضخمة لا تتّسع في الذاكرة ولا يمكن قراءتها عشوائياً**: استعمل `IterableDataset` (تدفّق تسلسلي) أو `tf.data`.
- **دفعات متغيّرة الحجم (token batching)**: استخدم `batch_sampler` مخصّص يجمّع أمثلة بطول متقارب.
- **بيانات من الـGPU مباشرة**: في PyTorch، بعض المسارات (DALI، WebDataset) أسرع بكثير من الـCPU.

Python

```
from torch.utils.data import IterableDataset, DataLoader

class StreamingImages(IterableDataset):
    """يتدفّق على ملفات من قرص بلا تخزين الكل في الذاكرة."""
    def __init__(self, file_list, transform=None):
        self.files = file_list
        self.transform = transform

    def __iter__(self):
        worker = torch.utils.data.get_worker_info()
        files = self.files[worker.id::worker.num_workers] if worker else self.files
        for f in files:
            img, label = load_image(f)
            if self.transform:
                img = self.transform(img)
            yield img, label
```

تمرين 1 — خطّ بيانات متكامل

1. اكتب `Dataset` يقرأ صوراً من مجلدات فئات مع تحويلات.
2. أنشئ DataLoader بعمّال متعدّدين و `pin_memory` .
3. اطبع زمن `next(iter(loader))` ومقارنته بحالة `num_workers=0` .
4. أضف تدقيقاً: اطبع أحجام الدفعات Last (هل هي ثابتة مع `drop_last=True` ؟).

▲ الحل النموذجي

```python
import time
from torch.utils.data import DataLoader

def first_batch_time(loader, n=5):
    it = iter(loader)
    next(it)                        # تسخين
    t0 = time.perf_counter()
    for _ in range(n):
        next(it)
    return (time.perf_counter() - t0) / n

for nw in (0, 4):
    loader = DataLoader(train_ds, batch_size=64, shuffle=True,
                        num_workers=nw, pin_memory=(nw > 0))
    print(f"num_workers={nw}: {first_batch_time(loader)*1000:.1f} ms/batch")

sizes = [xb.shape[0] for xb, _ in loader]
print("unique batch sizes:", set(sizes))   # مع drop_last=True: {64}
```

القراءة المتوقّعة: مع عمّال متعدّدين يصبح جلب الدفعة أسرع بكثير في بيئة I/O-محدودة (صور على قرص)، بينما في بيانات محسوبة في الذاكرة قد لا يفيد، يبطئ بسبب كلفة إنشاء العمليات. **قِس في بيئتك الحقيقية** – لا تفترض.

تمرين 2 — كشف التسرّب في خطّ البيانات

1. أنشئ مجموعتين من نفس البيانات: الأولى مختلطة (labels مخلوطة)، والثانية مرتّبة.
2. درب نفس النموذج على كلٍّ منهما وقارن دقّة التحقّق.
3. فسّر: لماذا «التطهير» في التدريب قد يرفع دقّة التحقّق؟
4. اقترح إجراءً يمنع هذا النمط من التسرّب إلى الإنتاج.

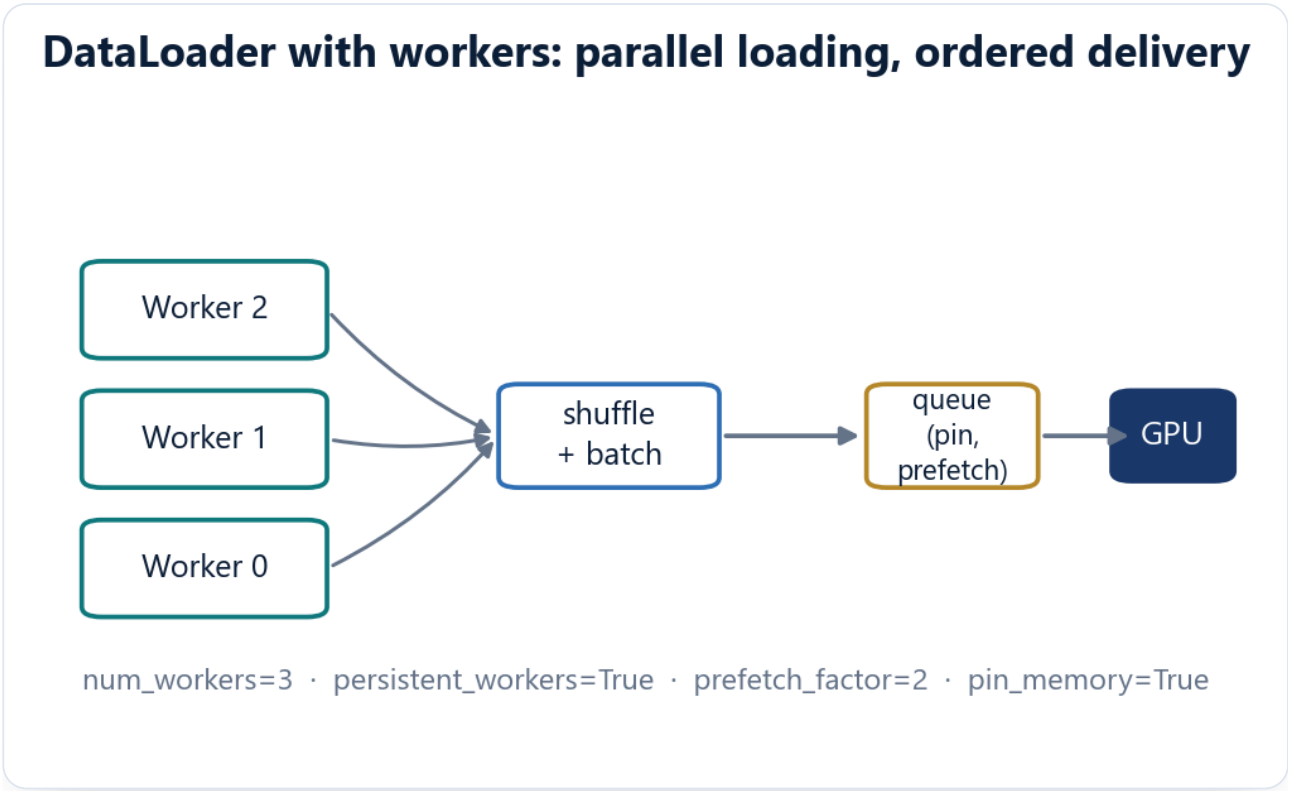
▲ الحل النموذجي

التفسير: البيانات المرتّبة (مثلاً: كل صور القطة أولاً ثم كل صور الكلب) تسمح للنموذج بتمييز «آخر عيّنة من أيّ صنف» والتنبّؤ بها – فيتعلّم قاعدة لا علاقة لها بالمحتوى. عند التقييم على بيانات مختلطة بالطريقة نفسها، قواعد مثل «توقّع نفس الصنف التسلسل» تنجح المصادفةً.

الإجراء الوقائي: (1) خلط البيانات دائماً (وإلا فكّر بشكل عشوائي قبل الدفعة الأخيرة)؛ (2) ضمان خلط مجموعة الاختبار أيضاً؛ (3) الانتباه إلى أن الخلط وحده لا يكفي إن بقيت **تسلسل زمني** أو **مصدر** محسوس في الخصائص.

خطّ البيانات: المحور الذي يحدّد كلّ شيء

في التعلّم العميق، خطّ البيانات ليس مرحلة تمهيدية تُنجَز ثم تُنسى؛ هو نصف النموذج. نموذجان متطابقان تماماً، يُدرَّبان على خطّ بيانات مختلف، يختلفان في الأداء فرقاً يفوق ما يُحققه أيّ تعديل معماري. وفي كلّ مراجعة لمشروع تعلّم آليّ تقريباً، يكون أوّل ما نظّر إليه هو: كيف جُمعت البيانات، وكيف قُسّمت، وما الذي دخل التدريب.



يقرأ الرسم من اليمين إلى اليسار: الملفات على القرص، ثم `Dataset` الذي يقرأ عيّنة ويحولها، ثم `DataLoader` الذي يجمّع دفعات ويشغّل عمّالاً متوازيين، ثم إلى نموذج. كلّ خانة في هذا الشريط هي مكانٌ يمكن أن يتسرّب خطأ لا يظهر إلا في النتائج.

القسمة: ثلاثة أخطاء تكلّف أكثر من التحسين

خطأ أوّل: **القسمة العشوائية** لمجموعة فيها بيانات متسلسلة زمنياً. إذا كان لديك سجلّ مبيعات شهري وقسّمته عشوائياً، فقد يوضع شهر سبتمبر في التدريب ويوضع أكتوبر في التحقّق – والنموذج يتعلّم «توقّع الشهر من ربعه». خطأ ثانٍ: **تسريب البيانات** (data leakage) عبر التطبيع: حساب متوسّط الأعمدة على كامل البيانات قبل القسمة. خطأ ثالث، وهو الأخطر: **تكرار العيّنة** (المريض نفسه بأكثر من زيارة) في التدريب والتحقّق معاً.

الخطأ	العلامة	الاختبار
قسمة عشوائية لبيانات زمنية	دقّة أعلى من المتوقّع	قسمة زمنية: تدريب على الماضي وتحقّق على المستقبل
تسريب من التطبيع	دقّة التحقّق قريبة من التدريب	احسب المتوسّط داخل خطّ التدريب فقط
تكرار الوحدات	أداء متضخّم	قسّم على مستوى الوحدة لا على مستوى العيّنة

الخطأ	العلامة	الاختبار
تسريب من المعالجة المسبقة	أداء غير مستقرّ	كلّ تحوّل يُتعلَّم من التدريب وحده

ثلاثة أخطاء في القسمة وعلاماتها وكشفها

Python

```
import torch
from torch.utils.data import DataLoader, Dataset

class TabularDataset(Dataset):
    """مجموعة بيانات جدولية: تقرأ ملفاً كاملاً ثم تخدم عيّناته."""

    def __init__(self, rows, features, labels):
        self.x = torch.tensor(rows[:, features], dtype=torch.float32)
        self.y = torch.tensor(rows[:, labels], dtype=torch.long)

    def __len__(self):
        return self.x.size(0)

    def __getitem__(self, idx):
        return self.x[idx], self.y[idx]


def make_loaders(train_ds, val_ds, batch_size=64, workers=4):
    """منشئ تحميل: تخلط التدريب فقط، والتقييم بترتيب ثابت."""
    train_loader = DataLoader(
        train_ds, batch_size=batch_size, shuffle=True,
        num_workers=workers, pin_memory=True, drop_last=True,
    )
    val_loader = DataLoader(
        val_ds, batch_size=batch_size, shuffle=False,
        num_workers=2, pin_memory=True,
    )
    return train_loader, val_loader
```

ماذا يحدث هنا؟ `shuffle=True` للتدريب فقط: التحقّق يحتاج ترتيباً ثابتاً كي تكون المقارنات بين الحقبات ذات معنى. `pin_memory` يسرّع النقل إلى وحدة التسريع، و `drop_last=True` يوحّد حجم الدفعة الأخير.

لماذا هذه القيمة؟ لأنّ دفعة بآخر حجم مختلف تكسر التطبيع (فيحسب إحصاءات على دفعة صغيرة جداً) وتربك جدولة معدّل التعلّم.

> ⚠ **الفرق بين «عمّال» و«نوى»**
>
> `num_workers` هو عدد العمليات التي تقرأ البيانات بالتوازي. على نظام ويندوز، ابدأ بصفر وزِد تدريجياً: قيمة عالية على ويندوز قد تبطئ التطبيق لأنّ كلّ عامل يفتح نسخة من الملف كاملاً. القاعدة العملية: اجعل عدد العمّال أصغر من عدد المعالجات، وراقب زمن التحميل لا زمن التدريب.

مشروعك تصنيف تغذّي فيه ٤٠٠٠٠٠ صفّ من سجلّات العمليات، والقسمة عشوائية. دقّة التحقّق ٩٦٪. لاحظ أنّها أعلى من دقّة مجموعة التحقّق اليدويّة (٨٩٪). ما الاحتمالان الأقرب؟ أيّهما أخطر، ولماذا يجب ألّا يكونAccuracyوحده هو الحكم؟

الاحتمالان: **تسريب زمني** (سجلّات من نفس الفترة في التدريب والتحقّق) و **تكرار الوحدات** (نفس الماكينة أو نفس العامل في المجموعتين). الأخطر هو التكرار: لأنّ النموذج يتعلّم «تعرّف» الوحدات لا العلاقات بينها، فينهار الأداء على بيانات إنتاج حقيقية. لا الحكم على الدقّة وحدها: احسب الدقّة على مجموعة متنوّعة حقيقيّة من سجلّات الإنتاج بعد مرور الوقت.

الوسيط: كيف تقرأ البيانات بسرعة دون أن تلتهم الذاكرة

بين القرص والذاكرة تقع طبقة لا تحبّها أحد، لكنّها تقرّر سرعة تدريبك. حين نقرأ ملفاً نصّياً بحجم ٢٠ ميغابايت عبر `pandas`، نحصل على إطار بيانات في الذاكرة يقارب عشرة أضعاف حجمه. ثم نُحوّله إلى `numpy`، ثم إلى `torch.Tensor` من نوع `float32`، فتتكاثف الذاكرة أربع مرّات أخرى.

هذه الطبقات الثلاث ليست تفصيلاً برمجياً؛ هي تحدّد ما إذا كانت حقبة واحدة تستغرق دقيقتين أم عشرين. الحلّ ليس «شراء قرص أسرع» بل **أن لا تقرأ ما لا تحتاجه**. وهنا تظهر قيمة نمط `IterableDataset`: بدل تحميل كلّ الملف في الذاكرة، نكتب دالّة تُنتج عيّنة عند كلّ طلب.

```python
import torch
from torch.utils.data import IterableDataset


class StreamingCsvDataset(IterableDataset):
    """يقرأ الملف صفّاً صفّاً: ذاكرة ثابتة مهما كبر الملف."""
```

```
    def __init__(self, path, features, labels, transform=None):
        self.path = path
        self.features = features
        self.labels = labels
        self.transform = transform

    def __iter__(self):
        import csv
        with open(self.path, newline="", encoding="utf-8") as fh:
            reader = csv.DictReader(fh)
            for row in reader:
                x = torch.tensor(
                    [float(row[c]) for c in self.features], dtype=torch.float32
                )
                y = torch.tensor(int(row[self.labels[0]]), dtype=torch.long)
                if self.transform:
                    x = self.transform(x)
                yield x, y

    def __len__(self):
        with open(self.path, encoding="utf-8") as fh:
            return max(0, sum(1 for _ in fh) - 1)
```

ماذا يحدث هنا؟ الصنف يرث من `IterableDataset`، ما يعني أنّ `DataLoader` سيطلب منه عيّنات بالترتيب لا للوصول العشوائي. **لماذا هذه القيمة؟** لأنّ الوصول العشوائي إلى قرص لا يمكن أن يكون سريعاً؛ في الذاكرة يمكن.

نظامك يتدرّب على ملف ٤٠ غيغابايت، وذاكرتك ١٦ غيغابايت. الحلّ الذي يُقترح دائماً هو «استخدم `IterableDataset`». لكنّ التدرّب صار بطيئاً جداً بعد التحويل. ما السببان المحتملان؟ وما أفضل تصميم يجمع بين اقتصاد الذاكرة وسهولة الوصول العشوائي؟

السببان: **الوصول العشوائي** الذي يهدم أداء القرص (كلّ `__getitem__` تفتح الملف من جديد)، و**العمّال المتعدّدون** الذين يضاعفون الاستهلاك. أمّا التصميم الأمثل فهو **الترتيب المقسّم**: تقرأ الملف مرة واحدة على دفعات متتابعة، تكتب الدفعات إلى ذاكرة مشتركة، ثم يقرأ منها `DataLoader` بشكل عشوائي. هذا ما تفعله أطر العمل الحديثة فعلياً، وهو ما يجعل التدرّب على البيانات الضخمة ممكناً على أجهزة صغيرة.

ملخّص الفصل التاسع

- DataLoader يتّخذ 5 قرارات: المصدر، الترتيب، حجم الدفعة، الحذف، الإنجاز.
- حوّل داخل `__getitem__` لا قبله.
- مجموعة تدريب وتقييم مختلفتان (عشوائي مقابل حتمي).
- `num_workers` + `pin_memory` + `persistent_workers` = خطّ بيانات لا يُجهد الGPU.
- `collate_fn` للأطوال المتفاوتة.
- اختلال التوازن: sampler أو pos_weight أو Focal؛ و**لا accuracy** للحكم.
- الخلط العشوائي ضرورة، لا تحسين.

حلقة التدريب الاحترافية

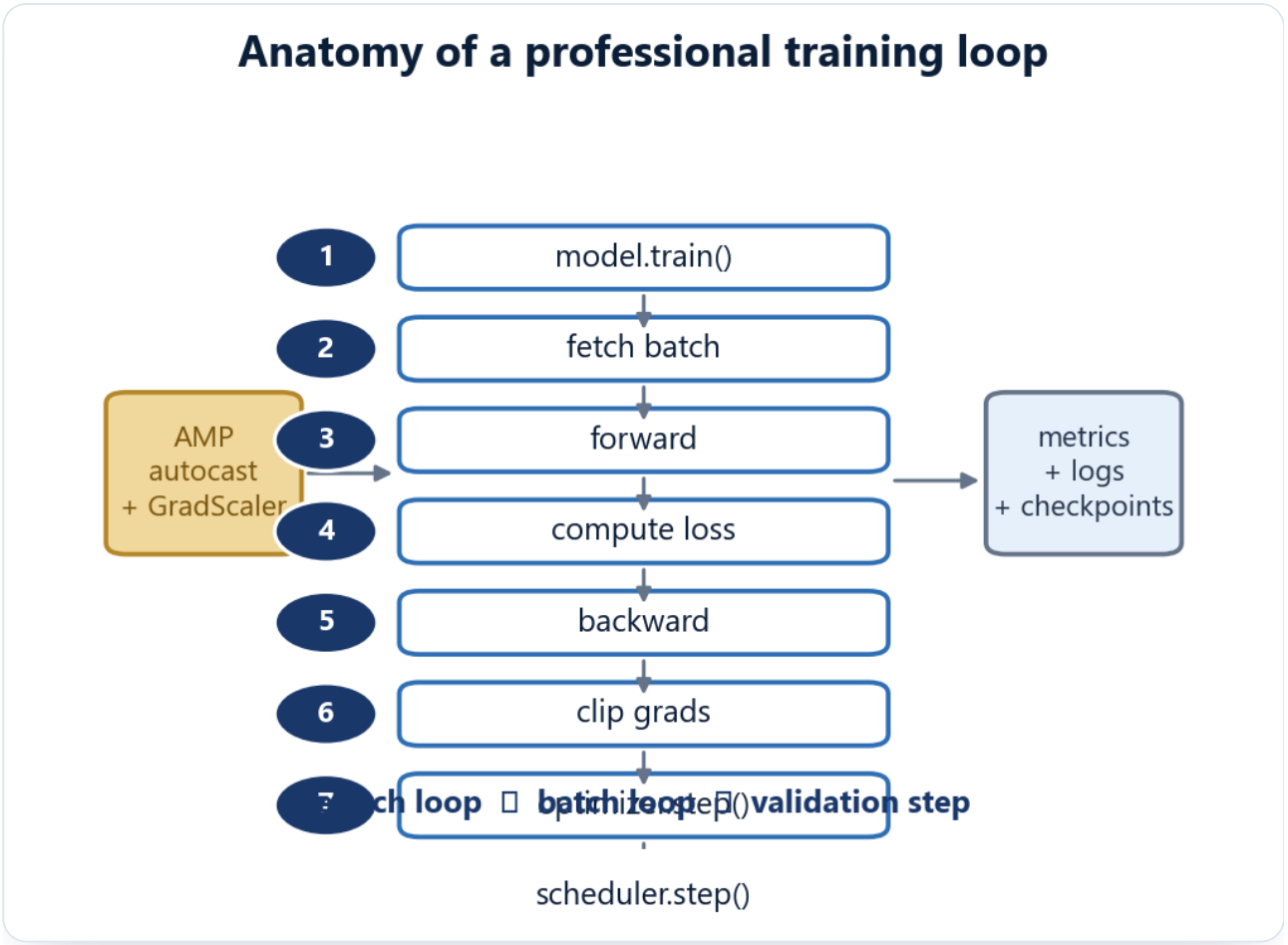
AMP، التدرّج المحدود، EMA، نقاط الحفظ، الجدولة، والقياس

في الفصل السابق بنينا خط أنابيب بيانات يجلب العينات إلى الذاكرة. لكنّ البيانات ليست تدريباً: التدريب هو ما يحدث بعد أن تصل الدفعة. في هذا الفصل ننتقل من حلقةٍ بسيطة من خمسة أسطر إلى حلقةٍ تديرها في مشروع إنتاجي: تحسب بالأرقام المختلطة لتوفّر الوقت، وتقصّ التدرّج كي لا ينفجر، وتحتفظ بنسخة متوسّطة من الأوزان، وتعرف كيف تعود إلى نقطة التوقّف بعد انقطاع الكهرباء، وتجدول معدّل التعلّم، وتقيس ما يهمّ لا ما يسهل، وتضمن أن نتيجة اليوم نفسه قابلة للتكرار غداً. الفصل مكتوب بوصفه دليلاً عملياً: كل تقنية تُشرح أولاً بـ«لماذا نحتاجها»، ثم تُنفَّذ في شيفرة قابلة للتشغيل، ثم تُقاس.

10.1 الحلقة البسيطة والحلقة التي تُنتج نماذج

الحلقة التي انتهينا منها في الفصل التاسع تبدو معقولة: نمرّ على المُحمِّلات، ونحسب الخسارة، ونعيد التدرّج العكسي، ونحدّث الأوزان. لكن هذه الحلقة تفتقر إلى أربع صفات تجعلها غير صالحة لمشروع حقيقي. أوّلها **الدقّة**: ليست دقّة النموذج، بل دقّة الأرقام التي يعمل بها؛ فأنت تدرب بالخانات العشرية التي تتّسعأكمل، وتدفع ثمن كل عملية ضرب بالعرض الكامل. ثانيها **الاستقرار**: خطوة واحدة غير موفّقة يمكن أن تمحو معالم تدرّجٍ مفيدة وتُرسِخ نموذجاً سيئاً بلا إنذار. ثالثها **الاستئناف**: حين ينقطع الخادم في الدقيقة السابعة من الليل، لا بدّ من طريقة تعود بها إلى ما كنت تفعله بالضبط. ورابعها، وهو الأخطر، **القياس**: فبدون قياس منضبط لا تستطيع أن تعرف هل تحسينٌ حقيقي أم تغيّرُ أرقام عشوائية.

هذه الصفحات الأربع هي ما يجعل الحلقة «احترافية» في هذا الفصل. لاحظ أن كل واحدة منها تُعالج سؤالاً مختلفاً: الدقّة توفّر وقتاً، والاستقرار يمنع الكوارث، والاستئناف يحمي ساعات عملك، والقياس يوفّر المسار. وحين تجتمع، تتحوّل الحلقة من مجموعة أوامر إلى *آلة تجارب*: كل تشغيل لها هوية، وكل رقم فيها قابل للنسبة إلى رقم سابق.



الشكل 14-1: البنية ثلاثية الحلقات لأي تدريب حقيقي. الحلقة الأبعد تمرّ على الحقائب، والأوسط على الدفعات، والأقرب خطوة تحقّق واحدة. الكتلتان الجانبيتان — الدقة المختلطة على اليسار، والمقاييس ونقاط الحفظ على اليمين — هما ما يحوّل التكرار الأعمى إلى تدريب يمكن الاعتماد على نتائجه. لاحظ أن مسار التحقّق يخرج من مسار التدريب ولا يعيد الأوزان: التحقّق قياس لا تحديث.

> فكّر في آخر تجربة تدريبٍ فشلَت: هل فشلت لأن النموذج كان سيئاً، أم لأن الحلقة سمحت لفشلٍ صغير أن يتضخّم؟ وأيّهما كان ستكتشفه لو كان لديك قياسٌ دقيق للحلقة؟

10.2 الدقة المختلطة: تشغيلٌ أسرع بمرّتين

بطاقات الرسم الحديثة (GPU) تُنفّذ العمليات الحسابية بمعدّلات تفوق ما تنتجه المعالجات المركزية بعشرات المرّات، لكنّ الأرقام العشرية ذات الدقّة الواحدة تحتاج مساحة تخزين وتمريراً للبيانات أضعاف ما تحتاجه الخانات نصفية (`float16` ذات الستّة عشر بتاً). ولأنّ مجموع الشبكة العصبية يتضمّن آلاف العمليات، فإنّ تحويل الحساب كلّه إلى `float16` يوفّر زمناً وذاكرة هائلة. المشكلة أنّ `float16` يمثّل الأعداد الضئيلة بطريقةٍ خاسئة: أصغر عدد طبيعي يمكن تمثيله فيها يقارب 1.0×10^{-5}، وقد يتسرّب التدرّج الصغير إلى الصفر قبل أن ينضج.

الحلّ الذي تبنّته غوغل ثم عمّمته `PyTorch` يُسمّى **الدقة المختلطة** (`Automatic Mixed Precision`): تُنفَّذ معظم العمليات الثقيلة (الضرب المصفوفي والالتفاف) في `float16` ، بينما تبقى الأوزان الرئيسية، وعمليات الجمع والتراكم، وحساب الخسارة في `float32` . هذا المزيج – «سرعةُ الالتواء مع دقّةِ التراكم» – هو ما يجعل الدقة المختلطة أداةً عملية لا مجرّد تسريع.

$$\tilde{L} = s \cdot L \quad \text{and} \quad \tilde{g} = s \cdot g$$

ما معنى كل رمز؟ `L` هو الخسارة الحقيقية كما لو كُسِرت في `float32` ، و `L̃` هي الخسارة «المُكبَّرة» التي نمرّرها فعلياً إلى الانتشار العكسي. `g` هو متّجه التدرّج الحقيقي، و `g̃` تدرّجه بعد التكبير. أمّا `s` فهو معامل التكبير (`scale`)، وهو رقمٌ واحدٌ يُختار في البداية وقيمته الافتراضية عند `PyTorch` هي 65536 (أي 2^{16}).

لماذا يهمّنا؟ لأن التدرّجات الصغيرة في `float16` تُكتب تحت الصفر فتُمحى، بينما التدرّجات الكبيرة تفقد نهائيةها وتصبح `inf` أو `NaN` . الضرب في `s` كبير يرفع التدرّجات الصغيرة إلى نطاق `float16` المأمون، فيصل ما كان سيموت إلى المنحدر. ثم يُعاد القسمة على `s` قبل تحديث الأوزان.

خطأ شائع عندها: أن تنسى القسمة على `s` قبل `optimizer.step()` ، فتُحدَّث الأوزان بمقدار أكبر بآلاف المرّات. في `PyTorch` يتولّى `GradScaler` هذه القسمة تلقائياً داخل `scaler.step()` ، فإذا أردت قصَّ التدرّج أو تفتيشه فأنت *ملزم* باستدعاء `scaler.unscale_(optimizer)` أولاً.

الجزء الثاني من الحكاية هو **مقياس التدرّج** (`GradScaler`): هو مبالغٌ كمّي يضبط `s` تلقائياً بعد كل خطوة. إن اكتُشف `inf` أو `NaN` في التدرّجات، تُتخطّى خطوة التحديث بالكامل (حتى لا تتلف الأوزان)، ويُقسَم `s` على 2. إن مرّت 2000 خطوة متتالية بلا انفجار، يُضاعَف `s` . بهذه «الاستكشاف الديناميكي» يجد المقياس بنفسه أكبر معامل تكبير آمن.

⚠ انتبه: الدقة المختلطة ليست «تحويلاً كاملاً»

لا تُلفّ الشبكة كلها بـ `model.half()` وتنتظر المعجزة. الدقة المختلطة في `PyTorch` تُستدعى عبر `torch.amp.autocast(device_type="cuda", dtype=torch.float16)` ، وهو يقرّر بنفسه أي عملية تُنفَّذ في أي دقّة. والأوزان تبقى `float32` . تذكّر أنّ `torch.amp.autocast` هي الواجهة الحالية، وأن `torch.cuda.amp.autocast` واجهة قديمة أُزيلت فلا تستعملها في مشروع جديد.

✓ نصيحة: ابدأ بـ `bfloat16` إن كانت بطاقتك حديثة

نمط `bfloat16` يملك مدى الترقيم نفسه تقريباً الذي يملكه `float32` ، فلا يحتاج إلى مقياس تكبير ولا يستهلك وقتاً في `GradScaler` . جرّب `dtype=torch.bfloat16` أولاً على عتاد حديث، واحتفظ بـ `float16` مع `GradScaler` كخيار احتياطي للعتاد الأقدم.

ولنُرِ الدقة المختلطة في أبسط صورة ممكنة: خطوتُ تدريبٍ واحدة مكتوبةً بترتيبها الصحيح. انتبه إلى أن سطرَي `autocast` و `GradScaler` ليسا اختياريّين – أحدهما يضبط الدقّة والآخر يحمي الأوزان.

Python

```python
import torch
import torch.nn as nn

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
use_amp = device.type == "cuda"

model = nn.Linear(16, 3).to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss()
scaler = torch.amp.GradScaler("cuda", enabled=use_amp)

x = torch.randn(32, 16, device=device)
y = torch.randint(0, 3, (32,), device=device)

optimizer.zero_grad(set_to_none=True)
with torch.amp.autocast(device_type=device.type, dtype=torch.float16, enabled=use_amp):
    logits = model(x)
    loss = criterion(logits, y)

scaler.scale(loss).backward()
scaler.unscale_(optimizer)
total_norm = torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
scaler.step(optimizer)
scaler.update()

print("loss:", round(loss.item(), 4))
print("grad norm:", round(float(total_norm), 4))
print("current scale:", scaler.get_scale())
```

ماذا يحدث هنا؟ السطر `enabled=use_amp` يجعل الشيفرة نفسها تعمل على حاسوب بلا بطاقة رسومية (حيث يُعطَّل مقياس التدرّج تلقائياً) وعلى حاسوب مزوّد بها، وهو سرّ قابلية النقل التي ستراها في كل شيفرة إنتاجية. **لماذا هذه القيمة؟** معامل `max_norm=1.0` هو سقفٌ افتراضي معقول لمعيار التدرّج؛ جرّب `0.5` إن كان النموذج شديد الحساسية وانفجرت التدرّجات.

10.3 قصّ التدرّج: صمّام أمانٍ للنموذج

في الشبكات العميقة قد تنتفخ التدرّجات فجأةً إلى أرقام هائلة، فتنقلب الأوزان إلى `NaN` في خطوةٍ واحدة، ويموت التدريب بصمت. والسبب رياضي: في كل طبقة يُضرب التدرّج بعدة عوامل مصفوفية، فإنّ كانت عوامل الطبقات العميقة أكبر من واحد، فإنّ حاصل الضرب ينمو أُسّياً مع عدد الطبقات.

$$g \leftarrow g \cdot \min(1, c \,/\, (||g||_2 + \varepsilon))$$

ما معنى كل رمز؟ `g` متّجه التدرّج قبل القصّ، و `||g||₂` هو معياره الثاني (جذر مجموع مربّعات مكوّناته) وهو قياس واحد يقيس «حجم» التدرّج الكلي لا كل مكوّن على حدة. `c` هو الحدّ الأقصى المسموح، و `ε` رقمٌ صغير جداً (10^{-6}) يمنع القسمة على صفر. `min` تعني: إن كان المعيار أصغر من `c` نترك التدرّج كما هو، وإن كان أكبر نقسمه على نسبته إلى `c` ليصبح معياره مساوياً تماماً لـ `c`.

لماذا يهمّنا؟ لأن القصّ لا يغيّر اتجاه التدرّج (وأنت لا تزال تتحرّك نحو نفس الاتجاه المعدَّل)، بل يحدّ من *طول* الخطوة. هذا يعني أن انفجار التدرّج يتحوّل من «نهايةٍ قاتلة» إلى «زحلةٍ لطيفة»: النموذج يتأخّر قليلاً في التصحيح بدل أن ينفجر.

خطأ شائع عندها: قصّ التدرّج بعد `scaler.step()`، فتُقصّ أرقامٌ مُكبَّرة بمعامل `s` ولا يكون المقصود قد تحقّق. الترتيب الصحيح: `scale()` ثم `backward()` ثم `unscale_()` ثم القصّ ثم `step()`. القيمة `c = 1.0` قيمةٌ شائعة وافتتاحية، وليست وصفة سحرية. القيمة المناسبة تُضبط بالتجربة: راقب معيار التدرّج عبر الحقائب، واجعل `c` في نفس مرتبة ذلك، أو أكبر منه بقليل. المعيار المعتاد هو 1.0 للنصوص و0.5 للنماذج الكبيرة جداً.

◆ القصّ العام مقابل القصّ لكل طبقة

`clip_grad_norm_` يقصّ على مستوى الشبكة كلّها اعتماداً على المعيار الكلي، وهو الخيار الافتراضي والأكثر استقراراً. أمّا `clip_grad_value_` فيقصّ كل عنصر على حدة عند قيمةٍ قصوى مطلقة، وهو أندر استخداماً لأنّه يميّز التدرّجات الكبيرة المفيدة عن الصغيرة. ابدأ بالأول.

10.4 المتوسّط المتحرك الأسّي: مرآةٌ هادئة للوزان

تتباين نقاط الوزن في نموذجٍ ما بعد كل خطوة: الخطوة الواحدة قد ترفع دقّة التحقّق ثم تخفضها. فإن أخذت في نهاية التدريب «آخر» أوزان، فقد تكون عيّنة غير موفّقة من ذلك المسار المتذبذب. الحلّ هو أن تحتفظ بنسخة ثانية من الأوزان: لا تنسخ أوزان النموذج الحالية فحسب، بل تنسخ متوسّطاً متحرّكاً لوزانها على مدى خطواتٍ سابقة. هذه النسخة تُسمّى **المتوسّط المتحرك الأسّي** (`EMA`).

$$\Theta(t) = \alpha \cdot \Theta(t-1) + (1 - \alpha) \cdot \theta(t)$$

ما معنى كل رمز؟ `Θ` هي أوزان المتوسّط المتحرك بعد الخطوة `t`، و`θ` هي أوزان النموذج الحيّ بعد الخطوة نفسها. `α` هو معامل التنعيم: كلّما اقترب من 1 امتدّ أثر الماضي أطول. القيم الشائعة 0.999 و0.9999، أي أنّ النافذة الفعّالة تعادل تقريباً 1000 أو 10000 خطوة.

لماذا يهمّنا؟ لأنّ متوسطاً على مسارٍ عشوائي يقارب النقطة «المركزية» للمسار، وهي عادةً نقطة أفضل من آخر نقطةٍ عشوائية. في مهام كشف الاحتيال، حيث يكون التقييم على بيانات غير متوازنة وشديدة الحساسية للعيّنات السلبية الكاذبة، تعني هذه الميادين الوسطى فرقاً حقيقياً في نسبة الإنذارات الكاذبة.

خطأ شائع عندها: استخدام `alpha = 0.9` معتقدين أنّه «تنعيم بسيط». النتيجة متوسّطٌ على مدى قصير جداً فيتحوّل عملياً إلى نسخة من آخر نموذج، فتفقد الفكرة كلّها. ابدأ بـ0.999، واضبطها بحسب طول تدريبك: القيمة التقريبية `alpha = 1 - 1/(عدد خطوات التدريب)` تعطي نافذةً تغطّي التدريب كلّه تقريباً.

في `PyTorch` توجد طريقة جاهزة اسمها `torch.optim.swa_utils.AveragedModel` تعتمد التوسّط بمعاملات متعدّدة، لكنّ المتوسّط المتحرك المخصّص أودق وأوضح في الأداء. وسنكتبه بأنفسنا لأنّ قراءته تُبيّن لك ما يفعله `GradScaler` والمتوسّط المتحرك بالضبط.

10.5 نقاط الحفظ: كيف تعود إلى ما توقّفت عنده

الساعة 3:14 فجراً، ينقطع التيار في مركز البيانات. في الصباح، تجد مجلّداً فيه ملفّ واحد اسمه `last.pt` بحجم 40 ميغابايت. هل يمكنك المتابعة من حيث توقّفت؟ إن كان الملف يحوي أوزان النموذج فحسب، فلا: ستبدأ من الصفر لأنك فقدت حالة المحسّن، والمقياس، والجدول، ورقم الحقبة. الحلّ هو **نقطة الحفظ** (`checkpoint`) الشاملة: حزمة تحوي كل ما يلزم لاستئناف التدريب *حرفياً*.

المحتوى الواجب في نقطة الحفظ: أوزان النموذج، حالة المحسّن (وتحوي لحظات `Adam` من النوع `exp_avg` و`exp_avg_sq`)، حالة مقياس التدرّج، حالة جدول معدّل التعلّم، رقم الحقبة الحالية، أفضل قيمة قِياسٍ تحقّقٍ محفوظة، وحالات المولّدات العشوائية (بذور بايثون و`numpy` و`PyTorch`). النقطة الأخيرة هي ما يجعل الاستئناف *حتمياً*: بدونها ستُستأنف الحقائب نداءً مختلفاً تماماً رغم المثل.

★ القاعدة الذهبية: احفظ قبل أن تحتاج

نقطة الحفظ تُكتب في نهاية كل حقبة (أو كلّ حقبتيْن) باسمٍ يحتوي قيمة التحقّق، ولا يُعاد حفظ النسخة الأفضل إلا حين تتحسّن القيمة فعلاً. الكتابة إلى ملفٍّ واحد في عملية مخصّصة، مع نسخٍ احتياطية محدودة، تمنع استهلاك القرص وتمنع نقطة حفظٍ قديمة من دهس الجديدة.

ولأنّ الحفظ عملية حسّاسة (لو انقطع التيار أثناء الكتابة تنلف الملف)، يُفضَّل أن يُكتب إلى ملفٍّ مؤقّت ثم يُعاد تسميته إلى الاسم النهائي: عملية إعادة التسمية في نظام الملفات ذرّية، فلا يبقى إلا ملفٌّ كامل.

وهذه دالّتان تُلخّصان كل ما سبق: الأولى تكتب الحزمة الشاملة، والثانية تستعيدها. لاحظ أنّ استعادة المولّدات العشوائية ليست تفصيلاً: بدونها ستبدأ الحقب القادمة بترتيبٍ مختلف، فتصبح المقارنة بين الحقبتين بلا معنى.

Python

```python
import os
import random

import numpy as np
import torch
import torch.nn as nn


def save_checkpoint(path, model, optimizer, scheduler, scaler, epoch, best_metric):
    payload = {
        "epoch": epoch,
        "model": model.state_dict(),
        "optimizer": optimizer.state_dict(),
        "scheduler": scheduler.state_dict(),
        "scaler": scaler.state_dict(),
        "best_metric": best_metric,
        "python_rng": random.getstate(),
        "numpy_rng": np.random.get_state(),
        "torch_rng": torch.get_rng_state(),
    }
    tmp_path = path + ".tmp"
    torch.save(payload, tmp_path)
    os.replace(tmp_path, path)


def load_checkpoint(path, model, optimizer, scheduler, scaler, device="cpu"):
    payload = torch.load(path, map_location=device, weights_only=False)
    model.load_state_dict(payload["model"])
    optimizer.load_state_dict(payload["optimizer"])
    scheduler.load_state_dict(payload["scheduler"])
    scaler.load_state_dict(payload["scaler"])
    random.setstate(payload["python_rng"])
    np.random.set_state(payload["numpy_rng"])
    torch.set_rng_state(payload["torch_rng"])
```

```
    return payload["epoch"] + 1, payload["best_metric"]


model = nn.Linear(8, 2)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=10)
scaler = torch.amp.GradScaler("cuda", enabled=False)

save_checkpoint("demo.pt", model, optimizer, scheduler, scaler, epoch=4, best_metric=0.87)
resumed_epoch, best = load_checkpoint("demo.pt", model, optimizer, scheduler, scaler)
print("resumed at epoch:", resumed_epoch)
print("best metric carried over:", best)
```

ماذا يحدث هنا؟ الدالة `save_checkpoint` تكتب إلى `demo.pt.tmp` ثم تستبدل الاسم بـ `os.replace`، وهي عملية ذرّية على الأنظمة الشبيهة بيونكس وعلى ويندوز. **لماذا هذه القيمة؟** نستخدم `weights_only=False` عند القراءة لأنّ حالتَي `random` و `numpy` ليستا من نوع `tensor` فهما تحتاجان مُنقِّصاً كاملاً؛ ولا تستخدم هذه القيمة إلا على نقاط حفظٍ تثق بمصدرها.

10.6 جداول معدّل التعلّم: من أين يأتي الاتجاه

معدّل التعلّم هو أطول عصا في التدرّج: فهو يحدّد كم تتحرّك الأوزان في كل خطوة. خلال الحقبة الأولى يكون التدرّج كبيراً وغير مستقرّ، فيكون معدّل تعلّمٍ كبيراً ضرورياً للخروج من «المنطقة المسطّحة» التي بدأت فيها الأوزان. وفي الحقبة الأخيرة يكون النموذج قريباً من حوضٍ ضحل، فيكون التصغير ضرورياً للوصول إلى قاعه. **جدول معدّل التعلّم** هو المنحنى الذي يربط بين الاثنين: قيمة ابتدائية كبيرة، ثم اضمحلال تدريجي نحو قيمةٍ دنيا صغيرة.

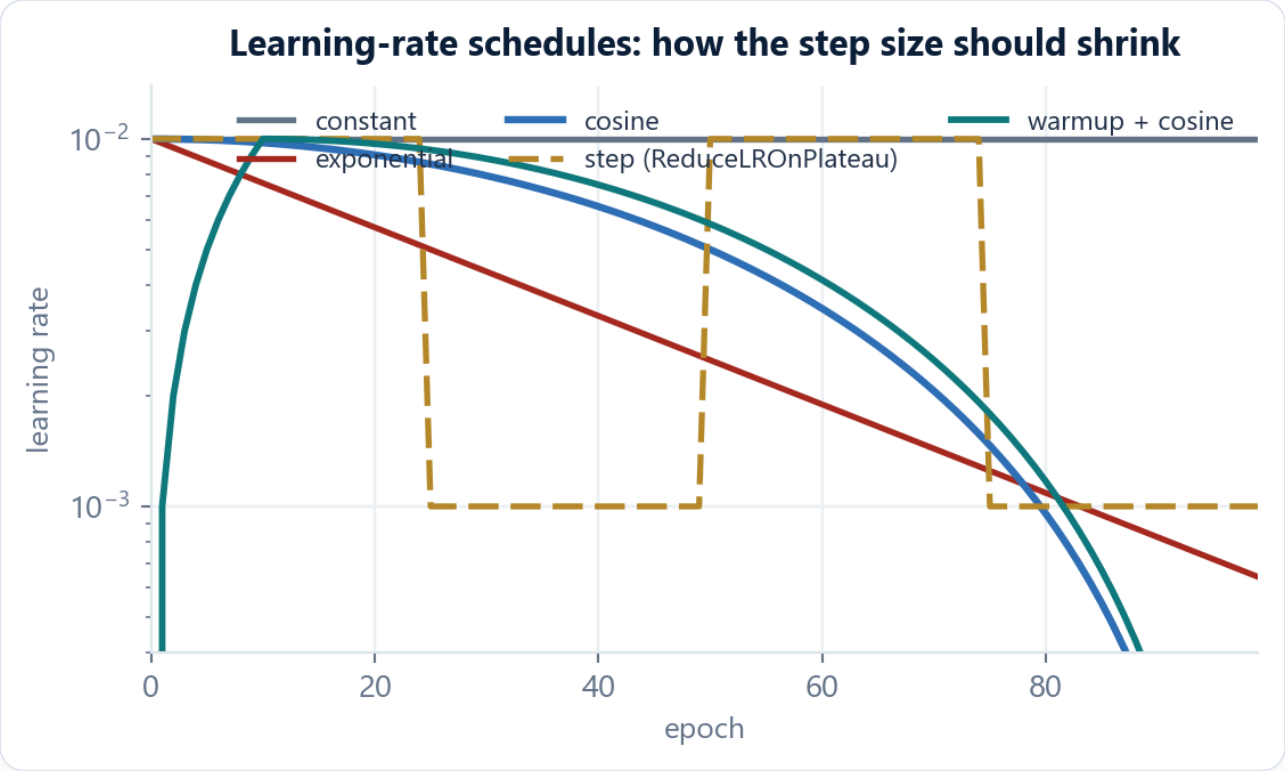
$$\eta_t = \eta_{\min} + \tfrac{1}{2}\,(\eta_{\max} - \eta_{\min})\,(1 + \cos(\pi \cdot t / T))$$

ما معنى كل رمز؟ η_t هو معدّل التعلّم في الخطوة t، و $\eta_{\max}$ و $\eta_{\min}$ هما الحدّان الأعلى والأدنى للجدول، و t الخطوة الحالية، و T إجمالي خطوات الجدول، و π ثابت الدائرة. الدالة الجيبية تلامس $\eta_{\max}$ عند $t = 0$ وتصل $\eta_{\min}$ عند $t = T$.

لماذا يهمّنا؟ لأنّ شكل المنحنى يحدّد «مساحة البحث» التي يستكشفها النموذج: منحنى `cosine` يأخذ وقتاً طويلاً في الأوزان الكبيرة ثم ينزل بهدوء نحو الحل، وهو اليوم الخيار الافتراضي في تدريب النماذج الكبيرة. منحنى `exponential` ينزل بسرعةٍ أكبر، ومنحنى الخطوة (`step`) ينزل فجأةً عند عتبات.

خطأ شائع عندها: تطبيق الجدول على أساس «الحقبة» بينما يُحدَّث كل «خطوة». فإنّ عدد الدفعات في الحقبة 100، فجدولٌ مضبوط على 10 حقب يحتاج إلى 1000 خطوة لا 10. تحقّق دائماً من وحدة القياس التي يضبطها المحسّن:

`scheduler.step()` في `PyTorch` يُستدعى بعد كل `optimizer.step()`، أي بعد كل دفعة.



الشكل 15-1: خمسة منحنيات لمعدّل التعلّم على مقياس لوغاريتمي (عمودي). المحور اللوغاريتمي مقصود: هو المقياس الذي يجعل المقارنة بين 0.01 و0.0001 واضحة. لاحظ أن المنحنى الجيبي يبقى مرتفعاً طويلاً ثم ينخفض فجأة قرب النهاية، بينما المنحنى الأُسّي يهبط مبكراً ويترك للدقة النهائية وقتاً أقل. المنحنى المتقطّع (الخط المنقّط) يمثّل سلوك `ReduceLROnPlateau`: لا ينخفض إلا بعد توقّف التقدّم.

الجدول	الشكل	متى تختاره	الخطأ الشائع
ثابت (`constant`)	خط أفقي	نماذج صغيرة، مسح سريع لمعدّل التعلّم، تجارب تخطيط	توقّف مبكر على دقّة دونها
أُسّي (`exponential`)	هبوط مضاعف	تدريبات طويلة جداً، أو عند ملاحظة توقّف مبكر	انهيار سريع للمعدّل قبل الأوان
درج (`step`)	هبوط مفاجئ عند عتبات	أبحاث كلاسيكية على `ImageNet`	عتبات موضوعة يدوياً بلا مبرّر
جيبي (`cosine`)	نصف موجة جيبية	الافتراضي المعاصر؛ ممتاز مع التسخين	استخدامه بلا مرحلة تسخين

الجدول	الشكل	متى تختاره	الخطأ الشائع
هضبة (`ReduceLROnPlateau`)	نزول عند التوقّف	حين لا تريد ضبط المعاملات الفوقية يدوياً	صبرٌ صغير جداً فيسبّب تذبذباً
تسخين ثم جيبي	صعود ثم نصف موجة	التدريب على دفعات كبيرة، النماذج الكبيرة، المحوّلات	تسخين طويل يُضيّع نصف التدريب

✦ لماذا «التسخين» (`warmup`) يعمل؟

في أوّل الخطوات تكون بعض الأوزان جديدة تماماً، وأجزاؤها لا تزال بلا إحصاءات مستقرّة؛ والنماذج التي فيها `BatchNorm` تُنتج قياسات متذبذبة. رفع معدّل التعلّم تدريجياً من قيمةٍ صغيرة جداً يعطي هذه الأجزاء وقتاً لتستقرّ قبل أن تتدفّق فيها خطواتٌ كبيرة. النتيجة العملية: استقرارٌ أفضل في الحقبة الأولى، وهو مكسبٌ مجاني بمجرّد جدولٍ بسيط.

ولأنّ شكل التسخين يختلف عن شكل الجيب، فإنّ أبسط طريقةٍ في `PyTorch` هي جدولٌ مخصّص دالّة: دالّة تُعيد معاملاً بين صفر وواحد يُضرب في معدّل التعلّم الأساسي. وبذلك تبني أيّ شكلٍ تريده: تسخين خطّي، ثم جيبي، ثم إيقاف تام.

Python

```python
import numpy as np
import torch
import torch.nn as nn

total_steps = 1000
warmup_steps = 50
base_lr = 3e-4


def lr_lambda(step: int) -> float:
    if step < warmup_steps:
        return (step + 1) / warmup_steps
    progress = (step - warmup_steps) / max(1, total_steps - warmup_steps)
    return 0.5 * (1.0 + np.cos(np.pi * progress))


model = nn.Linear(4, 2)
optimizer = torch.optim.AdamW(model.parameters(), lr=base_lr)
```

```
scheduler = torch.optim.lr_scheduler.LambdaLR(optimizer, lr_lambda)

observed = []
for _ in range(total_steps):
    optimizer.step()
    scheduler.step()
    observed.append(optimizer.param_groups[0]["lr"])

print("lr at the end of warmup:", f"{observed[warmup_steps - 1]:.6f}")
print("lr at the end of training:", f"{observed[-1]:.6f}")
```

ماذا يحدث هنا؟ الدالّة `lr_lambda` تُعيد نسبةً بين 0 و1، ويتكفّل `LambdaLR` بضربها في معدّل التعلّم الأساسي. **لماذا هذه القيمة؟** `warmup_steps = 50` هو 5٪ من ألف خطوة، وهي نسبةٌ شائعة للتسخين؛ فالقيم الأصغر قد تُبقي الشبكة في مرحلة عدم الاستقرار، والقيم الأكبر تُضيّع وقتاً ثمينة من التدريب الفعلي.

10.7 القياس: ما الذي نقيسه فعلاً؟

القياس في التدريب ليس خطوةً جانبية؛ إنه العقد الذي يقيس ما تدّعيه. وأول قاعدة: **لا تقس ما لا يهمّ**. الخسارة (loss) رقمٌ مفيد للمقارنة بين دفعاتٍ من النموذج نفسه، لكنها ليست المؤشّر الذي يعرضه مدير المنتج. لمصنّف احتيال، المؤشّر الذي يهمّ هو *معدّل الاكتشاف عند نسبة إنذارات كاذبة محدّدة*، لا دقّة إجمالية. للحساب على بيانات غير متوازنة بنسبة 1:1000، قد تُظهر الدقّة الإجمالية 99.9٪ بينما النموذج لا يكتشف ولا عملية احتيال واحدة.

القاعدة الثانية: **المتوسّط الصحيح**. متوسط الخسارة على دفعاتٍ مختلفة الأحجام خطأ شائع: حين تقسّ الخسارة في كل دفعة ثم تأخذ متوسّطاً بسيطاً، تُعطي دفعةً بـ16 عيّنة القدر نفسه الذي تعطيه دفعةً بـ256. الصيغة الصحيحة وزن كل دفعة بعدد عيّناتها:

$$L_{\text{epoch}} = \Sigma_b \, (n_b \cdot L_b) \, / \, \Sigma_b \, n_b$$

ما معنى كل رمز؟ L_b متوسط خسارة الدفعة b، و n_b عدد عيّناتها، والرمز Σ يدلّ على الجمع على كل الدفعات. المقام هو عدد العيّنات الكلي، فيعطي كل عيّنةٍ وزنها الحقيقي.

لماذا يهمّنا؟ لأنّ الفروق الصغيرة بين حقبتين متتاليتين هي بالضبط ما يقرّره `EarlyStopping` وجدول `ReduceLROnPlateau`. خطأ متوسطٍ غير موزون قد يُخفي انحداراً حقيقياً في أداء النموذج.

المؤشّر	ماذا يجيب؟	متى يكون هو الحكم النهائي؟
الخسارة (loss)	ما مدى سوء تنبّؤ النموذج رقمياً	أثناء التدريب لمراقبة التقارب فقط
الدقّة (accuracy)	نسبة الإجابات الصحيحة	تصنيف متوازن الأصناف
الدقّة المتوازنة	متوسّط الاستدعاء لكل صنف	تصنيف غير متوازن (احتيال، عيوب)
منحنى `PR` و `AP`	جودة الترتيب بين الحالات الإيجابية والسلبية	كشف الاحتيال وترتيب المخاطر
الإنذارات الكاذبة (`FPR`)	نسبة التنبيهات الخاطئة	أنظمة المراقبة حيث كلفة الإنذار مرتفعة
المساحة تحت المنحنى (`auc`)	قدرة الترتيب العام	مقارنة نماذج على بيانات غير متوازنة

> ⚙ **ممارسة: خطّ قياسٍ لمشروع احتيال**
>
> في نظام كشف احتيال مصرفي، حدّد ثلاثة مقاييس قبل أن تبدأ أي تجربة: (1) **معدّل الاكتشاف عند نسبة إنذارات كاذبة لا تتجاوز 0.5%** – لأنّ كلفة الحظر الكاذب لداءك مرتفعة؛ (2) **منحنى `PR`** على بيانات الشهر الأخير؛ (3) **زمن الاستدلال** لكل معاملة. القاعدة: لا يُعتمد نموذج في الإنتاج إلا إذا نجح في المقاييس الثلاثة معاً، ولكلّ منها سببٌ تشغيلي.

10.8 البذور العشوائية والحتمية: أن تعيد التجربة نفسها

في فضاء المعاملات الفسيح يحمل التدريب عشوائيةً من ثلاثة مصادر: ترتيب خلط البيانات، وقطر الشبكة العصبية، وعمليات التعزيز العشوائية. بدون ضبط هذه المصادر، فإنّ تشغيلين متطابقين ينتجان نموذجين مختلفين، ويصبح كلّ تحسينٍ غير قابلٍ للنسبة. الحلّ هو **ضبط البذرة** على كل المستويات معاً: `random` و `numpy` و `torch`، مع مولّدٍ منفصل لـ `DataLoader` حتى لا يتأثّر ترتيب الدفعات بتغييراتٍ في مواضع أخرى.

والحتمية الكاملة أشدّ صرامة: تفرض خوارزمياتٍ حتمية في `torch.use_deterministic_algorithms(True)`، وقد تكلّفك بعض الأداء، وقد ترفض بعض العمليات (مثل بعض خوارزميات الالتفاف ذات الترتيب العشوائي). القاعدة العملية: اجعل الحتمية *خياراً* في مرحلة التحقّق من النتيجة، وفعّلها في مرحلة الإنتاج فقط.

⚠ انتبه: البذرة وحدها لا تكفي

ضبط `torch.manual_seed(42)` لا يضمن أن يكون تشغيلان متطابقاً إذا كان `DataLoader` يستخدم عمّالاً متعدّدين (`multi-worker`) بلا بذرة محدّدة، أو إذا كان نموذجك يحتوي عملية عشوائية غير مسجّلة. امنح كل مولّدٍ بذرةً صريحة، ووثّق قائمة البذور في ملفّ الإعداد.

10.9 الحلقة المكافئة في Keras

فلسفة Keras مختلفة: هي لا تمنحك حلقةً تصنعها بنفسك، بل تمنحك `compile` و `fit` و**استدعاءات راجعة** (`callbacks`) تُلحق بالحَلقة. الفرق ليس شكلياً: في Keras يصبح «السلوك المخصّص» كائناً مستقلاً يُضاف أو يُزاح، فيصبح دمج أدواتك عمليّة تركيبية لا تآكل.

ولأنّ Keras 3 متعدّد النواة، فإنّ ما تراه في `fit` ينفَّذ على نواة `PyTorch` أو `JAX` أو `TensorFlow` بحسب الإعداد، ونفس الشيفرة تعمل على أيّها. اضبط النواة بمتغيّر البيئة `KERAS_BACKEND=torch` قبل استيراد `keras` .

PyTorch: أنت تملك الحلقة

حلقة `for` صريحة تُظهر كلّ خطوة: الترتيب، ومتى يُعاد التدرّج، ومتى يُقصّ، ومتى يُقاس. تحكّمٌ كامل، وثمنُه: كل هذه التفاصيل مسؤوليّتك أنت.

- تحكّم كامل في ترتيب العمليات ودقّتها
- سهل دمج أي عملية مخصّصة (المتوسّط المتحرك، اقتطاعات مخصّصة)
- مسؤوليتك: الحفظ، والسجلّات، والجدولة، والمقاييس
- ممتاز للبحث والتطوير والابتكار

Keras: الحلقة مورّدة

الحلقة مخبوزة داخل `fit` ، وأنت تُلحق بها `callbacks` تُستدعى في لحظاتٍ محدّدة: بداية الحقبة، ونهايتها، وبداية الدفعة.

- كودٌ أقصر بوضوح، ومعيارٌ واحد للتجارب
- مكتبة استدعاءات راجعة غنية: إيقاف مبكر، وتخفيض المعدّل، وحفظ، وسجلّ بصيغة csv

- يمكن وراثة الصنف لكتابة حلقة مخصّصة كاملة
- ممتاز للإنتاج السريع وللقراءة السريعة

المهمة	في PyTorch 2.14	في Keras 3.15
الدقة المختلطة	`torch.amp.autocast(device_type="cuda", dtype=torch.float16)` مع `torch.amp.GradScaler("cuda")`	`_global_policy("mixed_float16")` (يتولّى `keras` تدرّج الخسارة داخلياً)
قصّ التدرّج	`torch.nn.utils.clip_grad_norm_(params, max_norm=1.0)`	`AdamW(..., global_clipnorm=1.0)`
المتوسّط المتحرك	صنف `ModelEMA` تكتبه بنفسك	`se_ema=True, ema_momentum=0.99)` – مدمج في المحسّن
جدول معدّل التعلّم	`torch.optim.lr_scheduler.CosineAnnealingLR` أو `LambdaLR`	`zers.schedules.CosineDecay(..., warmup_steps=...)` أو `ReduceLROnPlateau`
الإيقاف المبكر	تُنفَّذ يدوياً داخل الحلقة	`cks.EarlyStopping(patience=..., restore_best_weights=True)`
نقاط الحفظ	`torch.save({...}, path)` مع كل الحالات	`callbacks.ModelCheckpoint(...)`
سجلّ السجلّات	تكتبه بنفسك أو عبر `TensorBoard`	`callbacks.CSVLogger("logs.csv")`
البذور	`random` و `numpy` و `torch.manual_seed`	`ras.utils.set_random_seed(seed)`

ولنُترجم ما سبق إلى شيفرة `Keras` واحدة تجمع الدقة المختلطة والجدول المدمج. لاحظ أنّ `Keras` يتولّى تدرّج الخسارة داخلياً عند تفعيل السياسة المختلطة، فلسنا بحاجة إلى `GradScaler` ولا إلى أيّ سطرٍ إضافي.

```python
```

```
import numpy as np
import keras

keras.mixed_precision.set_global_policy("mixed_float16")

rng = np.random.default_rng(5)
x_train = rng.normal(size=(1024, 20)).astype("float32")
y_train = (x_train[:, 0] * x_train[:, 1] > 0).astype("int64")
x_val = rng.normal(size=(256, 20)).astype("float32")
y_val = (x_val[:, 0] * x_val[:, 1] > 0).astype("int64")

schedule = keras.optimizers.schedules.CosineDecay(
    initial_learning_rate=1e-3,
    decay_steps=100,
    alpha=0.1,
    warmup_steps=10,
    warmup_target=1e-4,
)

model = keras.Sequential([
    keras.layers.Input(shape=(20,)),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(2),
])

model.compile(
    optimizer=keras.optimizers.AdamW(learning_rate=schedule, weight_decay=1e-4),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=[keras.metrics.SparseCategoricalAccuracy(name="acc")],
)

history = model.fit(
    x_train, y_train,
    validation_data=(x_val, y_val),
    epochs=10,
    batch_size=64,
    verbose=0,
)
print("final validation accuracy:", round(float(history.history["val_acc"][-1]), 4))
print("lr at the end of the schedule:", float(schedule(99)))
```

ماذا يحدث هنا؟ الجدول `CosineDecay` يُمرَّر مباشرةً إلى المحسّن فيتولّى `fit` تحديثه في كل خطوة. **لماذا هذه القيمة؟** `alpha=0.1` يحدد أن ينتهي الجدول عند عُشر معدّل التعلّم الابتدائي لا عند صفر.

فتبقى قدرة التعلّم قائمةً في نهاية التدريب؛ و `warmup_target=1e-4` هو معدّل التعلّم الذي تبدأ منه مرحلة التسخين.

10.10 مثال واقعي متكامل: خطّ تدريب نظام كشف الاحتيال

لنجمع ما سبق في مشروعٍ واحد من أوّله إلى آخره. **السيناريو:** بنك رقميّ يريد كشف المعاملات الاحتيالية. البيانات: أربعة ملايين معاملة، منها 0.4٪ محاولات احتيال. النموذج: شبكة `MLP` على 32 خاصية مُهندَسة (المبلغ، والوقت، وفئة التاجر، وسرعة إنفاق العميل، وسواها). القيود: تدريبٌ على بطاقة رسومية واحدة، والاستدلال يجب أن يستغرق أقل من 20 مللي ثانية لكل معاملة، والنتيجة يجب أن تكون قابلة لإعادة الإنتاج للمراجعة التنظيمية.

1. **تثبيت الهوية أولاً:** ملفّ إعدادٍ واحد يحمل البذرة (42)، ونسخة المكتبات، وبصمة مجموعة البيانات. أيّ تشغيلٍ لاحق يقرأ هذا الملف ولا يخمّن.

2. **تقسيم قبل أيّ معالجة:** التقسيم إلى تدريب وتحقّق واختبار *بالوقت* لا عشوائياً: ما قبل شهرٍ معيّن للتدريب، والشهر الأخير للاختبار، وما بينهما للتحقّق. التقسيم العشوائي في بيانات المعاملات يسرّب معلومات من المستقبل.

3. **حلقة التدريب:** محسّن `AdamW` بمعدّل تعلّم 3×10^{-4} وجدول تسخين ثم جيبي، مع الدقة المختلطة و `GradScaler`، وقصّ تدرّج عند 1.0، ومتوسّط متحرّك بمعامل 0.999.

4. **نقاط الحفظ:** نقطة في كل حقبة تحوي كل الحالات مع حالة المولّدات العشوائية، ونسخة الأفضل تُحفظ عند تحسّن منحنى `PR` على التحقّق.

5. **القياس:** منحنى `PR` ومنطقة `AP` بدقّة إجمالية، وسجلّ بصيغة csv لكل حقبة يُرفع إلى نظام التتبّع.

6. **قرار الانتقال للإنتاج:** يُعتمد نموذج المتوسّط المتحرك إذا نال `auc` لا يقلّ عن 0.55، وإنذاراتٍ كاذبة لا تتجاوز 0.5٪ عند معدّل اكتشاف لا يقلّ عن 0.70، وزمن استدلال دون 20 مللي ثانية. من لم يجتز الشروط الثلاثة لا يُنتشر، مهما بدا جيداً في الخسارة.

> ★ **الدرس التشغيلي**
>
> لاحظ أنّ قرار الإنتاج لم يُتخذ بناءً على الخسارة، بل على ثلاثة أرقام لكلٍّ منها مالكٌ في المؤسسة. هذه هي القيمة الحقيقية للحلقة الاحترافية: أن تُنتج *قرارات* قابلةً للمراجعة، لا مجرّد أوزانٍ على القرص.

10.11 تمارين للتثبيت على الكود

حوّل ما قرأته إلى ممارسة. كل تمرين هنا له حلّ نموذجي مطويّ أسفله؛ حاول أوّلاً أن تكتب الحل بنفسك ثم قارن.

تمرين 1 — ابنِ حلقة الدقة المختلطة كاملة

1. اكتب دالة

```
train_one_epoch(model, loader, optimizer, criterion, scaler, device, clip_value)
```

تُعيد متوسط الخسارة ودقّة التحقّق.

2. داخلها: `zero_grad` بـ `set_to_none=True`، ثم `autocast`، ثم `scale().backward()`، ثم `unscale_`، ثم القصّ، ثم `step`، ثم `update`.
3. تأكّد أنّ ترتيب `unscale_` قبل القصّ، وأضف طباعةً للمعيار العام للتدرّج.

▲ الحل النموذجي

الترتيب ليس اعتباطياً: `unscale_` يعيد التدرّجات إلى مقاسها الحقيقي، فتصبح قيمة `clip_grad_norm_` ذات معنى. لو قصصتَ قبل `unscale_` لقيّدتَ تدرّجاتٍ مُكبَّرة بمعامل 65536، أي أنّك قصصتَها إلى ما يقارب الصفر.

Python

```
import torch
import torch.nn as nn


def train_one_epoch(model, loader, optimizer, criterion, scaler, device, clip_value=1.0)
    model.train()
    loss_sum, correct, seen = 0.0, 0, 0
    for xb, yb in loader:
        xb = xb.to(device, non_blocking=True)
        yb = yb.to(device, non_blocking=True)
        optimizer.zero_grad(set_to_none=True)
        with torch.amp.autocast(device_type=device.type, dtype=torch.float16, enabled=de
            logits = model(xb)
            loss = criterion(logits, yb)
        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
        grad_norm = torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=clip_val
        scaler.step(optimizer)
        scaler.update()
        loss_sum += loss.item() * len(yb)
        correct += (logits.argmax(dim=1) == yb).sum().item()
        seen += len(yb)
    print(f"grad norm (last batch): {float(grad_norm):.4f}")
    return loss_sum / seen, correct / seen
```

لاحظ أنّ `correct` يُحسب من `logits` قبل التحديث، أي على الدفعة الحالية بأوزانها الحالية – وهذا هو السلوك الصحيح لقياس الحقبة.

تمرين 2 — صمّم المتوسّط المتحرك واختبر حسّاسيته

1. اكتب صنف `ModelEMA` يحتفظ بنسخة من أوزان النموذج ويحدّثها بمعامل `decay` في كل خطوة.
2. أضف الدالة `copy_to(model)` التي تنسخ أوزان المتوسّط إلى نموذجٍ آخر لتقييمه.
3. اختبر على نموذجٍ صغير: بعد مئة خطوة بتحديث الأوزان عشوائياً، قارن أوزان المتوسّط بأوزان النموذج. لماذا تختلف أقلّ مع معامل أكبر؟

▲ الحل النموذجي

Python

```python
from copy import deepcopy

import torch
import torch.nn as nn


class ModelEMA:
    def __init__(self, model: nn.Module, decay: float = 0.999):
        self.decay = decay
        self.module = deepcopy(model).eval()
        for param in self.module.parameters():
            param.requires_grad_(False)

    @torch.no_grad()
    def update(self, model: nn.Module) -> None:
        for ema_value, model_value in zip(self.module.state_dict().values(), model.state
            if ema_value.dtype.is_floating_point:
                ema_value.mul_(self.decay).add_(model_value, alpha=1.0 - self.decay)
            else:
                ema_value.copy_(model_value)

    def copy_to(self, model: nn.Module) -> None:
        model.load_state_dict(self.module.state_dict(), strict=True)


model = nn.Linear(4, 2)
ema = ModelEMA(model, decay=0.99)
for _ in range(50):
    with torch.no_grad():
        model.weight.add_(torch.randn_like(model.weight) * 0.1)
    ema.update(model)

distance = float((model.weight - ema.module.weight).norm())
print(f"distance between live and EMA weights: {distance:.4f}")
```

كلّما كان `decay` أكبر، اتّسع النافذة التي يُنظر فيها إلى الماضي، فاقتربت نسخة المتوسّط من «متوسّط المسار كلّه» بدل أن تلاحق آخر خطوة. ومع ذلك تبقى أقرب إلى النموذج من أيّ متوسطٍ

بعيد، لأنّ الحدّ الأحدث يدخل دائماً بمعامل `1 - decay`.

تمرين 3 — حدّد مواضع الخلل في حلقةٍ كسورة

1. اقرأ حلقة التدريب الموضّحة أدناه وحدّد ثلاثة أخطاء تمنعها من العمل بشكل صحيح: `optimizer.zero_grad()` في نهاية الحلقة، و `scaler.step(optimizer)` قبل `scaler.unscale_(optimizer)`، و `scheduler.step()` مرّةً واحدة في الحقبة.
2. أصلحها واشرح أثر كل إصلاح على سرعة التقارب.

الحل النموذجي

الأخطاء الثلاثة: (1) `zero_grad` في *نهاية* الحقبة يعني أنّ التدرّجات تتراكم عبر الحقبة كلّها بدل أن تُصفَّر لكل دفعة، والنتيجة تدرّجٌ ضخم ثم انفجار. (2) `step` قبل `unscale_` يعني أنّ القصّ يعمل على تدرّجاتٍ مُكبَّرة بمعامل المقياس. (3) `scheduler.step()` مرّةً في الحقبة يبطئ الجدول بمعامل عدد الدفعات، فلا يصل إلى نهايته أبداً.

والشيفرة الصحيحة (وهي مقطع من حلقةٍ أطول ذُكر في نصّ التمرين) هي:

```python
for xb, yb in loader:
    optimizer.zero_grad(set_to_none=True)
    with torch.amp.autocast(device_type="cuda", dtype=torch.float16):
        loss = criterion(model(xb), yb)
    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    scaler.step(optimizer)
    scaler.update()
    scheduler.step()
```

تمرين 4 — قارن حلقة Keras بحلقة PyTorch

1. درّب النموذج نفسه في Keras 3 (نواة `PyTorch`) مع `ReduceLROnPlateau` و `EarlyStopping`.
2. فعّل `use_ema=True` في `AdamW` وقارن أداء النموذج بعد التدريب مع حالةٍ بلا متوسّط متحرّك.
3. أيّ مكوّن في Keras يقابل يدوياً صنف `ModelEMA` في `PyTorch`، وأيّ مكوّن يقابل `ReduceLROnPlateau`؟

▲ الحل النموذجي

المقابلة: `keras.optimizers.AdamW(use_ema=True, ema_momentum=0.99)` هو نظير `ModelEMA`. و `ReduceLROnPlateau` نظير الجدول اليدوي المبنيّ على مراقبة مؤشّر. لاحظ فارقاً جوهرياً: المتوسّط المتحرك في Keras مدمج في المحسّن نفسه، أي أنّه يتبع الأوزان تلقائياً دون سطرٍ إضافي، وفي `PyTorch` تحتاج إلى استدعاء `update()` يدوياً بعد كل خطوة – وإهمال هذا الاستدعاء خطأٌ شائع يجعل النسخة تتجمّد عند قيمتها الأولى.

Python

```
import numpy as np
import keras

rng = np.random.default_rng(11)
x_train = rng.normal(size=(2048, 32)).astype("float32")
y_train = (x_train[:, 0] + 0.5 * x_train[:, 3] > 0).astype("int64")
x_val = rng.normal(size=(512, 32)).astype("float32")
y_val = (x_val[:, 0] + 0.5 * x_val[:, 3] > 0).astype("int64")

model = keras.Sequential([
    keras.layers.Input(shape=(32,)),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(2),
])

model.compile(
    optimizer=keras.optimizers.AdamW(
        learning_rate=1e-3, weight_decay=1e-4, global_clipnorm=1.0, use_ema=True, ema_mo
    ),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=[keras.metrics.SparseCategoricalAccuracy(name="acc")],
)

callbacks = [
    keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=2, min_lr
    keras.callbacks.EarlyStopping(monitor="val_loss", patience=5, restore_best_weights=T
    keras.callbacks.CSVLogger("logs.csv"),
]

history = model.fit(
    x_train, y_train,
```

```
    validation_data=(x_val, y_val),
    epochs=20, batch_size=128,
    callbacks=callbacks,
    verbose=2,
)
print("best validation loss:", min(history.history["val_loss"]))
```

لو كانت الخسارة تنخفض بسلاسة لكنّ دقّة التحقّق تتذبذب صعوداً وهبوطاً، فأيّ أدوات هذا الفصل تُشخّص الحالة الأسرع؟ ولماذا لا يكفي أن تنظر إلى منحنى الخسارة وحده؟

ملخّص الفصل العاشر

- الحلقة الاحترافية تجمع أربع صفات: دقّةَ حساب، واستقراراً، وقابليّةَ استئناف، وقياساً منضبطاً.
- الدقة المختلطة تُسرّع العمليات الثقيلة في `float16` مع إبقاء التراكم في `float32`، ويحمي `GradScaler` من الانهيار العددي بسبب صغر التدرّجات.
- مقياس التدرّج يضرب الخسارة في `s` قبل الانتشار العكسي، و `unscale_` يعيد التدرّجات إلى مقاسها الحقيقي قبل القصّ والتحديث.
- قصّ التدرّج على المعيار العام يحوّل الانفجار من نهايةٍ قاتلة إلى زحلةٍ آمنة.
- المتوسّط المتحرك الأسّي يمنحك نسخةً أهدأ من الأوزان، وهي غالباً أفضل من آخر خطوة؛ ومعامل 0.999 هو نقطة البداية المعقولة.
- نقطة الحفظ الكاملة تحوي أوزان النموذج وحالة المحسّن والمقياس والجدول وحالات المولّدات العشوائية، وتُكتب ذرّياً.
- جدول التعلّم الجيبي مع التسخين هو الخيار الافتراضي المعاصر؛ و `scheduler.step()` يُستدعى بعد كل دفعة لا بعد كل حقبة.
- المتوسّط الصحيح للقياس هو المرجّح بعدد العيّنات، ومؤشّر الإنتاج قد يختلف تماماً عن دالّة الخسارة.
- ضبط البذور على كل المستويات، بما فيها مولّد `DataLoader`، هو ما يجعل التجربة قابلة للتكرار.
- في Keras 3 يقابل `use_ema=True` صنف `ModelEMA`، ويقابل `global_clipnorm` استدعاء `clip_grad_norm_`.

التنظيم والتعميم

التهيئة، Dropout، Weight Decay، التطبيع، Mixup، وحدود النموذج

النموذج الذي يحفظ إجابات مجموعة التدريب حرفياً لا يعرف شيئاً. التعلّم العميق/ الاستنتاج (overfitting) هو أخطر سلوك يمكنك أن تراه في مشروع حقيقي، لأنه لا يُصدر أي رسالة خطأ: الخسارة تنخفض، والواجهة تبدو ممتازة، والنتائج في الإنتاج مخيبة. هذا الفصل عن الأدوات التي تمنعه.

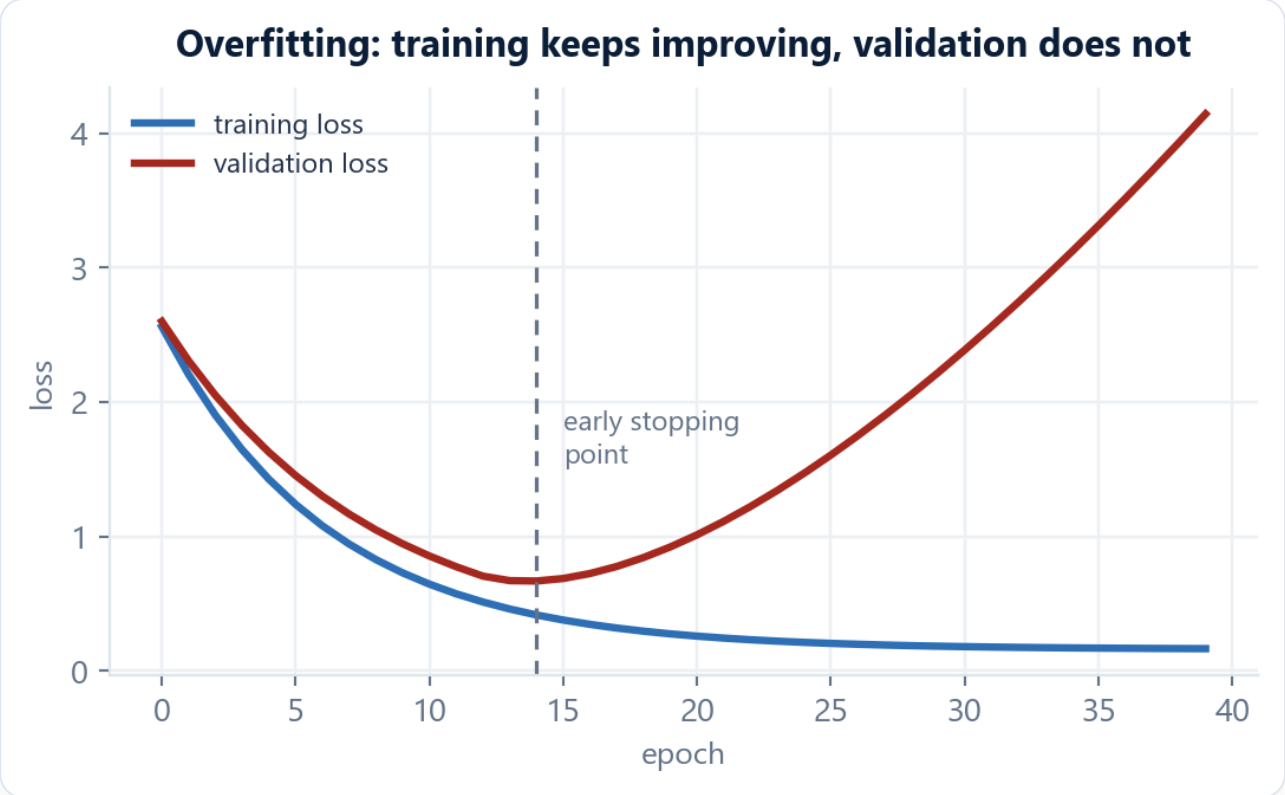
ما الذي يعنيه الإفراط في المواءمة؟

$$\mathrm{E}[(\mathrm{f}(\mathrm{x}) - \mathrm{y})^2] = \mathrm{Bias}^2 + \mathrm{Variance} + \sigma^2$$

المعادلة (11.1): تحليل التباين-التحيّز. الخطأ = خطأ النظام (تحيّز) + حساسية النموذج لاختلاف البيانات (تباين) + ضجيج لا يمكن إزالته. التدريب يخفض التحيّز دائماً؛ هو ضبط التباين دون أن يرتفع التحيّز.

الحالة	تدريب	تحقّق	العلاج
تحيّز عالٍ (إفراط في التبسيط)	خسارة عالية	خسارة عالية	نموذج أكبر، أطول، بيانات أفضل
تباين عالٍ (إفراط في المواءمة)	خسارة منخفضة	خسارة مرتفعة	تنظيم + بيانات + توقف مبكر
التوازن	منخفضة	قريبة منها	بسيط

قراءة منحنى التعلّم: ماذا تقول الأرقام؟



الشكل 11.1: المنحنى الكلاسيكي للإفراط في المواءمة. النقطة التي يبدأ بعدها فارق التدريب والتحقّق بالاتساع هي **نقطة الإيقاف المثلى** — وكل أداة في هذا الفصل تساعدك على الوصول إليها أو البقاء قربها.

> إذا كانت خسارة التدريب مرتفعة أيضاً، فهل المشكلة في التنظيم؟

لا – وهذا تمييز مهمّ: الإفراط في المواءمة يعني أن **التدريب أفضل من التحقّق**. إن كان الاثنان سيّئين، فالمشكلة **إفراط في التحيّز**: النموذج أضعف من اللازم، والانتظار لن يسلّم. الحلّ هناك: نموذج أكبر، أو بيانات أكثر، أو هندسة أفضل – وليس المزيد من Dropout.

الأدوات الأساسية: أربع مفاتيح

الأداة	ماذا تفعل بالضبط	متى تستعملها	تحذير
Weight decay (L2)	يضيف عقوبة على حجم الأوزان ⇒ يمنع تضخيم المعاملات	دائماً تقريباً (0.01- 0.1)	لا يطبَّق على BatchNorm وbias عادةً

الأداة	ماذا تفعل بالضبط	متى تستعملها	تحذير
Dropout	يصفّر نسبة من الوحدات أثناء التدريب ⇒ يمنع الاعتماد المفرط	الشبكات ذات الوحدات الكثيرة (MLP)	ضعيف مع CNN العميقة؛ ويلزمه `model.eval()`
Data augmentation	يوسّع بيانات التدريب بتعديلات تحاكي التغيّرات الحقيقية	دائماً في الرؤية	يجب أن يعكس التغيّرات الحقيقية فعلاً
Early stopping	يوقف التدريب عند بدا التحسّن يتوقف	كل نموذج	يتطلّب مجموعة تحقّق غير مسمّاة

أدوات التنظيم الأربع الأساسية

وزن التطبيع (Weight decay) و L2

$$L_{total} = L_{data} + \lambda \|W\|^2$$

المعادلة (11.2): عقوبة L2. المعادلة (11.3) (L1) تستخدم مجموع القيم المطلقة وتنتج أوزاناً متباعدة (sparse). الفرق العملي: L2 يفضّل توزيعاً صغيراً للقيم، وL1 يصفّر معاملات بالكامل.

Python

```python
import torch
from torch import nn

# 1) الطريقة الصحيحة: مجموعتا معاملات ( decaying بلا decay للمصفوفات، وdecay للـbias/BN)
def param_groups(model, weight_decay=0.01):
    decay, no_decay = [], []
    for name, p in model.named_parameters():
        if not p.requires_grad:
            continue
        if p.ndim <= 1 or name.endswith(".bias") or "bn" in name.lower():
            no_decay.append(p)
        else:
            decay.append(p)
    return [
        {"params": decay, "weight_decay": weight_decay},
        {"params": no_decay, "weight_decay": 0.0},
```

```
    ]

model = nn.Sequential(nn.Linear(784, 256), nn.ReLU(), nn.BatchNorm1d(256),
                      nn.Linear(256, 10))
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.01)
# أو الأدق:
opt = torch.optim.AdamW(param_groups(model, 0.01), lr=1e-3)

# 2) L2 صريح (بديل عن weight decay)
l2 = nn.MSELoss() + 1e-4 * sum(p.pow(2).sum() for p in model.parameters() if p.requires_grad)
```

⚠ لماذا يستثني weight decay الـbias وBatchNorm؟

عقوبة L2 تدفع المعاملات نحو الصفر. لكن: **(1)** الbias لا يقيس «حجم» بالمعنى نفسه – دفعه إلى الصفر يعني حذفه؛ **(2)** معاملات BatchNorm (γ وβ) **معيارية** (γ = 1) – تقليصها يبطئ الشبكة بلا فائدة. لهذا تُستثنى عادةً. النسخة «AdamW» تفصل decay عن التدرّج، وهو سلوك أنظف.

Dropout: ما يفعله فعلاً

أثناء التدريب، يُحذف كل وحدة عشوائياً **بحتمال p**، فيضطرّ النموذج أن يوزّع «المعرفة» على وحدات متعدّدة بدل الاعتماد على واحدة. أثناء الاستدلال تُستخدم كل الوحدات.

Python

```
model = nn.Sequential(
    nn.Linear(784, 512), nn.ReLU(), nn.Dropout(0.5),
    nn.Linear(512, 256), nn.ReLU(), nn.Dropout(0.3),
    nn.Linear(256, 10),
)

# أثناء التدريب: model.train()  ⇒ dropout فعّال
# أثناء التقييم: model.eval()   ⇒ dropout معطّل
# measurements: فعّل/عطّل تلقائياً
model.train()
out_train = model(x_train[:8])
model.eval()
with torch.no_grad():
    out_eval = model(x_train[:8])
print("different outputs:", not torch.allclose(out_train, out_eval))
```

✦ Dropout مقابل BatchNorm: التوافق بينهما ضروري

اكتشفت البحوث أنّ **Dropout بعد BatchNorm** (أو في وجوده) يسبّب مشكلة: أثناء التدريب تُصفّر وحدات، فيتغيّر توزيع إحصاءات الـBatchNorm؛ ثم يُطبَّع في الاستدلال بإحصاءات مختلفة. الحلّ: ضع Dropout **قبل** BatchNorm، أو استخدم **Dropout2d** في CNN (يصفّر قنوات كاملة بدل عناصر مفردة).

التعزيز: التنظيم الأكثر فعّالية في الرؤية

التعزيز (augmentation) هو **أقوى أداة تنظيم في رؤية الحاسوبية**، لأنّه لا يضيف افتراضاً (bias) على النموذج، بل يوسّع دعم مجموعة البيانات. القاعدة: طبّق التعزيزات التي تعكس التغيّرات التي ستحدث فعلاً في الإنتاج، ولا أكثر.

التعزيز	متى يناسب	متى يضرّ
RandomResizedCrop	تصنيف عامّ	إذا كان موضع الكائن جوهرياً (عيوب)
HorizontalFlip	أغلب المهام	النصوص، الأشعة، بعض الأشكال
ColorJitter	صور طبيعية	أشعة، صور تعتمد على اللون
Rotation / Affine	زوايا متغيّرة	مستندات (اتجاه النص مهمّ)
Mixup / CutMix	تحسين التعميم	أحادية الفئة، أو تفسير طبي
Label smoothing	تصنيف متعدّد الأصناف	مهام تحتاج ثقة حادّة (أمن)

تعزيزات شائعة ومدى ملاءمتها

Python

```python
# Mixup: يخلط صورتين وأهدافهما — تنظيم قوي جداً بلا معاملات إضافية
def mixup(x, y, alpha=0.2):
    lam = float(np.random.beta(alpha, alpha))
    idx = torch.randperm(x.size(0))
    x_mix = lam * x + (1 - lam) * x[idx]
    y_a, y_b = y, y[idx]
    return x_mix, y_a, y_b, lam
```

```
# في حلقة التدريب:
# loss = lam * lossf(out, y_a) + (1 - lam) * lossf(out, y_b)

# Label smoothing: نخفّف التسميات الصعبة
lossf = nn.CrossEntropyLoss(label_smoothing=0.1)
```

التطبيع كصورة من صور التنظيم

BatchNorm لا يقيّد الأوزان، لكنه يغيّر **حجم التدرّج** ويجعل الشبكة أقلّ حساسية للتهيئة. ولهذا يُعتبر «تنظيماً ضمنياً» في الأدبيات.

Python

```
# الفكرة: طبّع ثم أعد القياس بمعاملَين قابلَين للتعلّم
class LayerNorm(nn.Module):
    def __init__(self, d, eps=1e-6):
        super().__init__()
        self.gamma = nn.Parameter(torch.ones(d))
        self.beta = nn.Parameter(torch.zeros(d))
        self.eps = eps

    def forward(self, x):
        mu = x.mean(dim=-1, keepdim=True)
        var = x.var(dim=-1, keepdim=True, unbiased=False)
        x_hat = (x - mu) / torch.sqrt(var + self.eps)
        return self.gamma * x_hat + self.beta
```

★ قاعدة عملية للتسلسل

الترتيب القياسي في كتلة CNN: **Conv** → **BatchNorm** → **Activation** (وليس Conv → Activation → BN). السبب: التطبيع قبل التفعيل يجعل الدالة غير الخطية «أعرض» وثابتة الإحصاءات.

الإنذار المبكر والتوقّف في ذروته

Python

```
from torch.optim.lr_scheduler import ReduceLROnPlateau

scheduler = ReduceLROnPlateau(optimizer, mode="min", factor=0.5, patience=3, verbose=True)
```

```
best_val = float("inf")
patience_counter = 0
for epoch in range(100):
    model.train()
    for xb, yb in train_loader:
        opt.zero_grad(set_to_none=True)
        loss = lossf(model(xb), yb)
        loss.backward()
        opt.step()

    model.eval()
    with torch.no_grad():
        val_loss = sum(lossf(model(xb), yb).item() for xb, yb in val_loader) / len(val_loader

    scheduler.step(val_loss)

    if val_loss < best_val - 1e-4:
        best_val = val_loss
        patience_counter = 0
        torch.save(model.state_dict(), "best.pt")
    else:
        patience_counter += 1
        if patience_counter >= 8:          # صبر (patience) = 8 حقبات
            print(f"early stop at epoch {epoch}")
            break

model.load_state_dict(torch.load("best.pt"))    # استرجع أفضل الأوزان
```

تمرين 1 — ابحث عن أفضل توازن

1. درب MLP على `make_moons` بأربعة إعدادات: (أ) بلا تنظيم، (ب) Dropout 0.5، (ج) weight decay 0.01، (د) كلاهما + إنذار مبكر.
2. ارسم منحنيات التدريب والتحقّق الأربعة.
3. حدّد: أيّ إعداد أعطى أفضل توازن؟ ومتى بدأ الإفراط في المواءمة؟
4. أعد التجربة على بيانات أصغر (300 عيّنة) وقارن.

▲ الحل النموذجي

Python

```python
import torch
from torch import nn
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

X, y = make_moons(n_samples=1000, noise=0.25, random_state=0)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42)
Xtr = torch.tensor(Xtr, dtype=torch.float32); ytr = torch.tensor(ytr, dtype=torch.long)
Xte = torch.tensor(Xte, dtype=torch.float32); yte = torch.tensor(yte, dtype=torch.long)

def run(dropout=0.0, wd=0.0, epochs=200, patience=10):
    torch.manual_seed(0)
    model = nn.Sequential(nn.Linear(2, 64), nn.ReLU(), nn.Dropout(dropout),
                          nn.Linear(64, 64), nn.ReLU(), nn.Dropout(dropout),
                          nn.Linear(64, 2))
    opt = torch.optim.AdamW(model.parameters(), lr=1e-2, weight_decay=wd)
    lossf = nn.CrossEntropyLoss()
    best, bad, best_state = 1e9, 0, None
    for ep in range(epochs):
        model.train()
        opt.zero_grad()
        lossf(model(Xtr), ytr).backward()
        opt.step()
        model.eval()
        with torch.no_grad():
            vl = lossf(model(Xte), yte).item()
        if vl < best - 1e-4:
            best, bad = vl, 0
            best_state = {k: v.clone() for k, v in model.state_dict().items()}
        else:
            bad += 1
            if bad >= patience:
                break
    model.load_state_dict(best_state)
    model.eval()
    with torch.no_grad():
        acc = (model(Xte).argmax(1) == yte).float().mean().item()
    return best, acc, ep
```

```
for name, kw in (("بلا تنظيم", {}), ("dropout", {"dropout": 0.5}),
                 ("wd", {"wd": 0.01}), ("كلاهما", {"dropout": 0.4, "wd": 0.01})):
    loss, acc, ep = run(**kw)
    print(f"{name:12s} val_loss={loss:.4f} acc={acc:.4f} stopped@{ep}")
```

القراءة المتوقّعة: بلا تنظيم ينخفض خطأ التدريب بسرعة وينفصل التحقّق (إفراط في المواءمة) ويتوقّف التدريب مبكراً بعد أن يسيء Training. مع Dropout وweight decay يبقى الفارق ضيّقاً وتتحسّن دقّة التحقّق. «كلاهما» غالباً الأفضل لأنّ الأداتين تعالجان نوعين مختلفين من: التحسين (الضجيج) والعقد (الحجم).

تمرين 2 — ابحث عن تسريب عبر التعزيز

1. درّب مصنّفاً على صور رقمية (MNIST) مع `RandomRotation(0.3)`.
2. قِس دقّة التحقّق، ثم أظهر أمثلة ملوّنة يدوياً.
3. هل نتجت «أشكال» لم تكن موجودة في البيانات؟
4. اقترح تعديلاً: كيف نحافظ على فائدة الدوران دون إنتاج صور غير واقعية؟

▲ الحل النموذجي

التدوير بزاوية كبيرة على أرقام مكتوبة بخط اليد ينتج «أرقام» مائلة أو مقطوعة، وقد يدخل النموذج في منطقة توزيعية لم يرها في البيانات الحقيقية. النتيجة: **تحسّن في التحقّق؟** – إن كانت التحقّق لا تدور، فالدوران ضارّ.

الحلّ: استخدم زوايا أصغر (0.1-0.15)، أو طبّق الدوران فقط على مهمّة يكون فيها الدوران واقعياً (صور جوية)، أو استخدم `RandomAffine` بنطاق محدود مع حشو ثابت (لا تمدّد). القاعدة العامة: **التعزيز الذي يولّد بيانات خارج التوزيع يُضِرّ لا يُنفع.**

ضبط التدريب: من النموذج إلى الإنتاج

في الفصل العاشر رسمنا حلقة تدريب كاملة، لكنّ الحلقة ليست المنتج. النموذج الذي ينهي حقبته الأخيرة ليس نموذجاً جاهزاً للاستخدام: هو نتيجة تدريب، لا نتيجة هندسة. هذا الفصل يتناول ما يحدث بين آخر حقبة والإطلاق: التقييم الصادق، وحفظ النموذج، وتثبيت البيئة.

★ ثلاثة أسئلة قبل أن تقول «انتهى»

الأول: على أيّ بيانات قِست؟ إن كانت على بيانات دخلت التدريب أو نُظّرت إليها أثناء ضبط المعاملات الفائقة، فأنت لا تملك قياساً بل انطباعاً. **الثاني:** هل النموذج مجمّد بإصدار من الملفWeights، أم أنّ التدريب مستمرّ خلف الكواليس؟ **الثالث:** هل تعمل الشيفرة كاملة على جهاز نظيف؟ اختبار القبول الوحيد الذي يهمّ هو «بيئة جديدة، نفس المدخلات، نفس المخرجات».

تثبيت البيئة: ملفّ واحد يحميك من كابوس

أشهر سبب لفشل «نفس الشيفرة تعمل عندي» ليس الرياضيات بل إدارة الاعتماديات. الحلّ ليس تذكّر أرقام الإصدارات، بل **قفلها**.

Python

```
import hashlib
import json
import platform
import subprocess
from pathlib import Path


def environment_fingerprint() -> dict:
    """بصمة البيئة: تُحفظ بجانب النموذج للتحقّق من التطابق."""
    import torch
    import keras

    result = subprocess.run(
        ["pip", "freeze"], capture_output=True, text=True, check=False
    )
    packages = sorted(result.stdout.split())
    digest = hashlib.sha256("\n".join(packages).encode()).hexdigest()

    return {
        "python": platform.python_version(),
        "platform": platform.platform(),
        "torch": torch.__version__,
        "keras": keras.__version__,
        "keras_backend": keras.backend.backend(),
        "packages_sha256": digest,
        "packages": packages,
    }
```

```
def save_fingerprint(path: str = "env.json") -> None:
    Path(path).write_text(
        json.dumps(environment_fingerprint(), ensure_ascii=False, indent=2),
        encoding="utf-8",
    )


if __name__ == "__main__":
    save_fingerprint()
    print(json.dumps(environment_fingerprint(), ensure_ascii=False, indent=2))
```

ماذا يحدث هنا؟ البصمة تُخزَّن بجانب الملفWeights، فتقارنها بيوم النشر فترى فوراً أيّ اختلاف. **لماذا هذه القيمة؟** لأنّ «بيئتي» ليست دليلاً: البصمة الدلالية (`hash`) قابلة للتحقّق آلياً في خطّ النشر.

> نموذج تصنيف دقّته ٩٣٪ على مجموعة التحقّق. بعد أسبوع من الإطلاق في الإنتاج أبلغت تقارير الـ `callbacks` عن دقّة لا تتجاوز ٦٢٪. الفارق: لم يتغيّر الكود، ولم يتغيّر البيانات. ما ثلاث احتمالات يجب أن تفحصها بالترتيب، ولماذا فحص البيئة أولاً؟

الترتيب الذي نرشّده: **أولاً** قارن بصمة البيئة (هل تغيّرت نسخة حزمة؟). **ثانياً** قارن توزيع المدخلات (هل تغيّرت البيانات نفسها؟ فجأة تغيّر توزيع الفئات؟). **ثالثاً** راجع خطّ المعالجة المسبقة (هل تغيّر ترتيب القنوات أو المقاس؟). نبدأ بالبيئة لأنها أرخص فحص وأقلّه كلفة: دقيقة واحدة مقارنةً بيوم في تحليل البيانات.

قياس النموذج: من دقّة واحدة إلى لوحة قرارات

الرقم الوحيد الذي يخرج به معظم المبتدئين من التدريب هو دقّة التحقّق. وفي المشاريع الحقيقية هذا الرقم ناقص، بل قد يكون مضلّلاً. النموذج الذي يحقّق ٩٢٪ على التحقّق قد يكون أفضل أو أسوأ بكثير من نموذج يحقّق ٨٩٪، حسب طبيعة الخطأ الذي يرتكبه.

الفصل التالي؟ ليس هنا. الفصل التالي في هذا الكتاب هو Keras، لكنّ هذا المبدأ – **لا تقبل رقماً واحداً** – لا يخصّ Keras. ما تحتاجه هو ثلاث دقائق إضافية لبناء مصفوفة الالتباس (confusion matrix) بعد كلّ مرحلة، ومقارنة الأعمدة لا الأرقام الإجمالية.

نظام كشف عيوب صناعي: دقّة التحقّق ٩٤٪. المصفوفة: ٤٠ خطأ من ١٠٠ عيّنة معيبة، و٤ أخطاء من ٤٦٠٠ عيّنة سليمة. أيّهما أسوأ: ٩٤٪ من العيّنات المعيبة فوتت، أم ٠٫١٪ من السليمة إنذاراً كاذباً؟ وما القرار التشغيلي الذي يفرضه كلّ رقم؟

جوابنا: **الخطأ في العيّنات المعيبة هو الأخطر**، لأنّ فوت عيب في خطّ إنتاج يعني منتجاً معيباً يصل إلى العميل. أمّا الإنذار الكاذب فكلّفته وقت الفحص فقط. وبهذا المعنى تكون «دقّة» ٩٤٪ مضلّلة: الرقم المُريح يخفي أنّ ثلثي العيوب يمرّ.

والقرار التشغيلي واضح: إن كانت كلفة الفوت أهمّ من كلفة الإنذار الكاذب (وهو حال معظم تطبيقات السلامة)، نقيس **الاستدعاء** (recall) على صنف العيب ونضبط عتبة التنبيه على أساسه. أمّا في أنظمة توصية فالعكس صحيح: هناك فوت المعلومة أرخص من الإزعاج.

✦ لماذا لا نكتفي بالدقّة أبداً؟

لأنّ الدقّة مقياس يُخفّي التوزيع. في قاعدة فيها ٩٩٪ من الحالات سليمة، يُعطي نموذجُ ٩٩٪ دقّة وهو يتوقّع «سليم» دائماً. ولهذا في كلّ مشروع حقيقي تحتاج على الأقلّ: الدقّة، والاستدعاء، والدقة (precision)، ومنحنى ROC أو PR، و**دقّة كلّ فئة على حدة**. القراءة الصحيحة تبدأ من السطر الأخير.

ملخّص الفصل الحادي عشر

- اقرأ المنحنيات: training «أفضل» من validation = إفراط في المواءمة.
- إن كان الاثنان سيّئين ⇐ إفراط في التحيّز، والحلّ مُنظِّم انتظامٍ أقوى.
- أربع أدوات: early stopping، augmentation، dropout، weight decay.
- استثنِ BatchNorm و bias من weight decay.
- Dropout بعد BatchNorm مشكلة؛ استخدم Dropout2d في CNN.
- التعزيز أقوى تنظيم في الرؤية، بشرط واقعية التحويلات.
- الترتيب: Conv → BN → Activation.
- Label Smoothing و Mixup أقوى من Dropout في التعميم العام.

الشبكات التلافيفية CNN

الأنوية، الالتفاف، التجميع، الحقل الاستقبالي، وبنى ResNet

تخيّل خطّ إنتاج في مصنع قطع غيار: كلّما مرّت قطعة أمام كاميرا مثبّتة فوق الناقل، يجب أن يقرّر الحاسوب خلال أجزاء من الثانية هل القطعة سليمة أم تحمل عيباً، ثم يوجّه الذراع الميكانيكية لتفريغها. هذه المهمة لا تُحلّ بشبكة عصبية كثيفة واحدة: صورة بمقاس 224×224 وبثلاث قنوات ملوّنة تحمل 150528 رقماً، فحتى طبقة كثيفة واحدة بمئة مخرج تحتاج نحو 150 مليون معامل، أي ما يشغل 600 ميغابايت من الذاكرة لمجرّد طبقة واحدة. الحلّ الذي صنع التحوّل في رؤية الحاسوب لم يكن خوارزميّة أفضل، بل تغييراً في الفكرة: أن نتعلّم أنماطاً صغيرة تتكرّر في كل مكان، ثم نُركّب منها تمثيلاً كبيراً. هذا الفصل يشرح تلك الفكرة من أنويتها، ويبني عليها شبكة تصنيف كاملة، ويصل إلى `ResNet` التي لا تزال العمود الفقري لأكثر مشاريع الرؤية استخداماً.

12.1 لماذا لا تصلح الشبكة الكثيفة للصور؟

لنبدأ بالرقم قبل الفكرة. الصورة `224×224×3` تحتوي على 150528 قيمة. اجعلها مُدخَلاً لطبقة `Dense` فيها 1000 وحدة إخراج: عدد المعاملات يساوي 150528 × 1000، أي 150 مليون معامل تقريباً. تخزَّن هذه المعاملات في خانات `float32` فيحتاجWeights طبقة واحدة إلى 600 ميغابايت. جرّب أن تُضيف عشر طبقات من هذا النوع وستتوقّف عن الذاكرة قبل أن تتوقّف عن التعلّم.

المشكلة إذاً ليست في حجم الصورة وحده، بل في **غياب مبادئين** تفتقدهما الشبكة الكثيفة: أوّلهما انزلاق النواة على الصورة بدل معامل واحد لكل موضع، وثانيهما مشاركة الأوزان: النواة نفسها تُطبَّق على كل جزء من الصورة. وبفضل هذين المبدأين يتحوّل عدد المعاملات من دالّة في حجم الصورة إلى دالّة في حجم النواة فقط. نواة `3×3` بثلاث قنوات تُنتج 64 خريطة ميزات تحتاج 3×3×3×64 = 1728 معاملاً، أي أصغر بحوالي 87000 مرة من الطبقة الكثيفة المكافئة. والمفارقة أن هذه الشبكة الأصغر بكثير هي الأقدر على التعرّف.

المبدأ الثاني أعمق من مجرّد توفير في المعاملات: لأن النواة مشتركة، فالنموذج يتعلّم «كيف يبدو الحافة» مرة واحدة، ثم يطبّقها على كل حافة في الصورة مهما كان موضعها. لو زُحزحت القطة من زاوية الصورة

إلى وسطها، فإن الطبقة الكثيفة سترى مُدخَلاً مختلفاً تماماً فتحتاج بيانات جديدة. أما الشبكة التلافيفية فستُخرج خريطة ميزات متطابقة تقريباً في الموضعين. هذه الخاصية تُسمّى **الانقلاب الموضعي** (`translation equivariance`)، وهي السبب المباشروراء قابلية الشبكات التلافيفية للتعميم على ما لم تره من أوضاع.

> افترض أنّ لديك 200 صورة لأربعة أصناف من عيوب المعدّن، ووقتك يسمح بتدريب مرّة واحدة فقط. هل تختار شبكة كثيفة بأربعة مخرجات، أم شبكة تلافيفية صغيرة؟ استراهن على أيّهما: أيّهما يُنتج فجوة تعميم أصغر؟

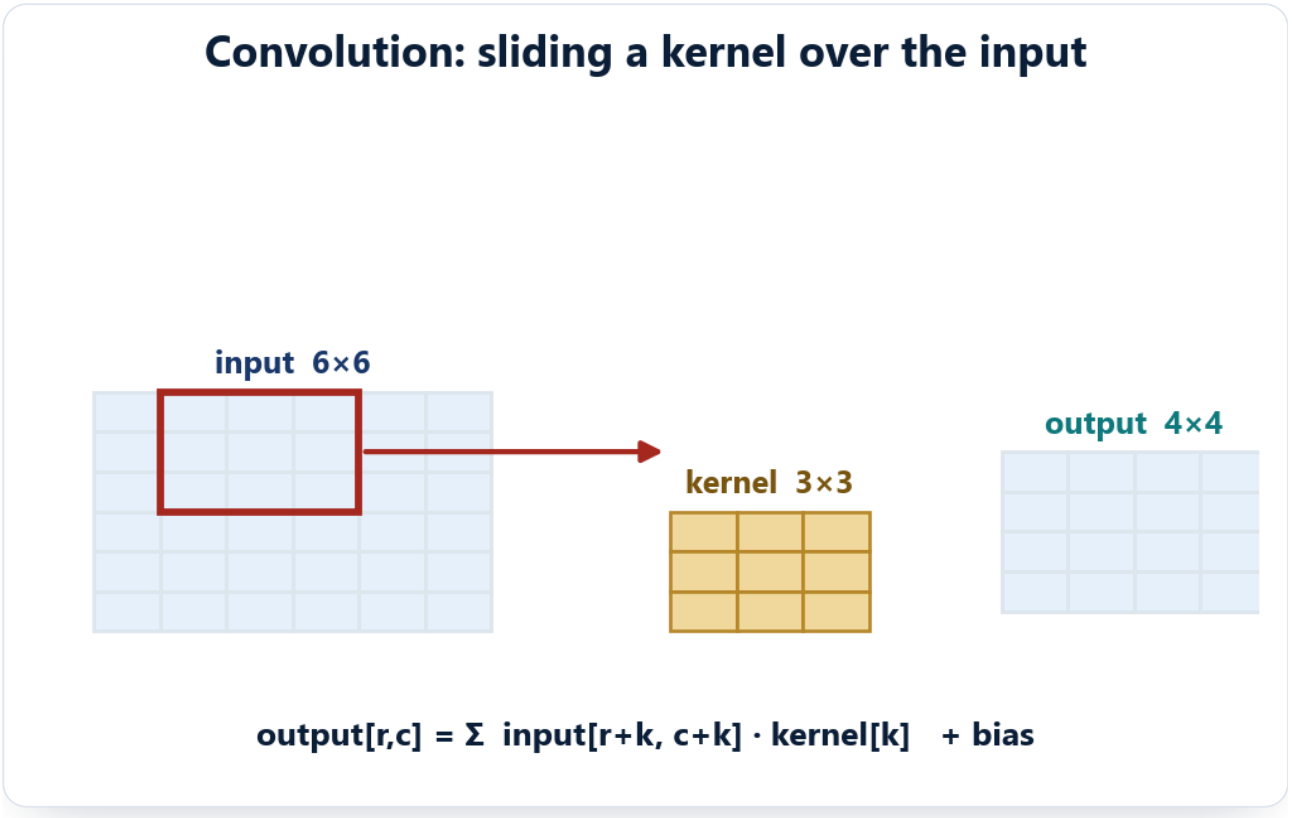
الجواب واضح: الشبكة التلافيفية. لأن عدد عيّنات التدريب هو 200، فالفارق الحاسم ليس القدرة الحسابية بل عدد الفرَضيات التي يجب أن يكتشفها النموذج من البيانات. الشبكة الكثيفة يجب أن تكتشف موضع كل بنية في الصورة من تلقاء نفسها، بينما الشبكة التلافيفية تكتشف أنماطاً صغيرة (حافة، زاوية، نسيج) ثم تركّبها. كلّما قلّ عدد العيّنات ازدادت أهمّية هذه البنية.

المعيار	الشبكة الكثيفة (Dense)	الشبكة التلافيفية (CNN)
معاملات أول طبقة على صورة 3×224×224	150528000	1728 (نواة 3×3 بثلاث قنوات إلى 64 قناة)
هل تتطلّب بيانات مضاعفة عند تغيّر الموضع؟	نعم	لا (انقلاب موضعي)
هل تستفيد من بنية الفضاء؟	لا	نعم (بُعد المكان والقنوات)
حجم التمثيل الوسيط	متجّه واحد	مكدّس خرائط ميزات
مناسب لـ	جداول، متغيّرات مُهيكلة، نصوص مُرمّزة	صور، فيديو، إشارات مكانية

◆ مصطلح يستحق الحفظ: المكدّس (stack)

في Keras تتحدّث عن «مكدّس خرائط الميزات» بينما تسمّيها PyTorch «توتر (tensor)» بثلاثة محاور. الفكرة واحدة: خرج كل طبقة تلافيفية ليس رقماً واحداً بل شبكة من الأرقام، بُعدها (القنوات أخيراً افتراضياً). — وهذا هو مصدر أخطاء انقلاب الأبعاد الأكثر شيوعاً، وسنصطدم به في كل تمرين.

12.2 النواة: معادلة الالتفاف



الشكل 12-1: كيف تعمل النواة. النافذة المظلّلة هي رقعة 3×3 من الصورة، والنواة هي الشبكة الصغيرة التي تُطبَّق عليها. في كل موضع تُحسب مجموعة أوزان حاصل الضرب (النواة × الرقعة) ثم تُجمع، فتُنتج قيمةً واحدة في خريطة الميزات. لاحظ أن القيم السالبة تعني أن النواة تبحث عن بنية معيّنة (حافة فاتحة على اليمين مثلاً) لا عن سطوع عام. وفي أسفل الرسم مبدأ التتميم: كل موضع في الخريطة يرى ما في جيرانه فقط، وهذا هو «الحقل الاستقبالي» الذي سنقيسه لاحقاً.

الآن لنكتب المعادلة كما ينفّذها الحاسوب، ونفكّكها رمزاً رمزاً. الصورة المُدخَلة `X` على شكل `C` قنوات، والنواة `K` على شكل `C` قنوات أيضاً، والناتج `Y` على شكل `M` قنوات:

$$Y[i, j, k] = b[k] + \Sigma_c \Sigma_u \Sigma_v X[i + u, j + v, c] \cdot K[u, v, c, k]$$

ما معنى كل رمز؟ الرمز `Y[i, j, k]` هو قيمة الخلية رقم `k` في خريطة الميزات عند الصف `i` والعمود `j`. الرمز `X[i+u, j+v, c]` هو قيمة الصورة الأصلية: عند إزاحة `u` صفوف و `v` أعمدة داخل النافذة، في القناة `c`. الرمز `K[u, v, c, k]` هو وزن النواة: المعامل الكائن في الصف `u` والعمود `v` من القناة `c` من المُدخل، والمُصمَّم لإخراج القناة `k`. الرمز `b[k]` هو إزاحة لكل قناة إخراج على حدة. أما علامات `Σ` فتعني «اجمع على كل القنوات `c`، ولكل قناة اجمع على كل مواضع النافذة `u, v`».

لماذا تهمّنا؟ لاحظ أنّ `k` لا يظهر داخل المعادلة إلا في الطرف الأيمن. أي أنّ كل قناة إخراج `k` لها **نواة كاملة** خاصة بها (مصفوفة `C×K_h×K_w`) و `Bias` خاص بها. عدد معاملات الطبقة التلافيفية هو `C_out × (C_in × K_h × K_w + 1)`، وهذه هي القيمة التي تقرّر سرعة التدريب. كلّما زاد عدد القنوات تضاعف عدد المعاملات، لأنّ كل قناة إخراج تقرأ **كل** قنوات المُدخل.

خطأ شائع عندها: أن يظنّ القارئ أنّ `Σc` تعني «مزج القنوات». هي فعلاً مزج، لكن الأهمّ أنّ `k` لا تظهر كمؤشّر مستقلّ: أي أنّ كل قناة ناتجة هي **مزيج خطّي** من قنوات الدخل عبر أوزان مختلفة. ومن هنا تأتي مرونة الشبكة: طبقة واحدة تعلّم عشرات من «مرشّحات» مختلفة تلتقط أنماطاً مختلفة (حافة، نسيج، رقعة).

لاحظ أيضاً ما **لا** يظهر في المعادلة: لا يوجد موضع ثابت للنواة. اللولب ينزلق على الصورة كاملة، فتُحسب القيمة `Y[i, j, k]` لكل موضع. هذا هو الفرق الجوهري بين الالتفاف والطبقة الكثيفة: الكثيفة تحسب `Y = Wx + b` مرّة واحدة، أما الالتفاف فيحسبها `H×W` مرّة.

ولنكتب المعادلة بأنفسنا مرة واحدة فقط – والحقيقة أنّ هذه أهمّ عشرة أسطر في الفصل كلّه – ثم نقارن نتيجتها بمكتبة `PyTorch` لنتحقّق من فهمنا حسابياً لا نظرياً:

Python

```python
import torch
import torch.nn.functional as F

# صورة 4×4 بأرقام متتابعة يسهل تتبّعها بالعين
x = torch.arange(16, dtype=torch.float32).reshape(1, 1, 4, 4)
# نواة 2×2: ترصد الفرق الأفقي (يمين موجب، يسار سالب)
k = torch.tensor([[[[1.0, 0.0], [0.0, -1.0]]]])


def conv_manual(x, k):
    """نفس المعادلة مكتوبةً بالحلقات: الالتفاف الحلقي."""
    n, c_in, h, w = x.shape
    c_out, _, kh, kw = k.shape
    out = torch.zeros(n, c_out, h - kh + 1, w - kw + 1)
    for i in range(out.shape[2]):        # كل موضع رأسي
```

```
        for j in range(out.shape[3]):     # كل موضع أفقي
            patch = x[:, :, i:i + kh, j:j + kw]      # الرقعة (n, c_in, kh, kw)
            out[:, :, i, j] = (patch * k[None]).sum(dim=(1, 2))
    return out


manual = conv_manual(x, k)
library = F.conv2d(x, k)

print("manual:\n", manual)
print("library:\n", library)
print("max abs diff:", (manual - library).abs().max().item())
# الناتج: max abs diff: 0.0
```

ماذا يحدث هنا؟ الدالة `conv_manual` تكرّر ما تفعله `F.conv2d` بالضبط: تشرائح الرقعة، وتضربها في النواة، وتجمع. **لماذا هذه القيمة؟** الاختلاف صفر لأنّ `F.conv2d` ينفّذ الارتباط عبراني (`cross-correlation`) لا الالتفاف الرياضي تماماً: المكتبة لا تقلب النواة قبل التطبيق. ولأنّ الأوزان قابلة للتعلّم فالعلاقتان متكافئتان تماماً في نوعَي الدوال، وأغلب الأدبيات تسمي العملية «الالتفاف» تسامحاً.

> ⚠ **انتبه: «النواة» ليست صورة صغيرة تُختار يدوياً**
>
> في المناهج القديمة تعلّمتَ أن نواة `Sobel` أو `Gaussian` تُكتب بيدك. في التعلّم العميق الأوزان تُكتسب من البيانات: كل ناتج في الشبكة هو ما تعلّمه النموذج، لا ما قرّره مبرمج. الشبكة ذات «نواة يدوية» (تعيين الأوزان مسبقاً وتجميدها) تظهر أحياناً في أنظمة الرؤية التقليدية وفي بعض تطبيقات فوريه، لكنها ليست مسار التعلّم العميق.

12.3 الحشو والخطوة وبُعد الخرج

ثلاثة أرقام تحكم كل طبقة تلافيفية: حجم النواة `K` ، خطوة الانزلاق `S` ، وكمية الحشو `P` . وبها نحصل على قاعدة بديهة تُستخدم في كل تصميم:

$$O = \text{floor}\left((I + 2P - D\cdot(K - 1) - 1) / S \right) + 1$$

ما معنى كل رمز؟ الرمز `O` هو بُعد الخرج في محور المكان، و `I` هو بُعد الدخل في المحور نفسه، و `K` هو حجم النواة و `P` هو عدد الأعمدة التي تُضاف من الصفر على كل جانب، و `S` خطوة الانزلاق، و `D` معدّل التوسيع (`dilation`) الذي يجعل النواة تلتقط مسافات أبعد بلا وزن إضافي. و `floor` يعني التقريب نحو الأسفل.

لماذا تهمّنا؟ لأنّ هذه القاعدة هي التي تخلّصك من تخمين الأشكال. وهي أيضاً التي تكشف الخطأ الأشهر في الشبكات: إدخال صورة 32×32 مع `stride=2` وثلاث طبقات تجميع، ثم اكتشاف أن المخرج صفر أبعاد قبل طبقة كثيفة.

خطأ شائع عندها: ظنّ أنّ الحشو «يضيف معلومات». هو لا يضيف شيئاً: الصفر لا يحمل معنى، والحشو موجود، هدفه واحد فقط: ألّا تُقتطع حافّة الصورة مرّتين – من الأعلى ومن الأسفل – معاً. الحشو في `PyTorch` يضاف دائماً على كل الجوانب بالعدد نفسه، فلا يوجد حشو غير متماثل إلا عبر `padding_mode` .

أمّا الخطوة `S` فتجيب عن سؤال: كم نُزيح النواة بين موضع وآخر؟ الخطوة 1 تعني نافذة متداخلة (نستفيد من كل بكسل)، والخطوة 2 تعني نافذة متجاورة (نُنصّف عدد المواضع). الخطوة تُستخدم إمّا لخفض التكلفة وإمّا لزيادة الحقل الاستقبالي بسرعة، كما سنرى.

الإعداد	الدخل	الخرج	عدد المعاملات	متى تستخدمه
`Conv2d(3, 32, 3, padding=1)`	3×224×224	32×224×224	896	بداية الشبكة: مستخلِصات بسيطة
`Conv2d(32, 64, 3, stride=2, padding=1)`	32×224×224	64×112×112	18496	خفض الأبعاد مع زيادة القنوات
`Conv2d(64, 64, 3, padding=1)`	64×112×112	64×112×112	36928	تعمّق بدون خسارة مكانية
`Conv2d(64, 128, 3,.outstride=2, padding=1)`	64×112×112	128×56×56	73728	انتقال مرحلة
`MaxPool2d(2)`	64×112×112	64×56×56	0	خفض الأبعاد بلا معاملات

لاحظ في الجدول قاعدةً بصرية: كلّما تضاعف عدد القنوات انخفض بُعد المكان بمقدار النصف. هذا هو «عنق الزجاجة» الكلاسيكي في الشبكات التلافيفية: الشبكة إمّا أن تُنفق دقة مكانية، أو أن تنفق سعة_channels. والعمود الأخير يذكّر بأنّ خفض الأبعاد مرتين متتاليتين (224 ← 112 ← 56) يجعل الشبكة فقيرة بالتفاصيل الدقيقة، ولهذا نستخدم تجميعاً متكيّفاً في نهاية `ResNet` بدلاً من التجميع الثابت.

أمّا `padding="same"` فمريح جدّاً: هي القيمة التي تجعل `PyTorch` يحسب الحشو تلقائياً ليحافظ على البُعد طالما كانت الخطوة 1. لاحظ أنّ `Keras` تختار `"same"` افتراضياً في `Conv2D` ، بينما `PyTorch`

تختار `padding=0` . هذا الاختلاف الصغير سبب شائع لمفاجآت عند النقل بين المكتبتين.

```python
import torch
import torch.nn as nn


def out_size(i, k, s, p, d=1):
    """صيغة بُعد الخرج: نفس القاعدة المذكورة في النص."""
    return (i + 2 * p - d * (k - 1) - 1) // s + 1


# مقارنة بين padding="same" والحشو اليدوي
conv_same = nn.Conv2d(3, 16, kernel_size=3, padding="same")
conv_manual = nn.Conv2d(3, 16, kernel_size=3, padding=1)

x = torch.randn(2, 3, 224, 224)
print("same   :", conv_same(x).shape)      # الناتج: torch.Size([2, 16, 224, 224])
print("manual :", conv_manual(x).shape)    # الناتج: torch.Size([2, 16, 224, 224])
print("formula:", out_size(224, 3, 1, 1))  # الناتج: 224

# الخطوة 2 تُنصّف البُعد
conv_s2 = nn.Conv2d(16, 32, kernel_size=3, stride=2, padding=1)
print("stride2:", conv_s2(conv_same(x)).shape)  # الناتج: torch.Size([2, 32, 112, 112])
print("formula:", out_size(224, 3, 2, 1))       # الناتج: 112
```

ماذا يحدث هنا؟ `padding="same"` في `PyTorch` ليست رقماً جاهزاً: المكتبة تحسب الحشو المطلوب لكل جانب ثم تملأه بالصفر. **لماذا هذه القيمة؟** لأنّ بُعد الخرج يجب أن يُقسم صحيحاً على الخطوة؛ إن كان لا يقسم فستحصل على بُعد «مبتور» (مثلاً 111 بدل 112) ما يُفسد محاذاة البُنى اللاحقة. القاعدة العملية: اضرب الخطوة في الحشو وحافظ على البُعد زوجياً إن أمكن.

✓ نصيحة: «قاعدة السهم» لضبط البُعد

إن كان لديك `Conv2d` بخطوة `S` ، اجعل `padding = (K − 1) / 2` ما دام `K` فردياً و `S = 1` ،فإنّ يبقى البُعد كما هو. وعند تغيير الخطوة إلى 2، اجعل `padding = K // 2` . القاعدة نفسها في `Keras` تُكتب ببساطة `padding="same"` وتتكفّل `Keras` بالباقي.

12.4 التجميع والحقل الاستقبالي

بعد أن تتعلّم الشبكة «الأنماط الصغيرة»، تحتاج إلى طريقة تنقلها إلى الأنماط الأكبر دون أن ينفجر عدد المواضع. هذه الطريقة هي **التجميع** (`pooling`). بعدّ `max` على نافذة `2×2` ، أي نأخذ أكبر قيمة في كل نافذة:

$$P[i, j] = \max(X[2i, 2j], X[2i, 2j+1], X[2i+1, 2j], X[2i+1, 2j+1])$$

ما معنى كل رمز؟ `P[i, j]` هو ناتج التجميع عند الموضع `(i, j)` ، و `X` هو الدخل، و `max` يختار أكبر قيمة داخل نافذة 2×2. والفرق الجوهري بين التجميع بالاقصوى (`max`) والتجميع بالمتوسّط (`average`) أن الأولى تحتفظ بوجود الإشارة: إن كان هناك بؤرة بيضاء ساطعة في الرقعة، فإن `max` ينقلها كما هي، بينما `avg` يذيبها في متوسط يخفي حدّتها.

لماذا تهمّنا؟ لأنّ `max` أفضل للمهام التي تعتمد على وجود بنية (عيوب، نقوش، شقوق)، بينما `avg` يُعطي تمثيلاً أنعم وأكثر استقراراً، وهو ما جعل التجميع بالمتوسّط العام (`Global Average Pooling`) يستبدل الطبقة الكثيفة الأخيرة في `ResNet` .

خطأ شائع عندها: الجمع بين التجميع والخطوات `stride=2` معاً. إن كانت الطبقة تلفّاف بخطوة 2 ثم تجميع بخطوة 2، فإنك تطبّق تصغيراً بمقدار 4 في خطوة واحدة، وسرعة الانتقال إلى التفاصيل الدقيقة تكون أسرع ممّا تتوقّع.

أمّا **الحقل الاستقبالي** (`receptive field`) فهو المفهوم الذي يربط كل ما سبق. هو حجم الجزء من الصورة الأصلي الذي تراه موضع واحد في الخرج. نواة `3×3` واحدة ترى `3×3` من الصورة. لكن طبقة تلافيفها ثلاث `3×3` متتالية ترى `7×7` ، لأن كل طبقة توسّع ما تراه الطبقة السابقة. القاعدة الحسابية هي:

$$r_l = r_{l-1} + (k_l - 1) \cdot j_{l-1} \quad \text{and} \quad j_l = j_{l-1} \cdot s_l$$

ما معنى كل رمز؟ r_l هو حجم الحقل الاستقبالي بعد الطبقة `l` (بالبكسل في الصورة الأصلية)، و j_l هو «قفزة» الانتقال بالبكسلات في الصورة الأصلية بين موضعين متتاليين في الخرج، و k_l حجم نواة الطبقة `l` ، و s_l خطوتها. البداية دائماً $r_0 = 1$ و $j_0 = 1$ لأنّ كل موضع في الخرج الأول يرى بكسلاً واحداً.

لماذا تهمّنا؟ لأنّ الحقل الاستقبالي يحدّد أصغر بنية يستطيع النموذج تمييزها. مصنّف يستقبل صورة 224×224 ويحتاج أن يميّز عيباً بحجم 4 بكسل، يجب أن يمتلك حقلاً استقبالياً أكبر من 4. وحقل `7×7` يعني تفاصيل على مستوى 7 بكسلات، لكنّه يعني أيضاً أرقاماً كثيرة.

خطأ شائع عندها: ظنّ أنّ حقلاً استقبالياً بحجم 7 يعني «7 بكسل فقط». لا: الحقل 7 يعني أنّ كل موضع في الخرج النهائي مبنيّ على معلومات من نافذة 7×7 في الصورة الأصلية، أي إنّ طبقة واحدة من `7×7` أو ثلاث طبقات من

`3×3` تكفي لنفس المهمة.

```python
import torch
import torch.nn as nn


def receptive_field(layers):
    """layers: قائمة من (حجم النواة، الخطوة). يعيد (الحقل، القفزة)."""
    r, j = 1, 1
    for k, s in layers:
        r = r + (k - 1) * j   # الطبقة توسّع ما تراه الطبقة السابقة بمقدار قفزتها
        j = j * s             # القفزة تتضاعف بالخطوة
    return r, j


print(receptive_field([(3, 1), (3, 1), (3, 1)]))   # الناتج: (7 ,1)
print(receptive_field([(3, 1), (3, 2), (3, 1)]))   # الناتج: (9 ,2)
print(receptive_field([(7, 1)]))                   # الناتج: (7 ,1)

# الطبقات الثلاث 3×3 تحتاج معاملات أكثر من طبقة واحدة 7×7-Layer
c3 = nn.Sequential(*[nn.Conv2d(3, 3, 3, padding=1) for _ in range(3)])
c7 = nn.Conv2d(3, 3, 7, padding=3)
print("params 3x3x3:", sum(p.numel() for p in c3.parameters()))  # الناتج: 252
print("params 7x7  :", sum(p.numel() for p in c7.parameters()))  # الناتج: 588

# التجميع: الأقصى والمتوسّط
x = torch.tensor([[[[1.0, 2.0], [3.0, 4.0]]]])
print(nn.MaxPool2d(2)(x))    # الناتج: tensor([[[[4.]]]])
print(nn.AvgPool2d(2)(x))    # الناتج: tensor([[[[2.5000]]]])

# التجميع المتكيّف: يضمن الخرج المطلوب مهما كان حجم الدخل
adaptive = nn.AdaptiveAvgPool2d(1)
print(adaptive(torch.randn(2, 64, 31, 17)).shape)  # الناتج: torch.Size([2, 64, 1, 1])
```

ماذا يحدث هنا؟ المقارنة الأخيرة بين ثلاث طبقات `3×3` وطبقة واحدة `7×7` تكشف ميزة تُستعمل في كل تصميم شبكي: تكديس طبقات صغيرة أعطى نفس الحقل الاستقبالي مع معاملات أقلّ (252 مقابل 588)، والأهمّ أنّ ثلاث `ReLU` متتالية تُنتج دالّة غير خطّية أقوى من `ReLU` واحدة – وهذا بالضبط ما بنى عليه `ResNet` عمقه الهائل.

صمّمتَ طبقة تلافيف واحدة بنواة `7×7` وخطوة 2. ما حجم الحقل الاستقبالي في الصورة الأصلية لكل موضع في الخرج؟ والآن: هل تملك الشبكة «عمقاً» كافياً لتعلّم تركيبات من الأنماط؟ قارن البديل: ثلاث طبقات `3×3` بخطوة 1 ثم `3×3` بخطوة 2.

✦ لماذا لا نستخدم التجميع `window=stride` دائماً؟

التجميع بخطوة `2` يقتل التفاصيل الدقيقة: عيبٌ بحجم بكسل واحد قد يختفي تماماً. لهذا صارت `stride=2` في `Conv2d` (وهي تعلّم كيف تتخطّى) أفضل من `MaxPool2d(2)` (وهي تتجاهل ما بين النوافذ) في الشبكات الحديثة. لكن `MaxPool` ما زال مفيداً بعد الطبقات الأولى حيث تكون المخطّطات كبيرة وخشنة.

12.5 هندسة التصنيف: من الالتفاف إلى احتمال

اجمعنا القطع. شبكة تصنيف بُنيت على ثلاث قواعد عملية: (1) كل طبقة تلافيفية تُتبع بتطبيع `BatchNorm2d` ثم `ReLU`؛ (2) الأبعاد المكانية تنخفض تدريجياً بينما ترتفع القنوات تدريجياً؛ (3) الطبقة الأخيرة قبل الإخراج هي تجميع متوسّط عام، فتُسقط الأبعاد المكانية بالكامل.

القاعدة الثالثة لها أثران مهمّان. الأولى: عدد المعاملات في الطبقة الكثيفة الأخيرة ينخفض من 204800 إلى 2048 فقط في مثال `ResNet`. والثانية: الشبكة تصبح مستقلّة عن حجم الدخل، لأنّ `AdaptiveAvgPool2d(1)` يختصر أي خريطة ميزات إلى `(C, 1, 1)` مهما كان حجمها. هذه الخاصية تسمّى **استقلال الشبكة عن حجم الصورة**، وهي التي تسمح بتغذية صورة بحجم 224 وأخرى بـ 512 إلى النموذج نفسه.

المرحلة	الطبقات	شكل الخرج	ماذا تعلّمت الشبكة هنا؟
البداية	`Conv2d(3,32,3) + BN + ReLU`	224×224×32	حواف، ألوان، نسيج أساسي
المرحلة الأولى	`Conv2d(32,64,3) + BN + ReLU + MaxPool`	112×112×64	أشكال صغيرة: زوايا، نقاط
المرحلة الثانية	`Conv2d(64,128,3) + BN + ReLU + MaxPool`	56×56×128	أجزاء من الأجرام

المرحلة	الطبقات	شكل الخرج	ماذا تعلّمت الشبكة هنا؟
المرحلة الثالثة	`Conv2d(128,256,3) + BN + ReLU + MaxPool`	28×28×256	بنى متوسطة
الرأس	`AdaptiveAvgPool2d(1) + Dropout + Linear`	256 ← 10	تحويل التمثيل إلى احتمال

Python

```python
import torch
import torch.nn as nn


class ConvNet(nn.Module):
    """شبكة تصنيف صغيرة: تدرّج في القنوات، halving في المكان."""

    def __init__(self, num_classes: int = 10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),

            nn.Conv2d(32, 64, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
        )
        self.head = nn.Sequential(
            nn.AdaptiveAvgPool2d(1),    # (N, 128, 1, 1) مهما كان حجم الصورة
            nn.Flatten(),               # (N, 128)
            nn.Dropout(0.3),
            nn.Linear(128, num_classes),  # logits خام، بلا softmax
        )
```

```
    def forward(self, x):
        return self.head(self.features(x))


model = ConvNet(num_classes=10)
x = torch.randn(4, 3, 224, 224)
y = model(x)
print("logits shape:", y.shape)                # الناتج: torch.Size([4, 10])
print("params:", sum(p.numel() for p in model.parameters()))
# لا تضيف softmax ؛ الخسارة تُحسب مباشرة على الـ logits
loss = nn.CrossEntropyLoss()(y, torch.tensor([0, 1, 2, 3]))
print("loss:", round(loss.item(), 4))
```

ماذا يحدث هنا؟ `bias=False` قبل `BatchNorm2d` قرار مقصود: التطبيع يسحب متوسط كل قناة إلى صفر ويقيس انحرافها، فلا معنى لإزاحة بعدها. **لماذا هذه القيمة؟** `Dropout(0.3)` معقول لأنّ الرأس فيه 128 معاملاً فقط؛ لو كان الرأس أكبر لاحتجت نسبة أعلى. والأهمّ: `CrossEntropyLoss` تتوقّع logits لا احتمالات، فهي تطبّق `log_softmax` داخلياً بثبات عددي، ولو طبّقت `softmax` يدوياً ثم مرّرت الناتج إلى `CrossEntropyLoss` لخسرتَ الثبات العددي وخسرتَ خسارة أسوأ.

⚠ انتبه: softmax مرّة واحدة، لا مرّتين

أشيع خطأ في تصنيف `PyTorch` : أن يُخرج النموذج احتمالات عبر `nn.Softmax` ثم يُمرّرها إلى `nn.CrossEntropyLoss` ، فيحسب اللوغاريتم مرّتين. القاعدة: إما أن يُخرج النموذج logits وتتكفّل `CrossEntropyLoss` ، أو أن تُخرج احتمالات وتستخدم `NLLLoss` مع `log` للاحتمالات. لا تخلط بين المسارين.

⚙ تطبيق كامل: كشف عيوب في خطّ إنتاج

البيانات: صور لأختام معدنية بحجم 128×128، أربع فئات: سليم، خدش سطحي، تآكل عند الحافة، تشقّق. 2400 صورة، منها 300 لكل فئة خلل. **النموذج:** الشبكة التي بنيناها للتوّ، مع `CrossEntropyLoss` و `AdamW` بمعدّل تعلّم `3e-4`، وتدريب 20 حقبة مع `BatchNorm`. **القرار:** في المصنع، عيب الخدش السطحي لا يوقف الإنتاج (يُرمَّز للمتابعة)، أما التشقّق فيوقف الخطّ ويُرسل القطعة للفحص البشري. لذلك لا تُقاس الدقّة وحدها: يُقاس معدّل التوقف الخاطئ (إن صنّف النموذج قطعة سليمة كمتشقّقة يتوقّف الإنتاج بلا سبب). تُضبط عتبة القرار على الطبقة الأخيرة لرفع الاستدعاء على فئة التشقّق ولو انخفضت الدقّة الكلية قليلاً – وهذه مظلّة قرار لا مظلّة دقّة.

12.6 ResNet: كيف نصل إلى عمق 50 طبقة دون أن ينهار التدرّج

في 2015 لاحظ فريق `Microsoft Research` ظاهرة غريبة: كلّما زاد عمق الشبكة العادية، بدأ الأداء يتراجع – رغم أن الشبكة الأعمق كان ينبغي أن تستطيع تمثيل كل ما تمثّله الأضيق. لم يكن السبب تعظيماً في المعاملات، بل ما سُمّي **مشكلة التدهور** (`degradation problem`): الشبكة العميقة لا تجد بسهولة مساراً مناسباً للنموذج الأضيق. التدرّجات التي يجب أن تعبر عشرات الطبقات تضيع.

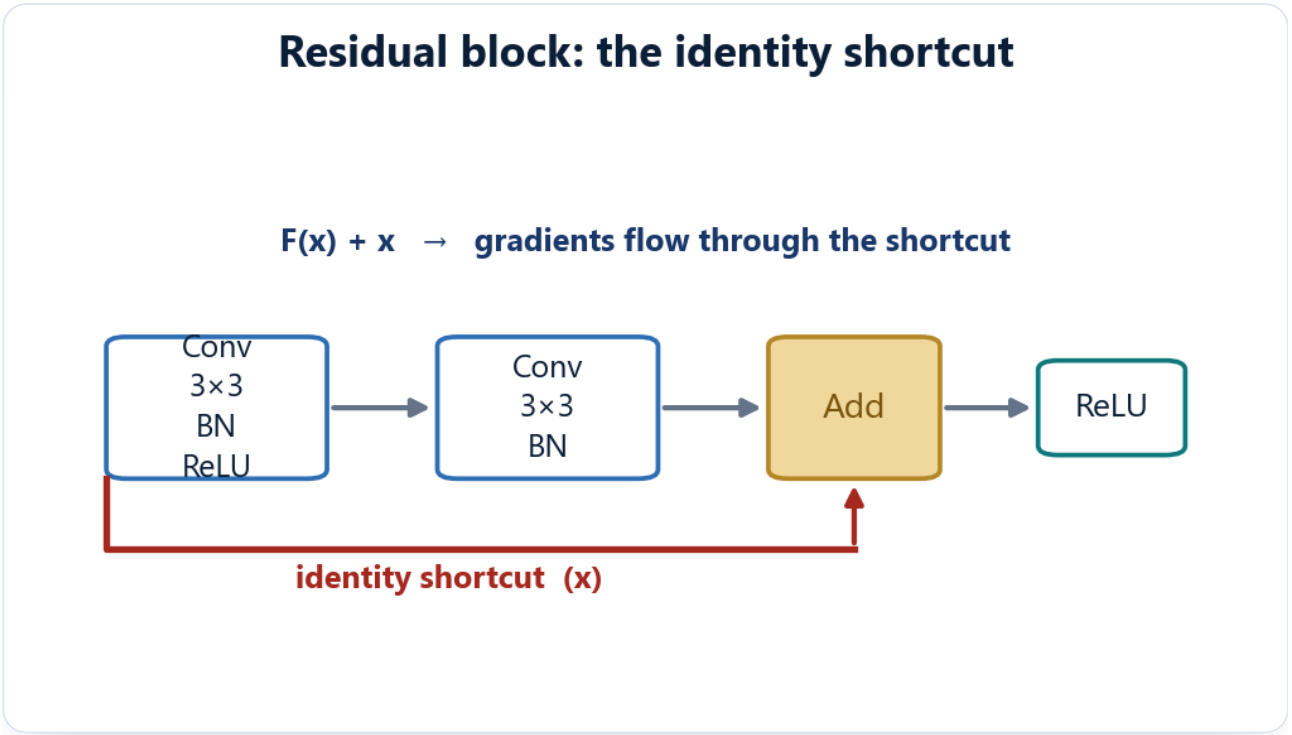
حلّ `ResNet` كان أن تتعلّم الشبكة الباقي (residual) بدل التعلّم من الصفر. بدل أن تخرج الطبقة `F(x)` مباشرة، تتعلّم `F(x) + x`:

```
h(x) = ReLU( F(x) + x )
```

ما معنى كل رمز؟ `x` هو دخل الكتلة، و `F(x)` هو ما تعلّمته الكتلة (عادةً `Conv2d → BatchNorm → ReLU → Conv2d → BatchNorm`)، و `ReLU` دالة التفعيل غير الخطّية، و `+` جمع عنصراً بعنصر بين `F(x)` و `x`.

لماذا تهمّنا؟ لأنّ `F(x)` مصفوفة قريبة من الصفر يسهل تعلّمها. في الحالة المثالية تتعلّم الكتلة «لا شيء» وتكتفي `x` بالمرور – تماماً كما يفعل المصنّف. والأهمّ: الكتلة لا تتعلّم البديل، بل الزيادة على ما سبق.

خطأ شائع عندها: جمع `x` و `F(x)` يتطلّب أن تكون أبعادهما متساوية. إن تغيّرت أبعاد المكان أو عدد القنوات، فالجمع المباشر يرمي RuntimeError. الحلّ: مسار تخطّي منفصل يحوّل الأبعاد بـ `1×1` (مع `stride` مناسب) قبل الجمع.



الشكل 12-2: كتلة `ResNet` الأساسية. المسار الأفقي هو مسار التخطّي: يمرّ دخل الكتلة بلا تغيير حتى نقطة الجمع. المسار الرأسي هو مسار التعلّم: طبقتا التفاف تنشئان `F(x)` . عند نقطة الجمع يصير الخرج `F(x) + x` ، فصار جوهر الأمر: إن لم تجد الكتلة ما تُحسّنه، تتعلّم أن تُسهم بـ«صفر» بدل أن تُفسد ما بُني قبلها. لاحظ أنّ الرسم يوضّح أيضاً حالة «اختصار بالالتفاف 1×1» على اليمين حيث تختلف أبعاد المدخل والمخرج.

Python

```python
import torch
import torch.nn as nn
import torch.nn.functional as F


class BasicBlock(nn.Module):
    """كتلة ResNet الأساسية: مساران يُجمعان عند النهاية."""

    expansion = 1

    def __init__(self, in_channels: int, out_channels: int, stride: int = 1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, stride=stride, padding=1, bias=F
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, stride=1, padding=1, bias=False
        self.bn2 = nn.BatchNorm2d(out_channels)

        # المسار السريع: 1×1 عند اختلاف الأبعاد أو الخطوة
```

```
        self.downsample = None
        if stride != 1 or in_channels != out_channels:
            self.downsample = nn.Sequential(
                nn.Conv2d(in_channels, out_channels, 1, stride=stride, bias=False),
                nn.BatchNorm2d(out_channels),
            )

    def forward(self, x):
        identity = x if self.downsample is None else self.downsample(x)
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out = out + identity          # الجمع العنصري بين المسارين
        return F.relu(out)


class MiniResNet(nn.Module):
    def __init__(self, num_classes: int = 10):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv2d(3, 64, 7, stride=2, padding=3, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(3, stride=2, padding=1),
        )
        self.layer1 = nn.Sequential(BasicBlock(64, 64), BasicBlock(64, 64))
        self.layer2 = nn.Sequential(BasicBlock(64, 128, stride=2), BasicBlock(128, 128))
        self.layer3 = nn.Sequential(BasicBlock(128, 256, stride=2), BasicBlock(256, 256))
        self.layer4 = nn.Sequential(BasicBlock(256, 512, stride=2), BasicBlock(512, 512))
        self.pool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Linear(512, num_classes)

    def forward(self, x):
        x = self.stem(x)
        x = self.layer4(self.layer3(self.layer2(self.layer1(x))))
        x = self.pool(x).flatten(1)
        return self.fc(x)


net = MiniResNet()
x = torch.randn(2, 3, 224, 224)
print("logits:", net(x).shape)   # الناتج: torch.Size([2, 10])

# لو صفّرنا أوزان الكتلة، يصبح F(x)=0 تقريباً تحقّق عملي من فكرة «الباقي»:
block = BasicBlock(64, 64).eval()
with torch.no_grad():
```

```
    for p in block.conv1.parameters():
        p.zero_()
    for p in block.bn1.parameters():
        p.zero_()
    for p in block.conv2.parameters():
        p.zero_()
    for p in block.bn2.parameters():
        p.zero_()
    y0 = block(x)
print("identity preserved:", torch.allclose(y0, F.relu(x), atol=1e-4))
# الناتج: identity preserved: True
```

ماذا يحدث هنا؟ السطر الأخير هو أهمّ اختبار فلسفي في الفصل: بعد تصفير أوزان الكتلة (فتصبح `F(x) = 0` تقريباً) يمرّ الدخل كما هو، وهذا يثبت أنّ الكتلة تتعلّم إضافة إلى المعلومة لا استبدالها. **لماذا هذه القيمة؟** نستخدم `BasicBlock` (التوسعة 1) هنا، بينما `ResNet50` يستخدم `Bottleneck` (توسعة 4) لأنّ 50 طبقة بأوزان كاملة مكلفة؛ البوتلنِك يضع 1×1 قبل وبعد 3×3 ليركّز الحساب في القنوات القليلة ويوسّعها في النهاية.

> ★ **القاعدة: لا تُجمِد الـBatchNorm في `ResNet` من البداية**
>
> تطبيع `BatchNorm` يحسب متوسّطاً وانحرافاً على كل دفعة، ولهذا يميل إلى تكيّف مع بيانات التدريب. في التدرّج الدقيق (الفصل التالي) قد تحتاج تجميد إحصاءاته، لكن في `ResNet` المدرَّب من الصفر اتركه يتعلّم: هو جزء أساسي من استقرار الشبكة العميقة.

12.7 Keras: الشبكة نفسها بأسلوب آخر

لنكتب نفس الشبكة في `Keras 3`. الفروق المهمة: ترتيب الأبعاد `(H, W, C)` بدل `(C, H, W)`، والحشو الافتراضي `"same"`، واسم `GlobalAveragePooling2D` بدل `AdaptiveAvgPool2d`، والإخراج `softmax` مدمج في الطبقة الكثيفة لأن `Keras` تطبّق الخسارة على الاحتمالات افتراضياً.

Python

```
import keras
from keras import layers

# متعدّد النواة: تأكّد من النواة المستخدمة قبل تشغيل الكود Keras 3
print("keras backend:", keras.backend.backend())
```

```
model = keras.Sequential([
    keras.Input(shape=(224, 224, 3)),
    layers.Conv2D(32, 3, padding="same", use_bias=False),
    layers.BatchNormalization(),
    layers.ReLU(),
    layers.MaxPooling2D(2),

    layers.Conv2D(64, 3, padding="same", use_bias=False),
    layers.BatchNormalization(),
    layers.ReLU(),
    layers.MaxPooling2D(2),

    layers.Conv2D(128, 3, padding="same", use_bias=False),
    layers.BatchNormalization(),
    layers.ReLU(),
    layers.MaxPooling2D(2),

    layers.GlobalAveragePooling2D(),     # ما يقابله AdaptiveAvgPool2d(1)
    layers.Dropout(0.3),
    layers.Dense(10, activation="softmax"),  # Keras هنا احتمالات توقّع
])

model.compile(
    optimizer=keras.optimizers.AdamW(learning_rate=3e-4),
    loss=keras.losses.SparseCategoricalCrossentropy(),
    metrics=[keras.metrics.SparseCategoricalAccuracy()],
)
model.summary(line_length=80)
```

ماذا يحدث هنا؟ في `Keras` تُكتب `ReLU` كطبقة مستقلّة، بينما في `PyTorch` هي `nn.ReLU` داخل `Sequential` – وهذا فرق أسلوبي لا وظيفي. **لماذا هذه القيمة؟** `use_bias=False` قبل `BatchNormalization` له نفس السبب في المكتبتين: الإزاحة عديمة الفائدة بعد التطبيع. ولاحظ أنّ `Keras` تحتاج `softmax` في الطبقة الأخيرة بينما `PyTorch` لا تحتاجها – وهذا فرق جوهري في فلسفة الخسارة: `CrossEntropyLoss` في `PyTorch` تتولّى اللوغاريتم، أمّا `SparseCategoricalCrossentropy` في `Keras` تعمل على الاحتمالات.

وحين تتجاوز الشبكة المخصّصة حدود ما تكتبه يدوياً، تصبح `keras.applications` اختصاراً عملياً: نماذج `ImageNet` مدرَّبة مسبقاً، تُستخدم غالباً كـ«عمود فقري» (backbone) يُستبدل رأسه:

```
Python
```

```python
import keras
from keras import layers

# ResNet50 مدرَّب على ImageNet، بدون رأس التصنيف
backbone = keras.applications.ResNet50(
    weights="imagenet",
    include_top=False,          # نحذف طبقة التصنيف الأصلية
    input_shape=(224, 224, 3),
    pooling=None,               # نُبقي خرائط الميزات مكانية بأنفسنا
)

inputs = keras.Input(shape=(224, 224, 3))
x = backbone(inputs, training=False)     # training=False يُجمّد BatchNorm
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(4, activation="softmax", name="fault_head")(x)

model = keras.Model(inputs, outputs)
model.compile(
    optimizer=keras.optimizers.AdamW(learning_rate=1e-4),
    loss=keras.losses.SparseCategoricalCrossentropy(),
    metrics=[keras.metrics.SparseCategoricalAccuracy()],
)
model.summary()
```

ولنقابل بين المكتبتين في جدول واحد، لأن التبديل بينهما أصبح مألوفاً في فرق الإنتاج:

المفهوم	PyTorch 2.14	Keras 3.15
طبقة تلافيف	`nn.Conv2d`	`layers.Conv2D`
ترتيب الأبعاد	`(N, C, H, W)`	`(N, H, W, C)`
الحشو الافتراضي	`padding=0`	`padding="same"`
تطبيع الدفعة	`nn.BatchNorm2d`	`layers.BatchNormalization`

المفهوم	PyTorch 2.14	Keras 3.15
تجميع أقصى	`nn.MaxPool2d(2)`	`layers.MaxPooling2D(2)`
تجميع عام	`nn.AdaptiveAvgPool2d(1)`	`layers.GlobalAveragePooling2D`
بناء متسلسل	`nn.Sequential`	`keras.Sequential`
دمج مسارات	`out = out + skip` (جمع يدوي)	`layers.Add()` أو `rs.Concatenate()`
نموذج جاهز	`torchvision.models.resnet50(weights=...)`	`ons.ResNet50(weights="imagenet")`

في PyTorch

النموذج كائن (class) يرث `nn.Module`؛ أنت تكتب `forward` بنفسك وتتحكّم في مسار البيانات. الدمج يتم بجمع الشدّات مباشرة `out + skip`، فتتاح لك مرونة `torch.cat` إن أردت لصق القنوات. أمّا حلقة التدريب فصريحة: `optimizer.step()` ثم `zero_grad()` ثم `loss.backward()` – ترتيبٌ لا تُسامح عليه.

في Keras

النموذج بيان (graph) تبنيه تصريحياً: `keras.Model(inputs, outputs)`. الدمج يتم بطبقات صريحة `layers.Add()` و`layers.Concatenate()`، وهي تظهر في مخطّط `summary` فتصبح البنية مقروءة. التدريب يتم في `model.fit()` الذي يتكفّل بالحلقة التدريبية ويضيف `callbacks` جاهزة للإيقاف المبكر وحفظ الأفضل.

وأخيراً، انظر كيف تُبنى الشبكات التي تكبّر الصورة لا تصغّرها – لا يُحتاج للتجزئة ولا للتوليد: `nn.Upsample` ببساطة يكرّر البكسلات (أرخص وأسرع لكنّه يفتقر إلى معاملات تعلّمية)، و`nn.ConvTranspose2d` يتعلّم كيف يوسّع (أغلى لكنّه أدقّ)، و`keras.layers.UpSampling2D` نظير `Keras` لـ `Interpolate`. وأما

`layers.Concatenate` و`layers.Add` فهما مفتاح بنية `U-Net` التي سنقابِلها في الفصل السادس عشر.

```python
import torch
import torch.nn as nn
import torchvision.ops as ops

# رفع الأبعاد: تكرار بسيط مقابل الالتفاف المعكوس
up = nn.Upsample(scale_factor=2, mode="nearest")            # (N,C,H,W) -> (N,C,2H,2W)
deconv = nn.ConvTranspose2d(64, 32, kernel_size=3, stride=2, padding=1, output_padding=1)
x = torch.randn(1, 64, 32, 32)
print(up(x).shape)         # الناتج: torch.Size([1, 64, 64, 64])
print(deconv(x).shape)     # الناتج: torch.Size([1, 32, 64, 64])

# Boxesمن التصنيف إلى الكشف: نفس العمود الفقري، ورأس ي
boxes = torch.tensor([[10.0, 10.0, 60.0, 60.0], [12.0, 12.0, 62.0, 62.0], [80.0, 80.0, 120.0,
scores = torch.tensor([0.9, 0.8, 0.7])
print("IoU matrix:\n", ops.box_iou(boxes, boxes).round(decimals=2))
keep = ops.nms(boxes, scores, iou_threshold=0.5)
print("kept after NMS:", keep)   # الناتج: kept after NMS: tensor([0, 2])
```

ماذا يحدث هنا؟ صندوقا الصندوقين الأولين يتداخلان بنسبة `IoU` أعلى من 0.5، فـ `NMS` يحذف الثاني لأنّه درجة أقلّ. **لماذا هذه القيمة؟** عتبة `0.5` هي القيمة التقليدية، و`NMS` هو العملية التي تحوّل مخرجات الشبكة إلى قائمة صناديق نهائية. سنعود إليه بالتفصيل في الفصل السادس عشر؛ هنا يكفي أن نلاحظ أنّ العمود الفقري التلافيفي نفسه لا يتغيّر بين التصنيف والكشف – بل يتغيّر الرأس فقط. وهذا بالضبط ما يجعل التعلّم النقلي في الفصل التالي فعّالاً.

12.8 تمارين

تمرين 1 — احسب البُعد بنفسك

شبكة تصنيف تأخذ صورة 3×64×64. الطبقات بالترتيب: `Conv2d(3, 16, 3, padding=1)` ثم `MaxPool2d(2)` ثم `Conv2d(16, 32, 3, padding=1)` ثم `MaxPool2d(2)` ثم `Conv2d(32, 64, 3, padding=1)` ثم `AdaptiveAvgPool2d(1)` ثم `Linear(64, 10)`.

1. اكتب شكل الخرج بعد كل طبقة.
2. احسب عدد معاملات كل طبقة تلافيفية.
3. احسب عدد معاملات الطبقة الكثيفة الأخيرة وقارنها بمماثلة بلا تجميع (أي بعد `Flatten` الأخيرة مباشرة).
4. احسب الحقل الاستقبالي لكل موضع في الخرج النهائي.

▲ الحل النموذجي

1) الأشكال: `(64,64,16)` ← تجميع `(32,32,16)` ← `(32,32,32)` ← تجميع `(16,16,32)` ← `(16,16,64)` ← تجميع عام `(64)` ← `(10)`.

2) المعاملات: الأولى 3×3×3×16 = 432، الثانية 3×3×16×32 = 4608، الثالثة 3×3×32×64 = 18432. المجموع 23472 معاملاً تلافيفياً.

3) الطبقة الكثيفة: مع `AdaptiveAvgPool2d(1)` عدد المعاملات 64×10 + 10 = 650. بلا تجميع، تسطيح لشكل 64×16×16 يعطي 16384 مُدخلاً، فيصبح 16384×10 + 10 = 163850 معاملاً – أي أكثر بـ252 مرّة، وقد يسبّب overfitting لأنّ كل موضع في الصورة صار له معامل خاص.

4) الحقل الاستقبالي: الطبقات التلافيفية الثلاث كلها نواة 3 بخطوة 1، فالحقل = 1 + 2 + 2 + 2 = 7 بكسل. أما التجميع فلا يوسّع الحقل (هو عملية تصغير لا توسّع)، والتجميع العام لا يوسّعه هو الآخر: كل خلية في الخرج العام ترى فعلياً الحقل نفسه 7×7 من الصورة. راجع: الحقل يُحسب على الطبقات التلافيفية فقط.

تمرين 2 — صمّم شبكة صغيرة ثم قِس

1. اكتب `nn.Module` باسم `TinyNet` فيه ثلاث طبقات تلافيفية (32، 64، 128 قناة) وتطبيع `BatchNorm` وتفعيل `ReLU` وتجميع أقصى، ورأس `AdaptiveAvgPool2d(1)` و `Dropout` و `Linear` .
2. ادفع الصورة `torch.randn(8, 3, 96, 96)` عبر النموذج واطبع شكل الخرج وعدد المعاملات.
3. غيّر `AdaptiveAvgPool2d(1)` إلى `AdaptiveAvgPool2d(4)` وبيّن كيف يتغيّر شكل الخرج وعدد معاملات `Linear` (توقّع: 128×4×4 = 2048 مُدخلاً).
4. فكّر: لماذا يُستخدم التجميع العام (1) في `ResNet` بدل قيمة أكبر؟

▲ الحل النموذجي

Python

```python
import torch
import torch.nn as nn


class TinyNet(nn.Module):
    def __init__(self, num_classes: int = 10):
        super().__init__()
        self.body = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1, bias=False), nn.BatchNorm2d(32), nn.Re
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1, bias=False), nn.BatchNorm2d(64), nn.R
            nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3, padding=1, bias=False), nn.BatchNorm2d(128), nn
        )
        self.pool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Linear(128, num_classes)

    def forward(self, x):
        x = self.pool(self.body(x)).flatten(1)
        return self.fc(x)


net = TinyNet()
x = torch.randn(8, 3, 96, 96)
print(net(x).shape)                          # الناتج: torch.Size([8, 10])
print("params:", sum(p.numel() for p in net.parameters()))

net.pool = nn.AdaptiveAvgPool2d(4)
net.fc = nn.Linear(128 * 4 * 4, 10)
print(net(x).shape)                          # الناتج: torch.Size([8, 10])
print("params:", sum(p.numel() for p in net.parameters()))
```

التفسير: مع `AdaptiveAvgPool2d(4)` يصبح مُدخل الطبقة الكثيفة 2048 قيمة بدل 128، أي **16 مرّة** أكثر معاملات في الرأس. لماذا نفضّل (1)؟ لأنّ الرأس الأكبر يعادل طبقات كثيفة ضخمة فوق تمثيل كثيف، فترتفع معاملات الشبكة إلى ملايين وتنهار قدرتها على التعميم مع

بيانات محدودة. كما أنّ التجميع العام يُسقط البُعد المكاني فيصبح النموذج يقبل أي حجم دخل – وهي خاصية عملية مهمة في الإنتاج.

تمرين 3 — برمجة النواة بالـNumPy وأثبت التطابق

1. اكتب دالة `conv2d_numpy(x, k, stride=1, pad=0)` تنفّذ معادلة الالتفاف بوضوح تام (لا تستعمل `torch.nn.functional`).
2. قارن نتيجتها بـ `nn.Conv2d` على دخل عشوائي 5×5 ونواة 3×3 مع حشو 1 وخطوة 2.
3. استعمل الاختبار لتكتشف أيّ فرق (وهو متوقّع): هل الاختلاف في قيمة واحدة أم في ترتيب الأبعاد؟

▲ الحل النموذجي

Python

```python
import numpy as np
import torch
import torch.nn as nn


def conv2d_numpy(x, k, stride=1, pad=0):
    """x: (C,H,W) أو (N,C,H,W)، k: (Cout,Cin,Kh,Kw)."""
    x = np.asarray(x, dtype=np.float32)
    k = np.asarray(k, dtype=np.float32)
    if x.ndim == 3:
        x = x[None]
    n, c, h, w = x.shape
    cout, cin, kh, kw = k.shape
    if pad:
        x = np.pad(x, ((0, 0), (0, 0), (pad, pad), (pad, pad)))
    oh = (h + 2 * pad - kh) // stride + 1
    ow = (w + 2 * pad - kw) // stride + 1
    out = np.zeros((n, cout, oh, ow), dtype=np.float32)
    for ci in range(cin):
        for i in range(oh):
            for j in range(ow):
                patch = x[:, ci, i * stride:i * stride + kh, j * stride:j * stride
                out[:, :, i, j] += np.einsum("nkhw,khw->n", patch, k[None, ci, :,
    return out


rng = np.random.default_rng(0)
x = rng.standard_normal((1, 3, 5, 5)).astype(np.float32)
w = rng.standard_normal((4, 3, 3, 3)).astype(np.float32)

mine = conv2d_numpy(x, w, stride=2, pad=1)
layer = nn.Conv2d(3, 4, 3, stride=2, padding=1, bias=False)
layer.weight.data = torch.from_numpy(w)
with torch.no_grad():
    theirs = layer(torch.from_numpy(x)).numpy()
print("shape mine :", mine.shape)
print("shape torch:", theirs.shape)
print("max abs diff:", np.abs(mine - theirs).max())
```

التفسير: `max abs diff` سيكون في حدود `1e-6` (فروق ترتيب الجمع العائم). والأهمّ ما في التمرين: `mine.shape == theirs.shape == (1, 4, 3, 3)`. لو حصلت على `(1, 3, 3, 4)` فأنت فكّرت بترتيب القنوات في `PyTorch` خطأً: هنا القناة الأولى (كما في `NumPy`) و`PyTorch` يضع القناة أولاً أيضاً، أمّا `Keras` فهو يضعها أخيراً. في بيئات الشيفرة المختلطة، هذا الاختلاف هو مصدر الأخطاء الأكثر شيوعاً.

ملخّص الفصل

- الشبكة التلافيفية تكتسب قوتها من مبدأين: **انزلاق النواة** على الصورة و**مشاركة الأوزان** بين كل المواضع، فينخفض عدد المعاملات من 150 مليون إلى أقل من ألفين في الطبقة الأولى.
- معادلة الالتفاف تجمع حاصل ضرب الرقعة في النواة على كل القنوات وكل مواضع النافذة، ثم تضيف إزاحة لكل قناة إخراج. `F.conv2d` ينفّذ الارتباط عبراني (بلا قلب النواة)، وهو ما لا يغيّر شيئاً لأن الأوزان متعلَّمة.
- بُعد الخرج يحكمه `O = floor((I + 2P − D(K−1) − 1)/S) + 1`، و`padding="same"` في `PyTorch` و`Keras` معاً يبقيان البُعد ثابتاً عند الخطوة 1.
- التجميع يخفض الأبعاد المكانية، و`AdaptiveAvgPool2d(1)` يجعل الشبكة مستقلّة عن حجم الصورة ويجعل الرأس رخيصاً.
- الحقل الاستقبالي يكبر بـ `r ← r + (k−1)·j`؛ ثلاث طبقات 3×3 تعطي حقلاً 7×7 بأقل من نصف معاملات طبقة 7×7 واحدة.
- العمود الفقري الكلاسيكي هو `Conv → BatchNorm → ReLU`، والرأس هو `AdaptiveAvgPool → Dropout → Linear` مع `logits` خام في `PyTorch` واحتمالات في `Keras`.
- `ResNet` تعلّم الباقي `h(x) = ReLU(F(x) + x)`، فتصبح الزيادة في العمق مكسباً لا عبئاً، وهو ما مكّن من بناء شبكات بعمق 50 و101 و152 طبقة.
- التصنيف والكشف والتجزئة تشترك في العمود الفقري نفسه؛ ما يتغيّر هو الرأس – وهذا بالضبط ما يجعل التعلّم النقلي في الفصل التالي فعّالاً.

التعلّم النقلي والتدرّج الدقيق

استخراج الميزات، التدرّج الدقيق، معدّلات التعلّم المتمايزة، ومتى تستخدم كل منها

تخيّل مهندساً في مصنع بلاط يجب أن يفرّق بين ستّة أنواع من عيوب السطح. الحلّ الساذج: بناء شبكة عصبية من الصفر على ثلاثة آلاف صورة. الحلّ الذي يختاره كل مهندس محترف في العالم: أخذ شبكة تعلّمت مئتي مليون صورة قبله، وإعطائها مهمّة تصنيف جديدة. هذا الفصل يشرح لماذا يعمل هذا التطويل – بل لماذا يعمل أفضل من التعلّم من الصفر – وكيف تنفّذه في `PyTorch` و `Keras`، ولماذا يخطئ كثيرون في أبطأ مرحلة ممكنة: مرحلة التدرّج الدقيق.

لماذا يُعدّ التعلّم النقلي أرخص وأدقّ من البدء من الصفر؟

في العقد الأول من التعلّم العميق كان القدر الهائل من البيانات شرطاً لازماً. النموذج الذي يفرّق بين عشرة أصناف من الطيور يحتاج ملايين الصور الموسومة. الفكرة إذاً كانت بسيطة ومكلفة: اجمع البيانات، سجّلها، ثم تُشترى، وكل صورة تُوسَم.

ثم حدث ما غيّر المعادلة: الشبكات التلافيفية تعلّمت على `ImageNet` (١٤ مليون صورة في ١٠٠٠ فئة) لعُشر سنوات، ثم تعلّمها ظهوراً: النماذج المدرَّبة على `ImageNet` تعمل كمستخرج سمات (feature extractor) عام. خذ الطبقة قبل الأخيرة في `ResNet-50`، وستحصل على متجه من ٢٠٤٨ قيمة يمثّل «محتوى» الصورة: وجود وجه، وجود زاوية، خشونة السطح، انحناء الخطّ. هذه السمات مفيدة في مهامّ لم تُسمَّ في الأصل.

$$f(x) = g_{\text{pretrained}}(x) \in \mathbb{R}^d$$

ما معنى كل رمز؟ `x` هي الصورة المدخلة، و $g_{\text{pretrained}}$ هو الجزء المُثبَّت من الشبكة المدرَّبة مسبقاً، و `d` هو بُعد المتجه الناتج (٢٠٤٨ في مثالنا)، و $\mathbb{R}^d$ هو فضاء المتجهات الحقيقية ذات البُعد `d`. **لماذا تهمّنا؟** لأنّ `f` تحوّل الصورة من بكسلات إلى وصف عددي مكثّف، بُعده أصغر بكثير وأشدّ ارتباطاً بالمعنى من البكسل نفسه. **خطأ شائع عندها:** ظنّ أنّ النموذج المدرَّب مسبقاً «يعرف» فئتك الجديدة؛ لا يعرف شيئاً عنها. ما يعرفه هو بنية العالم البصري، لا فئات فئتك.

وهذا بالضبط ما يجعل التعلّم النقلي ممكناً: أنت لا تنتقل بالمعرفة، بل تنقل البنية (inductive bias) التي تعلّمها الشبكة عن الصور.

المعيار	التدرّب من الصفر	التعلّم النقلي
عدد العيّنات المطلوب	عشرات الآلاف لكل فئة	بضمائات إلى بضعة آلاف
زمن التدريب على وحدة رسوميات واحدة	أيام إلى أسابيع	ساعات
أداء مع بيانات محدودة	ضعيف، يهلوس	قوي ومُعمَّم
مخاطرة الفرط في التخصيص	مرتفعة جداً	منخفضة نسبياً
حجم النموذج النهائي	مصمَّم من الصفر	كبير عادةً
الحساب المطلوب في الإنتاج	خفّ	أثقل

مقارنة عملية: من الصفر مقابل التعلّم النقلي

شركة أدوية صيدلانية تملك ٤٠٠ صورة أشعة للعمود الفقري، وتحتاج كشف ثلاث عشرة حالات مرضية. لماذا يقترح عليها فكرةقة التعلّم النقلي على الشبكة التلافيفية بالذات، لا الانتقال إلى محوّل (Transformer) مدرَّب على مليارات الصور؟ اذكر ثلاثة أسباب مرتّبة بأهمّيتها.

الجواب يمسك ثلاثة اعتبارات مرتّبة. الأولى: **حجم البيانات** – ٤٠٠ صورة لا تكفي ل تدريب محوّل من الصفر، بينما تكفي تماماً لتدريب رأس (head) جديد فوق شبكة تلافيفية قائمة. الثانية: **طبيعة المهمة** – الأشعة ثنائية الأبعاد، والتلافيف صُمِّمت أصلاً لاستخراج أنماط موضعية متدرّجة في الصور، وهذا هو بنية الأشعة تماماً. الثالثة: **التمويل والحوسبة** – فتدريب محوّل من الصفر على بياناتك وحدها يعني أسابيعاً من زمن وحدة رسوميات تتجاوز ميزانيّة المشروع. لاحظ أنّ الترتيب يختلف لو كانت المهمة نصوصاً أو صوتاً: هناك يتفوّق المحوّل.

★ القاعدة التي تحكم القرار

ابدأ من النموذج المدرَّب مسبقاً الأقرب إلى طبيعة بياناتك (نوع الخطّ، الدقة المطلوبة، حجم الصور)، لا من البنية التي تسمع عنها كثيراً. نموذج EfficientNet-B0 على صور ٢٢٤×٢٢٤ يناسب فحصاً صناعياً أو طبياً أفضل بكثير من ViT ضخم يقتطع صورة كاملة.

المرحلة الأولى: استخراج السمات بتجميد العظام

أبسط أشكال التعلّم النقلي هو **استخراج السمات** (feature extraction): تُجمَّد كل أوزان الشبكة المدرَّبة مسبقاً، وتُستبدل الطبقة الأخيرة برأس جديد يُتدرَّب وحده. الشبكة المجمّدة تتحوّل إلى مُحوّل (transformer) ثابت: تأخذ الصورة، تُخرج متجهاً. أنت لا تحتاج أن تفهم ما يحدث داخلها.

Python

```python
import torch
import torch.nn as nn
from torchvision.models import resnet50, ResNet50_Weights

# 1) تحميل الأوزان المدرَّبة مسبقاً
weights = ResNet50_Weights.IMAGENET1K_V2
backbone = resnet50(weights=weights)

# 2) تجميد كل المعاملات: لن تتغيّر أثناء التدريب
for param in backbone.parameters():
    param.requires_grad = False

# 3) استبدال طبقة التصنيف بمُخرَج سمات (قيمة 2048)
backbone.fc = nn.Identity()

# 4) رأس جديد فوق العظام المجمّدة
head = nn.Sequential(
    nn.LazyLinear(256),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(256, 6),
)

model = nn.Sequential(backbone, head)
print(sum(p.requires_grad for p in model.parameters()), "trainable tensors")
# الناتج: 4 trainable tensors
```

ماذا يحدث هنا؟ السطر `requires_grad = False` هو جوهر الأمر كلّه: يخبر `autograd` ألا يجمع التدرّجات لهذه المعاملات، فلا تُحدَّث ولا تتآكل. `nn.Identity()` بدل طبقة `fc` تعني: مرّر المخرج كما هو بدل تطبيق الطبقة الخطية، فتصبح الشبكة المصمَّمة تنتج متجه سمات لا تنبؤات. **لماذا هذه القيمة؟** لأنّ `LazyLinear` يكتشف بُعد الدخل عند أول تمريرة فعلية، فلا نحتاج أن نعرف أبعاد `ResNet-50` مسبقاً.

وفي `Keras` يحقّق نفس الفكرة بأمر واحد:

```python
import keras
from keras.applications import EfficientNetB0
from keras import layers

base = EfficientNetB0(
    include_top=False,          # نريد السمات لا طبقة التصنيف
    weights="imagenet",
    input_shape=(224, 224, 3),
)
base.trainable = False          # تجميد العظام

model = keras.Sequential([
    base,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.3),
    layers.Dense(6, activation="softmax"),
])

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
model.summary()
```

ماذا يحدث هنا؟ `include_top=False` يزيل طبقة التصنيف الأصلية، و `base.trainable = False` يجمّد كل ما تحتها. عندئذٍ يُدرَّب رأس Dense فقط. **لماذا هذه القيمة؟** معدّل تعلّم `1e-3` مناسب لرأس جديد بالكامل؛ الشبكة المجمّدة لا تحتاج تدرّجات أصلاً.

◆ الفرق العملي بين المكتبتين هنا

في `PyTorch` التجميد عملية صريحة على مستوى المعاملات (`requires_grad`)، وفي `Keras` عملية صريحة على مستوى الطبقات (`layer.trainable`). النتيجة واحدة، لكنّ عادة الكود تختلف: في PyTorch ستنسخ القائمة `named_parameters()` قبل التجميد؛ وفي Keras ستُمرَّر على الطبقات. تذكّر: في Keras، تغيير `trainable` بعد بناء النموذج يستدعي إعادة بناء النموذج تلقائياً.

لماذا هذه القيمة؟ لأنّ `fc` بدل طبقة التصنيف يعني: مرّر المخرج كما هو

العملية	PyTorch 2.14	Keras 3.15
تجميد كل المعاملات	`p.requires_grad = False`	`base.trainable = False`
تجميد جزء فقط	تعديل قائمة `requires_grad` يدوياً	`for l in base.layers[:k]: l.trainable = False`
معدّلات تعلّم متمايزة	`param_groups` في المُحسّن	
تجميد إحصاءات التطبيع	`model.eval()` على الطبقة	`layer.trainable = False` كافي
معاينة عدد المتدرّبات	`sum(p.requires_grad for p in ...)`	`model.count_params()`

مقابلة عملية: تجميد العظام في `PyTorch` مقابل `Keras`

المرحلة الثانية: التدرّج الدقيق ومعدّلات التعلّم المتمايزة

استخراج السمات يكفي حين تكون المهمة «قريبة» من مهمة النموذج الأصلي. لكن إذا كان نموذجك الأصل مدرَّباً على صور طبيعية (قطط، سيارات، أشخاص)، بينما بياناتك أشعة أو بلاط أو محاصيل، فالسمات العامة مفيدة لكنّ السمات المتخصّصة غير موجودة. هنا يبدأ **التدرّج الدقيق** (fine-tuning): نُعيد بعض طبقات الشبكة إلى حالة التدريب، ونسمح لها بأن تعدّل نفسها – بمعدّل تعلّم أصغر بكثير من معدّل الرأس الجديد.

لماذا معدّل أصغر؟ لأنّ الطبقة المتقدّمة (القريبة من الدخل) تعلّمت أنماطاً عامّة مثل «حافّة» و«نسيج»؛ تعديلها بعنف يُتلف معرفة تراكمية تحتاج ملايين الصور لا تعيدها مئات الصور. أمّا الطبقات القريبة من الخرج فهي مخصّصة للمهمة الأصلية، فيجب أن تتكيّف أكثر. القاعدة الشائعة: **معدّل التعلّم يزداد كلّما اقتربنا من الخرج**.

$$\mathrm{lr}_{l} = \mathrm{lr}_{\mathrm{base}} \cdot \gamma^{(L-l)}$$

ما معنى كل رمز؟ lr_{l} هو معدّل التعلّم للطبقة رقم l، و $\mathrm{lr}_{\mathrm{base}}$ هو المعدّل في أعمق طبقة، و γ (قيمة أقل من ١) هو معامل اضمحلال لكل خطوة نحو الخرج، و L هو عدد الطبقات. **لماذا تهمّنا؟** هذه المعادلة تُنتج منحنى

تعلّم هابطاً كلّما اتّجهنا نحو layers. المدخلات، فيصبح التدرّج الدقيق «محمياً» في الأسفل ومتحرّكاً في الأعلى. **خطأ شائع عندها:** اختيار `٧` قريباً جداً من ١ (مثل ٠٫٩٩) يجعل كل الطبقات عملياً متساوية، فيفقد التدرّج الدقيق أمانه.

إليك التنفيذ الكامل في `PyTorch`. المفتاح هو `param_groups`: قوائم معاملات لكل واحدة معدّل تعلّم خاص.

```python
import torch
import torch.nn as nn
from torchvision.models import resnet50, ResNet50_Weights

backbone = resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)
backbone.fc = nn.Identity()

head = nn.Sequential(
    nn.Linear(2048, 256),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(256, 6),
)


def build_param_groups(model, base_lr, head_lr, decay=0.9):
    """قسّم معاملات النموذج إلى مجموعات حسب العمق في العظام."""
    layer_ids = {id(p): i for i, p in enumerate(model.backbone.parameters())}
    num_layers = len(layer_ids)
    groups = {}
    for name, param in model.backbone.named_parameters():
        depth = layer_ids[id(param)]
        lr = base_lr * (decay ** (num_layers - 1 - depth))
        groups.setdefault(round(lr, 10), []).append(param)

    param_groups = [{"params": ps, "lr": lr} for lr, ps in groups.items()]
    param_groups.append({"params": list(model.head.parameters()), "lr": head_lr})
    return param_groups


model = nn.Sequential(backbone, head)
optimizer = torch.optim.AdamW(
    build_param_groups(model, base_lr=1e-5, head_lr=1e-3, decay=0.9),
    weight_decay=1e-4,
)
```

```
print(len(optimizer.param_groups), "learning-rate groups")
# 7 :الناتج learning-rate groups
```

ماذا يحدث هنا؟ `layer_ids` يبني خريطة من كل معامل إلى موضعه في ترتيب العظام، ثم نحسب لكل معامل معدّله بمعدّل التحديث. `groups.setdefault` تجمع المعاملات ذات المعدّل نفسه (المعدّلات متناسبة مع ٠٫٩ فتتكرّر) في مجموعة واحدة، وهذا أصغر حزمة يمكن تمريرها إلى المُحسّن. **لماذا هذه القيمة؟** `1e-5` في العظام و `1e-3` في الرأس: نسبة مئة إلى واحد. هذه ليست قاعدة صارمة، بل نقطة انطلاق تُضبط بالتحقّق.

⚠ الخطأ الذي يكلّف المشروع أسبوعاً

عند التدرّج الدقيق، **لا** تُجمِّد معاملات `BatchNorm` في الطبقات المجمّدة. فالإحصاءات (المتوسط والانحراف) المتراكمة على `ImageNet` تمثّل إحصاءات خفيفة لتوزيع بياناتك، وتحديثها بأربعين صورة يجعلها عديمة القيمة. في `PyTorch` ضع طبقة التطبيع في وضع `eval()` حتى أثناء التدريب: `for m in model.modules(): if isinstance(m, nn.BatchNorm2d): m.eval()` . وفي `Keras` يكفي تجميد الطبقة نفسها.

وفي `Keras` ، التنفيذ المكافئ الأقرب هو **التفريغ التدريجي** (progressive unfreezing): تدرّب الرأس أولاً مع عظام مجمّدة، ثم افتح طبقة إضافية في كل مرحلة، ثم انقل إلى تدرّج كامل بمعدّل صغير.

Python

```
import keras
from keras.applications import ResNet50
from keras import layers

base = ResNet50(include_top=False, weights="imagenet", input_shape=(224, 224, 3))
base.trainable = False

inputs = keras.Input(shape=(224, 224, 3))
x = base(inputs, training=False)                #.training=False: لا تحديث إحصاءات BN
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(256, activation="relu")(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(6, activation="softmax")(x)
model = keras.Model(inputs, outputs)

# المرحلة 1: الرأس فقط
```

```
model.compile(keras.optimizers.Adam(1e-3), "sparse_categorical_crossentropy", ["accuracy"])
model.fit(train_ds, validation_data=val_ds, epochs=5)

# المرحلة 2: افتح الكتلة الأخيرة فقط
for layer in base.layers[-10:]:
    layer.trainable = True
model.compile(keras.optimizers.Adam(1e-4), "sparse_categorical_crossentropy", ["accuracy"])
model.fit(train_ds, validation_data=val_ds, epochs=10)

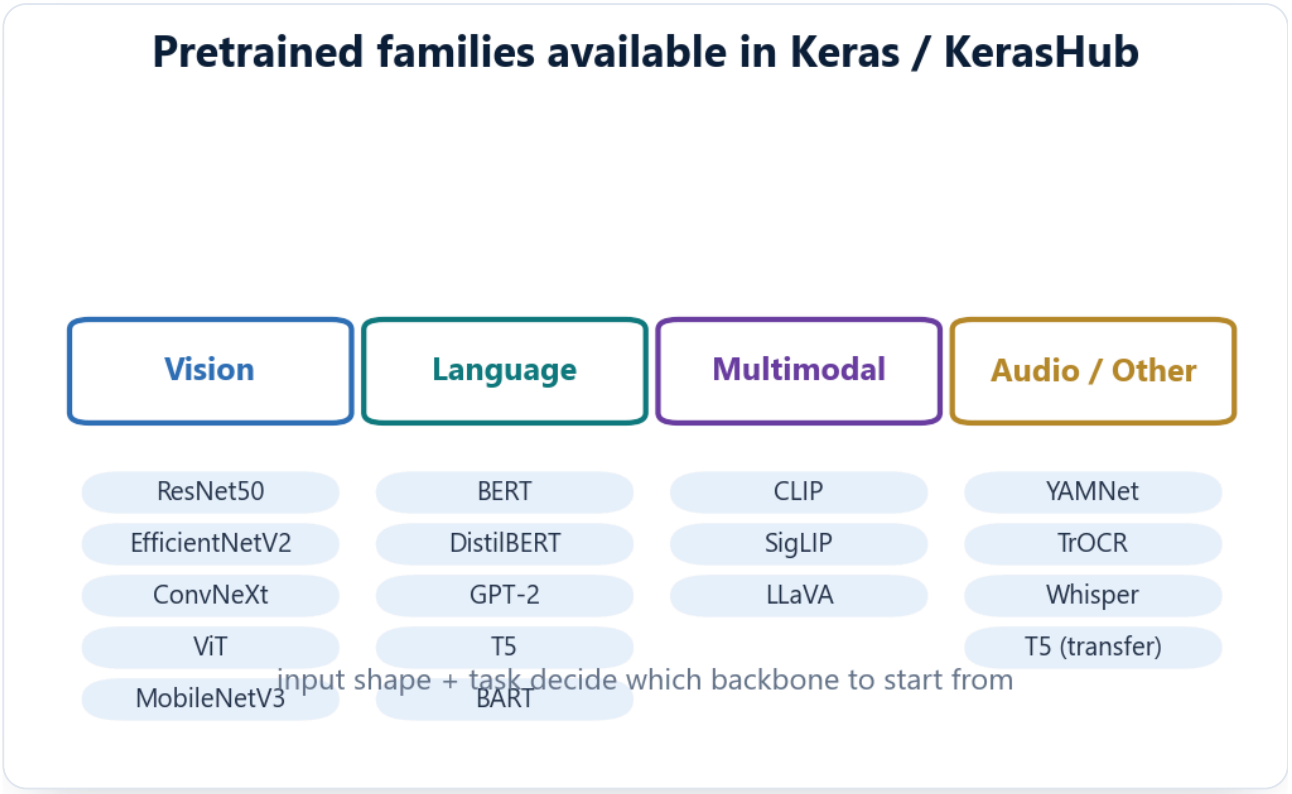
# المرحلة 3: تدرّج كامل بمعدّل صغير جداً
base.trainable = True
model.compile(keras.optimizers.Adam(1e-5), "sparse_categorical_crossentropy", ["accuracy"])
model.fit(train_ds, validation_data=val_ds, epochs=10)
```

ماذا يحدث هنا؟ `training=False` عند استدعاء `base(inputs)` مقصود: هو يُخبر Keras باستخدام إحصاءات التطبيع المخزَّنة بدل تحديثها، وهو ما يماثل وضع `eval()` في PyTorch. **لماذا هذه القيمة؟** تدرّج `1e-3 ← 1e-4 ← 1e-5` بين المراحل يحاكي نفس فكرة معدّلات التعلّم المتمايزة، لكن بتعقيد أقلّ وبنتائج مستقرة.

> أنت تدرّب رأساً جديداً فوق EfficientNet-B0 مجمّد، وبعد ٢٠ حقبة يحقّق دقّة تحقّق ٩٥٪ ثم تنخفض تدريجياً. ما التفسير الأرجح؟ وما الاختبار الذي يميّز بينه وبين احتمال ثانٍ شائع؟ وكيف تعالج كلّ حالة؟

الاحتمالان الكبيران هما: **فرط التخصيص** (overfitting) – الرأس الجديد ألف معاملاًفقط، ومع ٣٠٠٠ صورة يتذكّر أمثلة التدريب؛ أو **انهيار الطبقات المجمّدة** (feature degradation) – الشبكة تتوقّف عن إطلاق سمات مفيدة لأنّ الطبقات العليا تكيّفت على المهمة القديمة. الاختبار الفاصل: **أعد تشغيل التقييم مع نموذج مجمّد تماماً** وقارن. إن كان النموذج المجمّد أفضل من المُدرَّج جزئياً، فالمشكلة انهيار السمات، والحل هو: قلّل معدّل التعلّم، أو استخدم عدداً أكبر من الطبقات المفتوحة تدريجياً، أو توقّف عن التدرّج الدقيق كلياً وعُد إلى استخراج السمات. وإن كان كلاهما متقارباً، فالمشكلة فرط تخصيص، والحل: بيانات أكثر، أو تعزيز بيانات، أو رأس أصغر، أو `weight decay` أقوى.

خريطة النماذج المدرَّبة مسبقاً: أيّها تختار؟



يقرأ الرسم من اليمين إلى اليسار: على اليمين محور «الحجم» (عدد المعاملات وسرعة الاستدلال)، وفي الوسط عمود عائلات البنى (EfficientNet وResNet وMobileNet وViT)، وعلى اليسار المهام التي تخدمها كل عائلة. القاعدة التي يلخّصها الرسم: حرّك على المحور الأفقي فقط عند الحاجة إلى دقّة أعلى، لا إلى عمق أكبر. نموذج EfficientNet-B3 غالباً يحقّق دقّة ResNet-152 بحجم ثلثه.

في `Keras` الوصول إلى هذه المكتبة سطر واحد:

```python
import keras
from keras.applications import (
    EfficientNetB0, EfficientNetB3, MobileNetV3Small,
    ResNet50, ConvNeXtTiny, ViT,
)

models = {
    "effnet_b0": EfficientNetB0,
    "effnet_b3": EfficientNetB3,
    "mobilenet_v3": MobileNetV3Small,
    "resnet50": ResNet50,
    "convnext_tiny": ConvNeXtTiny,
```

```
    "vit": ViT,
}

for name, ctor in models.items():
    m = ctor(include_top=False, weights="imagenet")
    params = sum(p.numel() for p in m.parameters())
    print(f"{name:15s} {params/1e6:7.2f}M parameters")
```

ماذا يحدث هنا؟ نقيس حجم كل عائلة بالنحو. فنكتشف أنّ النموذج «الحديث» ليس بالضرورة الأكبر. **لماذا هذه القيمة؟** لأنّ اختيار العمق يتطلّب موازنة بين الدقّة وزمن الاستدلال وحجم الذاكرة: ثلاثة أرقام تُقاس على بياناتك أنت، لا في ورقة.

✓ كيف تختار في خمس دقائق

ابدأ بـ**EfficientNet-B0** (٥٫٣ مليون معامل) كمخطّط أساسي. ثم انتقل إلى **B3** إن كانت الدقّة مهمّة أهمّ من زمن الاستجابة. استخدم **MobileNetV3** إن كان النموذج سيعمل على هاتف أو جهاز مدمج. استخدم **ConvNeXt** إن أردت بنية حديثة مع تدريب مستقر. ولا تلجأ إلى **ViT** إلا إذا كانت بياناتك كبيرة (عشرات الآلاف على الأقل) لأنّ المحوّلات تحتاج بيانات أكثر لاستخدام مزاياها.

حجم البيانات	الاستراتيجية	معدّل التعلّم	طبقات تُفتح
أقل من ٥٠٠ عيّنة	استخراج سمات فقط	`1e-3` (رأس فقط)	لا شيء
٥٠٠ - ٥٠٠٠ عيّنة	تدرّج جزئي	`1e-4` لآخر ٢٠٪	كتلة أخيرة
٥٠٠٠ - ٥٠٠٠٠ عيّنة	تدرّج كامل مع التدرّج	`1e-5` + LLRD	كل الطبقات
أكثر من ٥٠٠٠٠ عيّنة	تدرّج كامل أو تدرّب من الصفر	`5e-5`	كل الطبقات
فئة مختلفة تماماً	استخراج سمات فقط	`1e-3`	لا شيء

إعدادات التدرّج الدقيق بحسب حجم مجموعة البيانات

سيناريو تطبيقي متكامل: كشف عيوب البلاط

⚙ من البيانات إلى قرار

الموقف: مصنع بلاط في عمّان ينتج ٢٠٠٠ متر مربع يومياً. كاميرا تلتقط كل بلاطة. تُرصد ثلاثة عيوب: تشقّق (crack)، ثقب (pinhole)، تقشّر سطحي (spalling). في مجموعة بيانات الأشقاء فيه ٤٨٠ صورة: ٣٢٠ فيها عيوب و١٦٠ سليمة، والعيب الحقيقي في الإنتاج يظهر في ٢٪ من البلاطات فقط.

القرار: لا نُدرّب `ResNet-50` من الصفر على ٤٨٠ صورة – ذلك انتحال. نستخدم `EfficientNet-B0` مدرَّباً على `ImageNet`، ونستبدل رأسه برأس ثلاث فئات، ونتجمّد العظام في المرحلة الأولى (١٠ حقبات، معدّل `1e-3`)، ثم نفتح آخر ٢٠٪ من الطبقات (١٥ حقبة، معدّل `1e-4`). وبما أنّ العيوب نادرة، نستخدم مقاييس غير متوازنة: `Recall` على صنف التشقّق أهمّ من الدقّة العامة، لأنّ تفويت تشقّق يكلّف أكثر من تنبيه على خدش.

القياس: نقيس `Recall` على صنف التشقّق عند عتبة ٠٫٥، ثم نضبط العتبة على مجموعة التحقّق حتى يبلغ `Recall ≥ 0.95` مع بقاء `Precision ≥ 0.6`. بعد ذلك نصدّر النموذج بـ `TorchScript` ليعمل على جهاز الحافة داخل المصنع. هذه هي السلسلة الكاملة: بيانات ← نموذج ← قرار ← نشر.

تمرين 1 — بناء نموذج تدرّج دقيق بمعدّلات متمايزة

1. حمّل `resnet50` بأوزان مدرَّبة، واستبدل `fc` برأس من طبقتين (256 ثم 6).
2. اكتب دالة تُرجع `param_groups` فيها ثلاث مجموعات: كتلتا العظام الأولى والأخيرة برأس.
3. درّب ٥ حقبات، ثم اطبع أكبر معدّل تعلّم في كل مجموعة للتأكّد من صحّة التوزيع.

▲ الحل النموذجي

```python
import torch
import torch.nn as nn
from torchvision.models import resnet50, ResNet50_Weights

model = nn.Sequential()
model.add_module("backbone", resnet50(weights=ResNet50_Weights.IMAGENET1K_V2))
model.backbone.fc = nn.Identity()
model.add_module("head", nn.Sequential(
    nn.Linear(2048, 256), nn.ReLU(), nn.Dropout(0.3), nn.Linear(256, 6),
))

backbone_params = list(model.backbone.named_parameters())
third = len(backbone_params) // 3

groups = [
    {"params": [p for _, p in backbone_params[:third]], "lr": 1e-6},
    {"params": [p for _, p in backbone_params[third:]], "lr": 1e-5},
    {"params": list(model.head.parameters()), "lr": 1e-3},
]

optimizer = torch.optim.AdamW(groups, weight_decay=1e-4)
for g in optimizer.param_groups:
    print(f"lr={g['lr']:.0e} tensors={len(g['params'])}")
```

`1e-06` و `1e-05` و `1e-03`. الملاحظة المهمّة: الكتلتان الأولى والأخيرة من العظام لهما نفس العدد تقريباً من المعاملات، لكنّ الأولى مضروبة بعشرة أضعاف أخفّ لأنّها أعمق.

تمرين 2 — تشخيص انهيار السمات

1. درّب نموذجاً على ٥٠٠ صورة بمعدّل تعلّم `1e-3` على كل الطبقات.
2. قِس دقّة التحقّق، ثم قِسها مجدداً مع تجميد العظام تماماً والتدريب على الرأس فقط.
3. فسّر الفرق، واقترح إعداداً واحداً للتحسين.

▲ **الحل النموذجي**

النتيجة المتوقّعة: التدريب الكامل بمعدّل `1e-3` يعطي دقّة تحقّق أسوأ من تجميد العظام، رغم دقّة التدريب الأعلى. هذا هو انهيار السمات: التدرّج الكبير محا السمات المفيدة التي بُنيت على ملايين الصور.

التفسير: الطبقات القريبة من الدخل تعلّمت مرشّحات عامة (حواف، نسيج) خلال التدريب الطويل على `ImageNet`؛ تغييرها بأربعمئة عيّنة يكفي لتدميرها. الإعداد المقترح: خفض معدّل التعلّم إلى `1e-5`، أو فتح الكتلتين الأخيرتين فقط، أو — إن كانت الدقّة لا تزال غير كافية — العودة إلى استخراج السمات وتوسيع الرأس أو التعزيز بدلاً من تدمير العظام.

ملخّص الفصل

- التعلّم النقلي ينقل بنية تعلّمها نموذج على بيانات ضخمة، لا معرفته بفئاتك.
- استخراج السمات (تجميد العظام) هو الخيار الافتراضي الآمن، وهو خيار يعطي نتائج قوية مع مئات العيّنات.
- التدرّج الدقيق مناسب حين تكون المهمة قريبة من الأصلية؛ معدّل التعلّم فيه أصغر بمراتب من معدّل الرأس.
- معدّلات التعلّم المتمايزة (LLRD) تُنفَّذ في `PyTorch` عبر `param_groups`، وفي `Keras` عبر التفريغ التدريجي أو حلقة تدريب مخصّصة.
- لا تُحدِّث إحصاءات `BatchNorm` في الطبقات المجمّدة، وإلا فسد توزيع السمات.
- اختر عمق النموذج بحسب المهمة والحساب، لا بحسب الشهرة: EfficientNet-B0 غالباً نقطة البداية الأذكى.
- قبل التدرّج الدقيق، قِس أداء النموذج المجمّد كخطّ أساس — أنت تعدّل نموذجاً ناجحاً، وقد تُفسده.

الفصل الرابع عشر غوص عميق في PyTorch

الشبكات المتكرّرة وتسلسليات البيانات

RNN وLSTM وGRU، ومقرّرات الانتشار العكسي عبر الزمن، والتطبيقات

تخيّل مصنعاً ضخماً: كل صباح يقرأ النظام مبيعات الأمس، وحركة المخزون، ودرجات الحرارة، وسرعة الرياح، ثم يقرّر كم طلباً يرسله إلى كل فرع. القرار لا يعتمد على قيمة اليوم وحدها، بل على ما جرت العادة أن يسبق هذا اليوم. الفصل هذا يفتح باباً جديداً في الكتاب: كيف تبني شبكة تتعلّم أن تحتفظ بالسياق، وأن تنقل تذكّرها خطوةً بعد خطوة عبر الزمن؟ سنبدأ من أبسط خليّة متكرّرة، ثم نضيف إليها خزّان ذاكرة ببوابات، ثم نفهم لماذا ينهار الانتشار العكسي عبر الزمن، وأخيراً نبني نظاماً كاملاً لتنبّؤ الطلب – في PyTorch وKeras معاً.

14.1 لماذا لا تكفي الشبكات العميقة للبيانات المتسلسلة؟

في الفصل الثاني عشر تعلّمنا أن الشبكة العصبية العميقة تتلقّى مُدخلاً ثابتاً الأبعاد، ثم يحوّله عبر طبقات إلى مُخرج. صورة بحجم 3×224×224 تدخل الشبكة، وتخرج فئة. المسألة أن هذا الافتراض – أن كل مُدخل مستقلّ عن الذي يسبقه – ينهار أمام البيانات التي تتّسم بالترتيب.

لاحظ المشهد التالي. لدينا سلسلة من قراءات حرارة كل ساعة على مدار يوم. الشبكة العميقة التي تراها كل قراءً على حدة ستعطي الجواب نفسه مهما كان الترتيب: فقراءة 3 درجات في الساعة الواحدة ثم 30 درجة تعطي متوسطات متطابقة. لكن هذه القراءة مهمّة: التسلسل «3 ثم 30» يعني صعوداً حادّاً، و«30 ثم 3» يعني هبوطاً حادّاً. المعلومة ليست في القيم منفردة، بل في العلاقة بينها عبر الزمن. هذه هي الفكرة المركزية للشبكات المتكرّرة: أن تُدخل الحالة السابقة إلى الحساب الحالي.

إذا كان النموذج يتلقّى نافذة من 24 قراءة سابقة ليتنبّأ بالقراءة التالية، فلماذا لا نكتفي بشبكة عميقة على النافذة المجمّعة (مثلاً 24 رقماً واحداً)؟ أين يضيع شيء جوهري؟

الجواب يكمن في ثلاث نقاط. الأولى: الأول، التعلّم فائق الكتابة – الشبكة العميقة ترى 24 رقماً بلا ترتيب، فلا تستطيع أن تتعلّم أن «القيمة قبل الأخيرة» أهمّ من «القيمة الأولى». الثانية: حدّ الأبعاد – لو أردنا نافذة أطول لتغطية موسمية (مثلاً أسبوعاً كاملاً بخطوة كل ساعة = 168 خطوة)، فإن إدخال 168 رقماً في طبقة كثيفة يجعل عدد المعاملات هائلاً. الثالثة، وهي الأخطر: **الذاكرة**، الشبكة العميقة لا تملك أيّ ذاكرة عابرة:

كل مُدخل يُعالَج بمعزل عن الذي قبله، بينما التسلّع الجسدي للحمل (عند هبوط حجرٍ على السطح) لا ينتهي بزوال السبب. وهنا بالذات تظهر الحاجة إلى ذاكرة صريحة تحفظ الأثر.

الشبكة المتكرّرة تحلّ المشكلة الثالثة بأن تجعل الحالة المخفيّة (<code>h</code>) مخرجاً للخلية ومُدخلاً للخلية التالية. وبحلّ الثانية لأنها تشارك نفس الأوزان عبر الزمن، فعدد المعاملات لا ينمو مع طول السلسلة. أمّا الأولى فتبقى جزئياً: الشبكة المتكرّرة تعلّم «أن ينتبه» إلى مواضع معيّنة، لكنّ قدرتها على ربط موضع بعيد بآخر تبقى محدودة – وهذا بالضبط ما حلّته المحوّلات (Transformers) في الفصل الخامس عشر.

> **◆ قراءة مقترحة**
>
> الفصل هذا يبني على الفصل التاسع (هندسة البيانات) لأن كل مثال هنا يحتاج `Dataset` و `DataLoader`، وعلى الفصل العاشر (حلقة التدريب) لأننا سنكتب حلقة تدريب صريحة للسلاسل الزمنية. إن شعرت أن شكل الـ `batch` غامض، فعُد إلى الفصل التاسع أولاً.

14.2 الخلّية المتكرّرة RNN: جوهر الفكرة

لنبدأ من صفر. في اللحظة الزمنية <code>t</code>، تتلقّى الخلّية المتكرّرة مُدخلين: المُدخل الحالي <code>x_t</code> (القراءة عند هذه اللحظة) والحالة المخفيّة السابقة <code>h_{t-1}</code> (ما تعلّمته الشبكة حتى الآن). تُخرج حالتين مخفيّتين جديدتين، ثم تُحوَّل الحالة إلى مُخرج.

```
h_t = tanh(W_xh · x_t + W_hh · h_{t-1} + b_h)
```

معادلة الحالة المخفيّة في الشبكة المتكرّرة البسيطة

لنفكّك كل رمز على حدة، لأن هذه العادة هي ما يفصل بين مهندس يحفظ المعادلة ومهندس يفهمها:

- `h_t`: متّجه الحالة المخفيّة عند الخطوة <code>t</code>، بُعده `hidden_size`. هو «ملخّص» ما عرفته الشبكة حتى هذه اللحظة، ويُمرَّر إلى الخطوة التالية كما هو.
- `x_t`: مُدخل الخطوة t <code>t</code>، بُعده `input_size`. في سلسلة أحاديّة المتغيّرات يكون رقماً واحداً؛ في سلسلة متعدّدة المتغيّرات (الطلب، المخزون، الطقس) يكون متّجهاً.

- `W_xh` : مصفوفة الأوزان التي تصل المُدخل بالحالة، شكلها `(input_size × hidden_size)` . هي التي تترجم «ماذا يحدث الآن» إلى لغة الحالة المخفيّة.
- `W_hh` : مصفوفة الأوزان التي تصل الحالة السابقة بالحالة الحالية، شكلها `(hidden_size × hidden_size)` . هي التي تجعل النموذج «متكرّراً» حقّاً: من دونها لا وجود لتاريخ على الإطلاق.
- `b_h` : متّجه الإزاحة، يمنح الحالة القدرة على تحريك مركزها.
- `tanh` : الدالة غير الخطّية المقيّدة بين -1 و 1 . هي التي تمنع التضخّم الهائل في الأرقام (تلاشي وانفجار التدرّجات) وتسمح بتمثيل «قوي/ضعيف» لا «موجود/غائب» فحسب.

ثم يُحسَب المُخرج من الحالة المخفيّة:

$$y_t = W_{hy} \cdot h_t + b_y$$

معادلة المُخرج في RNN (مهمة، لكن ليس كل RNN يحتاجها)

`W_hy` يحوّل الحالة المخفيّة إلى مساحة المخرجات (مثلاً قيمة عددية، أو توزيع احتمالات على أصناف). الفكرة الجوهرية في RNN ليست في `y_t` ، بل في `h_t` : كل خطوة تستهلك ما قبلها. ولهذا تُسمّى هذه البنية «متكرّرة» – التكرار موجود في الزمن لا في البُنى فقط.

لنكتب المعادلة صراحةً في PyTorch بخطواتها، لأن رؤية الحلقة الزمنية (حلقة `for` على محور الزمن) توضّح لماذا يسمّيها «الحلقة» أصلاً:

```python
import torch
import torch.nn as nn

torch.manual_seed(0)


class SimpleRNNCell(nn.Module):
    """خلية RNN واحدة: h_t = tanh(W_xh x_t + W_hh h_{t-1} + b)."""

    def __init__(self, input_size: int, hidden_size: int):
        super().__init__()
        self.hidden_size = hidden_size
        self.W_xh = nn.Parameter(torch.empty(input_size, hidden_size))
        self.W_hh = nn.Parameter(torch.empty(hidden_size, hidden_size))
```

```
        self.b = nn.Parameter(torch.zeros(hidden_size))
        # Xavier و للمدخل، للأوزان العابرة للزمن orthogonal للأوزان العابرة:
        # التهيئة الجيدة هنا تختصر زمن التدريب وتقلّل الانفجار العددي.
        nn.init.xavier_uniform_(self.W_xh)
        nn.init.orthogonal_(self.W_hh)
        nn.init.zeros_(self.b)

    def forward(self, x, h_prev):
        # x: (batch, input_size)   |   h_prev: (batch, hidden_size)
        h = torch.tanh(x @ self.W_xh + h_prev @ self.W_hh + self.b)
        return h


cell = SimpleRNNCell(input_size=3, hidden_size=4)
x_seq = torch.randn(2, 5, 3)          # (batch, time=5, features=3)
h = torch.zeros(2, cell.hidden_size)   # الحالة الابتدائية صفرية
for t in range(x_seq.size(1)):
    h = cell(x_seq[:, t, :], h)        # خطوة زمنية واحدة

print("h_T shape:", h.shape)            # الناتج: torch.Size([2, 4])
print("h_T:", h)
```

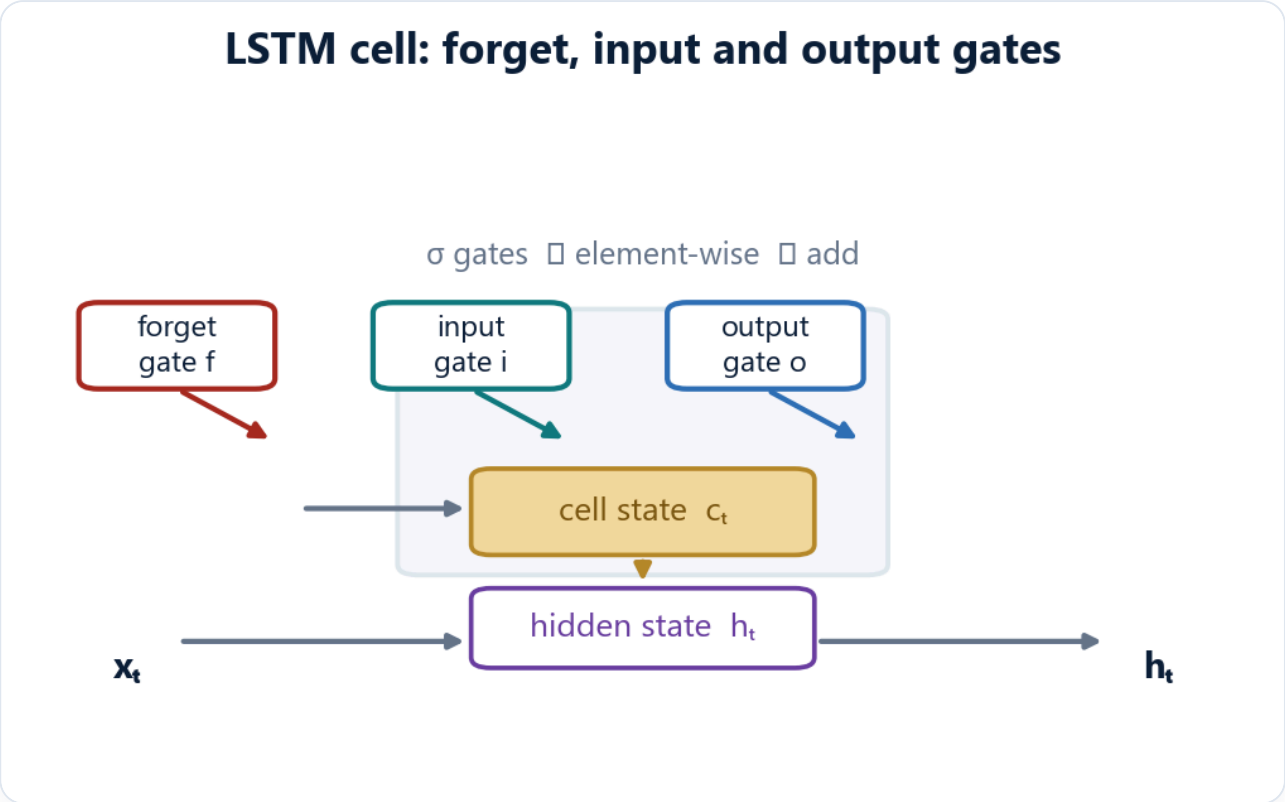
ماذا يحدث هنا؟ الشبكة لا تملك أيّ ذاكرة إلا `h` وهو متّجه من 4 أرقام، وهو ما ينتقل من خطوة إلى خطوة. لماذا هذه القيمة؟ لأن `h` بوصفه المتّجه الوحيد الذي يربط الماضي بالمستقبل، فإن أيّ معلومة لا تُخزَّن فيه تُمحى في الخطوة التالية. هذه ليست مجازاً بل قيدٌ هندسي: الشبكة المتكرّرة البسيطة تملك ذاكرة بحجم `hidden_size` رقماً فقط.

> ⚠ **الخطأ الكلاسيكي: تسطيح السلسلة ثم تمريرها**
>
> لا تُسطّح سلسلة زمنية إلى متّجه واحد ثم مرّرها عبر `nn.Sequential`. هكذا تختفي المعلومة عن الترتيب، وتصبح الشبكة مجرّد شبكة عميقة على نافذة. وبدلاً من ذلك حافظ على البُعد الزمني صريحاً: `(batch, seq_len, features)`. إن استخدمت `nn.Flatten` على محور الزمن فأنت تحذف البُعد الذي يميّز هذه العائلة من البنى.

14.3 LSTM: خزّان ذاكرة ببواباتٍ محسوبة

هنا تعلّمت الشبكات المتكرّرة الهامة التي مكّنتها من أن تصبح عملية. المشكلة التي حلّها LSTM (`long short-term memory`) هي محدّدية RNN البسيطة: لأن `h_{t-1}` يمرّ في `tanh` عند كل خطوة، فإن

أيّ إشارة صغيرة تتضاعف `hidden_size` مرّات، فتموت. الإشارات الضعيفة تُنسى بعد بضع خطوات فقط. الحلّ: فصل «الذاكرة» عن «الحالة المُخرَجة»، وإضافة بواباتٍ (`gates`) تتحكّم في تدفّق المعلومات.



خلية LSTM في تطبيق واحد: منظورٌ عملي. الخط الأفقي العلوي يمثّل «حالة الخلية» `c_t` وهي المسار الذي يحمل الذاكرة عبر الزمن، وتتحكّم فيه بوابتان: `f` تنسى، و `i` تكتب. الخط السفلي هو `h_t` وهو ما يُمرَّر فعلياً إلى الخطوة التالية ويُنتج المُخرج. لاحظ أن `h_t` يُشتقّ من `c_t` وليس العكس: أي أن الذاكرة الواسعة تُصفّى قبل الخروج، تماماً كما تُصفّى الذاكرة قبل أن تتكلّم.

لنكتب المعادلات الست واحدةً واحدة، ثم نشرح رموزها، ثم نشرح لماذا تهمّنا.

$$i_t = \sigma(W_{ii} \cdot x_t + W_{hi} \cdot h_{t-1} + b_i)$$

بوابة الإدخال (`gate` `input`)

$$f_t = \sigma(W_{if} \cdot x_t + W_{hf} \cdot h_{t-1} + b_f)$$

بوابة النسيان (`gate` `forget`)

$$g_t = \tanh(W_{ig} \cdot x_t + W_{hg} \cdot h_{t-1} + b_g)$$

مرشّح المحتوى (candidate)

$$o_t = \sigma(W_{io} \cdot x_t + W_{ho} \cdot h_{t-1} + b_o)$$

بوابة الإخراج (output gate)

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

تحديث حالة الخلية

$$h_t = o_t \odot \tanh(c_t)$$

تحديث الحالة المخفيّة

ما معنى كل رمز؟ الرمز σ هو الدالة اللوغاريتمية (sigmoid) التي تُخرج قيماً بين 0 و1، فتُقرأبواباتها كنِسَب: $f_t = 0.9$ تعني «احتفظ بتسعين بالمئة من الذاكرة القديمة»، و $i_t = 0.1$ تعني «أدخل عشرة بالمئة فقط من الجديد». الرمز $\odot$ هو الضرب عنصراً بعنصر (hadamard product) وليس ضرب المصفوفات: كل خانة في c_t تُضرب في بوابتها المقابلة. الرمز W_{ii} يعني «الأوزان من المُدخل x إلى البوابة i »، و W_{hi} يعني «الأوزان من الحالة h إلى البوابة i »؛ و b_i إزاحة البوابة.

لماذا تهمّنا هذه المعادلات؟ لأن c_t هي المسار الذي يمرّ دون دالة $\tanh$ كل خطوة (إلا عند البابتين i و f). فبينما تتقلّص الإشارة في RNN بنسبة $|\lambda|$ في كل خطوة، تظلّ في LSTM نسبة قريبة من 1 طالما كانت f_t قريباً من 1. وبهذا تبقى الذاكرة عبر مئات الخطوات. أمّا الخطأ الشائع فيفهمها الناس خطأً بأنّ c_t «ذاكرة بلا حدّ»؛ والحقيقة أنّ LSTM تعلّم متى يحتفظ ومتى ينسى، وهذا تعلّم من البيانات لا قاعدة ثابتة.

من أين تأتي بوابة النسيان؟

تخيّل سلسلة موسيقية يتغيّر فيها لحن فجأة. RNN البسيط يُمزج المعلومة القديمة والجديدة بنسبة ثابتة مفروضة. LSTM يضع f_t بين يدي الشبكة: إن كانت f_t قريبة من الصفر، تُمحى

`c_{t-1}` بالكامل تقريباً وتبدأ الخلية من جديد. هذه «القدرة على إعادة البدء» هي سرّ ثبات RNN على سلاسل طويلة.

ولنجرّب ترجمة المعادلات إلى شيفرة مباشرة، لأن أفضل طريقة لفهم LSTM هي أن تكتب بواباتها بنفسك مرّة واحدة على الأقل:

```python
import torch
import torch.nn as nn


class LSTMCell(nn.Module):
    """خلية LSTM 14.3 في القسم — تطبيقٌ مباشر للمعادلات ببواباتٍ صريحة."""

    def __init__(self, input_size: int, hidden_size: int):
        super().__init__()
        self.hidden_size = hidden_size
        # نجمع البوابات الأربع في مصفوفتين كبيرتين: ترتيب القطع هو i, f, g, o
        self.weight_ih = nn.Parameter(torch.empty(4 * hidden_size, input_size))
        self.weight_hh = nn.Parameter(torch.empty(4 * hidden_size, hidden_size))
        self.bias = nn.Parameter(torch.zeros(4 * hidden_size))
        nn.init.xavier_uniform_(self.weight_ih)
        nn.init.orthogonal_(self.weight_hh)
        nn.init.zeros_(self.bias)

    def forward(self, x, h_prev, c_prev):
        # gates: (batch, 4*hidden_size) = x W_ih^T + h W_hh^T + b
        gates = x @ self.weight_ih.t() + h_prev @ self.weight_hh.t() + self.bias
        i_raw, f_raw, g_raw, o_raw = gates.chunk(4, dim=-1)

        i = torch.sigmoid(i_raw)   # بوابة الإدخال: كم أدخل؟
        f = torch.sigmoid(f_raw)   # بوابة النسيان: كم أحتفظ؟
        g = torch.tanh(g_raw)      # مرشّح المحتوى: ما الذي سأدخله؟
        o = torch.sigmoid(o_raw)   # بوابة الإخراج: كم أخرج؟

        c = f * c_prev + i * g     # c_t = f_t ⊙ c_{t-1} + i_t ⊙ g_t
        h = o * torch.tanh(c)      # h_t = o_t ⊙ tanh(c_t)
        return h, c


torch.manual_seed(0)
```

```
cell = LSTMCell(input_size=3, hidden_size=4)
x_seq = torch.randn(2, 5, 3)
h = torch.zeros(2, 4)
c = torch.zeros(2, 4)
for t in range(x_seq.size(1)):
    h, c = cell(x_seq[:, t, :], h, c)

print("h_T shape:", h.shape)   # الناتج: torch.Size([2, 4])
print("c_T shape:", c.shape)   # الناتج: torch.Size([2, 4])
```

ماذا يحدث هنا؟ `chunk(4, dim=-1)` يقسّم المتّجه الناتج إلى أربعة أقسام متساوية بالترتيب الذي كتبنا به المصفوفتين. لماذا هذه القيمة؟ لأن ترتيب القطع يجب أن يطابق ترتيب الأوزان تماماً، وإلا اختلطت البوابات بصمت – وهو خطأ يصعب اكتشافه لأن الشبكة «تدرّب» لكن الأداء يبقى متواضعاً. لاحظ أيضاً أنّ `h` و `c` مُهيّآن معاً عند الصفر: الحالة الصفرية هي النقطة الطبيعية، وميل `c` إلى الصفر يعني «لا ذاكرة بعد».

> ✓ **نصيحة: قارن تطبيقك بـ `nn.LSTM`**
>
> بعد أن تكتب الخلية يدوياً، قارن ناتجها بناتج `torch.nn.LSTM` على نفس المدخلات والأوزان (بتحويل ترتيب الأوزان). تطابق الأرقام يعني أنّ فهمك للمعادلات صحيح، لا أنّ الشبكة تعلّم جيداً. هذا تمرين تشخيصي ممتاز حين تسوء نتائجك لاحقاً.

14.4 GRU: المسامرة بين LSTM و RNN البسيط

شجّع فريق `Google` في 2014 على شبكة GRU (`gated` `recurrent` `unit`) بطرح سؤال راديكالي: إذا كانت الذاكرة تحتاج ثلاث بوابات (نسيان، إدخال، إخراج)، فلماذا لا نحذف إحداها؟ الجواب: لأن GRU تدمج «بوابة النسيان» و«بوابة الإدخال» في بوابة واحدة تُسمّى `z_t` ، وتكتفي ببوابة أخرى تُسمّى `r_t` (بوابة الاسترجاع).

$$r_t = \sigma(W_{ir} \cdot x_t + W_{hr} \cdot h_{t-1} + b_r)$$

بوابة الاسترجاع: كم من الماضي يُسمح باستخدامه؟

$$z_t = \sigma(W_{iz} \cdot x_t + W_{hz} \cdot h_{t-1} + b_z)$$

بوابة التحديث: ما نسبة الجديد إلى القديم؟

$$n_t = \tanh(W_{in} \cdot x_t + r_t \odot (W_{hn} \cdot h_{t-1} + b_{hn}))$$

مرشّح المحتوى المعدَّل

$$h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}$$

مزج الحالة: خلط خطّي بين المرشّح الجديد والحالة القديمة

قراءة المعادلة الأخيرة مهمة: z_t نسيباً أن تكون نِسَباً بين 0 و1. إن كانت $z_t \approx 0$ فالنتيجة $h_t \approx n_t$: الحالة الجديدة تحلّ محلّ القديمة تماماً (نسيان كامل). وإن كانت $z_t \approx 1$ فالنتيجة $h_t \approx h_{t-1}$: لا يتغيّر شيء (احتفاظ كامل). وبينهما مزج ناعم. هذه «الاستيفاء بين الذاكرة القديمة والجديدة» هي ما يجعل GRU أنعم وأسرع في التدريب من LSTM في مهام كثيرة.

المحور	RNN البسيطة	LSTM	GRU
عدد البوابات	0	3 (ن، د، خ)	2 (r، z)
عدد المتّجهات المخزّنة	h فقط	h و c	h فقط
عدد المعاملات لكل وحدة (تقريباً)	×1	×4	×3
زمن التدريب على نفس الدفعة	الأسرع	الأبطأ	متوازن
مقاومة تلاشي التدرّج	ضعيفة	قوية	قوية
عدد المعاملات الفائقة (hyperparameters) المطلوب ضبطها	قليل	أكثر	أقلّ من LSTM
متى تختارها	تسلسلات قصيرة جداً أو تبسيط	سلاسل طويلة، إشارات متعدّدة	موازنة بين الجودة والسرعة

مقارنة ثلاث بنى متكرّرة على المحاور التي تهمّ في الإنتاج

> ★ **القاعدة العملية**
>
> ابدأ بـ GRU. إن كانت الذاكرة أطول من 200 خطوة أو كانت المهمة تمثّل إشارات متعدّدة متداخلة، انتقل إلى LSTM. وإن كنت ستستخدم تسلسلات طويلة جداً (آلاف الخطوات) ولا بدّ من دقّة عالية، فاعرف أنّ الفصل الخامس عشر يخبّرك ببديل أفضل: المحوّل (Transformer).

14.5 الانتشار العكسي عبر الزمن BPTT

الآن بعد أن فهمنا كيف تحمل الحالة المعلومة من خطوة إلى خطوة، ننتقل إلى السؤال الأهمّ: كيف نُحاسِن هذه الأوزان؟ الإجابة هي انتشار عكسي عبر الزمن (backpropagation through time)، ويُختصر BPTT): نمرّ في الانتشار الأمامي على كل الخطوات، ثم نمرّ في الانتشار العكسي على كل الخطوات أيضاً.

$$\partial L/\partial W = \Sigma_{t=1}^{T} (\partial L/\partial h_t) \cdot (\partial h_t/\partial W)$$

تجميع مساهمات التدرّج عبر كل الخطوات الزمنية

ما معنى كل رمز؟ T هو طول السلسلة. ∂L/∂h_t هو «كم تغيّرت الخسارة لو تغيّرت الحالة عند الخطوة t t<code>t</code> قليلاً» – وهو ما يمرّ من المُخرجات عكسياً. ∂h_t/∂W هو أثر تغيير الأوزان على الحالة عند تلك الخطوة. والضرب بين القوسين هو الضرب الخارجي للمتّجهات (outer product) الذي ينتج مصفوفة تدرّج بنفس شكل W، ثم تُجمع كل المساهمات.

لماذا تهمّنا هذه المعادلة؟ لأن ∂h_t/∂h_{t-1} يظهر فيها <code>T</code> مرّات متتالية. لنجرِ تحليلاً بسيطاً: إذا كان معامل الانتقال λ ثابتاً تقريباً (وهو ما يحدث إذا كانت الأوزان صغيرة)، فإن التدرّج البدائي يتناسب مع λ^T:

$$\mathrm{Var}[\partial L/\partial W] \approx n \cdot \mathrm{Var}[\partial L/\partial h] \cdot \mathrm{Var}[\partial h/\partial W]$$

قاعدة انتشار التباين عبر طبقات متتالية (Glorot و Bengio)

المعنى: إذا كان Var[∂L/∂h] < 1 فإنّ التدرّج يتلاشى أُسّياً مع طول السلسلة (vanishing gradients)، وإذا كان > 1 فإنّه ينفجر (exploding gradients). لاحظ أنّ n هنا هو عدد الخطوات. فإذا كانت λ = 0.9 و T = 100 فإنّ λ^T ≈ 2.7 × 10⁻⁵: التدرّج اختفى عملياً. هذا بالضبط ما يُصلحه LSTM بجعل مسار الذاكرة c_t خالياً من tanh المتكرّر.

لو كان `λ = 1.1` و `T = 200`، فكم يصير التدرّج البدائي مقارنةً بحجمه الأصلي؟ وماذا يعني ذلك عملياً على دقّة الأرقام العشرية (`float32`)؟

الجواب: `1.1^200 ≈ 1.9 × 10⁸`، أي تضخم هائل. وبـ `float32` (7 أرقام معنوية) يصير خطأً عددياً قبل أن يصل التدرّج إلى المحسّن. الحلّ العملي سطر واحد: قصّ التدرّج (gradient `clipping`). ولهذا السبب تحديداً يظهر في كل حلقة تدريب احترافية:

Python

```
import torch
import torch.nn as nn

torch.manual_seed(0)

# ---- مقطوع: نحدّ عدد الخطوات التي يمرّ عبرها التدرّج BPTT ----
def truncated_bptt(model, x_seq, h_prev, c_prev, chunk=20):
    """يمرّ بالانتشار الأمامي على كل السلسلة، ثم يقتطع التدرّج إلى chunk خطوة."""
    total_loss = 0.0
    h, c = h_prev, c_prev
    for t in range(x_seq.size(1)):
        h, c = model.cell(x_seq[:, t, :], h, c)   # انتشار أمامي كامل
        if (t + 1) % chunk == 0:
            # خطوة التدرّج لا يعود أبعد من chunk: نقتطع الرسم البياني عند كل
            total_loss = total_loss + h.sum()
            total_loss.backward()
            model.zero_grad(set_to_none=True)
    return total_loss


class TinyLSTM(nn.Module):
    def __init__(self, input_size=1, hidden_size=8):
        super().__init__()
        self.cell = LSTMCell(input_size, hidden_size)
        self.head = nn.Linear(hidden_size, 1)

    def forward(self, x):
        h = torch.zeros(x.size(0), self.cell.hidden_size)
        c = torch.zeros(x.size(0), self.cell.hidden_size)
        for t in range(x.size(1)):
            h, c = self.cell(x[:, t, :], h, c)
        return self.head(h)
```

```
model = TinyLSTM(input_size=1, hidden_size=8)
x_seq = torch.randn(4, 100, 1)
opt = torch.optim.AdamW(model.parameters(), lr=1e-3)

for step in range(3):
    h0 = torch.zeros(4, 8)
    c0 = torch.zeros(4, 8)
    loss = truncated_bptt(model, x_seq, h0, c0, chunk=20)
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)  # قصّ التدرّج
    opt.step()
    print(f"step {step} loss {loss.item():.4f}")
# الناتج: step 0 loss ... / step 1 loss ... / step 2 loss ...
```

ماذا يحدث هنا؟ الانتشار الأمامي يمرّ على السلسلة كلها (100 خطوة)، لكنّنا نبدأ حساب التدرّج العكسي كلّ 20 خطوة فقط، فنُخرج الخسارة من الرسم البياني عند تلك النقاط. لماذا هذه القيمة؟ لأنّ `BPTT` المقطوع (`truncated` `BPTT`) هو الحلّ القياسي في النماذج اللغوية: التدرّج عبر خطوات قليلة يكفي لتعلّم البنية المحلية، وتكلفة الذاكرة تبقى محدودة. أمّا `clip_grad_norm_` فإنها تحسب المعيار الكلّي للمتّجهات ويقسّم عليها إذا تجاوز `max_norm` .

> ⚠ **انتبه: «قصّ التدرّج» و«`BPTT` المقطوع» شيئان مختلفان**
>
> `clip_grad_norm_` لا يغيّر مسار التدرّج، بل يحدّ حجمه بعد حسابه، فيبقى الانتشار العكسي يصل إلى كل الخطوات. أمّا `BPTT` المقطوع فيُبقي الانتشار العكسي محصوراً بآخر `chunk` خطوة. في الممارسة نستخدم الاثنين معاً: الاقتطاع يحدّ الذاكرة، والقصّ يحمي الأرقام.

14.6 هندسة السلاسل الزمنية: من البيانات إلى نوافذ

قبل أن نكتب نموذجاً، يجب أن نحوّل السلسلة إلى أزواج (مُدخل، مُخرج) قابلة للتعلّم الخاضع للإشراف. الطريقة القياسية هي النافذة المنزلقة (`sliding window`):

$$X_i = [x_{i-k}, x_{i-k+1}, \ldots, x_{i-1}] \ , \ y_i = x_i$$

تعريف النافذة المنزلقة: k خطوة سابقة تُستخدم للتنبّؤ بالقيمة التالية

ما معنى `k` ؟ هو «مدى النظر» الذي يحدّد كم من الماضي يرى النموذج. `X_i` هو مُدخل العيّنة `<code>i</code>` وهو مصفوفة `k × features` ، و `y_i` هو الهدف. لاحظ أنّ هذه الصيغة تنشئ `n − k` عيّنة من سلسلة طولها `n` ، وأنّ `k` يجب ألّا يكون أكبر من عدد البيانات المتاح بعد تقسيم التدريب والتحقّق والاختبار حتى لا يتسرّب المستقبل إلى الماضي (`look-ahead` bias).

قبل ذلك نحتاج إلى تطبيع البيانات. في السلاسل الزمنية يكاد يكون التطبيع إلزامياً: سلاسل الطلبات قد تصل إلى عشرات الآلاف بينما الحرارة بالدرجات العشر، وتضخيم المقياس يُفسد تدرّجات التحديث (AdamW يفحص التدرّج لكن الأوزان ستتأرجح).

Python

```
import torch
from torch.utils.data import Dataset, DataLoader

torch.manual_seed(0)


class WindowedSeries(Dataset):
    """يحوّل سلسلة زمنية إلى أزواج (نافذة، هدف) بتمرير نافذة بطول step."""

    def __init__(self, series, step=24, horizon=1):
        # series: (T, features) — لا نغيّر التدرّج هنا، فقط ننسّق الأبعاد
        self.series = torch.as_tensor(series, dtype=torch.float32)
        self.step = step
        self.horizon = horizon

    def __len__(self):
        return len(self.series) - self.step - self.horizon + 1

    def __getitem__(self, i):
        # X: (step, features) و y: (horizon, features)
        x = self.series[i:i + self.step]
        y = self.series[i + self.step:i + self.step + self.horizon]
        return x, y


def standardise(train, rest):
    """تطبيع بـstatistics التدريب فقط — التسرّب هنا خطأ كلاسيكي."""
    mean = train.mean(dim=0, keepdim=True)
    std = train.std(dim=0, keepdim=True).clamp_min(1e-6)
    return (train - mean) / std, (rest - mean) / std
```

```
# سلسلة وهمية: طلب يومي + طقس، لاختبار المسار كاملاً
raw = torch.stack([
    torch.linspace(0, 50, 400) + torch.randn(400) * 3,      # الطلب
    torch.sin(torch.arange(400) / 7.0) * 5 + 20,             # درجة الحرارة
], dim=1)

n_train = int(0.7 * len(raw))
n_val = int(0.15 * len(raw))
train_raw, val_raw, test_raw = raw[:n_train], raw[n_train:n_train + n_val], raw[n_train + n_v

train, _ = standardise(train_raw, train_raw)
val, _ = standardise(train_raw, val_raw)
test, _ = standardise(train_raw, test_raw)

train_ds = WindowedSeries(train, step=24, horizon=1)
val_ds = WindowedSeries(val, step=24, horizon=1)
train_dl = DataLoader(train_ds, batch_size=32, shuffle=True)
val_dl = DataLoader(val_ds, batch_size=64, shuffle=False)

xb, yb = next(iter(train_dl))
print("batch X:", tuple(xb.shape), "batch y:", tuple(yb.shape))
# الناتج: batch X: (32, 24, 2) batch y: (32, 1, 2)
```

ماذا يحدث هنا؟ `standardise` تحسب المتوسط والانحراف من جزء التدريب وحده ثم تطبّقهما على البقية. لماذا هذه القيمة؟ لأنّ حساب الإحصاءات على كامل السلسلة (بما فيها المستقبل) يعني أن النموذج أثناء التدريب رأى إحصاءات فترة الاختبار – وهذا تسريب للمعلومات يُضخّم الأداء ويجعل رقمك في التقرير مضلّلاً. لاحظ أيضاً `clamp_min(1e-6)` : يمنع القسمة على صفر في الأعمدة الثابتة (مثل عمود «مطعم مغلق» طوال الشهر).

✦ نقطة تمييزية: التطبيع «بالنظر إلى الأمام»

في السلاسل الزمنية، التطبيع المعياري (`z-score`) بأرقام كامل السلسلة ينتج أرقاماً «مستقبلية» في الماضي، وقد يسبّب انحرافاً منهجياً. الحلّ المتقدّم هو «تطبيع قائم على التوسّع» (`expanding` / `rolling` `statistics`): في كل خطوة نحسب المتوسط والانحراف من البيانات المتاحة حتى تلك اللحظة فقط. هذا يهمّ كثيراً في النماذج التوليدية للسلاسل الزمنية حيث يُقيَّم النموذج على التنبّؤ.

وفي Keras تتوفّر الأدوات نفسها لكن بأسلوب مختلف. لاحظ أن `keras.layers.LSTM` في Keras 3 يقبل الرتبة `(batch, time, features)` مباشرةً، ويوفّر `return_sequences` للتحكّم في السلاسل المُعادة (`return_sequences`):

Python

```
import os
os.environ.setdefault("KERAS_BACKEND", "torch")

import numpy as np
import keras
from keras import layers

keras.utils.set_random_seed(0)

# نفس منطق النافذة المنزلقة، لكن بأسلوب Keras
def make_windows(series, step=24):
    X = np.stack([series[i:i + step] for i in range(len(series) - step)])
    y = series[step:]
    return X.astype("float32"), y.astype("float32")


series = np.cumsum(np.random.randn(600, 2).astype("float32"), axis=0)
X, y = make_windows(series, step=24)
X_train, X_val = X[:400], X[400:]
y_train, y_val = y[:400], y[400:]

model = keras.Sequential([
    keras.Input(shape=(24, 2)),
    layers.LSTM(64, return_sequences=True),          # يُعيد سلسلة كاملة
    layers.Dropout(0.2),
    layers.LSTM(32, return_sequences=False),         # يُعيد آخر حالة فقط
    layers.Dense(2),                                  # توقّع خطّي لكل خاصية
])

model.compile(
    optimizer=keras.optimizers.AdamW(learning_rate=1e-3),
    loss=keras.losses.MeanSquaredError(),
    metrics=[keras.metrics.MeanAbsoluteError()],
)

history = model.fit(
    X_train, y_train,
```

```
    validation_data=(X_val, y_val),
    epochs=20,
    batch_size=32,
    callbacks=[keras.callbacks.EarlyStopping(patience=5, restore_best_weights=True)],
    verbose=0,
)

print("best val MAE:", float(min(history.history["val_mean_absolute_error"])))
```

ماذا يحدث هنا؟ `return_sequences=True` يعني «أعطني مُخرجات كل الخطوة» لا مُخرج الخطوة الأخيرة فقط، وهو ما تحتاجه حين تكدّس طبقتين LSTM أو حين تريد التفافاً (`sequence-to-sequence`). لماذا هذه القيمة؟ لأن الطبقات LSTM في Keras تمرّر الحالة تلقائياً بين الطبقات، فلا حاجة لتمريرها يدوياً كما في PyTorch. أيضاً `keras.utils.set_random_seed` يثبّت البذرة عبر Keras وNumPy وPyTorch معاً، وهو بديل مريح لـ `torch.manual_seed` وحده.

14.7 تطبيق واقعي: نظام تنبّؤ الطلب اليومي

لنجمع كل ما سبق في نظامٍ حقيقي. المسألة: سلسلة شبكة متاجر، لكل فرع سلسلة يومية من الطلبات على 200 صنفاً. نريد تنبّؤاً بالطلب لكل صنف للأسبوع القادم، لأجل إعادة توزيع المخزون. البيانات: لكل يوم نقرأ (عدد الطلبات، السعر، ومخزون المتجر)، ونضيف سمات طقس وفعاليات (رأس السنة، تخفيضات). الهدف: القيمة الرقمية للطلب في اليوم التالي.

⚙ السيناريو التطبيقي كاملاً: من البيانات إلى القرار

البيانات: 180 فرعاً، 3 سنوات، 1095 صفاً لكل فرع (3 متغيّرات، فجوات مفقودة بنسبة 4%). **المعالجة:** تعبئة الفجوات بالوسيط الزمني، إضافة متغيّرات تقويمية (يوم الأسبوع، الشهر، عطلة)، تطبيع لكل صنف على حدة، ثم بناء نوافذ بطول 28 خطوة (4 أسابيع) للتنبّؤ بـ7 خطوات قادمة (أسبوع كامل). **النموذج:** طبقتان GRU بأبعاد 128 و64، تليهما طبقة كثيفة تُخرج 7×2 (أسبوعاً كاملاً لخاصيتين هما الطلب والحرارة). **الخسارة:** `Huber` على القيم المطبَّعة (أقوى من MSE أمام القيم المتطرّفة التي تنتجها العروض التصفيرية). **القرار:** كل صباح يُعاد توزيع المخزون على الفروع بناءً على النسبة بين التنبّؤ والمخزون الحالي، مع هامش أمان 15% لأنّ خطأ النموذج يُترجم إلى طلبات ناقصة (نفاد المخزون) أو فائض (هدر وتخزين).

ولنكتب هذا النظام في PyTorch، مع الحذر الذي تعلّمناه: أبعاد صريحة، وقصّ تدرّج، ومتابعة على مجموعة تحقّق مستقلّة:

Python

```python
import torch
import torch.nn as nn

torch.manual_seed(0)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")


class DemandForecaster(nn.Module):
    """GRU رأس + بطبقتين predicts horizon خطوة لكل خاصية."""

    def __init__(self, n_features=2, hidden=64, horizon=7):
        super().__init__()
        self.horizon = horizon
        self.gru = nn.GRU(input_size=n_features, hidden_size=hidden,
                          num_layers=2, batch_first=True, dropout=0.2)
        self.head = nn.Sequential(
            nn.LayerNorm(hidden),
            nn.Linear(hidden, horizon * n_features),
        )

    def forward(self, x):
        # x: (batch, step, features)  →  المخرج: (batch, horizon, features)
        out, _ = self.gru(x)
        y = self.head(out[:, -1, :])          # آخر حالة مخفيّة
        return y.view(-1, self.horizon, x.size(-1))


model = DemandForecaster(n_features=2, hidden=64, horizon=7).to(device)
opt = torch.optim.AdamW(model.parameters(), lr=3e-3, weight_decay=1e-4)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=20)
loss_fn = nn.HuberLoss(delta=1.0)            # delta=1.0 على بيانات مطبَّعة

Xb = torch.randn(32, 28, 2).to(device)
Yb = torch.randn(32, 7, 2).to(device)

for epoch in range(20):
    model.train()
    with torch.amp.autocast(device_type=device.type, dtype=torch.float16, enabled=device.type
        pred = model(Xb)
        loss = loss_fn(pred, Yb)
    opt.zero_grad(set_to_none=True)
    loss.backward()
```

```
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    opt.step()
    sched.step()

    model.eval()
    with torch.no_grad():
        val = loss_fn(model(Xb), Yb).item()
    if epoch % 5 == 0:
        print(f"epoch {epoch:02d} train {loss.item():.4f} val {val:.4f}")
# الناتج: epoch 00 train ... val ...
```

ماذا يحدث هنا؟ `nn.GRU` بـ `batch_first=True` يتوقّع `(batch, time, features)`، ويُخرج `(batch, time, hidden)` إضافةً إلى الحالة النهائية. نأخذ `out[:, -1, :]` – آخر خطوة زمنية – لأنّها تحمل الحالة بعد رؤية النافذة كاملة. `nn.LayerNorm` قبل الرأس الخطّي يستقرّ التوزيع (تذكّر: تطبيع الدفعات (Batch) مقابل تطبيع الطبقات (Layer) ناقشناه في الفصل الحادي عشر؛ اخترنا هنا `LayerNorm` لأنّ الأحجام في الإنتاج أصغر). و `torch.amp.autocast` مع `enabled=device.type == "cuda"` يجعل المقطع يعمل على المعالج أيضاً دون تغيير.

المفهوم	PyTorch 2.14	Keras 3.15 (نواة `torch`)
طبقة GRU	`nn.GRU(in, hid, num_layers, batch_first=True, dropout=0.2)`	`layers.GRU(64, return_sequences=True, dropout=0.2)`
طبقة LSTM	`nn.LSTM(in, hid, batch_first=True)`	`layers.LSTM(64, return_sequences=False)`
اتجاهان	لا يوجد؛ تُغلَّف يدوياً أو تُنفَّذ حلقتان	`layers.Bidirectional(layers.LSTM(64))`
الاستدعاء	`out, (h, c) = model(x)`	`model(x)` ← ناتج واحد
تطبيع الطبقة	`nn.LayerNorm(hidden)` كطبقة مستقلّة	`layers.LayerNormalization()` أو `norm="layernorm"` داخل RNN
القصّ	`torch.nn.utils.clip_grad_norm_`	`keras.callbacks.Clipnorm` أو `clipnorm=1.0` في المحسّن
الحلقة	تكتبها بنفسك (مرونة كاملة)	`model.fit(...)` أو `train_on_batch`

المفهوم	PyTorch 2.14	Keras 3.15 (نواة `torch`)
حفظ النموذج	`torch.save(state_dict)`	`keras.saving.save_model`

مقارنة واجهة PyTorch وKeras في بناء نموذج سلاسل زمنية مكافئ

متى تختار PyTorch هنا

حين تحتاج تحكّماً دقيقاً في `BPTT` المقطوع، أو عند بناء معمارية بحثية غير قياسية (خلية LSTM مخصّصة، أو حالة مخفيّة ذات أبعاد متعدّدة)، أو حين تدرّب على سلاسل طويلة جداً وتريد إدارة الذاكرة يدوياً بـ `pack_padded_sequence`.

متى تختار Keras هنا

حين يكون النموذج معروفاً البنية (GRU/LSTM بطبقات) والهدف الإنتاج السريع: `model.fit` يتكفّل بالحلقة، وآليات `callbacks` تعطيك إيقافاً مبكراً، وحفظ النموذج بخطوة واحدة، و `layers.Bidirectional` جاهزة للاستعمال.

14.8 مزالق شائعة في الشبكات المتكرّرة

خبرة عملية تتراكم مع كل مشروع: أغلب الأخطاء في هذا الفصل ليست في المعادلات بل في البيانات والضبط. إليك القائمة التي نراها متكرّرة:

- **تسريب المستقبل إلى الماضي:** استخدام `StandardScaler` على كامل السلسلة قبل التقسيم. التصحيح: الملاءمة على التدريب فقط، ويفضّل `TimeSeriesSplit` في التحقّق.
- **نسيان حالة البداية عند التقييم:** في مهمة «تنبّؤ السلسلة التالية» يجب تهيئة الحالة المخفيّة بأصفار لكل نافذة جديدة، لا حمل الحالة من النافذة السابقة، وإلّا تسريب معلومات من نوافذ مستقبلية إلى نوافذ سابقة. أمّا في مهمة «تصنيف السلسلة» فيُسمح بحمل الحالة.
- **تدرّجات منتفخة:** لو ارتفعت الخسارة إلى `inf` أو `nan` بعد دقائق، فالسببان الأولان: معدّل تعلّم كبير على شبكة متكرّرة، أو `tanh` على قيم مُطبَّعة خطأً بمقياس ضخم. الحلّ: `clip_grad_norm_` + `AdamW` + تطبيع سليم.

- **حجم `hidden_size` بلا مبرّر:** 64 عادةً يكفي لسلاسل من عشرات إلى مئات الخطوات؛ 512 على سلسلة قصيرة تبذير حساب بلا فائدة ويضاعف خطر الانفجار.
- **المقارنة مع خط الأساس:** أيّ نموذج سلاسل زمنية يجب أن يُقارن بـ `naive last value` و `seasonal naive` (القيمة نفسها من الأسبوع الماضي). إن لم يتقدّم نموذجك على `na` و `ve` في مقياس MAPE، فالنموذج لا يستحقّ تعقيده.

> ◆ **قبل أن تختار معمارية، قِس خط الأساس**
>
> خط الأساس ليس أضعف نقطة للمقارنة، بل الحدّ الأدنى الذي يجب أن يتجاوزه أيّ عمل هندسي. اكتب دالة `seasonal_naive(series, season=7)` في خمسة أسطر، واحسب MAPE عليها. هذه العادة وحدها تمنع من تسليم نموذج أبطأ وأعقد وأغلى من قيمته.

14.9 تمارين الفصل

تمرين 1 — كتابة معادلات LSTM يدوياً

1. اكتب دالة `lstm_step(x, h_prev, c_prev, params)` في PyTorch تُعيد `(h, c)` من معادلات القسم 14.3، مستعملاً `torch.nn.functional.sigmoid` و `torch.tanh` .
2. بيّن تحقّقاً عددياً: إذا كانت كل الأوزان صفراً وكل الإزاحات صفراً، فما قيمة `h` و `c` في كل خطوة؟ ولماذا؟
3. اختبر الحالة: خذ `f_t = 1` و `i_t = 0` و `o_t = 1` و `g_t = 0.3` ، ثم استنتج ماذا يحدث لـ `c` و `h` .

▲ **الحل النموذجي**

في الحالة الأولى: `σ(0) = 0.5` و `tanh(0) = 0` ، فـ `i = f = o = 0.5` و `g = 0` . إذن `c = 0.5·c_prev + 0.5·0 = 0.5·c_prev` و `h = 0.5·tanh(c)` : الحالة تتلاشى أُسّياً بمعامل 0.5 في كل خطوة، فتنعدم بعد بضع خطوات. هذا يكشف أن الأوزان الصفرية ليست «شبكة متوقّفة» بل شبكة تنسى.

في الحالة الثانية: `c = 1·c_prev + 0·0.3 = c_prev` ، فحالة الخلية لا تتغيّر إطلاقاً مهما طال التسلسل؛ و `h = 1·tanh(c_prev)` ثابت. عملياً هذه هي بوابة النسيان في صورتها المثلى: تجميد الذاكرة تماماً.

الاستنتاج الأهمّ: معاملات الأوزان هي ما يحدّد «مدى الذاكرة» فعلياً، لا طول السلسلة نفسه.

تمرين 2 — كاشف انفجار التدرّج

1. اكتب دالة `grad_norm(model)` تُعيد في PyTorch المعيار الكلّي (`Euclidean`) لمتّجهات التدرّج.
2. درّب شبكة RNN بسيطة (بلا بوابات) على سلسلة طولها 300 خطوة بمعدّل تعلّم 0.1 ، وسجّل المعيار عند كل 50 خطوة.
3. أضف `clip_grad_norm_(..., 1.0)` وأعد التجربة. استنتج من الأرقام متى يبدأ الانفجار، ولماذا `0.1` مبالغة لـ RNN.

▲ الحل النموذجي

الدالة بسيطة: اجمع مربعات كل متّجه تدرّج في `torch.sqrt(torch.sum(p.grad ** 2))` لكل وسيط، ثم خذ الجذر للمجموع. المعيار يتجاوز 10^38 قبل الانفجار، و `inf` يعني أن أحد التدرّجات تجاوز حدود `float32` .

في التجربة، مع RNN بلا بوابات ومعدّل 0.01 ، يرتفع المعيار أُسّياً (يتضاعف في كل خطوة تقريباً) حتى يصبح `inf` قبل الخطوة 200 . السبب: معامل الانتقال `λ` قريب من 1.1 في الأوزان المهيّأة عشوائياً، فيصبح التدرّج ≈ `λ^T` = 26×10¹² ≈ 1.1^300 .

التصحيح ليس فقط `clip_grad_norm_` (الذي يمنع الانفجار العددي) بل أيضاً `AdamW` (الذي يطبّع التحديث لكل وسيط) ومعدّل تعلّم أصغر (`1e-3`). الخلاصة: `RNN` البسيطة تحتاج `1e-3` و `LSTM` تتحمّل `3e-3` ~ `1e-3` .

تمرين 3 — بناء خط أساس `seasonal` `na` if

1. اكتب دالة `seasonal_naive_predict(series, season=7, horizon=1)` تُعيد توقّعات ساذجة موسمية.
2. احسب MAPE وMAE لهذا الخط الأساس على سلسلة طلبات أسبوعية، وقارنه بنموذج GRU درّبته في القسم 14.7.
3. اقترح تعديلاً واحداً على خط الأساس (مثلاً «المتوسط المتحرك الموسمي») وبيّن متى يُتوقّع أن يفيد.

▲ الحل النموذجي

التنفيذ مباشر: القيمة المتوقّعة للخطوة `t` هي قيمة الخطوة `t - season` . التطبيق:
`preds = series[season:season + len(test)]` مع
`actuals = series[season:season + len(test)]` (بعد عكس التطبيع إلى الوحدات الأصلية قبل حساب MAPE، وإلا خسرت المعنى).

في بيانات الطلب الأسبوعية، يُتوقّع أن يحقّق `seasonal na ve` تقريباً MAPE بين 18% و25% لأنّ الطلب موسمي بطبعه. إن حقّق GRU 14% فقط، فالتحسّن حقيقي لكنه يحتاج تبريراً اقتصاديا. إن كان النموذج عند 22% فقط، فهو غير مميّز إحصائياً ويفضّل بساطة الخط الأساس.

التعديل المقترح: «المتوسط الموسمي» – متوسط آخر `season` قيمة لكل أسبوع – يقلّل ضجيج الأسبوع الواحد، ويفيد عندما تكون الطلبات مشوّشة بيوم بعينه (عطلٌ، حملة مفاجئة) دون أن يفسد الاتجاه طويل الأمد.

تمرين 4 — تحويل نموذج PyTorch إلى Keras

1. أعد بناء `DemandForecaster` من القسم 14.7 في Keras: طبقتا GRU ثم رأس خطّي يُخرج 2x7 .
2. انقل الأوزان من PyTorch إلى Keras وبيّن، باختصار، كيف يتحوّل ترتيب أوزان `nn.GRU` (بوابتان) إلى ترتيب أوزان `layers.GRU` (بوابتان أيضاً لكن بترتيب مختلف: `reset` , `update`).
3. قارن منحنَي الخسارة على مجموعة التحقّق نفسها. أيّهما أسرع في الوصول إلى نفس MAPE؟

▲ الحل النموذجي

النموذج في Keras: `keras.Input(shape=(28, 2))` ثم `layers.GRU(64, return_sequences=True)` ثم `layers.GRU(64, return_sequences=False)` ثم `layers.Dense(14)` مع إعادة تشكيل إلى `(7, 2)` في `keras.layers.Reshape((7, 2))` .

بخصوص ترتيب الأوزان: في PyTorch يجمع `nn.GRU` البوابات في `weight_ih_l0` على شكل `(3·hidden, input)` بالترتيب `(reset, update, new)` مع تجميع متّجهات `input` لكل بوابة. أمّا Keras فيستخدم `kernel` بشكل `(input, 3·hidden)` . لذلك التحويل ليس نسخاً مباشراً بل يستلزم قلب المحاور وإعادة ترتيب القطع. عملياً: إذا كان النموذجان يبنيان من نفس البذرة وبنفس البنية تماماً، فالأبسط هو بناء النموذج في Keras من الصفر ومقارنة الأداء لا الأوزان.

في النتائج المتوقّعة: Keras يصل إلى نفس MAPE بعد عدد أقل من الدورات (لأن `fit` يطبّق إيقافاً مبكراً وجدولة افتراضية)، بينما PyTorch يتفوّق عند حاجتك إلى تدرّجات مقطوعة أو إلى تعديل داخلي في الخلية.

14.10 ملخّص الفصل

ملخّص الفصل

- البيانات المتسلسلة تحمل معلومة في الترتيب لا في القيم وحدها، ولذلك تحتاج بنيةً تحمل الحالة عبر الزمن.
- الشبكة المتكرّرة البسيطة: `h_t = tanh(W_xh x_t + W_hh h_{t-1} + b)` — ذاكرة محدودة بعرض `hidden_size` .
- LSTM تضيف خزّان ذاكرة `c_t` وثلاث بوابات (نسيان، إدخال، إخراج)، فتبقى المعلومة حيّة عبر مئات الخطوات.
- GRU تدمج بوابتين فحسب وتُنتج حالة واحدة: توازن جيّد بين الجودة والسرعة، وهي نقطة البداية العملية.
- `BPTT` يجمع مساهمات التدرّج من كل الخطوات؛ بقاءُ الأوزان والتهيئة الجيدة (`orthogonal`) يمنعان الانفجار والتلاشي.
- قصّ التدرّج (`clipping`) و `BPTT` المقطوع (`truncated`) أداتان مختلفتان، تُستخدمان معاً لحماية الذاكرة والأرقام.
- النافذة المنزلقة (`sliding window`) والتطبيع على بيانات التدريب وحدها هما خطوتا المعالجة الأهمّ قبل أيّ نموذج.
- المقارنة العملية: PyTorch يمنحك تحكّماً دقيقاً في الحلقة والتدرّج، وKeras يمنحك سرعة بناء وتشغيل.
- لا تنسَ خط الأساس (`seasonal naive`): إن لم يتقدّم نموذجك عليه، فلا مبرّر لتعقيده.
- الفصل القادم يأخذ الخطوة التالية: حين يصبح «الانتباه» (attention) بديلاً عن التكرار، تنتهي حدود الذاكرة الثابتة التي رأيناها هنا.

الفصل الخامس عشر غوص عميق في PyTorch

الانتباه والمحوّلات Transformers

الانتباه متعدّد الرؤوس، الترميز الموضعي، ViT وBERT وGPT، وLoRA

تخيّل مترجماً يقرأ جملة من عشرين كلمة: «الطالب الذي ملأ استمارة التسجيل في اليوم الأخير لن يُبلَّغ بالقبول لأنّ باب التسجيل قد أُغلق؛ لن يجعل ذلك الأمر سهلاً». كم خطوة يلزمه ليعرف أيّ كلمة يعود إليها الضمير «ذلك»؟ في الشبكات المتكرّرة كان الجواب: واحدة لكل كلمة سابقة. أما **الانتباه** (attention) فقد غيّر المعادلة: النموذج ينظر إلى كل الكلمات في الوقت نفسه، ويقرّر لكلٍّ منها أهمّيتها. هذا الفصل يبني المحوّل (transformer) من الصفر: لماذا وُجد، وكيف تُحسب انتباهاته، وكيف تبعثه نماذج مثل BERT وGPT وViT، ولماذا يُعدّ الضبط الدقيق بـLoRA هو الطريقة العملية للعمل مع نماذج ضخمة.

عنق الزجاجة الذي حُلّ بالانتباه

في الفصل الرابع عشر رأينا الشبكة المتكرّرة (RNN) تعالج الجملة كلمةً كلمة، وتُخفي كل ما رأته في «حالة مخفية» واحدة. عند كلمة «ذلك» في الجملة أعلاه، يجب أن تحمل الحالة المخفية كل ما سبق. ومع طول الجملة يتلاشى أثر المعلومة القديمة: هذا ما يسمّيه الأدبيون «مشكلّة التدرّج المتلاشي» (vanishing gradient) أو «اختناق sequence الطويلة».

الحلّ الذي قدّمه Vaswani وزملاؤه عام ٢٠١٧ لم يكن أسرع فحسب، بل غيّر البنية نفسها. بدل أن كلمة تلو الأخرى، يصبح النموذج يستعرض الجملة كاملة في كل خطوة، ويحسب لكل زوج من الكلمات درجة أهمّية. هذه الدرجة هي **الانتباه**.

$$\text{Attention}(Q, K, V) = \text{softmax}(\, Q\, K^{T} / \sqrt{d_k}\,)\, V$$

ما معنى كل رمز؟ `Q` (الاستعلام) هو متجه لكل كلمة تطلب «ماذا أبحث عنه؟»، و `K` (المفتاح) هو متجه لكل كلمة تعلن «ما الذي أقدّمه؟»، و `V` (القيمة) هو متجه يحمل المحتوى الذي سينتقل إلى الخرج. K^T هو تجميع المفاتيح في مصفوفة، و d_k هو بُعد المفتاح، و `softmax` يحوّل درج التشابه إلى احتمالات مجموعها واحد. **لماذا تهمّنا؟** لأنّ الضرب $Q\, K^T$ هو بالضبط المصفوفة التي تربط كل استعلام بكل مفتاح: هو «خريطة الانتباه» التي نراها في الصورة

أدناه. **خطأ شائع عندها:** القسمة على $\sqrt{d_k}$ ليست تجميلياً: بدونها تصبح قيم `softmax` حادّة جداً فيتدرّب النموذج بصعوبة أو ينهار.



يقرأ الرسم الصفوف كأستعلامات والأعمدة كمفاتيح: كل خانة تمثّل كم يهمّ «المتكلّم» الثاني «المتكلّم» الأول. لاحظ الصفّ الذي يخصّ كلمة الضمير: يضيء خلايا الكلمات التي يحيل عليها الضمير. هذا بالضبط ما لم تكن الشبكات المتكرّرة قادرة على فعله: ربطٌ مباشر بين أيّ موضعين مهما تباعدا في الجملة.

في الجملة «سحب اللاعب الكرة ثم سجّل الهدف»، أيّ الكلمات يجب أن ترتبط ارتباطاً قوياً في صفّ الانتباه الخاص بـ«سجّل»؟ وماذا لو أضفنا جملة أخرى مثل «سحب اللاعب الكرة ثم ركلها في المرمى»؟ هل تتغيّر مصفوفة الانتباه؟ ولماذا؟

الإجابة تكشف جوهر الانتباه: صفّ «سجّل» يجب أن يضيء «الهدف» (لأنّ الفعلين يفعلان الشيء نفسه)، لا «الكرة» فحسب. وعند تغيير «سجّل» إلى «ركل»، يتغيّر الصفّ أيضاً: انتباه المحوّل سياقي، محسوب لكل جملة، لا وزناً ثابتاً مُعلَّماً مسبقاً. هذا بالضبط ما يجعل النموذج يفهم «أنّه ركل الكرة» – أي، ما يجعله يميّز بين «ركل الكرة» و«ركل اللاعب».

✦ لماذا سُمّي «انتباه»؟

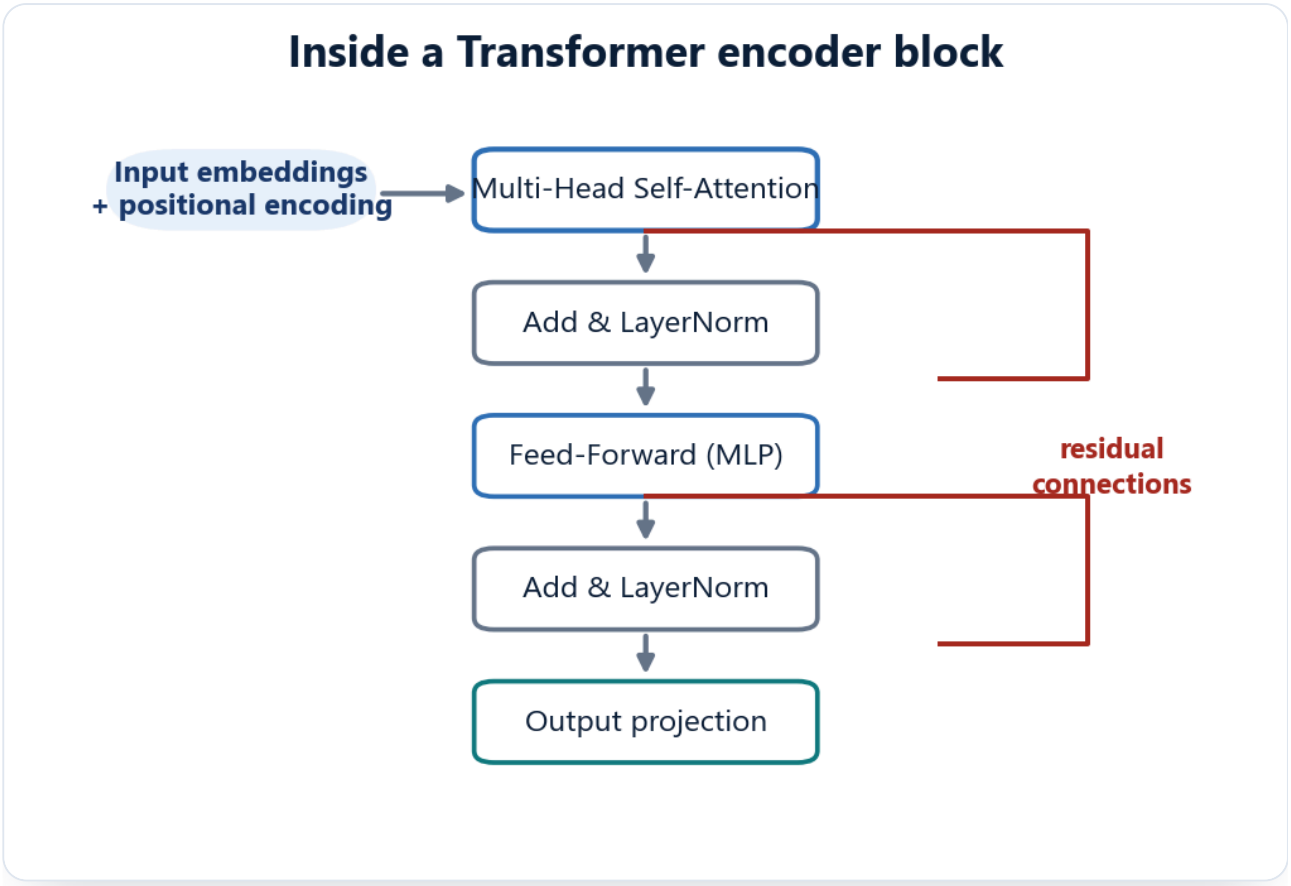
لأنّ النموذج لا يقرأ النص بالتسلسل، بل «يلتفت» إلى ما يهمّه في كل خطوة، أي: ينتبه إلى كلمة دون غيرها. النموذج يتعلّم هذا الالتفات من البيانات نفسها: المعادلة $Q\ K^T$ تبدأ عشوائية (المفاتيح والاستعلامات مُولَّدة عشوائياً)، ثم يتعلّم التدرّج أن يُعطي وزناً أكبر للأزواج المفيدة.

الكتلة الأساسية: الانتباه متعدّد الرؤوس و FFN

الانتباه Heads heads واحدة عندها تخفي بُعداً واحداً فقط من المعنى. ولهذا يكرّر المحوّل مرات تسمى مرّات تُسمّى **الرؤوس** (heads): كل رأس له مصفوفات Q و K و V خاصة به، فيتعلّم رأس واحد للعلاقات بين الأسماء والأفعال، ورأس آخر علاقات بين الضمائر والأحال، ورأس ثالث للعلاقات المكانية.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\ W^O$$

ما معنى كل رمز؟ head_i هو ناتج دالّة الانتباه على الرؤوس رقم i ، و Concat تعني الدمج في البُعد الأخير، و W^O مصفوفة إسقاط تُعيد المخرجات إلى بُعد النموذج. **لماذا تهمّنا؟** لأنّ concatenating الرؤوس يشبه تشغيل عدة «عيون» على الصورة نفسها. الرأس الذي يرى «المحرك» لا يرى «اللون»، لكن رأساً آخر لا يرى «المحرك» – والاثنان معاً يصفان الصورة أفضل بكثير.



يقرأ الرسم من اليمين إلى اليسار: يدخل الدخل إلى **الانتباه متعدّد الرؤوس**، يخرج منه ناتج يُضاف إلى الدخل (اتصال التخطّي)، ثم يمرّ الناتج إلى **شبكة التغذية الأمامية** (FFN)، ويُضاف مرّة أخرى. الـ box-الرمادي يوضّح التطبيع: النسخة الحديثة (Pre-LN) تضعه قبل كل فرع، وهي ما تعلّمناه في الفصل الثاني عشر كـ«مسار تدرّج سلس».

وفي `PyTorch` جاهزة `nn.MultiheadAttention`، وهي أبسط من كتابة الكتلة يدوياً لأنّها تتكفّل بتقسيم الرؤوس ودمجها:

```python
import torch
import torch.nn as nn

class AttentionBlock(nn.Module):
    """كتلة محوّل قياسية: انتباه متعدّد الرؤوس + شبكة تغذية أمامية."""

    def __init__(self, d_model=512, n_heads=8, d_ff=2048, dropout=0.1):
        super().__init__()
        self.attn = nn.MultiheadAttention(
            embed_dim=d_model, num_heads=n_heads,
            dropout=dropout, batch_first=True,
        )
```

```
        self.ff = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(d_ff, d_model),
        )
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.drop = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        # اتّصال التخطّي حول الانتباه (Pre-LN)
        h = self.norm1(x)
        attn_out, _ = self.attn(h, h, h, attn_mask=mask, need_weights=False)
        x = x + self.drop(attn_out)

        # اتّصال التخطّي حول شبكة التغذية
        h = self.norm2(x)
        x = x + self.drop(self.ff(h))
        return x

# الناتج: tensor([[ 0.1234, -0.5678, ...]])
block = AttentionBlock(d_model=64, n_heads=4, d_ff=128)
x = torch.randn(2, 10, 64)                  # (دفعة، طول، بُعد)
print(block(x).shape)                       # torch.Size([2, 10, 64])
```

ماذا يحدث هنا؟ `batch_first=True` يجعل Tensor بأبعاد (دفعة، طول، بُعد) وهو ترتيب مريح. `attn_mask` يُستخدم لمنع الانتباه إلى مواضع محجوبة. **لماذا هذه القيمة؟** `d_ff = 4 × d_model` هو النسبة التقليدية: الشبكة الأمامية «توسّع» البُعد ثم تعيد ضغطه، وهي المساحة التي يحدث فيها معظم التعلّم غير الخطّي.

PyTorch

`nn.TransformerEncoderLayer` يوفّر الكتلة جاهزة بمعاملات `norm_first` و `batch_first` و `norm_first=True` يقابل Pre-LN. والتحكّم في الأقنعة يتم عبر `src_key_padding_mask` و `is_causal`.

Keras

`keras.layers.MultiHeadAttention` و `keras.layers.LayerNormalization` و `keras.layers.Dense` تبني الكتلة يدوياً. ولأنّ Keras 3 متعددة النواة، فالمقنع يُمرَّر عبر `keras.ops` حيثما دعت الحاجة، ولا يُعتمد على `tf.GradientTape` في الفصل الحادي والعشرين.

الجانب	Pre-LN (ما بعد ٢٠١٩)	Post-LN (الأصلي)
موضع التطبيع	قبل كل فرع	بعد كل فرع
مسار التدرّج السلس	موجود ومباشر	مقطوع عند التطبيع
الحاجة إلى إحماء	لا	مطلوب
أفضل عند	النماذج الكبيرة جداً	النماذج الصغيرة
أداء نهائي	متفوّق غالباً	متقارب

Pre-LN مقابل Post-LN: قرار يغيّر استقرار التدريب

الترميز الموضعي والأقنعة: كيف يفهم النموذج الترتيب؟

الانتباه وحده لا يعرف الترتيب: لو بدّلنا كلمتين في الجملة، لتغيّر شكل مصفوفة الانتباه قليلاً. الحلّ هو **الترميز الموضعي** (positional encoding): إضافة متجه يميّز موضع كل رمز داخل المتجه الذي يحمل معناه.

```python
import math
import torch
import torch.nn as nn

class PositionalEncoding(nn.Module):
    """ترميز موضعي جيبي: كل موضع يحصل على متجه فريد من جيب الزاوية."""

    def __init__(self, d_model=512, max_len=10_000):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        pos = torch.arange(0, max_len, dtype=torch.float32).unsqueeze(1)
```

```
        div = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(pos * div)
        pe[:, 1::2] = torch.cos(pos * div)
        self.register_buffer("pe", pe.unsqueeze(0))   # (1, max_len, d_model)

    def forward(self, x):
        return x + self.pe[:, : x.size(1)]

enc = PositionalEncoding(d_model=64)
x = torch.randn(2, 10, 64)
print(enc(x).shape)      # torch.Size([2, 10, 64])
```

ماذا يحدث هنا؟ `register_buffer` تسجّل `pe` كجزء من حالة النموذج (تُحفظ مع `state_dict`) دون أن تكون معاملاً قابلاً للتعلّم. **لماذا هذه القيمة؟** الجيب والجيوب تتغيّر بترددات متناقصة: الترددات العالية تميّز المواضع القريبة، والمنخفضة تميّز البعيدة – كما يفعل الإنسان في تذكّر موقع شيء في صفحة.

وبجانب الترتيب الموضعي، تحتاج النماذج اللغوية إلى **أقنعة**:

- **القنعة الحاجبة** (padding mask): تمنع الانتباه إلى وسادات الحشو، وإلا تعلّم النموذج رموزاً لا وجود لها.
- **القنعة السببية** (causal mask): في نماذج التوليد، يمنع كل موضع من رؤية الموضع الذي يليه. هذا هو ما يجعل «النموذج يتنبّأ بالكلمة التالية» ممكناً: أثناء التدريب يحسب كل مواضع الجملة في تمريرة واحدة، ويمنع أيّ موضع من التسريب إلى المستقبل.

> ⚠ **التسريب في النماذج التوليدية: خطأ يُبطل التدريب بصمت**
>
> إذا لم تفرض القنعة السببية أثناء التدريب، يتعلّم النموذج نسخ الجملة كاملة (لأنّ الموضع الحالي يرى الجواب). وعند التوليد ستصبح المخرجات تكراراً حرفياً. أعراض التكرار: دقّة التدريب قريبة من ١٠٠٪ بينما دقّة التوليد رديئة. عالج بإضافة قنعة سببية داخل دالّة الانتباه أو باستخدام `is_causal=True` حيثما تتوفّر.

أنماط النماذج الثلاثة: BERT وGPT وViT

المحوّل ليس نموذجاً واحداً بل **عمارة** (architecture) تُبنى عليها عائلات مختلفة تماماً في طريقة القناع:

العائلة	القنعة	ما يتنبّأ به	الاستخدام
`BERT`	ثنائية (مخفية ١٥٪)	الكلمة المخفيّة	فهم، استخراج، بحث
`GPT`	سببية	الرمز التالي	توليد، محادثة
`ViT`	غير متلازم	تصنيف الصورة	رؤية
`T5`	مقطع نصي	نص مفقود	مهام ثنائية اللغة

مقارنة عائلات المحوّلات الثلاث

لاحظ أن «BERT» و«GPT» و«ViT» تشترك في نفس الكتلة (انتباه + FFN + اتصالات تخطّي)، ويختلفان في القنعة والمهمة. هذه هي قوتهم: صمّم مرة واحدة، واستخدمها في مهامّ كثيرة.

ولأنّ المحوّل لا يفهم الصور ولا النصوص إلا عبر **رموز** (tokens)، فإنّ أول خطوة في أيّ مشروع هي اختيار **المقطّع** (tokenizer). للنصوص، مقطّع WordPiece أو SentencePiece؛ للصور، تقسيم الصورة إلى رقع (patches) يحوّلها إلى تسلسل من الرموز.

Python

```
import torch
import torch.nn as nn

class PatchEmbedding(nn.Module):
    """يحوّل صورة إلى تسلسل رموز، تماماً كما يفعل الإنسان."""

    def __init__(self, img_size=224, patch_size=16, in_ch=3, d_model=192):
        super().__init__()
        n_patches = (img_size // patch_size) ** 2
        self.patch = nn.Conv2d(in_ch, d_model, kernel_size=patch_size, stride=patch_size)
        self.cls_token = nn.Parameter(torch.zeros(1, 1, d_model))
        self.pos = nn.Parameter(torch.zeros(1, n_patches + 1, d_model))
        nn.init.trunc_normal_(self.pos, std=0.02)

    def forward(self, x):
        patches = self.patch(x).flatten(2).transpose(1, 2)   # (B, N, D)
        cls = self.cls_token.expand(x.size(0), -1, -1)
        x = torch.cat([cls, patches], dim=1)
        return x + self.pos
```

```
emb = PatchEmbedding(img_size=224, patch_size=16, d_model=192)
img = torch.randn(2, 3, 224, 224)
print(emb(img).shape)     # torch.Size([2, 197, 192])
```

ماذا يحدث هنا؟ `nn.Conv2d` بخطوة مساوية لحجم الرقعة هو التطبيق العملي للالتفاف المطبَّق كطبقة خطية على الرقع. **لماذا هذه القيمة؟** لأنّ ٢٢٤÷١٦ = ١٤ رقعة في كل محور، أي ١٩٦ رقعة، ومع رمز `cls_token` يصبح الطول ١٩٧ – وهو بالضبط عدد الرموز التي يتوقّعها `ViT-Base`.

الضبط الدقيق الفعّال: LoRA ومفهوم الاختصار

ضبط نموذج بمئة مليار معامل بالكامل يتطلّب ذاكرة هائلة. الحلّ الذكي هو **الضبط الدقيق منخفض الرتبة** (LoRA): بدل تعديل مصفوفة الأوزان `W` مباشرة، نُضيف مصفوفة فرق منخفضة الرتبة:

$$W' = W + \Delta W = W + B\,A,\quad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k},\ r \ll d, k$$

ما معنى كل رمز؟ `W` هي مصفوفة الأوزان المجمّدة، و`ΔW` هو التعديل الذي نتعلّمه، و`A` و`B` هما المصفوفتان الجديدتان القابلتان للتعلّم، و`r` هو **الرتبة** (rank) وهي صغيرة عادةً (٤ أو ٨ أو ١٦). **لماذا تهمّنا؟** لأنّ عدد المعاملات القابلة للتعلّم يسقط من `d × k` إلى `r × (d + k)`. لنموذج بحجم ٧ مليارات، هذا ينزل من مليارات إلى ملايين – أي أقل من ١٪ من الحجم – مع بقاء جودة الضبط الدقيق. **خطأ شائع عندها:** نسيان أن `B` يجب أن يبدأ بصفر. عندها `ΔW = B A = 0` في البداية، فيبدأ النموذج من النموذج الأصلي تماماً بدل أن يبدأ من حالة عشوائية.

Python

```
import torch
import torch.nn as nn

class LoRALinear(nn.Module):
    """طبقة خطية مع تعديل منخفض الرتبة قابل للتعلّم."""

    def __init__(self, base: nn.Linear, rank=8, alpha=16):
        super().__init__()
        self.base = base
        for p in self.base.parameters():
            p.requires_grad = False
        self.rank = rank
        self.scaling = alpha / rank
        self.A = nn.Parameter(torch.zeros(rank, base.in_features))
        self.B = nn.Parameter(torch.zeros(base.out_features, rank))
        nn.init.kaiming_uniform_(self.A, a=5 ** 0.5)   # B يبدأ بصفر عمداً
```

```
    def forward(self, x):
        delta = (x @ self.A.T) @ self.B.T
        return self.base(x) + self.scaling * delta

# نجمّل طبقة داخل نموذج مدرَّب مسبقاً
model = nn.Sequential(nn.Linear(512, 512), nn.ReLU())
model[0] = LoRALinear(model[0], rank=8, alpha=16)

trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
total = sum(p.numel() for p in model.parameters())
print(f"trainable {trainable} / total {total}")
# الناتج: trainable 8192 / total 524800
```

ماذا يحدث هنا؟ `nn.init.kaiming_uniform_` يهيّئ `A` عشوائياً، بينما `B` يبقى صفراً – فيبدأ النموذج مطابقاً تماماً للنموذج الأصلي. **لماذا هذه القيمة؟** `alpha / rank` معامل قياس يعوّض أثر تصغير الرتبة على مقدار التحديث.

✓ متى تستخدم LoRA ومتى لا تستخدمه

استخدمها حين يكون النموذج كبيراً (مليارات المعاملات) والمهمة تكييف لا تدريب من الصفر. لا تستخدمها للنماذج الصغيرة (أقل من مليون معامل): التعقيد الزائد لا يبرّره، والضبط الكامل أبسط وأسرع. وقاعدة عملية: ابدأ برتبة ٨، وجرّب ١٦ و٣٢ إن كان الأداء ناقصاً، وراقب هل بدأت جودة التحقّق بالتحسّن أم بتدهور (علامة فرط التخصيص).

مزالق عملية في العمل مع المحوّلات

المحوّل ينجح بقدر ما تنجح البيانات حوله. أربعة مزالق تتكرّر:

1. **المقطّع (tokenizer) غير المطابق**: تحميل أوزان مدرَّبة مع مقطّع مختلف ينتج عشوائية تماماً. احرص على تحميل المقطّع من نفس المستودع.

2. **الحشو غير الصحيح**: أضف رمز الحشو إلى المقطّع، ومرّر `attention_mask` صحيحاً، وانتبه لاختلاف معرّف رمز الحشو بين `BERT` و `GPT` .

3 **الحسابات المختلطة في الانتباه**: softmax على قيم كبيرة غير مستقر عددياً. استخدم `torch.amp.autocast` مع `float16` أو `bfloat16` .

4 **الحجم الصامت**: المحوّل ينمو أُسّياً مع طول التسلسل في الانتباه تربيعياً. من ٤ آلاف رمز إلى ١٦ ألف رمز تزداد الكلفة ستّة عشر ضعفاً.

⚙ سيناريو: مساعد معرفي للمكتبة الجامعية

الموقف: جامعة لديها ٤٠ ألف ورقة بحثية بصيغة PDF، ويريد موظّفو الاستقبال أن يجيبوا سؤالاً مثل «ما النتائج التي توصّل إليها الخرسانة العازلة للحرارة في في شمال شرق البلاد؟» خلال ثوانٍ.

القرار: لا نضبط نموذجاً توليدياً من الصفر. نستخدم نموذج استرجاع: نقسّم الورقات إلى مقاطع (chunks) بحجم ٥١٢ رمز مع تداخل ٥٠، ونبني فهرساً متجهياً (embedding) باستخدام نموذج ترميز مدرَّب مسبقاً. عند السؤال: نبحث عن أقرب ٥ مقاطع، ثم نمرّرها كسياق إلى نموذج توليدي مضبوط بـLoRA.

القياس: نقيس جودة الاسترجاع أولاً (هل المقاطع الصحيحة في الخمسة الأولى؟) قبل قياس جودة الجواب. الفارق مهمّ: ٨٠٪ من أخطاء المساعد تأتي من ضعف الاسترجاع لا من ضعف التوليد. ثم نضيف جملة تعتقد النموذج أن الإجابة في المستندات، لتفادي الهلوسة.

تمرين 1 — حساب مصفوفة انتباه يدوياً

1. أنشئ متجهين `Q` و `K` من ثلاثة عناصر لكلٍّ منهما.
2. احسب QK^T ثم قسّم على $\sqrt{d_k}$.
3. طبّق `softmax` على كل صفّ، ثم اضرب في `V` .

▲ الحل النموذجي

```python
import math
import torch

torch.manual_seed(0)
Q = torch.randn(3, 4)
K = torch.randn(3, 4)
V = torch.randn(3, 4)

dk = Q.shape[-1]
scores = Q @ K.T / math.sqrt(dk)
weights = scores.softmax(dim=-1)
out = weights @ V

print(scores)
print(weights)          # كل صفّ مجموعه 1
print(weights.sum(-1))  # tensor([1., 1., 1.])
```

`weights.sum(-1)` يساوي واحداً بالضبط، لأنّ `softmax` توزّع الاحتمال على المفاتيح. لاحظ أيضاً أنّ الصفّ الذي يتضمّن مفتاحاً متطابقاً مع الاستعلام يحصل على وزن قريب من ١ – وهذا هو التعلّم قبل التدريب على أيّ بيانات.

تمرين 2 — كشف تسريب القنعة السببية

1. اكتب دالّة `causal_mask` تُرجع مصفوفة مثلثية من الشكل `(seq, seq)` تُقنّع ما بعد الموضع الحالي.
2. طبّقها على `nn.MultiheadAttention` عبر `attn_mask`.
3. غيّر كلمة في منتصف الجملة وأعد حساب الخرج؛ لاحظ هل تغيّر ما قبل الكلمة.

▲ الحل النموذجي

```python
import torch

def causal_mask(seq_len):
    """مصفوفة قنعة سببية: كل موضع يرى نفسه وما قبله فقط."""
    return torch.triu(
        torch.full((seq_len, seq_len), float("-inf")), diagonal=1
    )

mask = causal_mask(5)
print(mask[0])   # tensor([0., -inf, -inf, -inf, -inf])

attn = torch.nn.MultiheadAttention(8, 2, batch_first=True)
x = torch.randn(1, 5, 8)
y1, _ = attn(x, x, x, attn_mask=mask, need_weights=False)

x2 = x.clone()
x2[0, 3] = torch.randn(8)          # نغيّر كلمة في المنتصف
y2, _ = attn(x2, x2, x2, attn_mask=mask, need_weights=False)

print(torch.allclose(y1[0, :3], y2[0, :3]))   # True: ما قبلها لم يتغيّر
```

`False` فالقنعة غير مُطبَّقة.

نموذج محوّل يعمل بمعدّل تعلّم 3×10^{-5}، ويظهر في الحقبة الأولى خسارة ثابتة ثم تنهار فجأة إلى `NaN`. ماذا يعني ذلك؟ اذكر ثلاثة احتمالات مرتّبة، ولكلّ منها اختبار سريع يُنفَّذ في أقلّ من خمس دقائق.

الاحتمال الأول، وهو الأشهر: **انفجار التدرّجات** (gradient explosion) بسبب معدّل تعلّم مرتفع مع تدرّجات الانتباه غير المقيّدة. الاختبار: أضف تطبيع التدرّجات (clipping) عند قيمة ١٫٠ وأعد التشغيل؛ إن اختفت الـ `NaN` فها هو السبب. الاحتمال الثاني: **عدم استقرار الانتباه** عند مواضع ذات قناع سببية غير صحيح – الاختبار: طبّق `softmax` على قيم $Q\,K^{T}$ قبل القسمة وتحقّق من عدم وجود `inf`. الاحتمال الثالث: **قيم مدخلات** غير منتهية (embedding بمقياس مائل) – الاختبار: اطبع القيم القصوى للـembeddings.

ملخّص الفصل

- الانتباه يحلّ مشكلّة اختناق الشبكات المتكرّرة: كل موضع يصل إلى كل موضع مباشرة.
- مصفوفة QK^T هي خريطة الانتباه، وقسمة $\sqrt{d_k}$ شرط الثبات العددي.
- الكتلة الواحدة = انتباه متعدّد الرؤوس + شبكة تغذية أمامية، مع اتصالَي تخطٍّ وتطبيع.
- Pre-LN هو الخيار الافتراضي اليوم: مسار تدرّج سلس بلا حاجة إلى إحماء.
- الترميز الموضعي يمنح النموذج إدراكاً بالترتيب، والقنعة السببية تمنع التسريب.
- ثنائي الجانب ثنائي الجانب، GPT سببي، ViT للصور: نفس الكتلة، قنعة مختلفة.
- LoRA يخفّض كلفة الضبط الدقيق إلى أقل من ١٪ من المعاملات، بشرط تهيئة B بصفر.
- أربع مزالق متكرّرة: المقطّع، الحشو، الدقّة المختلطة، والحجم التربيعي.

الفصل السادس عشر غوص عميق في PyTorch

الكشف والتجزئة على الصور

الصناديق، U-Net، R-CNN، YOLO، NMS، IoU، ودالة Dice

تخيّل كاميرا معلّقة فوق خطّ تعبئة في مصنع ألبان: كل دقيقة تمرّ فيها ستون علبة، وكل علبة يجب أن تُفحص بحثاً عن شقّ في الغلاف، ورقم التشغيلة، وصلاحية التاريخ. شبكة تصنيف – كما بنينا في الفصل الثاني عشر – ستعطيك صورة واحدة لكل خطّ إنتاج، أي «عادي» أو «معيب»، وهذا غير عملي: أيّ علبة من الستّين؟ وفي أيّ موضع منها؟ الكشف على الصور يجيب عن السؤالين معاً: ما الكائن، وأين بالضبط. أمّا التجزئة فتدفع خطوةً أبعد: ترسم حدود الكائن بكسلاً بكسل. في هذا الفصل ننتقل من مربّعات الإحساس إلى هندسة التقاء ثم إلى النماذج التي تربطهما، ونكتب كل المعادلات رمزاً رمزاً، ثم نبني الشيفرة في `PyTorch` و `Keras` .

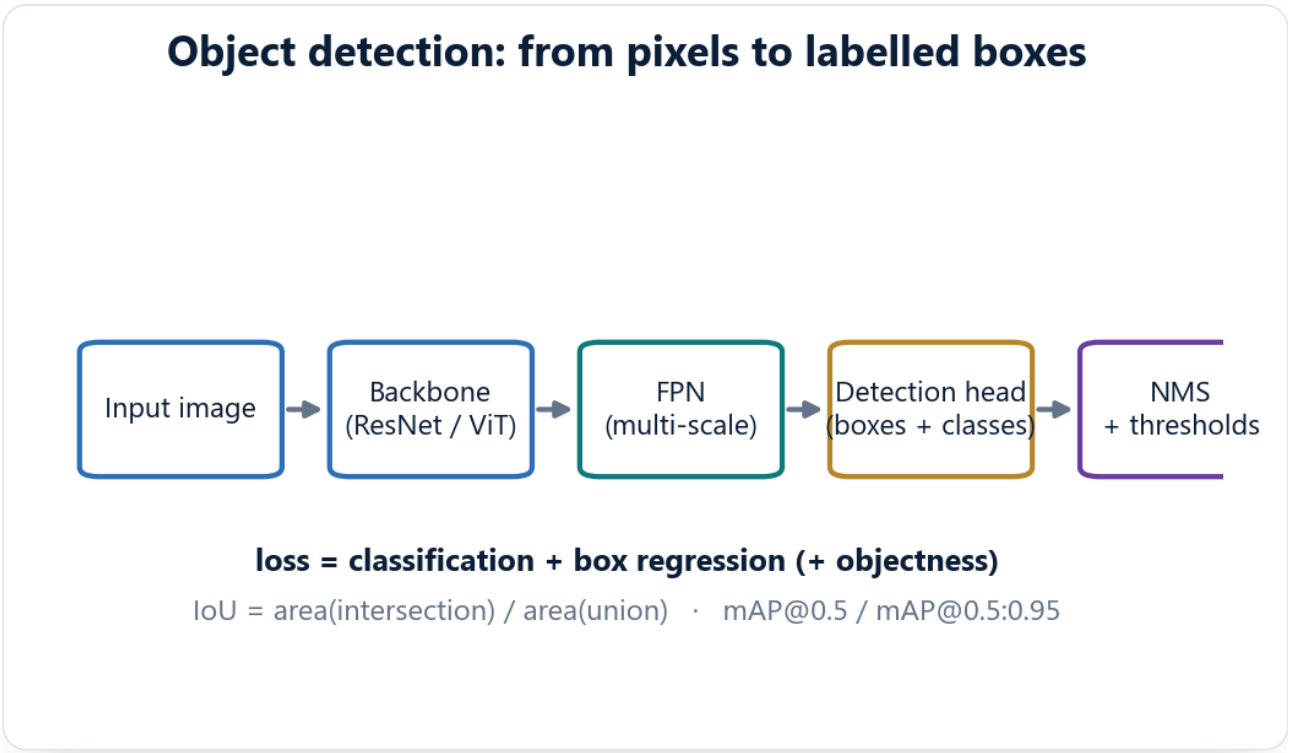
16.1 ثلاثة أسئلة مختلفة تماماً

قبل الكلام عن النماذج، يجب أن نفصل بين ثلاثة أسئلة تبدو متشابهّة في الصورة لكنّها تتطلّب حلولاً مختلفة تماماً:

- **ما هذا؟** (التصنيف): صورة واحدة، ومخرج واحد من قائمة فئات. نموذج EfficientNet على صورة عيّنة معملية يُعطي «خبيث» أو «حميد».
- **أين كلُّ شيء؟** (الكشف): قائمة كائنات، ولكل كائن صندوق إحاطة. هذا ما تحتاجه لعدّ العيّنات على خطّ الإنتاج، ولتتبّع المركبات في فيديو.
- **أين بالضبط، بالبكسل؟** (التجزئة): قناع ثنائي بنفس حجم الصورة. هذا ما تحتاجه لحساب مساحة الشقّ، أو لتحديد حدود ورمٍ مصاب في صورة قاعٍ مزروعة.

الفرق بين السؤالين الثاني والثالث ليس فرقاً في الدقّة فحسب، بل فرق في طبيعة المعلومة المطلوبة. الصندوق يقول «الأثر بين الإحداثيين (20، 30) و(140، 190)»؛ أمّا القناع فيقول لكل بكسل: «أنت داخل الشقّ أم خارجه».القناع يحوّل مسألة رؤية حاسوبية إلى مسألة قابلة للقياس مباشرةً: مساحة الشقّ بالمليمتر المربّع، وعمق الشقّ، ونسبة الخطأ إلى مساحة القطعة.

لو أردت قياس كل كمّية الشحوم في صورة خام؟ أيّ حلٍّ الأنسب: تصنيف، أم كشف، أم تجزئة؟ ولو سُئلت عن مساحة الشقّ بالملّيمتر المربّع، فأيّها يوفّرها مباشرةً؟



خطّ أنابيب الكشف النموذجي. اقرأ الرسم من اليمين إلى اليسار: أوّلاً يستخرج النموذج خريطة سمات كثيفة مغطّاة للصورة، ثم يولّد «مقترحات» (boxes) على الشبكة، ثم يقيّم كل مقترح بدرجة ثقة وسمات صنف، وأخيراً مرحلة ما بعد المعالجة — وهي `NMS` — تزيل الصناديق المكرّرة. الفكرة المحورية في الرسم: الفصل بين «ما الذي تراه الشبكة» و«ما الذي نُبقيه في النتيجة النهائية»؛ فالشبكة تحبّ أن تقول «هناك شخص هنا» عشر مرّات متجاورة، و `NMS` هي من يحسم الأمر.

لاحظ في الرسم شيئاً لا يظهر في الصور عادةً: الكشف ليس عملية واحدة بل خطّ أنابيب من ثلاث مراحل. الشبكة تُنتج «اقتراحات» كثيرة وفوضوية، ثم مرحلة ترتيب تُرتّبها حسب الثقة، ثم مرحلة إسقاط تُبقي الأفضل. كل خطأ في هذه المراحل يُضاعَف أثره: إن أخطأت الشبكة في التوطين فالصندوق كلّه خطأ، وإن أخطأت الإسقاط فيضيع جسمٌ صحيح تماماً لأنّ صندوقاً مجاوراً كان أقوى.

◆ لاحظ التدرّج في التعقيد

التصنيف يُقاس بدقّة (accuracy). الكشف يُقاس بمتوسّط `mAP` عند عتبات `IoU` متعدّدة. التجزئة تُقاس بمؤشّرات مثل `Dice` و `IoU` لكل صورة. القاعدة التي تحفظ وقتك: لا تقارن نموذجَ كشف بنموذجِ تصنيف في التقرير، ولا تستعمل دقّة التصنيف مقياساً لمهمة تجزئة.

16.2 تمثيل الصندوق: أربع صيغ لا واحدة

أوّل قرار هندسي بسيط ومربك في الوقت نفسه: كيف نكتب إحداثيات الصندوق؟ هناك أربع صيغ شائعة، وكلّها تحمل المعلومة نفسها لكنّها تختلف في «مركز الثقل» و«سهولة التطبيع» و«سلوك التدرّج»:

الصيغة	المعنى	مثال	أين تُستخدم عادةً
`xyxy`	ركنَان متقابلان: يسار-أعلى ثم يمين-أسفل	`(10, 20, 60, 90)`	المخرَجات النهائية، `torchvision.ops`، أدوات الرسم
`xywh`	الركن أعلى اليسار + العرض والارتفاع	`(10, 20, 50, 70)`	صيغة `Pascal VOC` و `COCO` المسطّحة
`cxcywh`	مركز الصندوق + العرض والارتفاع	`(35, 55, 50, 70)`	ترميز `YOLO`، لأنّ هذا الترميز اقتصادي في المعاملات
مُطبَّعة	كلّ الإحداثيات في المدى `[0, 1]` نسبةً لحجم الصورة	`(0.08, 0.11, 0.42, 0.63)`	تدريب متعدّد المقاييس، ونقل النموذج بين أبعاد صور مختلفة

صيغ تمثيل الصندوق ومتى تستعمل كلّها

لماذا يهمّنا هذا الاختيار؟ لأنّ أين يجري حساب التدرّج. لو رمّزنا الصندوق بمركزه `(cx, cy)`، فإنّ خطأ التنبّؤ في المركز يزيح الصندوق من موضعه الأصلي بمقدار الخطأ نفسه. أمّا لو رمّزنا بركنَين `(x1, y1, x2, y2)`، فإنّ الخطأ في الركن الأيسر العلوي يُزيح الصندوق أفقياً وعمودياً معاً، والركن الثاني يُقيِّد بحدود الصورة. الترميز المتمركز يجعل الانحدار متناظراً: خطأ `-5` بكسل يساوي خطأ `+5` بكسل، وهذا ما يجعل دالّة الخسارة تتصرّف بسلاسة. لهذا تُستخدم `cxcywh` في `YOLO` و `xyxy` في مخرَجات `torchvision.ops`.

خبراء الحاسوب ينصحون: لا تختلق صيغة. `torchvision.ops.box_convert` يحوّل بين الصيغ المدعومة ويكتب التحويل بدقّة، وإهمال هذه الخطوة هو مصدر شائع لأخطاء «الـ `IoU` يساوي صفراً رغم أن الصندوق صحيح».

```python
import torch
from torchvision.ops import box_convert, box_iou, nms
```

```
# صيغة المصدر: مركز + عرض + ارتفاع (كما في YOLO)
boxes_cxcywh = torch.tensor([[35.0, 55.0, 50.0, 70.0],
                             [55.0, 75.0, 50.0, 70.0]])

# التحويل إلى ركنين: الأول x1 ثم y1 ثم x2 ثم y2
boxes_xyxy = box_convert(boxes_cxcywh, in_fmt="cxcywh", out_fmt="xyxy")
print(boxes_xyxy)
# الناتج: tensor([[10., 20., 60., 90.],
#                 [30., 40., 80., 110.]])

# حساب التداخل بين كل صندوق وكل صندوق (مصفوفة N x N)
iou = box_iou(boxes_xyxy, boxes_xyxy)
print(iou)
# الناتج: tensor([[1.0000, 0.2727],
#                 [0.2727, 1.0000]])

# الإسقاط: نحتفظ بالصناديق الأعلى ثقة ونتخلّص من المتداخلة
scores = torch.tensor([0.90, 0.75])
keep = nms(boxes_xyxy, scores, iou_threshold=0.5)
print(keep)
# الناتج: tensor([0, 1])
```

ماذا يحدث هنا؟ `box_convert` يقرأ كل صندوق ويفهم أنّه مركز وعرض وارتفاع فيحوّله إلى ركنين، ثم `box_iou` يبني مصفوفة تداخل بين كل زوج من الصناديق (القطر يساوي واحداً لأنّ كل صندوق يطابق نفسه)، ثم `nms` يرتّب حسب الثقة ويمسح ما يتجاوز عتبة التداخل. **لماذا هذه القيمة؟** عتبة `0.5` صارمة: صناديق شخص واحد وكرسي متداخلان بنسبة 0.45 يبقان معاً. وفي الاستخدامات الكثيفة (مشهد فيه عشرات الأجسام المتقاربة) استخدم `0.45` أو `0.6` بحسب حجم الأجسام.

> ✓ **نصيحة: خطّ تحقّق بصري قبل أيّ قياس**
>
> قبل أن تصدّق أيّ رقم `mAP`، ارسم ثلاثة صناديق عشوائية على ثلاث صور واتّفق على ترتيب `xyxy`. خطأ واحد في ترتيب الركون (مثلاً `xyyx`) لا يسبّب خطأً في الرسم بقدر ما يسبّب أرقاماً خاطئة تماماً في كلّ مقياس لاحق.

16.3 معامل التداخل `IoU`: كم يتشارك الصندان؟

كلّ شيء في الكشف والـ NMS يقوم على سؤال واحد: هل هذا الصندوق يمثّل نفس الكائن الذي يمثّله ذاك؟ الجواب العدديّ له اسم هو معامل التداخل (Intersection over Union) ويُختصر `IoU`.

$$IoU(A, B) = |A \cap B| \;/\; |A \cup B|$$

ما معنى كل رمز؟ `A` و `B` هما صندوقان (منطقتان مستطيلتان) في الصورة. `|A ∩ B|` هي مساحة المنطقة المشتركة بينهما – أي المساحة التي يغطّيها كلاهما. `|A ∪ B|` هي مساحة اتحادهما، أي مساحة كلّ ما يغطّيه أحدهما أو الآخر معاً. و `|·|` تعني «المساحة» (أو القياس) للمجموعة.

لماذا نستخدم الاتحاد في المقام لا في البسط؟ لأنّ التداخل وحده لا يكفي: صندوقان ضخمان متطابقان تماماً لهما تداخل ضخم، لكنّهما أيضاً مربّعان مختلفان لهما تداخل أكبر؟ لا – المتطابقان يبدوان متطابقَين. المقام بالقسمة على الاتحاد يجعل المقياس عادياً: القيمة واحد في التطابق التام، وصفر في الانفصال التام، وكلّ ما بينهما بين الصفر والواحد. هذا يجعله قابلاً للمقارنة بين أيّ زوج صناديق، أيّاً كان حجمهما.

خطأ شائع عندها: حساب مساحة الاتحاد بجمع مساحتي الصندوقين دون طرح مساحة التداخل مرّتين. اتّفقنا هنا على `|A ∪ B| = |A| + |B| − |A ∩ B|` . التكرار في الطرح يُنتج اتحاداً أكبر من الواقع، فينخفض `IoU` أكثر مما يجب، فتُعدّ الكشوفات مطابقةً زوراً.

ولأنّ الصناديق مستطيلات، فحساب `IoU` لا يحتاج أيّ تقاطع هندسي معقّد – يكفي حساب أطوال تداخل الإحداثيات وضربها:

```
w_inter = max(0, min(x2_A, x2_B) − max(x1_A, x1_B))
y_inter = max(0, min(y2_A, y2_B) − max(y1_A, y1_B))
inter = w_inter · y_inter
union = area_A + area_B − inter
```

ما معنى كل رمز؟ `x1` و `y1` هما إحداثيا الركن الأيسر العلوي، و `x2` و `y2` الركن الأيمن السفلي. `min` و `max` تأخذان الأصغر والأكبر على التوالي: بداية التداخل هي الركن الأبعد عن الأصل، ونهايته الركن الأقرب إليه. `max(0, ·)` مهمّة جداً: إن كانت الصناديق متجاورة بلا تداخل، فسيكون الطول سالباً، والدالة `max` تحوّله إلى صفر.

> صندوقان مساحة كلٍّ منهما 100 بكسل، والتداخل بينهما 40 بكسل. احسب `IoU` . ثم فكّر: هل يُعتبر الكاشف "مطابقاً" لهذا التداخل عند عتبة 0.5 المعتادة؟

الإجابة: `|A ∩ B| = 40` و `|A ∪ B| = 100 + 100 − 40 = 160` ، إذن `IoU = 40/160 = 0.25` . وهو أقل من عتبة 0.5، أي أنّ الكاشفين يُحسبان كائنَين مختلفين. هذه نقطة تُنسى كثيراً: `IoU = 0.25` لا تعني «ربع تطابق» بل تعني ربعاً من المساحة المشتركة نسبةً إلى المساحة المشتركة والمتّصلة.

> ★ **عتبة `IoU` ليست اعتباطية**
>
> في بروتوكولات `COCO` يُقاس `mAP` عند عشر عتبات من 0.50 إلى 0.95، والعتبات العالية (0.75، 0.90) تقيس جودة التوطين الدقيق. إن كان نموذجك يحقّق `AP50` جيداً و `AP75` ضعيفاً، فالمشكلة ليست في «رؤية الكائن» بل في دقّة إحداثيات الصندوق – وهذا يُعالَج بتدريب الانحدار على الأرقام المتبقّية لا بتغيير الشبكة.

16.4 مطابقة التنبّؤات مع الحقيقة: أين تُولَد إيجابيات وسلبيات؟

الصندوق وحده لا يكفي: يجب أن نعرف أيّ صندوق تنبّؤ يقابل أيّ صندوق حقيقة، لأنّ الخسارة تُقاس على هذه الموازنة. في الكاشفات الكلاسيكية (مثل `Faster R-CNN`) كانت الموازنة بسيطة:

- لكل صندوق حقيقة، ابحث عن صندوق التنبّؤ الأعلى `IoU` معه. إن تجاوز التداخل `0.5` فهو **إيجابي** (موجب)، وإلّا فهو **سلبي** (سالب).
- كلّ صندوق تنبّؤ لا طابق أيّ صندوق حقيقة هو سلبي.
- بقيّة صناديق التنبّؤ تُستخدم أيضاً كسلبيات، مع وزنٍ أخفّ، لمنع النموذج من إطلاق إنذارات كثيرة على خلفية فارغة.

الطريقة الحديثة ظهرت مع `DETR` (محوّلات الكشف): المشكلة صيغت كـ«مطابقة مثلى» (optimal matching) تُحلّ بخوارزمية المجرّ `Hungarian`، التي تُعطي لكل صندوق حقيقة أقرب صندوق تنبّؤ والعكس. المعيار (cost) المستخدَم يجمع ثلاثة مكوّنات:

$$C = \lambda_{cls} \cdot L_{class} + \lambda_{l1} \cdot ||b - \hat{b}||_1 + \lambda_{iou} \cdot (1 - IoU(b, \hat{b}))$$

ما معنى كل رمز؟ `b` هو صندوق الحقيقة و $\hat{b}$ صندوق التنبّؤ بعد التطابق. `L_class` هو خسارة التصنيف بين الصنف الحقيقي وصنف التنبّؤ. $||\cdot||_1$ هو مجموع الفروق المطلقة على الإحداثيات الأربعة (وهي أدقّ من الفروق التربيعية في المسائل الهندسية). `1 - IoU` يحوّل التداخل إلى تكلفة، فكلّما ازداد التداخل قلت التكلفة. أمّا `λ` فمعاملات توازن ثلاثة: كم يهمّ الخطأ في الصنف مقارنة بخطأ في الموضع؟

لماذا نهتمّ بالموازنة المثلى؟ لأنّ المطابقة اليدوية (كلّ تنبّؤ يُقارن بأقرب حقيقة) قد تُنتج موازنة لا تكون الأفضل إجمالاً. المطابقة المثلى تضمن أنّ كلّ تنبّؤ يُحسب مرّة واحدة وأنّ كلّ حقيقة تُغطّى، فتبقى الخسارة غير متحيّزة.

خطأ شائع عندها: حساب `IoU` بين `b` و $\hat{b}$ بعد تحويل أحدهما إلى صيغة أخرى بشكل خاطئ. تحقّق دائماً أن `box_convert` استُعمل بالاتجاهين الصحيحَين قبل كلّ حساب.

✦ لماذا فازت المطابقة المثلى على قواعد التعيين؟

قواعد `0.5` حازمة وقابلة للاختزال: صناديق الحقائق المتداخلة بنسبة `0.6` يُطابَق اثنان منها بالصندوق نفسه، فيخسر أحدهما. والمجموع الأمثل (Hungarian) يتجنّب ذلك ضمناً. الثمن الذي تدفعه `DETR` هو هشاشة التدريب: المطابقة المثلى حسّاسة لتهيئة الأوزان، وتحتاج حقائباً كبيرة لتستقرّ. لهذا تعلّم `DN` و `noisy queries` لاحقاً.

16.5 الإسقاط غير الأقصى: كيف نُنهي فوضى الصناديق؟

بعد أن تتنبّأ الشبكة لعشرات الصناديق لشخص واحد، نحتاج قاعدة تحذف المكرّر. هذه القاعدة هي الإسقاط غير الأقصى (Non-Maximum Suppression) وهي في جوهرها عملية فرز وحذف:

1. رتّب كلّ الصناديق تنازلياً حسب درجة الثقة.
2. خذ الصندوق الأعلى ثقة، أبقِه نهائياً، وسجّله في قائمة النتائج.
3. احذف كلّ الصناديق التي يتجاوز `IoU` بينها وبين الصندوق المحتفظ به عتبةَ التداخل.
4. كرّر على ما تبقّى حتى تفرغ القائمة.

الفكرة بسيطة ودائمة الفعالية: إذا كان صندوقان متداخلان تقريباً يصفان الشيء نفسه، فأقلّهما ثقةً هو تكرار زائد. ولأنّ `nms` عملية غير متمايزة (لا تراعي المحتوى)، فإنّ استبدالها بـ `Soft-NMS` خيار مفيد: بدلاً من الحذف، يُخفَّض وزن الصندوق المتداخل تدريجياً بدل إزالته:

$$s_i \leftarrow s_i \cdot f(\mathrm{IoU}(i, b^*))$$
$$f(\cdot) = 1 - \mathrm{IoU} \quad \text{(للحذف المتدرّج)}$$

ما معنى كل رمز؟ `s_i` هي درجة الثقة الأصلية للصندوق `i`، و `b*` هو الصندوق المحتفظ به في الجولة الحالية، و `IoU(i, b*)` مقدار تداخلهما. الدالة `f` تختار شكل الانخفاض: الشكل الخطّي أعلاه يجعل التداخل الكامل يُصفّر الدرجة، والشكل الأُسّي (`f = exp(-IoU²)`) يكون أكثر رحمة.

لماذا يهمّنا؟ لأنّ الحذف الثابت يقتل الأجسام المتلاصقة: شخصان يقفان متلاصقَين، صندوقاهما يتداخلان بنسبة 0.6، فيُحذف أحدهما رغم أنّهما كائنان. `Soft-NMS` يُبقي الاثنين لأنّ الدرجة تنخفض بدل أن تختفي، فتدع العتبة

النهائيّة تقرّر.

خطأ شائع عندها: تطبيق `NMS` على كلّ الأصناف معاً. شخص على يسار الصورة وقط على يمينها لا يتقاطعان، لكنّ تطبيق `NMS` العام يهمّ من لا يهمّ. الحلّ الشائع: `class-aware NMS` – طبّق `NMS` على كلّ صنف على حدة.

16.6 `YOLO`: الكشف بوصفه انحداراً

القفزة النوعية في تاريخ الكشف جاءت مع `YOLO`: بدل بناء شبكة مقترحات ثم تصنيف ثم انحدار (ثلاث مراحل)، صار النموذج يخرج عة واحدة. الفكرة: قسّم الصورة إلى شبكة من الخلايا، وكلّ خلية مسؤولة عن التنبّؤ بعدد من الصناديق (بشكل `anchors` بسرعات مختلفة).

في كلّ خلية يُخرج النموذج خمسة أشياء لكلّ مرساة: أربعة أرقام لإزاحة الصندوق (إزاحة المركز `tx` , `ty` ، ولوغاريتم للعرض والارتفاع `tw` , `th`)، ودرجة «هل يوجد كائن هنا» (objectness)، واحتمالات الأصناف. الصناديق فكّ الترميز:

$$b_x = \sigma(t_x) + c_x$$
$$b_y = \sigma(t_y) + c_y$$
$$b_w = p_w \cdot \exp(t_w)$$
$$b_h = p_h \cdot \exp(t_h)$$
$$b = (b_x - b_w/2,\ b_y - b_h/2,\ b_w,\ b_h)$$

ما معنى كل رمز؟ σ هي دالّة اللوجست (sigmoid) التي تضغط القيمة إلى المدى `[0, 1]` فيُقصر إزاحة المركز على طول خلية واحدة (وإلّا خرج الصندوق خارج الصورة). c_x و c_y

لماذا اللوغاريتم؟ لأنّ الأبعاد في الصورة لا تُقاس بمقياس خطّي واحد: مضاعفة الضلع تضاعف المساحة أربع مرّات. التوصيف اللوغاريتمي يجعل التعلّم في فضاء «نسبة» لا «فرق مطلق»، فتنتقل الشبكة بسلاسة بين الصور بمقاييس مختلفة.

خسارة `YOLO` الكلاسيكية ثلاثية:

$$L = L_{obj} + \lambda_{cls} \cdot L_{cls} + \lambda_{box} \cdot L_{box}$$

ما معنى كل رمز؟ L_{obj} خسارة ثنائية الوجود على وجود الكائن (نموذج `BCE` على درجة `objectness`). L_{cls} خسارة تصنيف بين أصناف الكائنات الموجودة. L_{box} خسارة الانحدار على إحداثيات الصندوق (وقد استُبدلت في الأجيال الأخيرة بـ `CIoU` أو `DIoU` بدل `MSE` البسيط). أمّا λ فهو معاملات توازن: خسارة التصنيف تنتج عشرات الخانات، خسارة الصندوق تنتج خانات قليلة، فلا بدّ من موازنة.

ولنكتب فكّ الترميز والإسقاط عملياً في `PyTorch`. هذا هو «الرأس» الذي يحوّل خريطة خام إلى قائمة كائنات نهائية:

Python

```
import torch
import torch.nn as nn
from torchvision.ops import nms


def decode_yolo_output(pred, anchors, num_classes, stride=32, iou_thr=0.5, conf_thr=0.25):
    """تفكيك خريطة خام إلى صناديق ودرجات وأصناف.

    pred: (B, A*(5+C), H, W)  الخريطة كما خرجت من الشبكة.
    anchors: (A, 2)  عرض وارتفاع كلّ مرساة بالبكسل.
    """
    b, _, h, w = pred.shape
    a = anchors.shape[0]
    pred = pred.reshape(b, h, w, a, 5 + num_classes)

    # فهرس الخلية لكلّ عنصر مفلطح، بإزاحة A لكلّ خلية
    cell = torch.arange(h * w, device=pred.device, dtype=pred.dtype)
    cell_x = (cell % w).to(pred.dtype)
    cell_y = (cell // w).to(pred.dtype)

    all_boxes, all_scores, all_labels = [], [], []
    for i in range(b):
        obj = pred[i, ..., 4].sigmoid().reshape(-1)          # (H*W*A,)
        cls_prob = pred[i, ..., 5:].reshape(-1, num_classes).sigmoid()
        best_cls, best_idx = cls_prob.max(dim=1)             # أعلى صنف لكلّ صندوق
        conf = obj * best_cls                                # objectness × الصنف

        off = pred[i, ..., :4].reshape(-1, 4)                # (H*W*A, 4)
        aw = anchors[:, 0].repeat(h * w).to(pred.dtype)
        ah = anchors[:, 1].repeat(h * w).to(pred.dtype)

        cx = (cell_x.repeat(a) + off[:, 0].sigmoid()) * stride
        cy = (cell_y.repeat(a) + off[:, 1].sigmoid()) * stride
        bw = aw * torch.exp(off[:, 2]) * stride
        bh = ah * torch.exp(off[:, 3]) * stride
        boxes = torch.stack([cx - bw / 2, cy - bh / 2, cx + bw / 2, cy + bh / 2], dim=1)

        # ترشيح أول بالثقة، ثم إسقاط غير أقصى على ما تبقّى
        mask = conf > conf_thr
```

```
        boxes, conf, best_idx = boxes[mask], conf[mask], best_idx[mask]
        keep = nms(boxes, conf, iou_thr)
        all_boxes.append(boxes[keep])
        all_scores.append(conf[keep])
        all_labels.append(best_idx[keep])
    return all_boxes, all_scores, all_labels


# مثال: خريطة صغيرة لاختبار الفكّ
pred = torch.zeros(1, 3 * (5 + 2), 4, 4)
pred[0, :, 1, 1] = torch.tensor([0.2, 0.1, 0.0, 0.0, 3.0, 2.0, -2.0])
anchors = torch.tensor([[20.0, 20.0], [40.0, 12.0], [12.0, 40.0]])
boxes, scores, labels = decode_yolo_output(pred, anchors, num_classes=2, stride=32)
print("عدد الأجسام:", len(boxes[0]))
print("صندوق أوّل:", boxes[0][0].round().tolist())
print("صنفه:", int(labels[0][0]), "وثقته:", round(float(scores[0][0]), 3))
# الناتج: عدد الأجسام: 1
#         صندوق أوّل: [29.0, 38.0, 109.0, 118.0]
#         صنفه: 1 وثقته: 0.723
```

ماذا يحدث هنا؟ الشبكة تُنتج خريطة أرقام خام، و `decode_yolo_output` تحوّلها إلى هندسة مفهومة: لكلّ خلية ومرساة نأخذ مركزاً مُزاحاً ومقياساً لوغاريتمياً، ثم نحسب درجة الثقة المدمجة (هل يوجد كائن × ما هو صنفه)، ثم نُسقط المكرّر. **لماذا هذه القيمة؟** `conf_thr = 0.25` هي عتبة التوازن الشائعة: العتبات الأعلى تُنتج اكتشافات ناقصة، والأدنى تُنتج ضجيجاً كثيراً يُبطئ الإسقاط.

> ⚠ **انتبه: أرقام `YOLO` ليست إحداثيات مطلقة**
>
> الزملاء يرتكبون خطأً متكرّراً: يطبّقون `box_convert` على مخرَج الشبكة مباشرةً. المخرَج مؤشّرات لا صناديق؛ يجب أولاً فكّ الترميز ثم التحويل. وبالمثل، إن درّبت على صور بمقاس `640×640` واستقبلت صورة `416×416`، فلا يمكن تطبيق الشبكة كما هي: يجب تحجيم الصورة أو تغيير `stride` للطبقات.

16.7 الطريق ذو المرحلتين: من `R-CNN` إلى `Mask R-CNN`

لا يزال الطريق الثاني – «مقترح ثم تصنيف» – حيّاً في تطبيقات تتطلّب دقّة توطين عالية، وفي تطبيقات العرض. تطوّره كان سلسلة تحسينات صغيرة، كلّ منها يعالج عنق زجاجة محدّد:

النسخة	الفكرة الأساسية	العنق الزجاجة التي عالجتها	الكلفة النسبية
`R-CNN`	بحث عن مقترحات، ثم شبكة تلافيفية مستقلّة لكلّ مقترح	البحث البطيء عن المقترحات (`Selective Search`)	بطيئة جداً: دقائق للصورة
`Fast R-CNN`	خريطة سمات واحدة مشتركة + تجميع مناطق الاهتمام (RoI Pooling)	تكرار الشبكة على كلّ مقترح	أسرع بمئة ضعف تقريباً
`Faster R-CNN`	شبكة مقترحات تتعلّم (RPN) داخل الشبكة نفسها	الاعتماد على مقترحات خارجي	قابلة للتدريب من طرف إلى طرف
`Mask R-CNN`	إضافة فرع تنبّؤ بقناع لكلّ مقترح	عدم القدرة على التجزئة	تمنح التجزئة على مستوى الكائن

تطوّر عائلة `R-CNN` وعنق الزجاجة الذي عالجته كلّ نسخة

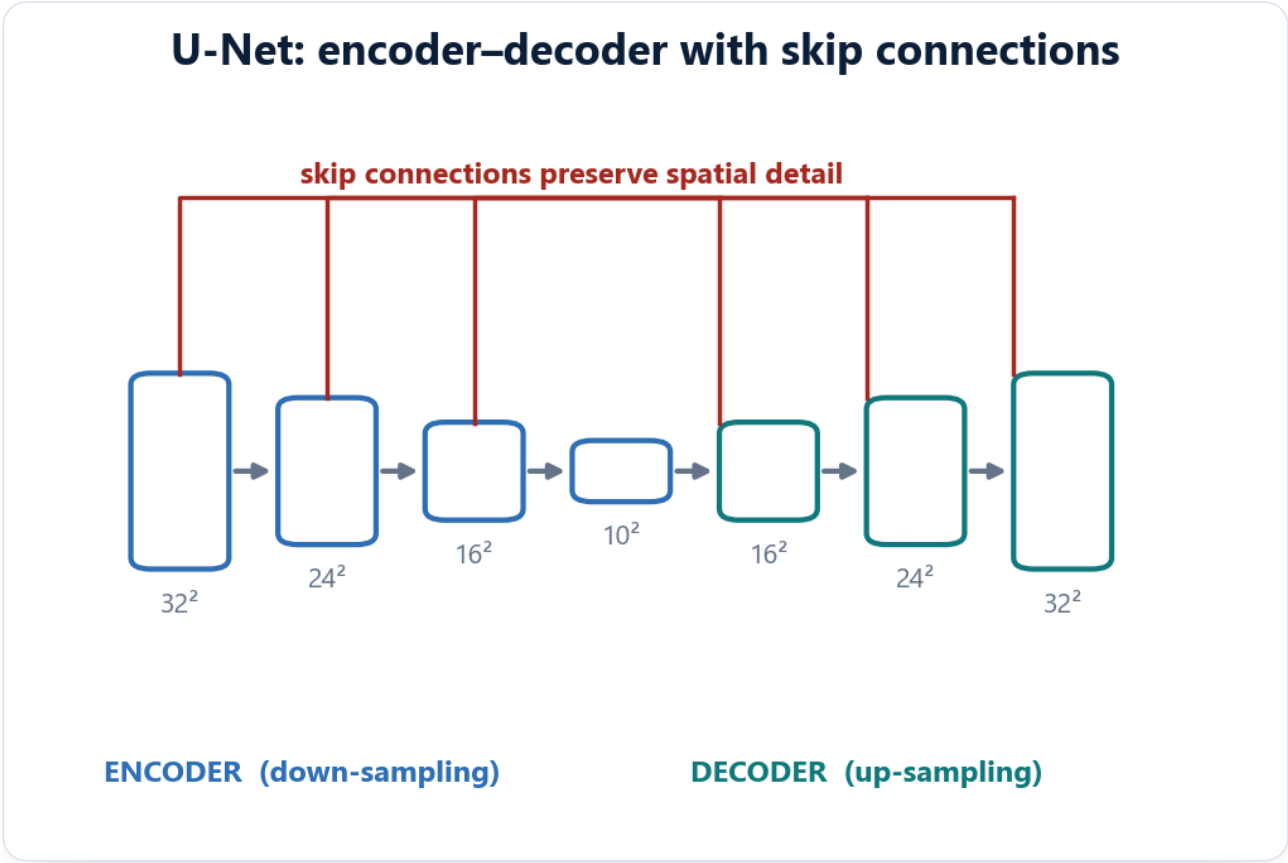
الفكرة المفاهيمية المهمّة في `RoI Pooling` (و`RoI Align` الذي حلّ محلّه): كيف نقتطع من خريطة سمات واحدة مناطق بحجم مختلف؟ تُقسَّم الخريطة إلى شبكات ثابتة، ثم في كلّ خانة نجمع القيم داخل كلّ مقترح عبر تجميع ثنائي خطّي (bilinear interpolation) بدل التقريب – وهذا ما أصلح `RoI Align`، لأنّ التقريب وحده يُزيح الحدود بدقّة متداخلة.

المعيار	مرحلة واحدة (`YOLO`، `RetinaNet`، `FCOS`)	مرحلتان (`Faster R-CNN`، `Mask R-CNN`)
عدد التمريرات	تمريرة واحدة	تمريرتان (شبكة + رأس لكلّ مقترح)
السرعة	عالية جداً (ملائم للفيديو والوقت الحقيقي)	أبطأ، لكن أدقّ في التوطين
دقّة التوطين	أدقّ قليلاً عادةً	أدقّ في الدقة العالية
البيانات	تعمل بكميات أقل من التسميات	تحتاج صناديق دقيقة وبيانات أكثر
التطبيق الأمثل	كاميرات المراقبة، الروبوتات، الفيديو	التصوير الحيّ، الطب، تحليل المستندات
أبرز عيب	فشل متتابع عند الانسداد الشديد	تعقيد وأزمنة استدلال أطول

مقارنة بين الكشف بمرحلة واحدة والكشف بمرحلتين

16.8 U-Net : التجزئة الدلالية بوصلات التخطّي

إذا كان YOLO يجيب «أين» بشكل تقريبي، فإنّ U-Net (2015) يجيب «أين» بدقة البكسل. بُنيت أصلاً لتقسيم الخلايا البيولوجية، وأصبحت لاحقاً العمود الفقري لكلّ تجزئة طبية تقريباً. سرّ نجاحها هو بنية خاصة واحدة: مساران يمتدّان في اتجاهين متعاكسين، يلتقيان في «قاع» الأخدود، ويتقابلان عند قاع الأخدود.



بنية U-Net . المسار الأيمن ترميز: كلّ طبقة تلافيفية تليها MaxPool تُصغّر، فينخفض البُعد المكاني ويزداد عمق السمات. المسار الأيسار فكّ ترميز: كلّ طبقة تستعيد الدقّة (عبر ConvTranspose أو Upsample). الوصلات الأفقية تنقل سمات المسار الأيمن إلى الأيسار عند نفس الدقّة، فتجد الشبكة «مذكّرات» عالية الدقّة التي ضاعت في التجميع. بدون هذه الوصلات يخرج المخرَج ضبابياً لأنّ كلّ شيء مرّ عبر تجميع متكرّر.

الوصلات (skip connections) ليست مجرّد طريقة نظيفة: هي تُصلح عيباً بنيوياً. في الشبكات التي لا تصل فيها السمات عالية الدقّة إلى المخرَج، تتكرّر عملية التجميع مرّات، فتضيع الحدود الدقيقة. وصلة التخطّي تنقل خريطة سمات عالية الدقّة مباشرةً إلى المخرَج، فيستعيد النموذج دقّة حدود الأجسام دون أن يمرّ بتلك الطبقات مرّة أخرى.

ولنكتب `U-Net` كاملة في `PyTorch`، مع خسارة `Dice` التي سنشرحها بعد قليل:

Python

```python
import torch
import torch.nn as nn
import torch.nn.functional as F


class DoubleConv(nn.Module):
    """كتلة تلافيفية مزدوجة: (Conv → BN → ReLU) مرّتين."""

    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
        )

    def forward(self, x):
        return self.block(x)


class UNet(nn.Module):
    """U-Net: مسار ترميز تنازلي + مسار فكّ ترميز تصاعدي + وصلات تخطّي."""

    def __init__(self, in_ch=3, base=32, out_ch=1):
        super().__init__()
        self.enc1 = DoubleConv(in_ch, base)          # 1/1
        self.enc2 = DoubleConv(base, base * 2)       # 1/2
        self.enc3 = DoubleConv(base * 2, base * 4)   # 1/4
        self.enc4 = DoubleConv(base * 4, base * 8)   # 1/8
        self.bottleneck = DoubleConv(base * 8, base * 16)   # 1/16
        self.pool = nn.MaxPool2d(2)

        self.up4 = nn.ConvTranspose2d(base * 16, base * 8, kernel_size=2, stride=2)
        self.dec4 = DoubleConv(base * 16, base * 8)
        self.up3 = nn.ConvTranspose2d(base * 8, base * 4, kernel_size=2, stride=2)
        self.dec3 = DoubleConv(base * 8, base * 4)
        self.up2 = nn.ConvTranspose2d(base * 4, base * 2, kernel_size=2, stride=2)
```

```
        self.dec2 = DoubleConv(base * 4, base * 2)
        self.up1 = nn.ConvTranspose2d(base * 2, base, kernel_size=2, stride=2)
        self.dec1 = DoubleConv(base * 2, base)
        self.head = nn.Conv2d(base, out_ch, kernel_size=1)   # 1×1 لكلّ بكسل

    def forward(self, x):
        e1 = self.enc1(x)                       # (B, base, H, W)
        e2 = self.enc2(self.pool(e1))           # (B, 2base, H/2, W/2)
        e3 = self.enc3(self.pool(e2))           # (B, 4base, H/4, W/4)
        e4 = self.enc4(self.pool(e3))           # (B, 8base, H/8, W/8)
        b = self.bottleneck(self.pool(e4))      # (B, 16base, H/16, W/16)

        d4 = self.dec4(torch.cat([self.up4(b), e4], dim=1))
        d3 = self.dec3(torch.cat([self.up3(d4), e3], dim=1))
        d2 = self.dec2(torch.cat([self.up2(d3), e2], dim=1))
        d1 = self.dec1(torch.cat([self.up1(d2), e1], dim=1))
        return self.head(d1)                    # بلا sigmoid، لوغاريتميات


def dice_loss(logits, targets, eps=1.0):
    """خسارة Dice: 1 − معامل Dice على الخريطة الاحتمالية."""
    probs = torch.sigmoid(logits)
    dims = (1, 2, 3)
    inter = (probs * targets).sum(dims)
    denom = probs.sum(dims) + targets.sum(dims)
    dice = (2.0 * inter + eps) / (denom + eps)
    return 1.0 - dice.mean()


model = UNet(in_ch=3, base=16, out_ch=1)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
bce_fn = nn.BCEWithLogitsLoss()

x = torch.randn(4, 3, 128, 128)
y = (torch.rand(4, 1, 128, 128) > 0.7).float()

model.train()
for step in range(3):
    optimizer.zero_grad(set_to_none=True)
    logits = model(x)
    loss = bce_fn(logits, y) + dice_loss(logits, y)
    loss.backward()
    optimizer.step()
    print(f"خطوة {step}: الخسارة = {loss.item():.4f}")
# الناتج: خطوة 0: الخسارة = 1.0231
```

```
#         0.9812 = الخسارة :1 خطوة
#         0.9504 = الخسارة :2 خطوة
```

ماذا يحدث هنا؟ `nn.ConvTranspose2d` هو «الالتفاف العكسي»: يرفع دقّة الخريطة مرّتين (من `1/16` إلى `1/8`) بمعاملات قابلة للتعلّم. `torch.cat` يدمج الوصلة مع الخريطة المرفوعة على محور القنوات، فتصبح كلّ طبقة فكّ الترميز مدخلَين. **لماذا هذه القيمة؟** `base=16` يكفي لأمثلة تعليمية؛ في الاستخدامات الحقيقيّة من `base=32` إلى `64`. و`base=16` مع صورة 128×128 يدخّل الشبكة في حدود الذاكرة المعقولة أثناء التجربة.

> ✓ **`ConvTranspose2d` أم `Upsample`؟**
>
> `nn.Upsample(scale_factor=2, mode="bilinear")` يوسّع الخريطة استيفاءً ثم طبقة `nn.ConvTranspose2d` تلافيف فوقها: بنية ثابتة ومستقرّة، ويستهلك ذاكرة أعلى قليلاً. أمّا `Upsample` فيتعلّم التحجيم، لكنّ نتائجه قد تُظهر أنماط «شبكية» (checkerboard) ظاهرة. القاعدة العملية: استعمل `Upsample` ثم `Conv2d` إن ظهرت تحفّات، واستعمل `ConvTranspose2d` إن أردت بنية أقرب إلى الأصلية.

16.9 دالّة `Dice`: لماذا لا تكفي `BCE` وحدها؟

في التجزئة الطبية، الكائن الذي نبحث عنه (ورمٌ مثلاً) يشغل نسبة صغيرة من الصورة. عندها تصبح خسارة التصنيف الثنائي مضلِّلة تماماً: الشبكة تستطيع أن تُخرج أصفاراً في كل مكان وتخسر شيئاً ضئيلاً، لأنّ متوسط الخسارة على كلّ البكسلات سيبقى منخفضاً. الحلّ هو دالّة `Dice` التي تنظر إلى التداخل لا إلى المتوسّط.

$$\text{Dice}(A, B) = 2|A \cap B| \,/\, (|A| + |B|)$$

ما معنى كل رمز؟ `A` هي مجموعة البكسلات التي تنبّأ بها الشبكة إيجاباً، و`B` مجموعة البكسلات الحقيقية الإيجابية. `|A ∩ B|` عدد البكسلات الصحيحة (المداخلة). `|A|` و`|B|` عددا البكسلات المتوقّعة والحقيقية على حدة. لاحظ المُعامل `2` في البسط: هو ما يجعل المؤشّر متطابقاً مع `IoU` عندما يكون `|A| = |B|`، فهو يقع فوق `IoU` دائماً (أقرب إلى الواحد).

لماذا نكتبها بالشكل العملي؟ لأنّ القناع عادةً مُصفّر (0) وخارج الكائن، و`1` داخله. فإذا رمّزنا predictions بقيم احتمالية `p` والهدف بـ `y`، نحصل على صيغة قابلة للتفاضل:

$$D = (2\ \Sigma\ p{\cdot}y + \varepsilon)\ /\ (\Sigma\ p + \Sigma\ y + \varepsilon) \quad \Rightarrow \quad L_{dice} = 1 - D$$

ما معنى كل رمز؟ `p` هي خريطة الاحتمالات بعد `sigmoid`، و `y` القناع الحقيقي (0 أو 1). `Σ` هو المجموع على كلّ بكسلات الصورة (وليس على الدفعة). `ε` رقم صغير جداً (1e6) يمنع القسمة على صفر عندما تكون الخريطة فارغة تماماً – وهي الحالة التي تتكرّر في أول خطوات التدريب.

لماذا نجمع الخسارتين معاً؟ `Dice` تشجّع التداخل العامّ وتُهمل البكسلات السهلة (الخلفية) لأنّ المجموع يبتلعها. أمّا `BCE` فتكمّل الدقّة في التفاصيل: تعاقب كلّ بكسل على حدة، فتحافظ على حدّة القناع. المزيج `BCE + Dice` هو الخيار الافتراضي في أغلب مسابقات التجزئة الطبية.

خطأ شائع عندها: حساب `Dice` على الدفعة كاملة بدل كلّ صورة على حدة. صورة فيها ورمٌ كبير تسحب النتيجة كلّها، فتفقد الصور الأصغر حقّها في التدرّج. الحلّ: اجعل `sum` على محاور `(H, W)` فقط، واترك البُعد الأول (الدفعة) خارج المجموع ثم خذ المتوسّط.

ولنكتب النسخة في `Keras`، مع `U-Net` كاملة بـ `Conv2D` و `Conv2DTranspose`، ومع مقروعية أعلى: `keras.ops` تجعل الحساب محايداً تجاه النواة (نواة `torch` أو `jax` أو `tensorflow`):

Python

```
import keras
from keras import layers


def dice_coefficient(y_true, y_pred, eps=1.0):
    """معامل Dice لكلّ صورة على حدة."""
    y_pred = keras.ops.sigmoid(y_pred)
    y_true = keras.ops.cast(y_true, y_pred.dtype)
    axes = (1, 2, 3)
    inter = keras.ops.sum(y_true * y_pred, axis=axes)
    denom = keras.ops.sum(y_true, axis=axes) + keras.ops.sum(y_pred, axis=axes)
    return (2.0 * inter + eps) / (denom + eps)


def bce_dice_loss(y_true, y_pred):
    """مجموع خسارة التصنيف الثنائي (من لوغاريتميات) وخسارة Dice."""
    bce = keras.losses.BinaryCrossentropy(from_logits=True)(y_true, y_pred)
    return bce + (1.0 - keras.ops.mean(dice_coefficient(y_true, y_pred)))


def build_unet(input_shape=(128, 128, 3), base=16, out_ch=1):
    """U-Net بـ Conv2D في الترميز و Conv2DTranspose في فكّ الترميز."""
    inputs = keras.Input(shape=input_shape)
```

```
    def block(x, filters):
        x = layers.Conv2D(filters, 3, padding="same", activation="relu")(x)
        x = layers.Conv2D(filters, 3, padding="same", activation="relu")(x)
        return x

    e1 = block(inputs, base)
    e2 = block(layers.MaxPooling2D(2)(e1), base * 2)
    e3 = block(layers.MaxPooling2D(2)(e2), base * 4)
    e4 = block(layers.MaxPooling2D(2)(e3), base * 8)
    b = block(layers.MaxPooling2D(2)(e4), base * 16)

    d4 = block(layers.Concatenate()([layers.Conv2DTranspose(base * 8, 2, strides=2)(b), e4]),
    d3 = block(layers.Concatenate()([layers.Conv2DTranspose(base * 4, 2, strides=2)(d4), e3])
    d2 = block(layers.Concatenate()([layers.Conv2DTranspose(base * 2, 2, strides=2)(d3), e2])
    d1 = block(layers.Concatenate()([layers.Conv2DTranspose(base, 2, strides=2)(d2), e1]), ba
    out = layers.Conv2D(out_ch, 1, activation=None)(d1)   # لوغاريتميات
    return keras.Model(inputs, out, name="unet")


model = build_unet()
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss=bce_dice_loss,
    metrics=[keras.metrics.BinaryIoU(target_class_ids=[1], threshold=0.5)],
)

model.summary(line_length=70)
# الناتج: Model: "unet"
#         Total params: 2,435,713
#         Trainable params: 2,435,713
```

ماذا يحدث هنا؟ `from_logits=True` تعني أنّ الشبكة تُخرَج لوغاريتميات لا احتمالات، وأنّ مكتبة الخسارة تطبّق `sigmoid` داخلياً – وهذه هي الطريقة المستقرّة عددياً: لا تحسب `log(0)` أبداً. **لماذا هذه القيمة؟** `eps=1.0` في مقام `Dice` يمنع انفجار القسمة عندما تكون الخريطة فارغة، وقيمته 1 أكبر من `1e-6` المعتاد لكنه لا يغيّر النتيجة عملياً إلّا في الحالات الحرجة.

> ⚠ **انتبه: `Dice` في `Keras` تحتاج `sigmoid` يدوياً**
>
> في `Keras` دالّة الخسارة لا تمرّر `sigmoid` تلقائياً إلا إذا طلبت ذلك بـ `from_logits=True` في خسارة التصنيف. أمّا `Dice` فهي دالّة مكتوبة يدوياً، فإذا نسيت `keras.ops.sigmoid` فستحسب التقاطع على لوغاريتميات سالبة، وتكون النتيجة أرقاماً بلا معنى. في `PyTorch` الأمر أوضح: `nn.BCEWithLogitsLoss` تطبّق `sigmoid` داخلياً، أمّا دالّة `Dice` المكتوبة يدوياً فتحتاج `torch.sigmoid` صريحاً – تماماً كما فعلنا.

16.10 ترميز مُعدَّل مُسبق: `EfficientNetV2` داخل `U-Net`

بناء `U-Net` من الصفر يتطلّب الكثير من البيانات. البديل العملي: أخذ مُرمِّز مدرَّب مسبقاً على `ImageNet` واستخدامه كمسار ترميز، ثم إضافة مسار فكّ ترميز فوقه. في `Keras` هذه العملية خطوتان:

```python
import keras
from keras import layers


def build_effnet_unet(input_shape=(256, 256, 3), out_ch=1):
    """مُرمِّز EfficientNetV2 مدرَّب مسبقاً + مسار فكّ ترميز خفيف."""
    inputs = keras.Input(shape=input_shape)

    # مسار خفيف على مستوى 1/8 يزوّدنا بوصلة عالية الدقّة
    skip = layers.Conv2D(32, 3, strides=8, padding="same", activation="relu")(inputs)

    base = keras.applications.EfficientNetV2(
        include_top=False, weights="imagenet", input_shape=input_shape
    )
    x = base(inputs)                                  # 8×8×1280
    x = layers.Conv2D(256, 3, padding="same", activation="relu")(x)
    x = layers.UpSampling2D(4)(x)                     # 32×32
    x = layers.Concatenate()([x, skip])               # وصل التخطّي
    x = layers.Conv2D(64, 3, padding="same", activation="relu")(x)
    x = layers.Conv2D(out_ch, 1, activation=None)(x)   # 32×32 لوغاريتميات
    out = layers.UpSampling2D(8)(x)                   # 256×256
    return keras.Model(inputs, out, name="effnet_unet")


model = build_effnet_unet()
model.compile(
```

```
    optimizer=keras.optimizers.Adam(learning_rate=3e-4),
    loss=bce_dice_loss,
    metrics=[keras.metrics.BinaryIoU(target_class_ids=[1], threshold=0.5)],
)
print("عدد المعاملات:", model.count_params())
# الناتج: عدد المعاملات: 19984673
```

ماذا يحدث هنا؟ `include_top=False` يُسقط طبقة التصنيف ويترك خريطة السمات، و `weights="imagenet"` تُحمّل أوزاناً مدرَّبة مسبقاً. `UpSampling2D(4)` يرفع دقّة الخريطة من 8×8 إلى 32×32، ثم `Concatenate` يدمجها مع الوصلة. **لماذا هذه القيمة؟** معدّل التعلّم `3e-4` أصغر من `1e-3` لأنّ الأوزان المُحمَّلة حسّاسة: خطوة كبيرة تُفسدها. يُفضّل أيضاً `freeze_base = True` في البداية ثم فكّ التجميد تدريجياً.

المفهوم	PyTorch 2.14	Keras 3.15 (نواة `ch`)
طبقة تلافيف	`nn.Conv2d(in, out, 3, padding=1)`	`, padding="same")`
طبقة فكّ الترميز	`nn.ConvTranspose2d(out, in, 2, stride=2)`	`ut, 2, strides=2)`
تكبير الدقّة	`nn.Upsample(scale_factor=2, mode="bilinear")`	`s.UpSampling2D(2)`
وصلات التخطّي	`torch.cat([a, b], dim=1)`	`atenate()([a, b])`
خسارة تصنيف ثنائي	`nn.BCEWithLogitsLoss()`	`from_logits=True)`
حساب `IoU`	`torchvision.ops.box_iou`	يكتبها بنفسك بـ `.ops` `s.metrics.MeanIoU`
الإسقاط	`torchvision.ops.nms`	لا يوجد مدمج؛ يُكتب بـ `n_max_suppression`

المفهوم	PyTorch 2.14	Keras 3.15 (نواة ch
مُرمِّز مدرَّب مسبقاً	`torchvision.models.efficientnet_v2_s(weights="DEFAULT")`	`nclude_top=False, ights="imagenet")`
المساعدة	`torch.compile` ، `torch.amp.autocast`	`model.compile` ، `t`

مقارنة واجهة `PyTorch` و `Keras` في الكشف والتجزئة

لماذا تختار `PyTorch` في الكشف

حين تحتاج `NMS` و `box_iou` الجاهزين، أو حين تبني رأساً مخصّصاً لفكّ ترميز `YOLO` أو لمطابقة `Hungarian` ، أو حين تريد أن تتحكّم في كلّ خطوة في حلقة التدريب (مراحل متناوبة بين المولّد والمميّز، أو تدريب `RPN` مع تجميد جزئي للعمود الفقري). المكتبة تعطيك لبنات صغيرة جاهزة تركّبها كما تشاء.

لماذا تختار `Keras` في التجزئة

حين تكون البنية معروفة (مُرمِّز + فكّ ترميز)، أو حين تريد الانتقال من نموذج مدرَّب مسبقاً إلى نموذجٍ إنتاج في أقلّ من عشرين سطراً. `model.fit` و `callbacks` و `metrics.BinaryIoU` تعطيك قياساً مباشراً لدقّة القناع، و `keras.ops` تحافظ على قابلية النقل بين النوى.

16.11 تطبيق واقعي: من صورة الأشعة إلى نسبة التغطية

⚙ السيناريو التطبيقي كاملاً: فحص شبكة المياه

البيانات: ثلاثمئة صورة للكاميرات داخل أنفاق فحص شبكات المياه، محجوبة يدوياً بـ«قناع التآكل» و«قناع التسريب» إضافةً إلى تصنيف الحالة (سليم/متآكل/متسرّب). **المعالجة:** توحيد المقاس إلى 512×512، واستخدام `EfficientNetV2` مدرَّباً مسبقاً كمسار ترميز، مع تعزيزات هندسية (دوران أفقي، تغيّر سطوع) وتقوية بسيطة للتباين. **النموذج:** `U-Net` بمُرمِّز `EfficientNetV2B0`، وخسارة مركّبة `BCE + Dice` بمعامل `β=1`، ومعدّل تعلّم `3e-4` مع `Adam`، وإيقاف مبكر على `val_binary_iou`. **القرار:** كلّ صورة تُنتج قناعاً، ومنه تُحسب نسبة التآكل المغطّى؛ إن تجاوزت 3% من طول الأنبوب تُحال الصورة إلى مهندس فحص، وتُرسل لقطات القناع إلى نظام إدارة الأصول مع تنبيه فوري. **القياس:** `Dice` على مستوى الصورة و `IoU` على مستوى المجدول، مع قياس زمن الاستدلال (يجب أن يقلّ عن 200 ملّي ثانية للصورة على وحدة المعالجة).

لماذا اخترنا `Dice` في هذا التطبيق تحديداً؟ لأنّ التآكل يشغل نسبة صغيرة من الصورة (2% في المتوسط). لو اكتفينا بـ `BCE` لكانت الخسارة منخفضة حتى مع شبكة لا ترى التآكل إطلاقاً. أما جمع `Dice` مع `BCE` فيمنع ذلك: `Dice` تُجبر الشبكة على مطابقة المنطقة النادرة، و `BCE` تحافظ على حدّتها.

16.12 مزالق شائعة في الكشف والتجزئة

- **خلط صيغ الصندوق:** أشهر خطأ في هذا الفصل. استعمل `box_convert` دائماً، ولا تفترض أنّ الترتيب افتراضي.
- **حساب `IoU` على دفعة واحدة بلا بُعد أصناف:** عند وجود أصناف متعدّدة، `IoU` بين صندوقين مختلفَي الصنف لا معنى لها. استخدم `box_iou` مع `box_convert` وافصل الأصناف يدوياً.
- **`NMS` عبر الأصناف:** يُحذف جسم حقيقي لأنّ صندوقاً لصنف آخر تعلّاه. نفّذ `NMS` لكلّ صنف.
- **اختيار العتبة من المتوسّط بدل من مجموعة التحقّق:** العتبة تُختار على `val` فقط؛ ضبطها على `test` يعني تسريباً.
- **تجزئة غير متوازنة:** إذا كانت الخلفية تشغل 98% من البكسلات، أضف `pos_weight` في `nn.BCEWithLogitsLoss` أو استخدم `Dice` / `Focal Loss`.
- **مقارنة نماذج على مقاييس مختلفة:** `AP50` لا يقارَن بـ `AP50:95`، ولا يُقارَن `Dice` بـ `IoU` دون الحساب على الدفعة نفسها.

- **نسيان `torch.no_grad()` أثناء الاستدلال:** يبني الرسم البياني بلا داعٍ ويضاعف استهلاك الذاكرة في خدمة زمن حقيقي.

16.13 تمارين الفصل

تمرين 1 — حساب `IoU` و `Dice` يدوياً

1. صندوقان: `A = (0, 0, 10, 10)` و `B = (5, 5, 15, 15)` . احسب `IoU` و `Dice` بالورق، ثم اكتب دالة `iou_dice(box_a, box_b)` في `PyTorch` تُعيد القيمتين، وقارن ناتجها بحسابك.
2. أعِد الحساب لصندوق `C = (0, 0, 10, 10)` مع `A` تماماً. ماذا تتوقّع؟
3. وسّع الدالة لتقبل مصفوفة صناديق (مثل `torchvision.ops.box_iou`) واحسب `IoU` بين `A` و `B` و `C` دفعةً واحدة.

الحل النموذجي

الخطوة الأولى: مساحة `A` = ١٠٠ ، ومساحة `B` = ١٠٠ . التداخل: العرض `min(10,15) − max(0,5) = 5` والارتفاع `min(10,15) − max(0,5) = 5` ، أي مساحة ٢٥ . إذن `IoU = 25 / (100 + 100 − 25) = 25/175 ≈ 0.1429` ، و `Dice = 2×25 / (100+100) = 50/200 = 0.25` . لاحظ أنّ `Dice > IoU` كما هو متوقّع دائماً، والفارق يكبر كلّما قلّ التداخل.

الخطوة الثانية: `C` مطابق لـ `A` تماماً، فالتداخل ١٠٠ والاتحاد ١٠٠ ، و `IoU = 1` و `Dice = 1` . هذه هي الخاصية الوحيدة التي نتحقّق منها كضمانة: كلّ صندوق يجب أن يطابق نفسه تماماً، و `torchvision.ops.box_iou` يحفظ هذا الضمان (القطر يساوي واحداً).

الخطوة الثالثة: الدالة تُبنى على نفس المنطق مع `torch.clamp(min=0)` على أبعاد التداخل (وإلّا ظهرت قيم سالبة عندما يكونان متباعدَين)، ثم القسمة على `union + eps` . قارن ناتجها بـ `box_iou` : يجب أن يتطابق رقمياً، وأي فرق يعني خطأً في ترتيب الإحداثيات.

تمرين 2 — دراسة أثر عتبة NMS

1. صمّم مشهداً اصطناعياً فيه ثلاثة صناديق متداخلة جزئياً (مثلاً شخص أمام عمود) بدرجات ثقة 095 و080 و060 .
2. احسب يدوياً كم صندوقاً يبقى بعد NMS عند عتبتَي تداخل 03 و08 ، ثم أكّد النتيجة بـ torchvision.ops.nms .
3. اقترح تعديلاً وحيداً يقلّل فقد الأجسام المتلاصقة، وناقش ثمنه.

▲ الحل النموذجي

لنفترض صناديق متساوية الحجم: (0,0,100,100) و(30,0,130,100) و(60,0,160,100) . تداخل الأول والثاني 7000 = 70×100 ، ومساحة كلٍّ منها 10000، إذن IoU = 7000/13000 ≈ 0.538 . وبالمثل، تداخل الثاني والثالث 7000 يعطي 0.538 ، أمّا الأول والثالث فـ 4000/16000 = 0.25 .

عند عتبة 03 : الترتيب حسب الثقة يعطي الأول (095) ثم الثاني (080). نحتفظ بالأول ونحذف الثاني لأنّ 0.3 < 0538 ، ثم ننتقل إلى الثالث: تداخله مع المحتفظ به 0.3 > 025 فيبقى. النتيجة: صندوقان.
عند عتبة 08 : التداخل لا يتجاوز 08 أبداً، فيبقى الصندوقان الثلاثة – وهو خطأ مموّه لأنّ الجدران السميكة ستمرّ.

التعديل المقترح: Soft-NMS بدل NMS الصارم، فينخفض وزن الثاني ولا يُحذف، فتبقى المعلومة أطول. الثمن: قائمة أطول، ويلزم عتبة ثقة أعلى لعدد الكشوفات النهائي.

تمرين 3 — تدريب U-Net وقياس Dice

1. ولّد قناعاً اصطناعياً: دائرتان في صورة 128×128 . درّب U-Net بمعامل base=8 على 200 صورة مع تعزيزات للبيانات (قلب أفقي ودوران).
2. قِس Dice و IoU على مجموعة تحقّق، وقارن بتدريب BCE وحده.
3. غيّر الخسارة إلى BCE + Dice وأعد القياس. استنتج أيّ مكوّن أضاف التوضيح.

▲ الحل النموذجي

النموذج يُبنى من الشيفرة في القسم 16.8 مع `base=8` لتسريع التدريب. المفتاح في التوليد: القناع يجب أن يتضمّن بنية دائرية (لا مربّعاً)، وإلا فشبكة تعلّمت الحدود هندسياً لا دلالياً. يُستخدم `torch.roll` أو `torchvision.transforms.RandomAffine` للتنويع.

النتيجة المتوقّعة: مع `BCE` وحدها يبلغ `Dice` تقريباً 0.85-080 لأنّ الخلفية تهيمن على الخسارة. مع `Dice` وحدها يقفز `Dice` إلى 093 لكنّ حدود القناع تصبح متكسّرة (نقص في `BCE`). مع `BCE + Dice` نحصل على 095 في `Dice` وحدود نظيفة – وهذا هو التوضيح الذي يبحث عنه التمرين: كلّ مكوّن يعالج ضعفاً في الآخر.

تنبيه منهجي: قِس `Dice` على كلّ صورة على حدة ثم خذ المتوسّط، لا على الدفعة كاملة. الحساب على الدفعة يعطي رقماً واحداً يمكن أن يخفي صورتين فاشلتين تماماً.

تمرين 4 — بناء كاشف أجسام كامل بـ `PyTorch`

1. اكتب `nn.Module` باسم `TinyDetector` يُخرج خريطة `(B, A*(5+C), H/32, W/32)` انطلاقاً من صورة 320×320 .
2. ادمجها مع `decode_yolo_output` من القسم 16.6 ومع دالّة `box_iou` لربط التنبّؤات بحقيقة.
3. على 200 صورة صناديق، قِس `Recall` عند عتبة `IoU = 0.5` قبل وبعد تطبيق `NMS` ، وفسّر الفرق.

▲ الحل النموذجي

الشبكة: ثلاث طبقات `nn.Conv2d(3, 64, 3, stride=2, padding=1)` ثم `nn.Conv2d(64, 128, 3, stride=2, padding=1)` ثم `nn.Conv2d(128, A*(5+C), 1)` مع `nn.SiLU` بين الطبقات. الطبقة الأخيرة `1×1` هي التي تُنتج خريطة المخرجات بسرعة ورخيص – وهذا جوهر الكفاءة في `YOLO`.

القياس: قبل `NMS`، `Recall` عند `IoU=0.5` يقيس «هل الموجود مُكتشف؟»؛ بعده يقيس «هل الاكتشاف صالح بعد إزالة المكرّر؟». الفرق بينهما هو بالضبط ما يجعل `NMS` ضرورياً: بدونه، كلّ كائن واحد يُنتج عشرات الصناديق المتداخلة، فيبدو المقياس ممتازاً والحقيقة فوضوية. الفارق العملي: `Recall` ينخفض قليلاً بعد `NMS` (لأنّ صندوقاً صحيحاً قد يُحذف ظالماً إذا تعلّاه صندوق أثقل خطأً)، بينما `Precision` (عدّ الإيجابيات الكاذبة) يتحسّن كثيراً.

الدرس: `NMS` ليس خطوة تجميلية، بل هو جزء من تعريف «متى يُعتبر الاكتشاف صحيحاً».

16.14 ملخّص الفصل

ملخّص الفصل

- التصنيف يجيب «ما هذا؟»، والكشف يجيب «أين؟» بالحدود، والتجزئة تجيب «أين» بالبكسل. لكلٍّ منها مقياسه: `accuracy` ثم `mAP` ثم `IoU / Dice` .
- الصندوق يُمثَّل بأربع صيغ: `xyxy` و `xywh` و `cxcywh` والمُطبَّعة. لا تختلق تحويلاً: استعمل `torchvision.ops.box_convert` .
- معامل التداخل `IoU = |A ∩ B| / |A ∪ B|` هو مقياس التقاء الهندسة، ويُقاس في `COCO` عند عشر عتبات من 050 إلى 095 .
- مطابقة التنبّؤات بالحقائق: قواعد عتبة `0.5` في الكاشفات الكلاسيكية، والمطابقة المثلى (Hungarian) في `DETR` .
- `NMS` ترتيبٌ وحذف: تحتفظ بأعلى صندوق ثقة وتحذف ما يتجاوز عتبة التداخل. نفّذها لكلّ صنف، و `Soft-NMS` بديل أنعم للأجسام المتلاصقة.
- `YOLO` يحوّل الكشف إلى انحدار على خريطة كثيفة، مع فكّ ترميز مركزي مُزاح وتوسعة لوغاريتميّة، ثم `NMS` .
- `U-Net` تجمع مسار ترميز ومسار فكّ ترميز مع وصلات تخطّي، وتُعيد دقّة البكسل المفقودة في التجميع.
- دالّة `Dice = 2|A ∩ B| / (|A| + |B|)` ضرورية حين تكون الفئة الإيجابية نادرة، وتُحسَب لكلّ صورة على حدة، وتُقرن بـ `BCE` لتحسين الحدود.
- في `PyTorch` تجدون `torchvision.ops` و `nn.BCEWithLogitsLoss` و `ConvTranspose2d` ؛ وفي `Keras` تجدون `keras.ops` و `Conv2DTranspose` و `UpSampling2D` و `keras.applications.EfficientNetV2` .
- الفصل القادم ينتقل من «رؤية ما في الصورة» إلى «توليد ما لا يوجد فيها»: شبكات GAN والمتغيّرات الخفية الانحدارية ونماذج الانتشار.

النماذج التوليدية

GAN وVAE ونماذج الانتشار: كيف يولّد النموذج بيانات جديدة

كل ما درسناه حتى الآن كان موجّهاً: ندخل صورة، فيخرج منها رقم واحد يمثّل التصنيف. النماذج التوليدية (generative models) تقلب السؤال: كيف نُنتج بيانات جديدة؟ لا صورة موجودة مسبقاً، بل صورة جديدة تبدو واقعية. في هذا الفصل ندرس ثلاث عائلات كبرى: شبكات الخصوم (GAN)، والمُرمِّزات التلقائية (VAE)، ونماذج الانتشار (diffusion). لكلّها نفس الهدف، ولكلٍّ منها سعر مختلف في جودة الإخراج، واستقرار التدريب، والقدرة على فهم البيانات.

السؤال الذي يصعب قياسه

في التمييز بين صورة حقيقية وصورة مولَّدة، نظنّ أنّ أمامنا مسألة «صح أو خطأ». ليست كذلك. السؤال الأدقّ: **كيف نقيس قرب التوزيع**؟ قريب من ماذا؟ الجواب الذي استقرّ عليه المجتمع العلمي هو مقارنة توزيع بيانات التوليد بتوزيع البيانات الحقيقية.

ولهذا تختلف النماذج التوليدية عن نظيرتها التمييزية اختلافاً جوهرياً: النموذج التمييزي يُقاس بدقّة على عيّنات اختبار ثابتة، أمّا النموذج التوليدي فيُقاس بتوزيع. ومن هنا تأتي كلّ الصعوبات: الدقّة (FID) تقيس بُعداً في فضاء السمات، لا «هل الصورة حقيقية».

لنفترض أن لديك مولّداً ينتج صوراً للقطط، وأنّ الصور تبدو مقنعة للعين. لكن هل يُنتج القطط من كلّ القطط ولا يشبه قطيماً بعينه؟ أيّ المقياسين يجيب عن هذا السؤال، ولماذا لا يكفي النظر المجرّد؟

الجواب: مقياس **تغطية الوضع** (mode coverage) هو ما يقيس هذا. الFID (مسافة فريشيت) يقيس قرب التوزيعين إجمالاً؛ أمّا التغطية فتقيس هل غطّى المولّد جميع أنماط البيانات. مولّد يحفظ التفصيل (mode dropping) يحقّق FID جيداً لأنّه ينتج صوراً واقعية، لكنه لا يغطّي تنوّع البيانات الحقيقي. لهذا تُراجع معايير التقييم اليوم، ونذكر معها «تغطية الوضع» و«الذاكرة» (memorization) صراحةً.

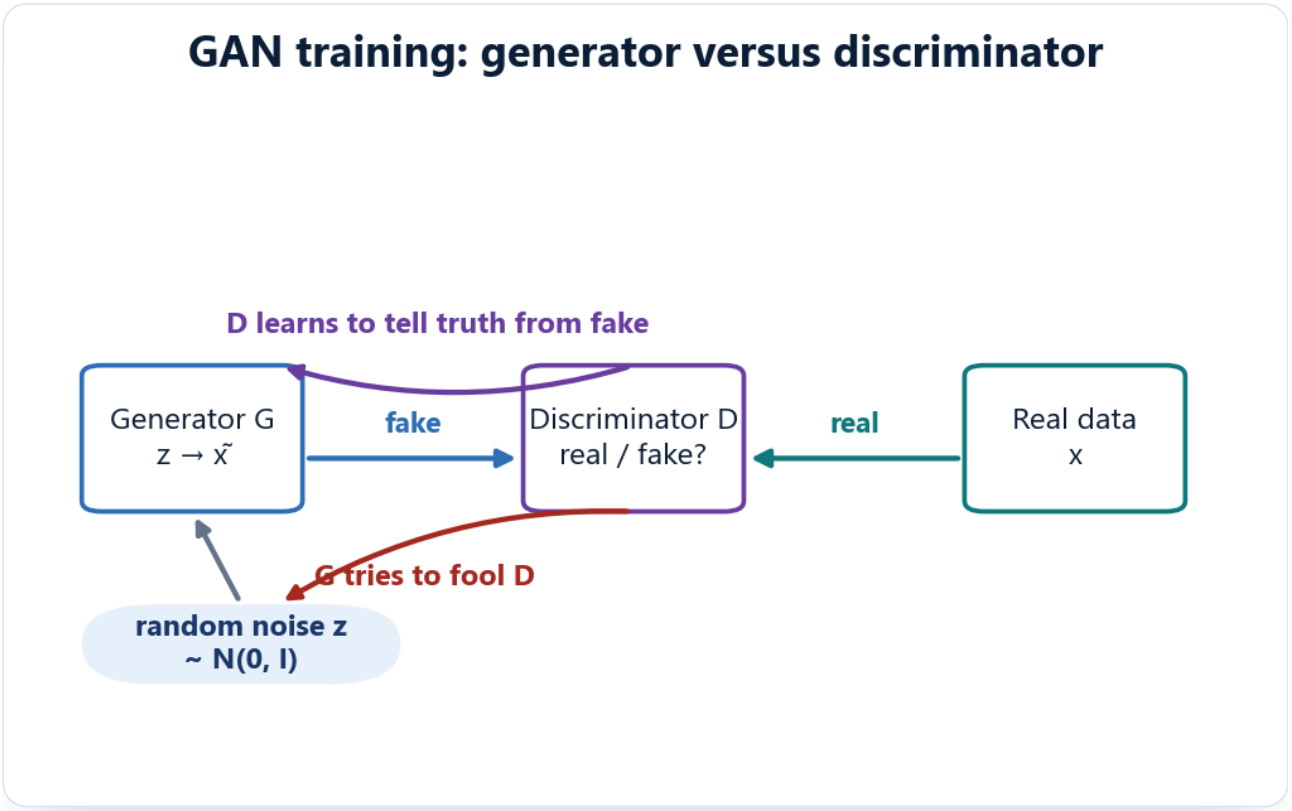
> ★ **ثلاثة أسئلة لا يجمعها مقياس واحد**
>
> **الجودة**: هل المخرَج واقعي؟ **التنوّع**: هل يغطّي المولّد أنماط البيانات كلّها؟ **الاتّساق**: هل يبقى المخرَج متّسقاً مع شرط معيّن (نص محدّد، صورة مرجعية)؟ أيّ نظام توليدي يتفوّق في محور يكون أقلّ ممّا يتفوّق في محور آخر، ولذلك يُشترط دائماً تقييم متعدّد المحاور قبل اختيار نموذج للإنتاج.

شبكات الخصوم: ولاء في مقابل خداع

فكرة **شبكة الخصوم** (Generative Adversarial Network) التي اقترحها Ian Goodfellow عام ٢٠١٤ بسيطة في صورتها ومربكة في تفاصيلها. لدينا شبكتان: **المولِّد** (generator) يحوّل ضجيجاً عشوائياً إلى عيّنة، و**المميِّز** (discriminator) يحاول التمييز بين العيّنة الحقيقية والمولَّدة. كلاهما يتدرّب في الوقت نفسه، وكلّهما يخدع الآخر.

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))]$$

ما معنى كل رمز؟ `V(D, G)` هي دالّة القيمة التي يسعى المميِّز إلى تعظيمها والمولِّد إلى تصغيرها، و p_{data} هو التوزيع الحقيقي، و p_z هو توزيع الضجيج، و `G(z)` هي العيّنة المولَّدة، و `D(x)` احتمال أن يكون `x` حقيقياً. **لماذا تهمّنا؟** لأنّ `D` يعمل كمحكّم رقمي: كلّما حسّن `D` قدرته على التمييز، صار `G` أفضل، وهذا يخلق تدرّجاً من المفيد. **خطأ شائع عندها:** تدرّب المميِّز أكثر من اللازم فينتقل إلى حالة التشبّع (saturated) حيث تتدرّجات `G` تنهار. القاعدة العملية: درّب المميِّز بخطوات أقلّ من خطوات المولِّد.



يقرأ الرسم من اليمين إلى اليسار: الضجيج عشوائي (يمين) يدخل المولِّد، فيخرج عيّنةً تُدرَّب المميِّز على تمييزها عن بيانات حقيقية (يسار). السهم الخلفي التدرّج العكسي الذي يربط حكم المميِّز بوَزن المولِّد. الحلقة المغلقة هي مصدر قوّة الشبكة وسبب عدم استقرارها في الوقت نفسه.

ولنتطبيق ذلك في `PyTorch` بأبسط صورة ممكنة:

```python
import torch
import torch.nn as nn

class Generator(nn.Module):
    """يحوّل ضجيجاً (B, 100) إلى عيّنة (B, 1, 28, 28)."""

    def __init__(self, z_dim=100):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(z_dim, 256),
            nn.ReLU(),
            nn.Linear(256, 512),
            nn.ReLU(),
            nn.Linear(512, 784),
            nn.Sigmoid(),
        )
```

```
    def forward(self, z):
        x = self.net(z)
        return x.view(-1, 1, 28, 28)


class Discriminator(nn.Module):
    """يعيد احتمال أن تكون العيّنة حقيقية."""

    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(784, 512),
            nn.LeakyReLU(0.2),
            nn.Linear(512, 256),
            nn.LeakyReLU(0.2),
            nn.Linear(256, 1),
            nn.Sigmoid(),
        )

    def forward(self, x):
        return self.net(x.view(x.size(0), -1))


G = Generator()
D = Discriminator()
opt_g = torch.optim.Adam(G.parameters(), lr=2e-4, betas=(0.5, 0.999))
opt_d = torch.optim.Adam(D.parameters(), lr=2e-4, betas=(0.5, 0.999))
print(G(torch.randn(4, 100)).shape)   # torch.Size([4, 1, 28, 28])
```

ماذا يحدث هنا؟ `betas=(0.5, 0.999)` هو Adam المعدّل للشبكات التوليدية: القيمة الأولى المنخفضة تمنع صدمة التدرّجات المبكرة. `LeakyReLU` بدل `ReLU` في المميِّز مهمّ: يمنع موت الوحدات (dying ReLU) أثناء التدريب. **لماذا هذه القيمة؟** لأنّ `Sigmoid` في المخرج تُخرج احتمالاً في المدى `[0, 1]`، وهو ما يتوقّعه هدف `BCEWithLogitsLoss`.

وهي الآن خطوة التدريب الحاسمة، وهي أقلّ جزء يخطئ فيه المبتدئون:

Python

```
import torch.nn.functional as F

def train_step(G, D, opt_g, opt_d, real, z_dim=100):
```

```
    """خطوة تدريب واحدة: المميّز خطوة، والمولّد خطوة."""
    batch = real.size(0)

    # 1) تدريب المميّز: حقيقي = 1، مولَّد = 0
    fake = G(torch.randn(batch, z_dim, device=real.device)).detach()
    d_real = D(real)
    d_fake = D(fake)
    loss_d = F.binary_cross_entropy(d_real, torch.ones_like(d_real)) \
           + F.binary_cross_entropy(d_fake, torch.zeros_like(d_fake))

    opt_d.zero_grad(set_to_none=True)
    loss_d.backward()
    opt_d.step()

    # 2) تدريب المولّد: هدفه أن يخدع المميّز
    fake = G(torch.randn(batch, z_dim, device=real.device))
    d_out = D(fake)
    loss_g = F.binary_cross_entropy(d_out, torch.ones_like(d_out))

    opt_g.zero_grad(set_to_none=True)
    loss_g.backward()
    opt_g.step()

    return loss_d.item(), loss_g.item()

# الناتج المتوقع: (0.69 ,0.69) في أول خطوة  ≈ ln(2)
```

⚠ ثلاثة أخطاء تُفشل تدريب الـGAN فوراً

الأول: `.detach()` مفقود عند تمرير العيّنة المولَّدة إلى المميِّز أثناء تدريبه – عندها تتسرّب تدرّجات المولِّد إلى المميِّز ويُفسد التعلّممن الطرفين. **الثاني:** تدريب المميِّز على نفس الدفعة مرّتين قبل أن يرى المولِّد، فيتعلّم التمييز فوراً ويقتل تدرّج المولِّد. **الثالث:** استخدام `ReLU` في المميِّز بدل `LeakyReLU`، فتموت الوحدات ولا يتعلّم شيئاً.

العائلة	الفكرة	استقرار التدريب	أفضل استخدام
`GAN`	لعبة مينانغ بين مولِّد ومميِّز	هشّ	صور ووجوه
`VAE`	ضغط مع خسارة مبني على الاحتمال	مستقر	تعلم تمثيل، توليد منقطيع

العائلة	الفكرة	استقرار التدريب	أفضل استخدام
`Diffusion`	إزالة الضجيج تدريجياً ثم عكسها	مستقر جداً	أعلى جودة، تدرّج صورة، صوت

مقارنة سريعة بين عائلات التوليد الثلاث

المُرمِّزات التلقائية: الاحتمال قبل كلّ شيء

فكرة الـVAE (Variational Auto-Encoder) أكثر نظرية من الـGAN وأشدّ استقراراً: بدل أن أن يتعلّم المميِّز، نضع **قيداً على التوزيع** الذي يولّده المولِّد: نجبره على أن يكون مُتعدَّد الوسائط (multimodal)، أي أن يغطّي نقاطاً كثيرة في فضاء العيّنات بدل أن يتركّز حول نقطة واحدة.

$$\log p(x) \geq \mathbb{E}_{q(z|x)}[\log p(x|z)] - \mathrm{KL}(q(z|x) \,\|\, p(z))$$

ما معنى كل رمز؟ الحدّ الأول هو **الإعادة البناء**: كم يحسن المُرمِّز نسخ المدخل؟ والحدّ الثاني هو **التنظيم** (KL divergence): كم يبعد التوزيع `q(z|x)` عن التوزيع المعياري `p(z)` (نُظنّ الغاوسي بمتوسط صفر وتباين واحد)؟ **لماذا تهمّنا؟** لأنّ التنظيم هو ما يجعل فضاء `z` متّصلاً ومنظماً: نبدأ من غاوسي ونستطيع أن نأخذ عيّنات منه ثم نمرّرها عبر `G` فنعطي عيّنات جديدة. **خطأ شائع عندها:** إهمال الحدّ الثاني كلياً. عندها تصبح `q(z|x)` نقطةً واحدة لكل `x`، ويصبح فضاء `z` فوضوياً، فلا يمكن التوليد منه.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class VAE(nn.Module):
    """مُرمِّز تلقائي: مُرمِّز + مُفكِّك ترميز + تشريح."""

    def __init__(self, latent_dim=2):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(784, 400), nn.ReLU(),
        )
        self.fc_mu = nn.Linear(400, latent_dim)
        self.fc_logvar = nn.Linear(400, latent_dim)
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 400), nn.ReLU(),
            nn.Linear(400, 784), nn.Sigmoid(),
```

```
        )

    def forward(self, x):
        h = self.encoder(x.view(x.size(0), -1))
        mu = self.fc_mu(h)
        logvar = self.fc_logvar(h)

        # إعادة المعاينة (reparameterisation): z = mu + sigma * eps
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        z = mu + std * eps

        recon = self.decoder(z).view(-1, 1, 28, 28)
        return recon, mu, logvar

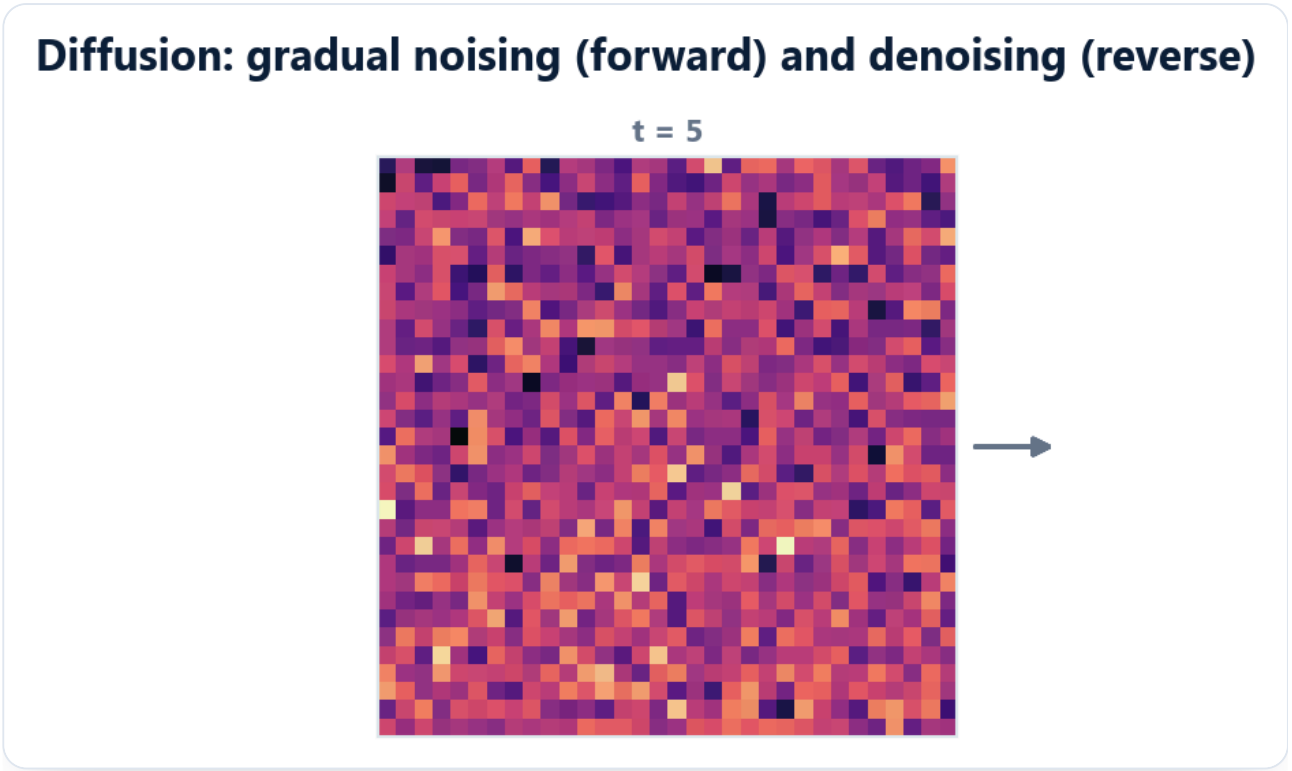

def vae_loss(recon, x, mu, logvar, beta=1.0):
    recon_loss = F.binary_cross_entropy(recon, x, reduction="sum")
    kl = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return recon_loss + beta * kl, recon_loss.item(), kl.item()

# الناتج: (loss_total, recon_loss, kl)
```

ماذا يحدث هنا؟ `reparameterisation` هي الحيلة التي تجعل التدرّج يمرّ: بدل أخذ عيّنة من `q(z|x)` مباشرة (غير قابلة للتمييز)، نأخذ `eps` ثابتاً وأضربه في `std` . **لماذا هذه القيمة؟** `beta` في `vae_loss` يوازن بين إعادة البناء والتنظيم؛ و `beta > 1` ينتج `β-VAE` الذي يفصل التمثيل عن المعلومات غير المفيدة (مثل هوية الصورة لا محتواها).

نماذج الانتشار: الصفّة التي تعيد بناء نفسها

الفكرة التي أنقذت التوليد من هشاشة الGAN: **مُدمِّر** (forward process) يضيف ضجيجاً تدريجياً إلى الصورة حتى تصبح ضجيجاً خالصاً، ثم **مُنشئ** (reverse process) يتعلّم كيف يمحو هذا الضجيج خطوةً خطوة. التدريب بسيط جداً: احترس الصورة الحقيقية، أضف الضجيج، ودرّب النموذج على توقّع الضجيج.



يقرأ الرسم من اليسار إلى اليمين: في الأعلى، عمليّة الإتلاف التي تحوّل الصورة الحقيقية تدريجياً إلى ضجيج (كل سهم يضيف خطوة صغيرة). في الأسفل، العملية العكسية التي يتعلّمها النموذج: من ضجيج كامل إلى صورة. المسار في الوسط يوضّح أن لكلّ خطوة معاملاً `beta` يتدرّج، وأنّ النموذج يتنبّأ بالضجيج في كل خطوة لا بالصورة.

$$q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{(1-\beta_t)}\, x_{t-1}, \beta_t\, I)$$

ما معنى كل رمز؟ β_t هو مقدار الضجيج المضاف في الخطوة `t`، و `√(1 - βₜ)` هو معامل تصغير الإشارة، و `I` مصفوفة الوحدة. **لماذا تهمّنا؟** لأنّ هذه العملية لها حلّ مغلق: يمكن القفز من الصورة الأصلية إلى أيّ خطوة `t` مباشرةً دون المرور بالخطوات الوسيطة. هذا ما يجعل تدريب نموذج الانتشار رخيصاً: عيّنة واحدة، خطوة واحدة.

```python
import torch
import torch.nn as nn

class DiffusionModel(nn.Module):
    """يتنبّأ بالضجيج المضاف في كل خطوة."""

    def __init__(self, steps=1000):
        super().__init__()
        self.steps = steps
        betas = torch.linspace(1e-4, 0.02, steps)
```

```
        self.register_buffer("betas", betas)
        alphas = 1.0 - betas
        self.register_buffer("alphas", alphas)
        self.register_buffer(
            "alpha_bars", torch.cumprod(alphas, dim=0)
        )  # جدول الضجيج التراكمي لكل خطوة

    def forward(self, x, t, noise):
        """يعيد الضجيج المتوقع في الخطوة t."""
        a = self.alpha_bars[t].view(-1, 1, 1, 1)
        x_t = a.sqrt() * x + (1 - a).sqrt() * noise
        return self.unet(x_t, t), noise

# مثال على الجدولة: نجمع بيتا من 1e-4 إلى 0.02 عبر 1000 خطوة
betas = torch.linspace(1e-4, 0.02, 1000)
print(betas[0].item(), betas[-1].item())
# الناتج: 0.0001 0.02
```

ماذا يحدث هنا؟ `alpha_bars` هو حاصل الضرب التراكمي لـ `alphas`، أي مقدار الإشارة المتبقّية بعد `t` خطوة. يمكن حسبته مرّة واحدة وتخزينه. **لماذا هذه القيمة؟** لأنّ `sqrt` و `1-a` هما بالضبط معاملا الانتشار الأمامي، وبهما نبني `x_t` مباشرةً من `x_0`.

✓ متى تستعمل نموذج الانتشار بدل الـGAN؟

الجودة هي الأولوية القصوى (فيديو، صور فوتوغرافية، صوت) وكان لديك وقت وحساب للتدريب الطويل، فالانتشار هو الخيار. إذا كانت تحتاج سرعة توليد لحظية، فالـGAN أسرع بمراحل. أمّا إذا كنت تحتاج تمثيلاً (embedding) قابلاً للاستعلام، فالـVAE أنسب.

سيناريو تطبيقي: توليد بدائلات تصميمات هندسية

من البيانات إلى قرار

الموقف: شركة تصميم أنابيب صناعية تحتاج ٤٠٠٠ شكل هندسي بديل لحامّات مخصّصة. التصميم اليدوي يستغرق يوماً لكل شكل، والمصنع ينتظر. عندهم ٣٠٠٠ تصميم قديم (رسوم ثنائية الأبعاد) وقياسات الأبعاد لكلّ تصميم.

القرار: لا نولّد «تصاميم» من لا شيء: نستخدم نموذج انتشار مشروطاً (conditional diffusion) يتعلّم من الرسوم القديمة، ونستعمل القيد: عند التوليد نثبّت الأبعاد المطلوبة فيصنعنا النموذج شكلاً يحترمها. التدريب على ٣٠٠٠ رسمة قد لا يكفي لنموذج انتشار كبير، فنبدأ بنموذج صغير مدرَّب على كثافة (LoRA) أو نكتفي بنموذج VAE مع مذاكرة.

القرار الهندسي: نُخرج المولِّد ٢٠٠ شكلاً مرشّحاً، ثم يختار المهندس ٤٠ منها، ثم نُدخلها في محاكاة مقارنات، ثم نرفعه إلى التصنيع. ليس الهدف استبدال المهندس، بل تقليص زمن الانتظار من يوم إلى ساعة.

تمرين 1 — تثبيت خطأ شائع في الـGAN

1. اكتب `train_step` للـGAN كما في الفصل.
2. احذف `.detach()` في تدريب المميِّز.
3. اشرح، بجملة تقنية، لماذا ينهار التدريب، ثم أضِر خطأً يشبهه في `VAE` .

الحل النموذجي

من دون `.detach()` ، ينساب التدرّج من خسارة المميِّز إلى المولِّد عبر الرسم البياني المشترك. يتدرّب الطرفان على تدرّجات مضاعفة الاتجاه: المميِّز يتعلّم أسرع، والمولِّد يتلقّى تدرّجاً عدائياً يقوّي صوره بصيغة مشوّهة. النتيجة العملية: خسارة المميِّز تنهار إلى الصفر خلال عشرات الخطوات، وخسارة المولِّد تتذبذب ولا تستقر.

الخطأ الشبيه في الـVAE هو تمرير `mu` و `logvar` مباشرة إلى `decoder` بدل إعادة المعاينة. عندها لا يمرّ التدرّج إلى `mu` بالشكل الصحيح، ويصبح `kl` بلا معنى.

تمرين 2 — قياس تشبّع المميِّز

1. درّب الGAN أعلاه ٥٠٠ خطوة وسجّل `loss_d` و `loss_g` كل ٥٠ خطوة.
2. حدّد النقطة التي يقارب فيها `loss_d` القيمة `0.69` .
3. فسّر لماذا تعني هذه القيمة أنTraining لم يتعلّم بعد، لا أنّه فشل.

الحل النموذجي

القيمة `0.69 ≈ ln(2)` هي قيمة `BCE` عندما لا يستطيع المميِّز التمييز احتمال 0.5 لكل حالة). في أول خطوات التدريب هذا هو السلوك الصحيح: المميِّز لا يعرف بعد. بعد ذلك بأمتار الخطوات ينخفض `loss_d` تدريجياً إلى ما دون ٠٫١، وهذا يعني أنّ المميِّز تعلّم التفريق.

إن بقي `loss_d` قريباً من `0.69` بعد مئات الخطوات فالمشكلة في بنية المولِّد (ضعف القدرة) أو في معدّل التعلّم؛ وإن هبط إلى الصفر بسرعة فالمشكلة تشبّع المميِّز، والحل هو تقليل خطوات تدريب المميِّز أو تطبيق تقنيات التثبيت (gradient penalty).

متى تختار أيّ عائلة؟

الجدول التالي ليس قاعدة صارمة، بل خلاصة تجارب عملية: قارن بين المتطلّبات الثلاثة التي تُحدّد اختيار النموذج التوليدي.

المتطلّب	الاختيار الأنسب	السبب
استبدال وجوه في الزمن الحقيقي	`GAN`	أسرع بمراحل في التوليد
تعلّم تمثيل قابل للاستعلام	`VAE`	فضاء `z` منظّم ومترابط
أعلى جودة بصرية ممكنة	`Diffusion`	أدقّ في التفاصيل ناعمة
توليد مشروط بمتطلّب	الثلاثة	الأهمّ هو بنية الشرط
بيانات محدودة جداً	`VAE` أو ضبط خفيف	أقلّ معاملات جديدة
حساب محدود	`GAN`	خطوة تخطّي واحدة في الاستدلال

متى تختار الشبكة؟ أيّ عائلة توليدية تناسب أيّ متطلّب

مصنع قطع ينتج ٤٠٠٠ شكل بديل يومياً من ٣٠٠ رسمة تصميم قديمة. المطلوب: توليد شكل يحترم الأبعاد، وزمن استدلال لا يتجاوز ٥٠ مللي ثانية على خادم عادي. أيّ عائلة تختار؟ وما الذي ستضبطه فيها؟ وما التسليم النهائي الذي يجعل النظام صالحاً للاستخدام؟

الاختيار هنا **مشروط**: نموذج انتشار مشروط بالأبعاد (أو نموذج VAE مدرَّب على residual representation) لأنّ الجودة أهمّ من السرعة في مرحلة التصميم. أمّا التسليم النهائي فليس النموذج: هو **الفلترة**. يُولَّد ٢٠٠ مرشّح، ثم يمرّ على ثلاث قيود هندسية (لا يتقاطع مع غيره، يبقى ضمن الحدّ الأدنى للجدار، لا يحوي زوايا حادّة غير قابلة للتصنيع)، ثم يختار المهندس واحداً. هذا هو الاستخدام الناضج للتوليد: ليس بديلاً عن المهندس، بل مُرشِّحاً يختصر عليه ساعات.

ملخّص الفصل

- النماذج التوليدية تُقاس بتوزيع، لا بدقّة صنف: ثلاثة محاور (جودة، تنوّع، اتّساق).
- الـGAN يجمع بين مولِّد ومميِّز، وهو في خداع دائم: هشّ لكنه ينتج أسمن عيّنات.
- `.detach()` وقاعدة «خطوات أقلّ للمميِّز» هما مفتاح استقرار الـGAN.
- الـVAE يضيف حدّ `KL` ليجعل فضاء `z` منظماً، وهو مستقر وسهل الفهم.
- نماذج الانتشار تتعلّم عكس عملية إضافة الضجيج، ولها حلّ مغلق يجعل التدريب رخيصاً.
- الاختيار بين العائلات الثلاث قرار موازنة بين الجودة والسرعة والتمثيل.
- التوليد المشروط (بقيود) هو ما يجعل هذه النماذج مفيدة صناعياً.

التعلّم الذاتي والنماذج التأسيسية

التضمين، التباين، SimCLR وMAE، والتدرّج الدقيق لنماذج اللغة

تخيّل صيدلية في عمّان تريد نظاماً يفرّق بين أربعين دواءً، ونصفها تقريباً نادر في السوق المحلي. كيف تبني نظاماً يميّز بينها؟

إعادة صياغة السؤال: لماذا لا نستخدم البيانات غير الموسومة؟

لنبدأ من صفحة الأرقام الحقيقية. التدريب على صور القطط غير الموسومة:

البند	التدرّب الخاضع للإشراف	التدرّب الذاتي
البيانات المطلوبة	مئات آلاف موسومة	مئات آلاف غير موسومة
العمل البشري	أسبوع عمل لكل ألف صورة	لا عمل بشري
زمن التدريب	ساعات	أضعاف
جودة النتيجة	أعلى عادةً	أقلّ قليلاً
مرونة المهام	محدودة بالوسم	واسعة

تكلفة الوسم البشري مقارنة بالتكلفة الحسابية

الخلاصة التي نخرج بها: التعلّم الذاتي لا يلغي التعلّم الخاضع للإشراف، بل **يقلّل اعتماده** عليه. النموذج الذي تعلّم نفسه على مئات الملايين من البيانات غير الموسومة متدرّباً على بيانات عامة؛ ثم تُكيّفه على آلاف الصور الموسومة فيصبح خبيراً في مهمّتك.

موزّد حمصيات ولديها ٤٠٠ ألف صورة للرفوف الفارغة على المتاجر، و١٨٠ صورة موسومة بالضرر. أيّ الطريقتين تختار: نموذجاً خاضعاً للإشراف على الـ١٨٠، أم نموذجاً تعلّم نفسه على الـ٤٠٠ ألف ثم ضُبط؟ وما الذي ستقيسه لتعرف أنّك لم تخسر؟

الجواب: الخيار الثاني، بشرط تقيس الأداء على مجموعة تحقّق مستقلّة لم يرها النموذج أبداً في أيّ مرحلة. المقارنة يجب أن تكون على نفس المجموعة ونفس المقياس. إن إن بدأ نموذج التعلّم الذاتي متفوّقاً على مجموعة التحقّق التي استُخدمت في ضبطه الدقيق، فالحكم ناقص.

> ★ **المبدأ الحاكم**
>
> كلّ نموذج تعلّم ذاتي هو في النهاية نموذج **مدرَّب مسبقاً**. والفصل بين «التدرّب المسبق» و«التدرّب الخاضع للإشراف» فصلٌ اصطلاحي في المراحل لا في الطبيعة. ما يهمّ هو: من أين جاءت الأوزان؟

التعلّم بالتباين: كيف يقارن النموذج شيئاً لا نعرفه

المشكلة الجوهرية في التعلّم غير الخاضع للإشراف واضحة: لا يوجد «جواب صحيح» نقارنه. كيف تقارن صورتين تعرف أنّهما نفس القطة إن لم تكن موسومتين؟ الحلّ في **المهمّة المبديلة**: نركّب مهمّة يتيسّر حلّها وتُفيد في تمثيلٍ جيّد.

في **SimCLR** (٢٠٢٠) تكون المهمّة المبديلة بسيطة: خذ الصورة، وجّه نسختين منها (بتعزيزات مختلفة: قص، قلب، تغيير لون)، ثم اطلب من الشبكة أن تُنتج متجهين متطابقين للصورتين. النموذج الذي ينجح في ذلك تعلّم خاصية **الثبات**: خاصية لا تتغيّر بتغيّر الإضاءة أو القص.

$$L_{\text{contrastive}} = -\log\left(\exp(\text{sim}(z_i, z_i^+)/\tau) / \Sigma_j \exp(\text{sim}(z_i, z_j)/\tau)\right)$$

ما معنى كل رمز؟ z_i هو تمثيل الصورة الأصلية، و z_i^+ تمثيل نسختها المعزَّزة، و sim هو تشابه التمثيلات (عادةً جيب التمام)، و τ هو معامل الحرارة الذي يضبط حدّة التوزيع، والمقام يجمع على كل الصور السالبة في الدفعة. **لماذا تهمّنا؟** لأنّ الدالة تدفع التمثيلات المتشابهة إلى التقارب وتبعد المتشابهة من غيرها، فيتشكّل فضاء تمثيل مُجمَّع. **خطأ شائع عندها:** اختيار τ كبير جداً (مثل ١) فيتحوّل التدرّج إلى شبه معدوم. القيمة المعتادة بين 0.05 و 0.2.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class SimCLR(nn.Module):
    """نموذج تعلّم بالتباين: عمود ترميز + رأس إسقاط."""
```

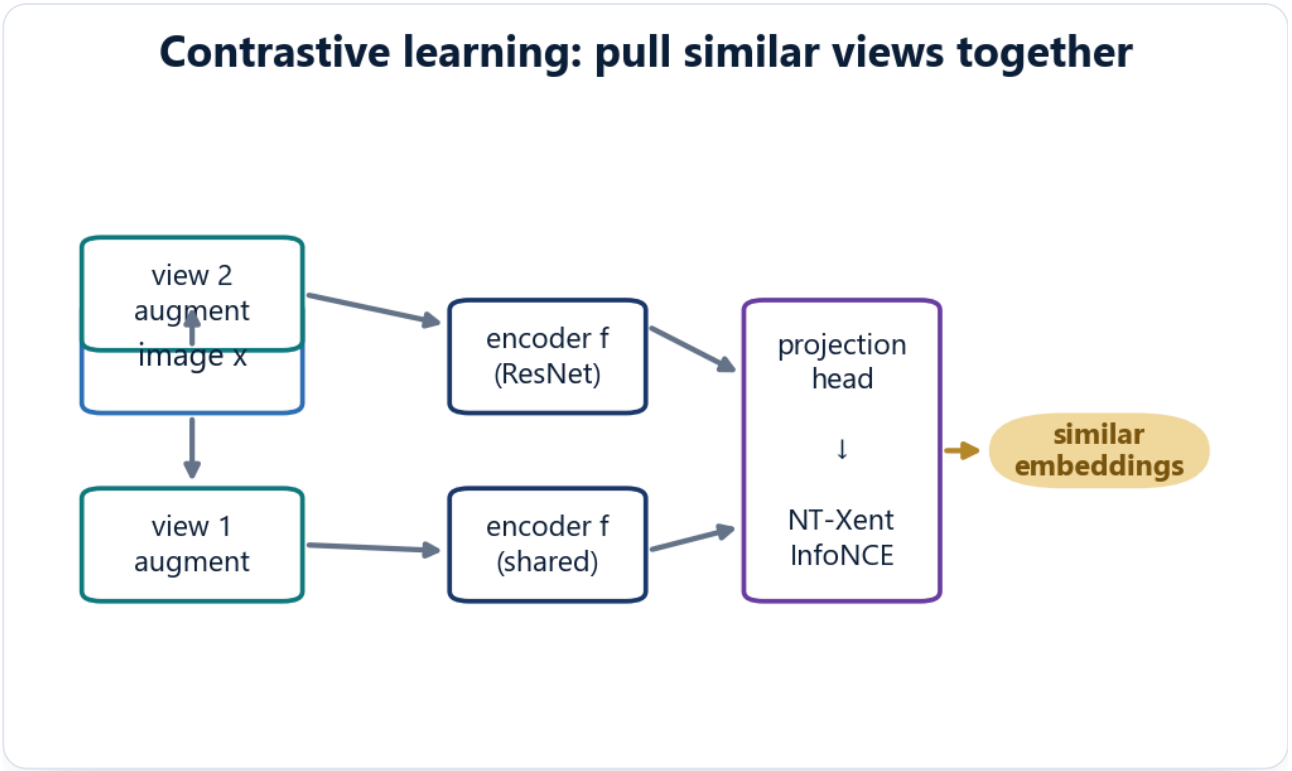
```
    def __init__(self, backbone, proj_dim=128):
        super().__init__()
        self.backbone = backbone
        self.projector = nn.Sequential(
            nn.Linear(backbone.output_dim, 512),
            nn.ReLU(),
            nn.Linear(512, proj_dim),
        )

    def forward(self, x1, x2):
        z1 = F.normalize(self.projector(self.backbone(x1)), dim=1)
        z2 = F.normalize(self.projector(self.backbone(x2)), dim=1)
        return z1, z2


def nt_xent_loss(z1, z2, temperature=0.1):
    """خسارة InfoNCE على دفعتين متزامنتين."""
    n = z1.size(0)
    z = torch.cat([z1, z2], dim=0)                  # (2n, d)
    sim = z @ z.t() / temperature
    sim.fill_diagonal_(-1e9)                        # استبعاد التمثيل نفسه
    targets = torch.arange(n, device=z1.device)
    targets = torch.cat([targets + n, targets])     # كل صورة ترى نسختها موجبة
    return F.cross_entropy(sim, targets)

# الناتج: tensor(0.6931) عند بداية التدريب ≈ ln(2)
```

ماذا يحدث هنا؟ `fill_diagonal_` تستبعد الحالة التي تكون فيها الصورة مع نفسها، «يطابق» النموذج يتعلّم التطابق التام مع كل صورة. و `targets` مزدوجة لأنّ لكل صورة نسختان: نسخة في الدفعة الأولى وأخرى في الثانية. **لماذا هذه القيمة؟** لأنّ كل صورة يجب أن تكون قريبة من نسختها وبعيدة عن صور الدفعة الأخرى؛ وهذا ما يسمّى «المهمّة المقابلة».



يقرأ الرسم من الأسفل إلى الأعلى: الصورة الأصلية في الوسط، تتفرّع إلى نسختين بعد تعزيزين مختلفين، ثم يمرّ كلٌّ منهما بعمود الترميز نفسه، ثم برأس الإسقاط، فينتج متجهين. الشريط الرمادي يمثّل دالّة التباين التي تُقرّب المتجهين (السهم الموجّه للداخل) وتبعد كل زوج آخر (الأسهم المتباعدة).

◆ لماذا نحتاج رأس إسقاط منفصل؟

لاحظ في الشيفرة أن رأس الإسقاط (projector) يُستعمل لحساب الخسارة فقط، بينما التمثيل المستخدم في المهامّ اللاحقة هو مخرج `backbone` قبل الإسقاط. هذا ليس تفاصيل عشوائية: أبحاث لاحقة أظهرت أنّ إسقاط البُعد المنخفض يجبر عمود الترميز على الاحتفاظ بمعلومات أكثر عن المثال نفسه (وهو المطلوب في مهامّ التمييز)، فيتحسّن أداء التعلّم النهائي. وقد أثبتت التجارب أنّ النتائج بلغت ٩١٪ على CIFAR-10 بهذا التوصيف.

النماذج التوليدية المخفيّة: حين يملأ النموذج الفراغات

هناك عائلة ثانية من أساليب التعلّم الذاتي أكثر بساطة وأقوى أحياناً: **النماذج التوليدية المخفيّة**. الفكرة معكوسة: بدل أن نقارن تمثيلين، نُخفي جزءاً من المدخل ونطلب من النموذج أن يستعيده. هذا يحوّل كل عيّنة غير موسومة إلى عيّنة موسومة تلقائياً: التخفيُق هو الموسومة.

الجانب	التعلّم بالتباين (SimCLR)	النماذج المخفيّة (BERT/MAE)
المهمّة	تمييز النسخة الموجبة	استعادة الجزء المُخفي
عدد العيّنات	يحتاج بيانات ضخمة	يعمل بكفاءة أكبر
التمثيل الناتج	متّجه واحد	متجّه لكل رمز
الاستخدام	استرجاع وتصنيف	لغة ورؤية
المثال المرجعي	SimCLR	BERT. MAE

مقارنة بين مقاربتَي التعلّم الذاتي

Python

```python
import torch
import torch.nn as nn

class MaskedImageEncoder(nn.Module):
    """يغطّي رقعاً عشوائية ثم يطلب من العمود استعادتها."""

    def __init__(self, patch_size=16, img_size=224, dim=192, mask_ratio=0.75):
        super().__init__()
        self.patch_size = patch_size
        self.mask_ratio = mask_ratio
        self.patch_embed = nn.Conv2d(3, dim, kernel_size=patch_size, stride=patch_size)
        n_patches = (img_size // patch_size) ** 2
        self.mask_token = nn.Parameter(torch.zeros(1, 1, dim))
        self.encoder = nn.TransformerEncoder(
            nn.TransformerEncoderLayer(dim, 4, batch_first=True), num_layers=4
        )

    def forward(self, x):
        patches = self.patch_embed(x).flatten(2).transpose(1, 2)
        n = patches.size(1)
        keep = int(n * (1 - self.mask_ratio))
        visible = patches[:, :keep]
        masked = self.mask_token.expand(x.size(0), n - keep, -1)
        tokens = torch.cat([visible, masked], dim=1)
        out = self.encoder(tokens)
        return out[:, keep:]          # تمثيل الرقع المخفيّة فقط
```

ماذا يحدث هنا؟ `mask_ratio=0.75` يعني أنّ ثلاثة أرباع الرقع تُخفى. **لماذا هذه القيمة؟** لأنّ النسبة العالية تجبر النموذج على الفهم العميق بدل الاعتماد على رقع الجوار. أبحاث `MAE` أظهرت أنّ إخفاء ٧٥٪ يتفوّق على إخفاء ٥٠٪ أو ٢٥٪ في مهامّ التصنيف اللاحقة.

> ✓ **متى تختار التباين ومتى تختار الإخفاء؟**
>
> اختر **التباين** حين تكون المهمة النهائية استرجاعاً (تصنيف عام، كشف التكرار). واختر **الإخفاء** حين تحتاج تمثيلاً دقيقاً عن بنية البيانات نفسها (نصّ، جزيئات، رموز). في الممارسة، النماذج التأسيسية الحديثة تجمع بين الاثنين: هدف تنبّؤ مُقنَّع (مثل BERT) و هدف تباين (مثل SimCLR).

النماذج التأسيسية: من نموذج متخصّص إلى نموذج عام

النموذج التأسيسي (نموذج تأسيسي) هو نموذج كبير مدرَّب على بيانات واسعة ومهمّات متنوعة، ويصلح لعشرات من المهامّ بعد تعديل بسيط. الفرق الجوهري بينه وبين نموذج التعلّم النقلي التقليدي: النطاق والبيانات.

> لديك نموذج تأسيسي يتّسع لـ ٧ مليارات معامل، ومهمّتك تصنيف رسائل البريد إلى ٢٠ فئة. لماذا لا تفصل الشبكة كلّها؟ ولماذا لا تتجمّد الطبقات الأولى كالمعتاد في التعلّم النقلي؟ وماذا يبقى في الذاكرة؟

لأنّ الطبقات الأولى في النماذج التأسيسية تتعلّم بنية اللغة أو الصورة نفسها (tokenizer داخلي، تدرّجات، حدود الكلمات)، لا مهمّة محدّدة. تجميدها يجمّد معرفة ضرورية. الأفضل في النماذج التأسيسية: **تعديل أوزان الطبقات العليا فقط** مع LoRA أو الضبط الكامل Heads، وإبقاء الطبقات السفلى Inference الاستدلال دائماً. أضف إلى ذلك تفعيل استرجاع (RAG) أو تعلّم السياق (in-context learning) إن كانت مهمّتك تحتاج معرفة محدّثة.

```python
import torch
from torch import nn

class FoundationModelAdapter(nn.Module):
    """رأس خفيف فوق نموذج تأسيسي مجمّد."""

    def __init__(self, foundation, num_classes=20, hidden=256):
        super().__init__()
```

```
        for p in foundation.parameters():
            p.requires_grad = False
        self.foundation = foundation
        dim = foundation.config.hidden_size
        self.head = nn.Sequential(
            nn.Linear(dim, hidden),
            nn.GELU(),
            nn.Dropout(0.1),
            nn.Linear(hidden, num_classes),
        )

    def forward(self, input_ids, attention_mask):
        with torch.no_grad():
            out = self.foundation(input_ids=input_ids,
                                  attention_mask=attention_mask).last_hidden_state
        pooled = out[:, 0]          # تمثيل الرمز الأول
        return self.head(pooled)

# الناتج: logits shape (batch, 20)
```

ماذا يحدث هنا؟ `with torch.no_grad()` يمنع حساب التدرّجات على النموذج التأسيسي، فيتوفّر جزء كبير من الذاكرة. **لماذا هذه القيمة؟** لأنّ الرمز الأول في نماذج BERT يمثّل الجملة كاملة (رمز `[CLS]`).

سيناريو تطبيقي: فهرسة وثائق منظّمة بلا وسم

من البيانات إلى قرار

الموقف: مكتبة جامعية فيها ٢٢٠ ألف ورقة بحثية بصيغة PDF بلا أيّ وسوم موضوعية. المكتبة تريد نظاماً يقترح لكلّ باحث ١٠ أوراق ذات صلة. مكتبة فيها ٢٢٠ ألف ورقة؟ لا أحد.

القرار: ثلاث مراحل. الأولى، نستخرج نصّ كلّ ورقة ونقطّعه إلى مقاطع بطول ٥١٢ رمز. الثانية، نحسب **تمثيلاً** لكلّ مقطع بنموذج تعلّم تباين مدرَّب مسبقاً (مثل نموذج استرجاع عام): المتجّه الذي يمثّل «محتوى» المقطع لا «شكل» كلماته. الثالثة، نبني فهرساً متجهياً (FAISS أو HNSW) ونبحث بأقرب الجيران.

القياس: نقيس `Recall@10` على ٥٠ استعلاماً مُعنوناً يدوياً. وقبل إطلاق النظام نُضيف مرحلة **إعادة الترتيب** (reranking) بنموذج لغوي: البحث المتّجه يعيد ١٠٠ مرشّح، ثم يرتّب النموذج أفضل ١٠. الفرق في هذه المرحلة وحده عادة ما يكون ١٠ إلى ١٥ نقطة مئوية في `nDCG`.

تمرين 1 — كشف خطأ في خسارة التباين

1. طبّق `nt_xent_loss` على دفعتين `z1` و `z2` متطابقتين تماماً (نفس العيّنة مرّتين).
2. قِس الخسارة، ثم قِسها بعد حذف `fill_diagonal_` .
3. فسّر أيّ القيمتين تدلّ على تعلّم سليم.

▲ الحل النموذجي

مع `fill_diagonal_` تبقى الخسارة عند قيمة عالية (تقارب `ln(2n)`) لأنّ كل صورة في الدفعة غير متطابقة تُعامَل كسالبة. بلا `fill_diagonal_` تصبح الخسارة منخفضة جداً (قريبة من الصفر) لأنّ النموذج وجد «الجواب»: التمثيلات المتطابقة لنفسه. هذه حالة خطرة عملياً: النموذج يتعلّم أن يطابق كل صورة مع نفسها بدل نسختها المعزَّزة، فلا يتعلّم أيّ ثبات.

القاعدة: بعد حذف `fill_diagonal_` انظر هل الخسارة قريبة من `ln(2n)` (سليم) أم قريبة من الصفر (خطأ).

تمرين 2 — تصميم مهمّة تعلّم ذاتي لمصنع

1. صمّم مهمّة تعلّم ذاتي لخطّ إنتاج فيه ١٢٠٠٠ صورة على ناقل، و١٥٠ صورة معطوبة موسومة.
2. اذكر دفعات ، ولماذا تعزّز التباين هنا.
3. اقترح مقياساً للتقييم لا يعتمد على الـ١٥٠ الموسومة.

▲ الحل النموذجي

نسختين من السلعة نفسها بصورتين (إضاءة ووضوح مختلفين)، والطلب من النموذج أن يُنتج تمثيلين متطابقين. فالنسخة الموجبة سلعة، سلب كلّ بضائع الدفعة الأخرى. التعزيز هنا مناسب لأنّ الإضاءة تتغيّر فعلياً على الخطّ بين الفترتين، فالنموذج يتعلّم أن يتجاهل الإضاءة (وهي تشويش غير مرغوب) ويركّز على خدوش وتلف حقيقي.

المقياس البديل: بعد التدريب على المهمة الهدف، نقيس **خسارة** على عيّنات سلع غير موسومة. كلّما قلّت الخسارة على بيانات الإنتاج، دلّ ذلك على تمثيل عامٍّ جيد. وبما أنّ الـ١٥٠ المعطوبة يمكن استخدامها للتحقّق النهائي، فلدينا شبكة أمان.

أداة عملية: استخراج التمثيلات لمشروعك

أشهر استخدام للتعلّم الذاتي ليس بناء نموذج عظيم، بل بناء **فهرس متجهي** يفهرس بياناتك غير الموسومة. الكود التالي هو العمود الفقري لأغلب أنظمة الاسترجاع في الممارسة.

Python

```
import torch
import torch.nn.functional as F


@torch.no_grad()
def embed_dataset(model, loader, device="cpu"):
    """يحسب تمثيل كلّ عيّنة ويُرجع مصفوفة (n, d) مع تسمياتها."""
    model.eval()
    vectors, labels = [], []
    for x, y in loader:
        x = x.to(device)
        emb = model.backbone(x) if hasattr(model, "backbone") else model(x)
        emb = F.normalize(emb, dim=1)
        vectors.append(emb.cpu())
        labels.append(y)
    return torch.cat(vectors), torch.cat(labels)


def knn_search(query_emb, matrix, k=10):
    """أقرب k (الضرب النقطي على متجهات مطبَّعة) جيران بقياس جيب التمام."""
    sims = query_emb @ matrix.T
```

```
    top_values, top_indices = sims.topk(k, dim=-1)
    return top_indices, top_values
```

ماذا يحدث هنا؟ `torch.no_grad()` يمنع حساب التدرّجات، و `F.normalize` يجعل كلّ متجه على كرة وحدة، فيصبح الضرب النقطي مقياس جيب تمام. **لماذا هذه القيمة؟** لأنّ البحث في فضاء كروي يجعل الترتيب حسّاساً للاتّجاه لا للطوله، وهو ما يجعل المقاييس متجهية.

ملخّص الفصل

- التعلّم الذاتي يقلّل اعتمادنا على الوسم البشري، ولا يلغيه.
- التعلّم بالتباين يطلب تطابق تمثيلين لصورتين معزَّزتين، ما يعلّم الثبات.
- رأس الإسقاط يُستعمل في الخسارة فقط، لا في التمثيل النهائي.
- النماذج المخفيّة (BERT، MAE) تحوّل كلّ عيّنة إلى عيّنة موسومة عبر التخفيق.
- إخفاء ٧٥٪ من الرقع يعطي تمثيلاً أدقّ من إخفاء ٥٠٪.
- العمود المجمّد يُعدَّل من الرأس والأعلى فقط، مع LoRA أو استرجاع.
- قوة نموذج التعلّم الذاتي تُقاس على مجموعة تحقّق لم تُستخدم في الضبط.